

2-1 Agent

Agent八股整理¹

LLM与大模型应用基础

- 01 Transformer 与 LLM 基础（重要）
- 02 推理机制与采样参数（了解）
- 03 上下文窗口、KV Cache 与长上下文问题
- 04 幻觉、对齐与指令跟随
- 05 Prompt Engineering
- 06 Context Engineering
- 07 结构化输出与 Function Calling（重要）
- 08 模型选型与路由

Agent核心范式

- 01 Agent定义、边界与适用场景
- 02 Workflow vs Agent
- 03 ReAct、Plan-and-Execute、CodeAct（重要）
- 04 Task Decomposition 与 Planning
- 05 Tool Use 与 Action Execution
- 06 Memory体系
- 07 Multi-Agent 架构
- 08 Human-in-the-Loop
- 09 Agent 状态机与终止条件

知识-记忆-检索

- 00-高频面试题（汇总）
- 01-RAG基础
- 02 Chunking-Embedding-Rerank
- 03 Hybrid-Retrieval与Query-Rewrite（了解）
- 04 RAG-vs-Memory（重要）
- 05 长期记忆与用户记忆（重要）
- 06 知识库权限与多租户隔离（了解）

07 Agentic-RAG与GraphRAG（重要）

08 RAG常见失败模式

协议-runtime-工程化

00 协议-runtime-工程化高频面试题（汇总）

01 Function Calling 与 Tool Schema（重要）

02 MCP 原理与实践（重要）

03 A2A 与 Agent 互操作（前沿了解）

04 Session-State 与 Context 生命周期

05 Workflow-Orchestration 与 Durable Execution

06 任务调度、重试、幂等、回滚（重要）

07 异步任务、长任务、流式执行（重要）

08 Agent Runtime 设计（重中之重）

09 Skills、插件化与能力封装（重要）

01 Transformer 与 LLM 基础（重要）

阅读目标：把为什么是 Transformer、LLM 到底学到了什么、为什么 Decoder-only 成为主流讲清楚，做到既能回答八股，也能把它和工程实践连起来。

agent开发岗位了解即可，当然时间充裕还是希望大家细品，agent开发中比较偏算法的八股可能就这几个了

一、这篇在面试里到底考什么

校招里问 Transformer 与 LLM 基础，面试官通常不是想听你复述Attention Is All You Need那篇论文的摘要，而是想判断四件事。

第一，你是否真的理解 LLM 的工作对象是 token 序列，而不是句子，知识点这种人类语义单位。很多同学会直接说大模型理解自然语言，但工程上模型首先处理的是离散 token，经 tokenizer 编码后进入 embedding，再通过一层层注意力与前馈网络做变换，最后输出下一个 token 的概率分布。你只要把这个链条说顺，面试官会立刻判断你不是只会背概念。

第二，面试官在看你有没有从架构到工程的连接能力。比如为什么 Transformer 能替代 RNN/CNN，不只是因为效果更好，更关键是它取消了时间步递归依赖，训练时更容易并行；而推理时虽然生成仍是逐 token 自回归，但注意力机制让模型在每一层都能直接访问历史上下文。你如果能继续补一句代价是注意力随序列长度增长会带来较高的时间与显存开销，这也是长上下文、KV Cache、prefix cache 等工程优化出现的背景，那就从算法题切到了工程题。

第三，面试官想确认你理解 LLM 的能力从哪里来。如果你只会说参数量大，所以智能，这属于不及格答案。更好的回答应该是：自回归预训练让模型在海量语料上学习统计规律；规模扩展让模型获得更强的模式归纳和 in-context learning 能力；后续的指令微调、偏好对齐、工具接入，则把原始语言能力变成更可用的产品能力。也就是说，LLM 的能力不是某个单点 magic，而是架构 + 数据 + 训练目标 + 规模 + 后训练共同作用的结果。

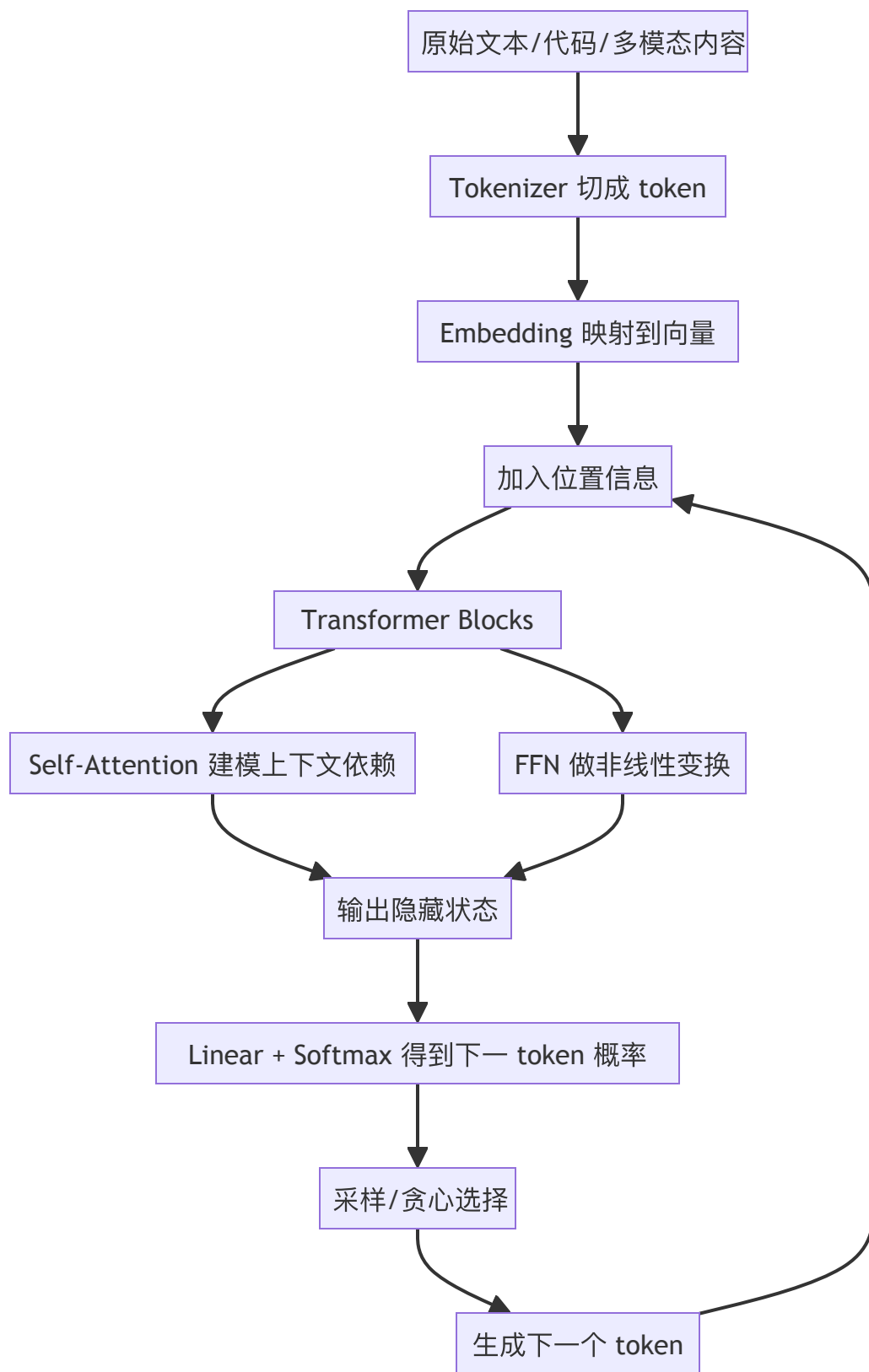
第四，面试官经常借这道题看你后续能不能接更深的问题。比如你如果说到 RoPE，可能会被追问长上下文；说到 decoder-only，可能会被问为什么生成式应用偏爱它；说到 tokenizer，可能被问中文分词、代码 token、上下文成本；说到 scaling，可能被问为什么更大模型并不总是生产最优解。所以这道题表面是基础，实际是 02 章后续所有问题的入口。

一个比较好的答法是：先用一句话定义 LLM，再讲 Transformer 的核心部件，再讲为什么大多数生成式产品采用 decoder-only 架构，最后补一嘴训练与后训练阶段。这样层次最清楚，也最容易被继续追问。

给大家一个面试的口语版话术：

大语言模型本质上是基于 Transformer 的自回归语言模型，核心任务是根据已有 token 预测下一个 token。Transformer 相比 RNN 更适合大规模并行训练，核心模块包括 embedding、位置编码、self-attention、FFN、残差和归一化。现代生成式产品大多采用 decoder-only，因为 next-token 目标和开放式生成高度一致。LLM 的能力来自预训练、规模扩展和后训练对齐，不是单纯因为参数大。它强在通用模式归纳和 in-context learning，但也因为参数里不是显式知识库，所以会出现幻觉、过时和格式不稳定，这就是为什么后续需要 prompt engineering、context engineering、RAG 和工具调用。

二、总体看一下这块的思维导图



从应用工程视角看，LLM 最核心的事情其实只有一句：给定一个 token 序列，预测下一个最可能出现的 token。模型并不知道摘要问答SQL 生成Agent 规划这些任务标签，它只是通过训练把大量任务都转写成基于上下文继续生成的统一接口。提示词工程、上下文工程、结构化输出、工具调用，本质上都是在操控这个统一接口。

因此理解 LLM 最好的方式，不是把它看成神秘智能体，而是把它看成一个非常强的条件概率模型：

$P(\text{next_token} \mid \text{previous_tokens}, \text{parameters})$

这条公式背后藏着三层重要认识。第一，模型能力高度依赖上下文组织方式，这就是为什么 prompt/context engineering 重要。第二，模型输出不是一个唯一答案，而是一个概率分布，所以采样参数会影响生成风格与稳定性。第三，模型参数里存的是压缩后的统计规律，不是一条条可显式索引的事实表，所以它会幻觉、会过时、会在缺证据时自信生成。

三、什么是语言模型，什么又是大语言模型

语言模型不是从大模型时代才出现的。传统 n-gram、RNN/LSTM、Transformer 其实都可以做语言建模。所谓语言建模，就是估计一个 token 序列出现的概率，或者更具体一点，给定前文预测后文。大语言模型中的大，首先体现在参数规模、训练数据规模和训练算力规模上，其次体现在它不是只针对单一任务训练，而是在通用语料上做大规模预训练，再通过后训练适配更多任务。

这里要特别区分三个层次。

第一层是 tokenizer。模型并不直接读取字和词，而是读取 token。不同 tokenizer 策略会影响上下文成本、跨语言表现、代码表现与边界处理。比如中英文混合、特殊符号、长数字串、URL、代码片段，经常会被切成很不同的 token 序列，直接影响成本和效果。面试里如果问为什么同一段中文在不同模型里 token 数不一样，根本原因就在 tokenizer 不同。

第二层是 base model。它通过预训练学到语言分布和广泛世界知识，但未必擅长听指令，也未必稳定遵循格式。你可以把它理解为很会续写，但不一定好用。

第三层是 instruction/aligned model。通过监督微调、偏好优化、规则约束等方法，模型更愿意遵循系统指令、拒绝不当请求、输出更贴近产品预期的回答。很多校招同学把 LLM 与 Chat 模型画等号，这是不精确的。Chat 只是 LLM 的一个对话化产品形态。

从面试答题看，一个非常好的表述是：大语言模型不是简单把参数堆大，而是把通用语言建模、可迁移能力、上下文内学习、对齐与工具接入叠加到了同一个模型接口上。这样既体现理论理解，也自然过渡到后续 Agent 话题。

四、Transformer与RNN/LSTM/CNN的对比

Transformer 成为 LLM 底座，有两个最重要的理由：表达能力与并行效率。

RNN/LSTM 的问题在于，序列建模天然按时间步递归，虽然可以处理变长序列，但训练时很难大规模并行，而且长距离依赖容易衰减。CNN 虽然并行性更好，但在建模远距离依赖时通常需要堆很多层，感受野扩展不够直接。Transformer 用自注意力直接让当前位置与序列中其他位置建立联系，从路径长度上看，任意两个位置之间的信息交互都更短，这对捕获长程依赖非常关键。论文《Attention Is All You

Need》给出的核心贡献并不只是提出了 attention，而是把序列建模的主干彻底换成了 attention-first 的结构。

但只说注意力更强还不够，面试官一般更想听到工程含义。Transformer 在训练阶段的并行性非常好，因为整个输入序列可以同时经过线性变换、同时计算 attention score、同时做前馈网络。这使得它更适配 GPU/TPU 这种大规模并行硬件，成为后续 scaling 的前提。很多时候不是某个架构理论最优，而是它在硬件上最容易被放大，最终就赢了。

当然，Transformer 也不是没有代价。标准自注意力的计算与显存开销通常会随着序列长度增长而显著增大，所以大上下文成本很高。今天大家讨论长上下文、稀疏注意力、滑动窗口、KV Cache、prefix caching，本质都在修这个代价。也就是说，Transformer 赢在统一、强大、可扩展，但它的工程负担从来没有消失，只是被后续优化体系接住了。这是一种工程上的 trade-off

五、Transformer 的核心模块，面试要讲到什么粒度

1. Embedding：把离散 token 映射到连续空间

Tokenizer 输出的是 token id，本身只是整数。Embedding 层把每个 token 映射成高维向量，让模型可以在连续空间里学习相似语义、相似模式、相似上下文角色。比如北京和上海的某些维度可能更接近，for 和 while 在代码语料里也会形成相似模式。

面试里一般不需要把 embedding 讲得太数学化，但最好强调两点。第一，embedding 不是词典释义，而是训练中不断更新的分布式表示。第二，embedding 只解决是什么 token，不解决它在第几个位置，位置问题需要单独编码。

2. Position Encoding / RoPE：让模型知道顺序

如果只有 token 向量，没有位置信息，那么我喜欢你和你喜欢我对模型来说几乎只是同一堆词袋。Transformer 本身没有天然顺序感，所以必须引入位置编码。早期论文使用的是绝对位置编码，后来很多 LLM 使用相对位置编码或 RoPE (Rotary Position Embedding)。RoPE 的常见优点是更适合外推到更长上下文，在现代 LLM 中非常常见。

面试里不一定非要背 RoPE 公式，但你要知道：位置编码不是可有可无，而是序列建模不可缺的顺序先验。如果被问为什么长上下文外推会出问题，你也可以从位置编码泛化能力切入。

3. Self-Attention：Transformer 的心脏

Self-attention 的直觉非常重要。对序列中某个位置来说，它在生成新的表示时，不只看自己，还会参考当前上下文里哪些 token 更相关。通过 Q、K、V 三组向量，模型先计算我该关注谁，再汇聚这些位

置的信息，得到新的上下文表示。

经典公式可以写成：

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

你不一定要把公式背得一字不差，但建议知道每一项的意义：

Q 是我当前想找什么，K 是别人身上有哪些可匹配的特征，V 是别人真正携带的信息。

softmax 把相关性归一化成权重，最后对 V 做加权求和。

面试里最容易被追问的是两个点。第一，为什么要除以 $\sqrt{d_k}$ ？因为维度增大时点积幅度会变大，softmax 更容易饱和，缩放能让训练更稳定。第二，为什么 multi-head 有用？因为不同 head 可以学习不同关系，比如语法依赖、共指关系、代码作用域、局部模式、长程依赖等。不是每个头都负责一种固定语义，但多头机制确实扩大了表征子空间。

4. FFN、Residual、LayerNorm：不是配角

很多同学只会讲 attention，却忽略 FFN、残差连接和归一化。其实在现代 Transformer 里，这些模块都很关键。FFN 为每个位置提供更强的非线性变换能力，可以理解为 attention 负责信息路由，FFN 负责局部加工。残差连接帮助梯度稳定传播，也让网络更容易训练更深。LayerNorm 则帮助数值分布更稳定。

如果面试官问 Transformer 为什么能堆很深，你完全可以回答：不仅因为 attention 机制本身强，还因为有残差、归一化、优化器和训练工程共同支撑。真正的大模型从来不是单模块胜利，而是系统工程胜利。

5. Masked Self-Attention：为什么 decoder-only 能做生成

对于生成式模型，训练和推理都必须保证当前位置不能偷看未来 token。这就是 causal mask / autoregressive mask 的意义。它会把未来位置屏蔽掉，使模型只能基于历史上下文预测下一个 token。于是模型学到的是严格的左到右生成机制。

这点非常关键，因为很多生成式产品本质上依赖的就是这个机制：输入 prompt，模型按顺序往后续写。面试里如果问为什么 decoder-only 架构适合聊天生成，你只要抓住三点：统一的 next-token 目标、天然适合开放式生成、工程生态成熟。

六、为什么今天大多数生成式产品偏爱 decoder-only

BERT 这类 encoder-only 模型非常擅长理解任务，比如分类、匹配、抽取；T5/BART 这类 encoder-decoder 模型在翻译、摘要、文本到文本任务上也很强。但在通用聊天、代码生成、Agent 执行、开放式续写这些场景里，decoder-only 最终成为主流，原因主要有四个。

第一，训练目标统一。decoder-only 直接做自回归语言建模，和实际生成任务非常一致。它不用把训练目标拆成复杂的多任务形式，预训练与部署链路更统一。

第二，扩展性强。随着参数、数据和算力规模增大，decoder-only 在通用能力、few-shot/in-context learning、代码与工具使用上的收益非常明显。GPT-3 把这种只靠大规模语言建模就出现强迁移能力的现象推到了行业中心。

第三，产品接口简单。对上层应用来说，给上下文，继续生成就是一个足够统一的接口。无论问答、写作、规划、JSON 抽取还是工具调用，都可以映射到这个接口上。

第四，生态成熟。推理框架、KV Cache、prefix cache、流式输出、采样控制、函数调用等能力，今天基本都围绕 decoder-only LLM 形成了完整工业链。

这并不意味着 encoder-only 或 encoder-decoder 过时。做 rerank、embedding、分类、小模型理解任务时，它们仍然很重要。校招面试里更稳妥的表述是：生成式主流程偏好 decoder-only，但生产系统往往是多模型协同，而不是一个 decoder-only 模型包打天下。

七、预训练、规模扩展与能力形成

1. 预训练目标：next-token prediction 为什么足够强

很多初学者会疑惑：只做预测下一个 token 这么简单的任务，为什么能学出问答、翻译、代码、推理甚至工具使用能力？这恰恰是 LLM 最反直觉也最重要的地方。因为海量文本本身已经隐含了知识、任务格式、推理痕迹、程序模板、对话模式和世界规律。模型在压缩这些分布时，学到的不是某个单独任务，而是任务之间共享的统计结构。

当然，这里一定要讲清楚边界。next-token prediction 学到的是分布规律，不是严格逻辑引擎，也不是事实数据库。所以模型可以看起来会推理，但并不保证每次都对；可以像是知道知识，但知识可能过时或混杂。你在面试里把这层边界说出来，会显得非常清醒。

2. Scaling Laws：为什么大模型突然好用了

经验上，模型规模、数据规模、训练计算量增加，性能会遵循一定的可预测趋势。Kaplan 等人的 scaling laws 说明 loss 会随着模型、数据、算力扩展而按幂律下降；Hoffmann 等人的 Chinchilla 工作进一步强调，在固定算力预算下，参数量和训练 token 都要一起扩展，很多只堆参数不喂足数据的模型其实是 undertrained。这个认知对工程很重要：不是模型越大越值钱，而是要看训练是否 compute-optimal。

对校招面试来说，不需要你背论文细节，但最好知道两个结论。第一，大模型能力的跃迁很大程度上来自规模化训练，而不是某一个小技巧。第二，规模扩展不是无脑增参数，数据量和计算预算同样关键。

你如果能顺口补一句今天生产选型更关注单位成本智能，而不是绝对参数规模，会和 08 章模型路由自然衔接。

3. In-Context Learning：为什么 few-shot 会生效

GPT-3 让行业意识到：模型可以仅通过上下文里的指令和示例，在不更新参数的情况下快速适应新任务。这种能力通常被称为 in-context learning。你可以把它理解为模型在运行时临时模拟某种任务分布，而不是把新知识真的写回参数。

这个点对 Agent 和应用开发非常重要。因为提示词、示例、工具 schema、检索结果，本质都是在利用模型的 in-context learning 能力进行运行时编程。面试里如果被问 Prompt Engineering 为什么有用，根子就在这里。

八、现代 LLM 不止一种：你至少要知道这些分类

1. 按任务接口分：encoder-only / decoder-only / encoder-decoder

最常见的问法是：三种架构怎么区分，分别适合什么任务？

一个稳的回答如下：

- encoder-only：更擅长理解输入整体语义，常用于分类、检索、匹配、embedding、rerank；
- decoder-only：更擅长开放式生成，是聊天、代码生成、Agent 的主流底座；
- encoder-decoder：把输入理解与输出生成分开，在翻译、摘要、结构化文本转换任务中很自然。

2. 按参数激活方式分：Dense 与 MoE

Dense 模型每个 token 都走全部参数路径，MoE（Mixture of Experts）则只激活部分 expert。MoE 的核心优势是以较低推理成本获得更大总参数容量，缺点则是训练、路由、系统复杂度更高。校招基础面不一定深挖，但如果你面的是平台/推理方向，这个点常会被问到。

3. 按输入模态分：纯文本、多模态、工具增强

到 2026 年，主流商用模型普遍已支持文本、图像甚至音视频等多模态输入，同时提供结构化输出、函数调用、Web/File/Computer Use 等工具能力。这里要注意一个面试陷阱：多模态不等于 Agent。多模态只是输入输出能力扩展；Agent 还涉及规划、状态、工具循环、终止条件等更高层机制。

九、面试高频题：怎么答更像工程同学

问题 1: Transformer 为什么比 RNN 更适合大模型时代?

回答重点不是更先进，而是训练可并行、长程依赖路径更短、更适合 GPU/TPU 扩展。再补一句代价是长序列成本高，所以需要缓存与长上下文优化，就有工程味了。

问题 2: Self-Attention 在干什么?

一句话解释：对每个位置，模型都在根据当前查询内容，从上下文里找最相关的 token，并把它们的信息加权汇总成新的表示。然后再看公式与 Q/K/V 含义。

问题 3: 为什么要多头注意力?

因为单一注意力子空间不够，多头让模型从多个表示子空间并行捕捉不同类型的关系，提高表达能力与稳健性。

问题 4: 为什么大多数聊天模型是 decoder-only?

因为训练目标和开放式生成高度一致，工程接口统一，生态成熟，扩展性强。不要只回答因为 GPT 成功了。

问题 5: next-token prediction 为什么能学到推理?

因为训练语料中隐含了大量任务模式与推理轨迹，模型通过压缩这些分布获得可迁移能力。但这不是严格逻辑保证，所以推理能力强不代表永远正确。

问题 6: 参数越大越好吗?

不是。能力通常会随规模增长，但是否值得取决于数据、训练是否充分、推理成本、延迟预算、业务指标。生产上追求的是质量/成本/延迟的 Pareto 最优，而不是盲目追大。

问题 7: Tokenizer 为什么很重要?

因为它直接影响 token 成本、跨语言表现、代码表现和上下文利用率。同样内容在不同 tokenizer 下 token 数可能差很多，成本与效果都会变。

问题 8: 位置编码为什么必须有?

因为注意力本身对顺序不敏感，没有位置信息就无法区分你喜欢我和我喜欢你这种顺序差异。

问题 9：LLM 里的知识是怎么存的？

更准确地说，是以分布式参数形式压缩在模型权重里。它不是可直接查询的数据库，所以会过时、会混淆、会在细粒度事实题上出错。这也是 RAG、工具调用、检索验证存在的原因。

问题 10：为什么说 Transformer 是基础，但不是全部？

因为今天的可用 LLM 产品除了模型架构，还依赖 tokenizer、训练数据、后训练对齐、推理系统、缓存、工具协议、评测与治理。只会讲架构，但讲不出系统，就很难做真实应用。

十、这一块八股怎么结合自己的项目讲

如果面试官问你做过什么 LLM 项目，不要上来就说我们用某某模型做了智能问答。更好的讲法是把底层原理映射到设计决策。

例如你可以这样说：

我们当时选 decoder-only 指令模型作为主生成器，因为目标是开放式问答和工具调用，而不是单纯分类。之所以特别关注 tokenizer 和上下文长度，是因为业务输入里有大量中英文混合文档和代码片段，token 成本很敏感。上线后我们把系统 prompt、知识片段、工具 schema 都视为运行时上下文的一部分来管理，本质上是在利用模型的 in-context learning 能力，而不是把所有业务逻辑都写死在参数里。

这段话的价值在于，它把 Transformer/LLM 基础翻译成了真实项目语言。面试官一听就知道你不是背论文，而是真的理解为什么这些基础会影响产品设计。

十一、常见误区与追问陷阱

第一个误区，是把 attention 说成模型在理解重要词。这太拟人了。更准确的说法是：attention 是一种可学习的信息路由与权重分配机制，不保证可解释，不等于人类注意力。

第二个误区，是把参数量当成唯一指标。今天很多小模型在特定任务上性价比更高，尤其在工具调用、结构化抽取、分类与路由场景。工程上看的是任务适配，而不是排行榜崇拜。

第三个误区，是把模型知道很多误解成模型内置事实库。这会让你在幻觉、RAG、工具调用等问题上全线崩掉。一定要记住：参数存的是压缩分布，不是可直接核验的知识表。

第四个误区，是把 Transformer 讲成只有 attention。如果你能主动补充 FFN、残差、归一化、mask、自回归目标，回答立刻比普通八股高一个档次。

十二、截至 2026-04，你还应该补上的现代视角

到 2026 年，大模型基础认知和 2023 年相比有一个明显变化：大家不再只讨论模型会不会，而开始讨论模型以什么代价、在什么上下文、通过什么工具链完成任务。

OpenAI 当前文档把模型区分为 reasoning 与非 reasoning 家族，并把模型选型、reasoning effort、structured outputs、tool use 都纳入统一接口；Anthropic 在 2025 年后明显强化了 context engineering、advanced tool use、programmatic tool calling 等工程主题；Google Gemini 文档则把 long context、thinking、function calling、structured outputs、context caching 作为第一层能力来讲。你会发现，行业的关注点已经从 Transformer 架构本身转向围绕 Transformer 的整套应用系统。

但这并不意味着基础不重要。恰恰相反，越是到 Agent 时代，越需要把基础打牢。因为你做任何一层能力封装，最终都绕不开几个底层事实：模型处理的是 token；生成依赖自回归；上下文有预算；长上下文有代价；参数里不是可靠知识库；推理系统的缓存和路由决定了成本与延迟。LLM与大模型应用基础这一章后面所有内容，其实都只是这些底层事实在不同工程问题上的展开。

资料来源与延伸阅读

官方与论文

1. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017
<https://arxiv.org/abs/1706.03762>
2. Brown et al., *Language Models are Few-Shot Learners*, 2020
<https://arxiv.org/abs/2005.14165>
3. Kaplan et al., *Scaling Laws for Neural Language Models*, 2020
<https://arxiv.org/abs/2001.08361>
4. Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022
<https://arxiv.org/abs/2203.15556>
5. OpenAI Reasoning Best Practices / Reasoning Guides
<https://developers.openai.com/api/docs/guides/reasoning-best-practices/>
<https://developers.openai.com/api/docs/guides/reasoning/>
6. Anthropic Models Overview
<https://docs.anthropic.com/en/docs/about-claude/models>
7. Gemini Models 文档
<https://ai.google.dev/gemini-api/docs/models>

02 推理机制与采样参数（了解）

阅读目标：把模型为什么会想、为什么会随机、为什么参数一改结果就变讲清楚，尤其是 reasoning model 与普通 GPT 模型在使用方式上的区别。

一、面试官在考什么

这一题表面看像参数题，实际上面试官主要在考你有没有把模型推理过程拆成几个不同层面来看。

第一层是模型内部计算与外部可见生成的区别。很多同学把模型输出的最终文本，当成模型全部计算过程本身。实际上推理请求往往至少包含两部分：一部分是模型内部为了得到答案而进行的计算与表示更新，另一部分才是最终输出到用户侧的文本或结构化结果。到 2025—2026 年，各家厂商都在把 reasoning/thinking 作为正式能力暴露出来：OpenAI 文档区分 reasoning models 与 GPT models，提供 reasoning.effort；Anthropic 在 Claude 4 之后强调 thinking / adaptive thinking；Gemini 文档则提供 thinkingBudget。面试官问推理机制，通常就在试你是否知道：模型的会不会想和想多久是可以作为工程控制量的。

第二层是 logits 到 token 的解码过程。模型前向计算完，并不会直接吐出一句完整答案，而是先给出下一个 token 的概率分布。然后系统再通过 greedy、temperature、top_p、top_k、stop sequence、最大输出长度等机制控制如何从分布里选 token。也就是说，采样参数不是模型知识的来源，而是把已有概率分布转成最终文本的阀门。

第三层是随机性与正确性的边界。校招面试里经常会问：把 temperature 调成 0，结果是不是就一定正确？标准答案当然是否定的。低温度只会让模型更偏向高概率 token，并不能把错误知识变成正确知识，也不能弥补缺失上下文、错误工具结果、差的 prompt 或模型本身的能力上限。

第四层是推理成本意识。reasoning effort、thinking budget、max tokens、流式输出、工具循环，都直接影响延迟与成本。能把推理质量与 token 成本、首 token 延迟、总时延联系起来的同学，更像真正做过产品，而不是只在 playground 里调过参数。

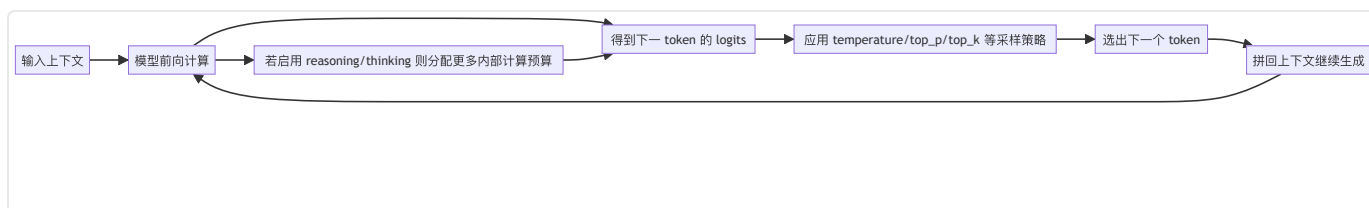
一个较好的项目表达是：

我们没有一上来就靠调 temperature 解决问题，而是先把任务分成‘理解、决策、输出’三段。对于抽取和路由这类确定性较强的步骤，我们降低随机性、加强结构化约束；对于生成候选方案或文案改写，允许更高多样性。复杂报表分析场景里我们还单独提高 reasoning budget，因为这类任务对多步一致性要求更高。最终不是追一个万能参数，而是根据任务阶段做差异化配置。

这段话为什么好？因为它体现出三个工程意识：

1. 参数是服务于任务阶段的；
2. 质量与延迟、成本一起考虑；
3. 推理与结构化输出不是同一个环节。

二、先把全流程想清楚



从工程实现看，一次完整生成通常是循环式的：输入上下文，模型算出 logits，解码器根据采样策略选 token，把新 token 再追加到上下文里，然后继续下一轮。直到命中停止条件，比如生成到终止符、达到最大 token 数、输出完整结构、或者工具调用流程结束。

所以推理这个词在大模型应用里常常混着三层含义：

1. 模型的内部分析能力；
2. 逐 token 生成的解码循环；
3. 在 Agent 中围绕工具、状态、计划展开的更长执行链路。

如果面试官没说清楚，你在回答时可以先主动界定：如果这里说的是模型 inference 的话，我会从 logits 解码和 reasoning budget 两层来讲；如果说的是 Agent 规划推理，那就要加上工具循环和状态管理。这类主动澄清非常加分。

三、什么叫推理机制：不要把会思考说得太玄

1. 训练阶段 vs 推理阶段

训练阶段是更新参数，推理阶段是固定参数、给定输入求输出。大模型应用开发里说推理服务推理成本，通常指 inference，而不是 human-like reasoning。很多同学第一次学 LLM 时会被 reasoning

model这个词带偏，觉得它和 inference 是两回事。其实 reasoning model 还是在做 inference，只不过它被设计成愿意在回答前消耗更多内部计算预算、执行更长的中间分析流程。

2. 显式推理提示、隐式推理预算与最终答案

传统 prompt engineering 常用 let's think step by step 这类提示去诱导模型显式输出中间步骤。现在主流商业模型越来越多地提供内部推理能力，也就是让模型在不完全暴露全部中间过程的情况下，先做更充分分析再输出结果。OpenAI 在 reasoning 文档中强调 reasoning.effort；Anthropic 的 Claude 4 系列在 extended/adaptive thinking 中区分思考预算与输出展示；Gemini 提供 thinkingBudget，并支持动态 thinking。

工程上你要知道这三者的区别：

- 显式推理文本：模型把思路写出来给你看；
- 内部推理预算：模型在内部花更多计算，但不一定完整暴露过程；
- 最终可见答案：真正返回给用户或程序消费的结果。

很多时候产品并不需要完整思维链暴露，只需要更高正确率、更稳结构和更低误答率。面试里如果被问要不要让模型把思考过程全吐出来，更稳的回答是：不一定。需要结合安全、稳定性、用户体验和成本来决定，很多生产场景更关心 reasoning budget 与最终输出质量，而不是完整中间文本。

3. 推理增强并不等于万能

更高 reasoning effort 通常有利于复杂数学、代码、多步规划、工具串联这类任务，但不意味着所有任务都该开高档。分类、抽取、简单改写、低风险 FAQ，往往没必要给高推理预算。面试里如果你说所有请求都用最强推理模型，最高 reasoning effort，这基本等于在告诉面试官你没有成本意识。

更成熟的说法是：推理预算是一个可调节的推理时计算资源，适用于高不确定性、高复杂度、多约束一致性任务；对简单、规则明确、对速度敏感的任务则应该下调，甚至路由到非 reasoning 模型或小模型。

四、采样参数到底控制了什么

1. Temperature：控制分布平滑程度

temperature 可以理解为放大或压缩概率分布差异的旋钮。温度低时，高概率 token 更容易被选中，输出更集中、更像最保守续写；温度高时，低概率 token 更有机会被选中，输出更发散、更有创造性。

要注意，temperature 作用在采样阶段，而不是改写模型知识。它不会让模型更懂业务，也不会修正事实错误。它主要改变输出多样性与稳定性。

一个非常常见的面试追问是：温度 0 是不是绝对确定？严格说，不同服务端实现、并发环境、浮点数细节、工具调用和后处理链路都可能引入微小差异，所以你不能把temperature=0理解成数学上的绝对确定性，只能理解成非常偏向高概率 token 的近似确定性设置。

2. Top-p：核采样

top-p，也叫 nucleus sampling。做法是先把 token 按概率从高到低排序，然后只保留累计概率达到 p 的最小候选集合，再在这个集合里采样。直觉上，它不是固定保留前 k 个，而是保留足够覆盖主要概率质量的那一撮候选。

top-p 的好处是它会随着分布形态自适应。当概率特别集中的时候，候选集会很小；当概率较分散时，候选集会更大。很多 API 文档都建议：temperature 和 top_p 一般不要同时大改，因为两个都在调随机性，叠加后更难预期。

3. Top-k：固定候选截断

top-k 先只保留概率最高的 k 个 token，再从这 k 个里采样。它的行为比 top-p 更硬：无论分布多么集中，都会只看前 k 个。某些开源推理框架和 Gemini/Vertex 体系会暴露 top-k，但不是所有厂商都把它作为主推荐参数。

面试里如果问 top-k 和 top-p 的区别，你最好回答成固定候选数 vs 固定累计概率质量，而不是简单说都差不多。

4. Max output tokens / stop sequences

这两个参数经常被低估。max output tokens 决定模型最多还能继续生成多长；stop sequences 则用于在命中特定字符串时提前结束。很多结构化输出失败、JSON 截断、工具循环超长、答案啰嗦，本质上都和这两个设置有关。

你在项目里如果遇到模型总是说废话，除了调 prompt，还要第一时间看：

- 最大输出长度是否给太大；
- 是否缺少明确停止条件；
- 是否要求了请详细思考并展开之类会放大冗长的指令；
- 是否在 Agent 循环里没有设置终止阈值。

5. Presence / frequency penalties 等附加参数

不同平台还会提供 presence penalty、frequency penalty 之类参数，用来降低重复、鼓励引入新 token。校招面试一般不会深挖，但你可以知道一个基本原则：这些参数是微调输出风格的辅助项，不是主控项。真正决定结果质量的通常还是模型本身、上下文、任务拆解、推理预算和输出约束。

五、为什么参数调优不能替代任务建模

很多人刚做应用时会陷入一个误区：结果不理想，就疯狂改 temperature、top_p、top_k。实际上，大模型结果不好，最常见原因往往不是参数，而是任务建模出了问题。比如：

- 任务太模糊，模型根本不知道目标；
- 上下文缺证据，模型只能猜；
- 工具 schema 不清晰，模型不知道何时调用；
- 输出格式没有约束，模型自由发挥；
- 示例分布和真实任务不一致；
- 模型选型不对，用小模型硬做高复杂度任务。

面试里如果对方向你一般怎么调参数，高质量回答不应该是我先把温度调低。更成熟的流程应该是：

先确认任务是否定义清楚；

再看模型选型是否匹配；

再看 prompt/context/工具设计；

最后才是用 sampling 参数做风格或稳定性微调。

这背后体现的就是工程方法论：先改大杠杆，再改小旋钮。

六、Reasoning models 与普通 GPT models：回答方式要变

OpenAI 的 reasoning best practices 文档明确区分 reasoning models 与非 reasoning GPT models。前者更适合多步推理、复杂代码、复杂决策与工具协作，后者通常在低延迟、通用聊天、普通生成任务里更经济。这个区分对面试很重要，因为它会影响你怎么 prompt。

1. 对 reasoning model，不要过度 micromanage

很多强推理模型在设计上已经会进行内部分析。如果你给它过于繁复、过细、甚至相互冲突的步骤指令，反而可能限制它。更稳的做法是把目标、约束、验收标准、可用工具、输出格式讲清楚，把怎么想留给模型自己完成。OpenAI 的新模型提示指导、Anthropic 的 prompt engineering 文档，都越来越强调明确目标和边界，而不是把中间思路写得极其死板。

2. 对小模型和非 reasoning 模型，要更显式

如果是 mini/nano 级别模型，或者本来就不擅长复杂推理的模型，你通常需要：

- 更明确地拆任务；
- 给更直接的格式要求；
- 提供更接近真实分布的 few-shot 示例；
- 避免一个 prompt 里塞太多隐含要求。

这也是为什么面试里经常会问同一个 prompt 在大模型和小模型上要怎么改。标准答案就是：小模型需要更强约束、更少歧义、更短上下文、更直接 schema。

七、截至 2026-04 的主流厂商参数差异，你需要知道这些

下面这张表不是为了背数值，而是帮你建立同名参数，不同家族不一定同义且推荐用法不同的意识。

厂商/家族	当前文档重点	你面试里该怎么说
OpenAI	区分 reasoning models 与 GPT models；支持 reasoning.effort；文档建议 temperature 与 top_p 不要同时大改	强调模型家族差异，复杂任务优先看 reasoning effort，而不是只盯 temperature
Anthropic Claude 4+	thinking / adaptive thinking；某些新模型上 manual budget_tokens 已逐步被 adaptive/effort 取代	强调思考预算是正式能力，并且 thinking 输出不等于全部内部推理
Gemini 2.5/3.x	提供 thinkingBudget；Gemini 3 文档多次强调 temperature 保持默认 1.0 更稳	说明 Gemini 上有时不要机械套用低温更稳的老经验，要遵循当前模型家族建议

这一段很适合在面试里展示你会看官方文档，不是死背旧博客。尤其是 Gemini 3 的文档明确强调 temperature 保持 1.0，低于 1.0 反而可能出现 looping 或复杂推理退化，这就是一个典型的旧经验不一定适用于新模型的案例。

八、推理质量、延迟与成本是怎么绑在一起的

1. 更高推理预算通常意味着更高成本

推理预算变高，内部 reasoning tokens 或内部计算过程会变多。哪怕你没把中间思路都显示出来，账单和延迟也可能会上升。Anthropic extended thinking 文档明确说明，费用与 thinking tokens 有关；Gemini pricing 也把 thinking token 计入输出价格口径；OpenAI reasoning effort 则直接体现为不同的推理时计算深度。

2. 首 token 延迟和总时延不是一回事

有些请求首 token 很快，但整段输出很长；有些 reasoning 请求首 token 慢，但最终答案更短更准。你在面试里如果能主动区分 TTFT (time to first token) 和总完成时长，会显得非常像做过线上系统的人。

3. 参数不是越多越好，预算也不是越高越好

推理预算本质是资源分配问题。面向用户的实时问答、语音交互、简单表单抽取，通常优先低延迟；复杂代码修复、报表分析、规划执行，则可能更愿意牺牲一些延迟换正确率。一个成熟系统的做法一般是路由，而不是全量最高配置。

九、高频面试题与参考回答要点

问题 1: temperature 调成 0，模型就不会错了吗？

不会。temperature 只影响采样随机性，不修复错误知识、缺失上下文或错误任务建模。

问题 2: top-p 和 top-k 有什么区别？

top-p 按累计概率质量截断候选集，top-k 按固定候选数量截断。前者更自适应，后者更硬。

问题 3: 为什么官方常说 temperature 和 top_p 不要同时改？

因为两者都在控制随机性，同时大改会让行为更难预测和调试。

问题 4: reasoning effort / thinking budget 有什么用？

它控制模型在回答前投入多少推理时计算资源。复杂任务更有价值，简单任务通常不值得高开。

问题 5：为什么低温度不等于高正确率？

因为正确率还取决于模型能力、上下文证据、任务定义、工具结果和输出约束。低温只是更保守。

问题 6：什么时候应该提高 temperature？

在文案生成、发散创意、标题 brainstorming、多个候选方案生成等需要多样性的任务里更有意义。

问题 7：什么时候不建议追求显式思维链输出？

当你只关心最终结构化结果、担心冗长输出、需要更稳产品体验或有安全与泄露风险时，不一定需要完整中间过程。

问题 8：为什么有时同一参数在不同模型上表现差很多？

因为模型家族、默认解码策略、内部 reasoning 机制、训练对齐方式都不同。参数名相同，不代表行为完全一致。

问题 9：项目里一般先调哪些参数？

先不急着重调参数，先确认任务定义、上下文、prompt、模型选型和输出约束，再对 temperature / max tokens / stop sequences 做必要微调。

问题 10：为什么模型会重复说车轱辘话？

常见原因包括最大输出太大、缺少停止条件、提示词鼓励展开、模型陷入高概率重复、或任务目标没有收束。

问题 11：推理模型是不是一定比普通模型更强？

不是。它通常在复杂多步任务上更强，但未必适合低延迟、低成本、简单结构化任务。

问题 12：Agent 场景里采样参数怎么设？

要看阶段。规划阶段可能允许适度发散，执行/结构化阶段往往更低随机性、更强 schema 约束。不要所有阶段一个参数打到底。

十、常见误区

第一个误区，是把采样参数当成主要优化手段。真正的大杠杆往往是模型、上下文、任务拆解和工具链。

第二个误区，是把更高 reasoning effort理解成更高智能。它更像愿意花更多时间做题，并不保证每题都对。

第三个误区，是用统一参数覆盖所有任务。实际系统往往至少区分：分类/抽取、对话生成、复杂分析、工具执行四类。

第四个误区，是忽视停止条件。很多线上事故不是模型不会，而是模型停不下来、JSON 没收口、工具循环没终止。

十一、总结

如果你只记住一句话，那就是：

采样参数控制的是怎么从概率分布里拿答案，推理预算控制的是模型在拿答案前愿意花多少计算去想，两者都重要，但都不能替代正确的任务建模。

先解决模型选型、上下文证据和输出约束，再去调 temperature/top_p/max_tokens，才是生产上更有效的顺序。

资料来源与延伸阅读

官方文档

1. OpenAI Reasoning Models / Best Practices
<https://developers.openai.com/api/docs/guides/reasoning/>
<https://developers.openai.com/api/docs/guides/reasoning-best-practices/>
2. OpenAI Responses API Reference (temperature、top_p 等参数)
<https://developers.openai.com/api/reference/resources/responses/methods/create/>
3. OpenAI 最新模型使用指南 (reasoning.effort)
<https://developers.openai.com/api/docs/guides/latest-model/>
4. Anthropic Extended Thinking
<https://docs.anthropic.com/en/docs/build-with-claude/extended-thinking>
5. Anthropic Prompt Engineering Overview
<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>

6. Gemini Thinking

<https://ai.google.dev/gemini-api/docs/thinking>

7. Gemini 3 Developer Guide / Text Generation

<https://ai.google.dev/gemini-api/docs/gemini-3>

<https://ai.google.dev/gemini-api/docs/text-generation>

8. Vertex AI 参数调节文档

<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/adjust-parameter-values>

相关资料

9. OpenAI GPT-5 系列 Prompt Guidance

<https://developers.openai.com/api/docs/guides/prompt-guidance/>

10. OpenAI Latency Optimization

<https://developers.openai.com/api/docs/guides/latency-optimization/>

03 上下文窗口、KV Cache 与长上下文问题

阅读目标：不仅知道context window 是什么，还要知道它为什么贵、为什么长上下文不等于会用长上下文，以及 KV Cache / prompt cache / context cache 各自解决什么问题。

一、这篇在面试里考什么

问上下文窗口，很多同学张口就是它表示模型一次能处理多少 token。这句话没错，但不够。面试官真正想知道的是：**你是否理解上下文预算会同时决定质量、成本、延迟和系统设计（最爱考的trade-off类问题）。**

第一，面试官会看你是否区分能放进去和能用得好。主流商用模型到 2026 年已经普遍支持几十万到百万级上下文，但学术和工业实践都在提醒我们：长窗口能力不等于对长窗口中每一段信息都同等敏感。Lost in the Middle 这类研究说明，把关键信息放在长上下文中间位置时，模型利用效果可能明显下降。也就是说，长上下文是容量问题，长上下文利用率是检索与注意力分配问题，这两者不能混为一谈。

第二，面试官想看你是否知道推理时最贵的部分之一是什么。很多人只关心模型参数量，却忽略了序列越长，前缀计算与 KV Cache 占用也越重。特别是在线推理服务里，长上下文不仅提高 token 账单，还会拉高首 token 延迟，压缩并发能力。所以上下文越大越好的说法非常外行。

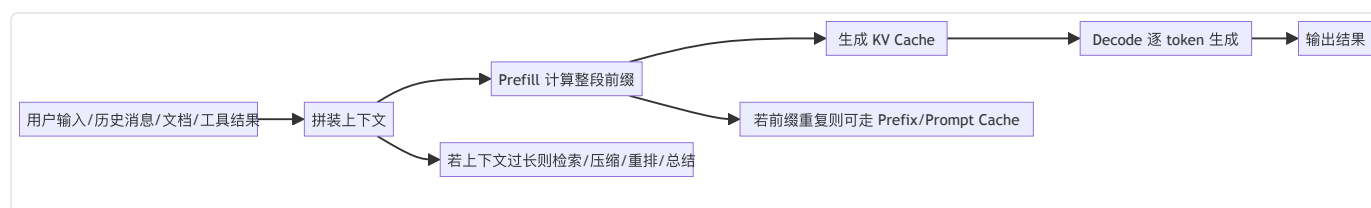
第三，这一题常常被用来过渡到缓存与优化。真正做过系统的同学会知道，长上下文时代最重要的工程技巧之一就是不要重复算同样的前缀。这背后就牵涉 KV Cache、prefix caching、prompt caching、context caching。它们名字像，但层次不同：有的是模型解码时的内部状态缓存，有的是服务端对重复前缀的复用策略，有的是应用层显式把大块上下文缓存成可复用对象。面试官问你这些概念，就是在看你有没有把推理服务、应用接口和成本控制串起来。

第四，长上下文问题还会自然连到 RAG、memory、context engineering。因为一旦你真正理解上下文预算有限、注意力分布不均、缓存成本可观，你就会知道为什么不能把整个知识库一股脑塞进 prompt，为什么需要检索、重排、摘要、压缩和按需加载。

你可以这样结合项目描述，让面试官觉得你是对这一块有独立思考的：

我们的业务一开始尝试把整篇文档直接塞进模型，但发现首 token 延迟明显上升，而且回答对中间章节引用不稳定。后来我们把系统 prompt、工具定义、产品规则做成稳定前缀，通过 prompt caching 提高命中率；知识内容则走检索与重排，只把高相关片段放进本轮上下文。多轮会话里再把早期历史压缩成结构化 summary/state，避免上下文越聊越脏。这样做后，质量、延迟和成本都更可控。

二、先建立一个工程视角的全景图



理解长上下文，最关键的是区分 prefill 和 decode。

- **Prefill**：模型先把整段已有上下文都读一遍，为每一层生成 K/V 状态。上下文越长，prefill 通常越重，TTFT（首 token 延迟）越高。
- **Decode**：之后每生成一个新 token，只需要基于历史缓存继续算新增部分。这个阶段 KV Cache 会极大降低重复计算。

所以如果面试官问为什么超长 prompt 首 token 慢，你就从 prefill 讲起；如果问为什么 KV Cache 能提速，你就从 decode 阶段避免重复计算讲起。把这两部分分清楚，回答会非常专业。

三、什么是上下文窗口，为什么它不是一个单纯的数字

1. 定义：一次请求中模型可处理的总 token 预算

上下文窗口（context window）通常指模型在一次请求中能处理的 token 总预算。这个预算一般同时覆盖输入和输出，某些平台还会受到 reasoning token、工具消息、系统提示等隐性成本影响。很多初学者只记住某模型是 128k/400k/1M，但真正线上做系统时，你要问的是：

- 系统提示词占了多少；
- 历史对话占了多少；
- 检索资料占了多少；
- 工具 schema 和工具结果占了多少；
- 还要给输出预留多少。

也就是说，context window 不是你能塞进去的文档大小，而是整个会话状态的预算上限。

2. Token 预算系统是系统资源，不是无成本便利

OpenAI、Anthropic、Gemini 等平台都在文档里强调 token 成本；Gemini 与 Anthropic 还分别提供 context/prompt caching 机制，OpenAI 则在新文档里强调 prompt caching 与 compaction。为什么大家都在做这些事？因为长上下文太贵了，而且越来越成为真实应用的主要瓶颈之一。

你在面试里最好能明确指出两个成本：

- **显性成本**：token 计费；
- **隐性成本**：显存占用、并发下降、首 token 延迟增加。

只会说会更贵是不够的，更好的表达是：更长输入不仅提高计费，还增加 prefill 计算与 KV Cache 占用，影响 TTFT 和吞吐，因此长上下文设计一定是质量与性能的平衡问题。

四、KV Cache 到底是什么

1. 直觉解释

在自回归生成中，如果每生成一个新 token 都把前面全部上下文从头算一遍，那代价会极大。KV Cache 的核心思想就是：在 previous tokens 已经算过的情况下，把各层 attention 需要的 Key 和 Value 状态缓存起来。下一步只需要为新 token 计算 Query，并与历史的 K/V 交互，就能继续生成。

你可以把它理解成：

历史读过的书页先做成索引，下次不再整本重读，只增量读新的一页。

2. 为什么它重要

KV Cache 直接决定：

- 解码速度；
- 显存占用；
- 最大并发；
- 可支持的有效上下文长度。

Google 关于 tiered KV cache 的工程文章直接指出，KV Cache 的 GPU 显存占用会成为上下文长度、并发度和总体吞吐的关键瓶颈。Hugging Face 的缓存文档也把动态缓存、静态缓存、量化缓存、offloading 等策略作为生成性能优化的重要主题。也就是说，今天的推理服务优化，很多时候已经不是算力够不够，而是 KV Cache 怎么放、怎么复用、怎么迁移的问题。

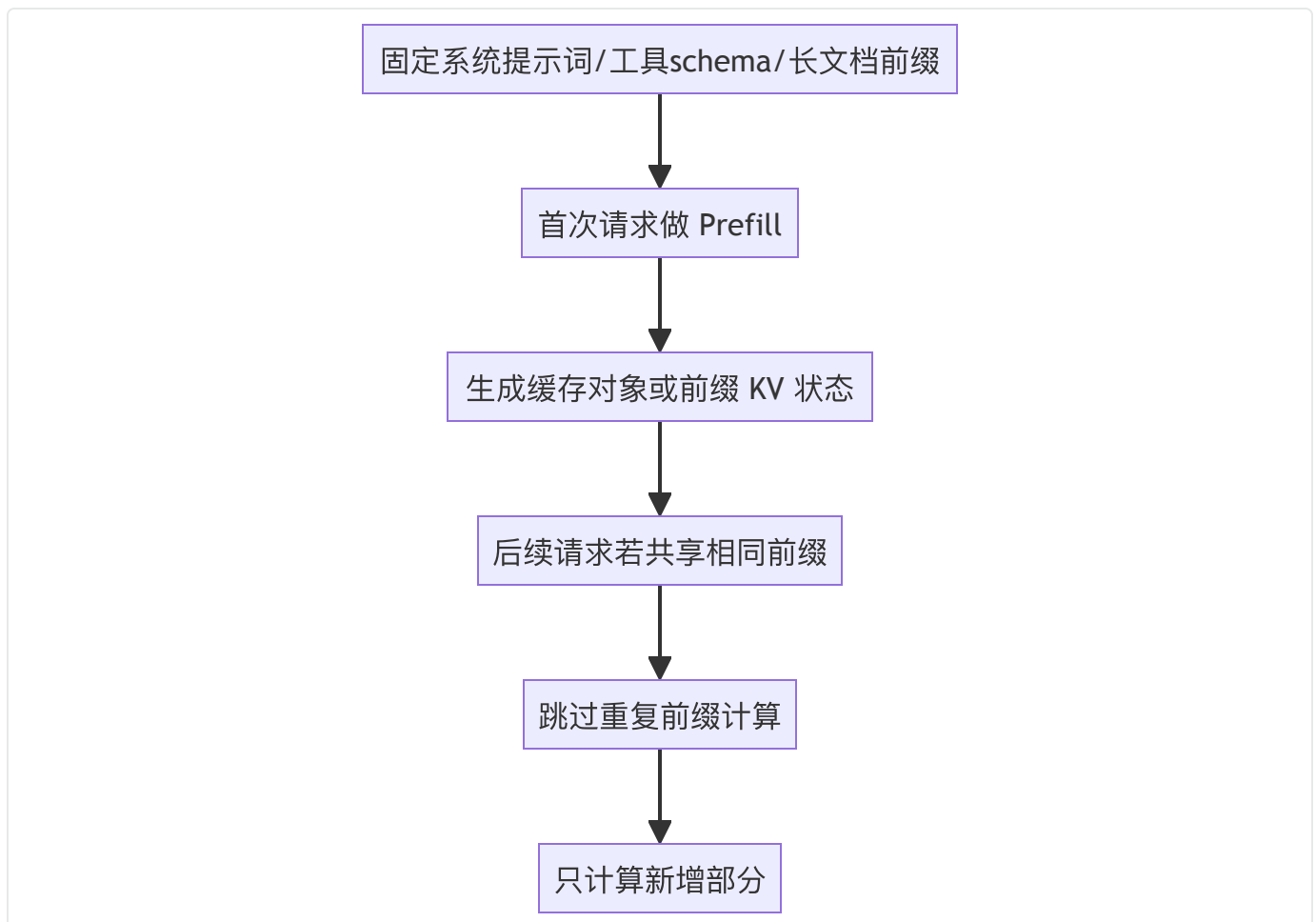
3. KV Cache 与 Prompt Caching 不是一回事

这是面试非常爱问的点。

- **KV Cache**: 通常指单次生成或持续会话中的模型内部状态缓存，主要优化 decode 过程。
- **Prefix / Prompt Caching**: 通常指当不同请求共享相同前缀时，服务端或框架复用已算过的前缀结果，避免重复 prefill。
- **Context Caching**: 更多是应用层或 API 层对大块上下文的显式缓存引用，例如 Google Gemini/Vertex 的 cache object。

你如果能清楚地区分模型内部状态缓存和跨请求前缀复用，面试官一般会默认你看过推理框架或官方文档。

五、Prefix Caching / Prompt Caching / Context Caching的区别



1. Prefix Caching

vLLM 文档把 automatic prefix caching 解释得非常直接：如果新请求与历史请求共享同样的前缀，就可以直接复用前缀对应的 KV 块，跳过共享部分的计算。vLLM 甚至明确指出，这种复用已经被很多公有端点和开源推理框架广泛使用，因为它几乎不改变输出，却能显著降延迟。

这对应用开发的启示是：

如果你的系统 prompt、工具 schema、产品规范、长文档前言高度稳定，就应该尽量把稳定前缀做成可缓存结构，而不是每次重新拼接一大坨略有差异的文本，白白浪费缓存命中率。

2. OpenAI Prompt Caching

OpenAI 文档已把 Prompt Caching 作为正式指南，说明 recent models 会自动启用缓存，并在 pricing 中区分 input 与 cached input 价格。这意味着从产品角度看，提示词工程不再只是怎么写得对，还变成怎么写得能命中缓存。比如频繁改动系统提示词、随机插入无关 metadata、把稳定前缀和用户动态输入混在一起，都可能降低缓存收益。

3. Anthropic Prompt Caching

Anthropic 文档在 2025—2026 年进一步细化了 prompt caching，支持 automatic caching，并明确说明 tools、system、user/assistant content、图像与文档、tool use/result 等大量内容都可以成为缓存对象。这个设计非常适合 Agent：因为 Agent 的系统提示、工具定义、权限说明、项目背景等通常都比较稳定，真正变化的是用户请求和局部上下文。

4. Gemini / Vertex Context Caching

Google Gemini 和 Vertex AI 文档强调 explicit caching：你可以先把一大块文本、音频、视频或文档缓存起来，后续请求引用该缓存对象，并只附加少量本轮输入。它更像把大上下文存成一个可复用资产，适合多轮围绕同一大文档做问答、分析或生成。Gemini 还支持 TTL 概念，说明 cache 不是永久资产，而是受生命周期管理的资源。

六、长上下文为什么不一定真的有用

1. Lost in the Middle：中间位置的信息最容易被忽略

《Lost in the Middle》是长上下文讨论里最经典的论文之一。它指出，很多长上下文模型在需要从很长输入中找到关键信息时，效果对位置信息非常敏感：开头和结尾的内容往往更容易被利用，而中间部分可能被显著忽略。这个现象对应用设计的启发非常直接：

- 不要把最关键证据埋在长 prompt 正中间；

- 长文档问答不要只靠整篇塞入，最好结合检索和重排；
- 最重要的信息应该尽量靠前、靠后或被显式标记；
- 单纯扩大窗口不能替代检索质量。

面试里如果问为什么 1M context 还要做 RAG，这就是最标准的回答素材。

2. 上下文污染与注意力稀释

上下文太长会带来另一个问题：噪声变多。无关历史对话、重复片段、失效工具结果、多个版本冲突的业务规则，都可能污染模型判断。很多线上系统不是上下文不够，而是上下文太脏。这时候真正需要的不是更大窗口，而是更好的上下文治理。

3. 长上下文与多轮会话并不等价

有些同学以为模型有百万窗口，就可以无限聊。但多轮会话除了窗口上限，还有状态漂移、历史冲突、旧指令污染、缓存失效、工具结果过时等问题。到 2025—2026 年，OpenAI 的 Responses compaction、Anthropic 的 context management、Gemini Live 的 session summary 等新能力都在说明：长会话要靠摘要、压缩、状态管理，不是简单无限堆历史消息。

七、长上下文的常见优化思路

1. 检索优先，而不是全量塞入

如果任务是从几十万字文档中找特定证据，最优策略往往是先检索、再精选片段、再按顺序组织，而不是把整本文件直接塞给模型。大窗口降低了必须极致裁剪的压力，但并没有消灭检索与重排的必要性。

2. 结构化组织比原文堆叠更重要

把上下文分成：

- 任务目标；
- 关键事实；
- 规则约束；
- few-shot 示例；
- 工具列表；
- 候选证据；

- 输出 schema;

通常比把十段文档原文依次贴上去效果更好。这其实已经进入 context engineering 的范畴：不是喂更多，而是喂得更有结构。

3. 显式压缩与会话总结

对于多轮对话，保留全部历史往往既贵又脏。更常见做法是：

- 只保留最近若干轮原文；
- 早期历史压缩成 summary/state；
- 把已完成的步骤用户偏好待办约束提取成结构化状态；
- 必要时做 compaction。

4. 让前缀稳定，提高缓存命中率

这是一条非常工程化、也非常容易在面试中脱颖而出的经验。

很多系统 prompt、权限说明、工具 schema、产品规则明明是稳定的，却每轮都加时间戳、随机 request id、无意义换行或顺序扰动，导致 prompt cache 很难命中。

真正懂系统的人会主动把稳定前缀和动态后缀拆开设计。

八、高频面试题与答题要点

问题 1：什么是 context window？

一次请求里模型可处理的总 token 预算，通常覆盖输入与输出，也会受到系统提示、工具消息、历史对话等共同占用。

问题 2：上下文窗口越大越好吗？

不一定。更大窗口意味着更高成本、更高首 token 延迟、更大缓存压力，而且模型对长上下文的利用率未必线性提升。

问题 3：为什么长 prompt 首 token 会慢？

因为 prefill 需要先对整段前缀做计算并生成各层缓存，上下文越长，prefill 通常越慢。

问题 4: KV Cache 是什么?

是自回归生成时缓存历史 token 的 Key/Value 状态, 避免每一步都从头计算历史上下文。

问题 5: KV Cache 和 prompt cache 有什么区别?

KV Cache 更偏单次生成内部状态; prompt/prefix cache 更偏跨请求前缀复用; context cache 则常是 API 层显式缓存大块输入。

问题 6: 为什么有了 1M context 还要做 RAG?

因为长上下文并不保证高效利用, 关键证据可能被埋没; 而 RAG 能减少噪声、提升证据密度、降低成本。

问题 7: Lost in the Middle 说明了什么?

说明模型对长上下文中不同位置的信息利用并不均衡, 中间位置的关键信息更可能被忽略。

问题 8: 为什么缓存能降成本?

因为重复前缀不必重新 prefill, 既减少计算也减少部分输入计费, 并改善延迟。

问题 9: 长会话为什么需要 compaction / summary?

因为无限保留原始历史会带来窗口占用、噪声累积、旧状态污染和成本上升。

问题 10: 项目里如何提高缓存命中率?

稳定 system prompt 和工具 schema, 减少无意义动态字段, 把固定前缀与用户动态输入分离。

十、常见误区

第一个误区, 是把上下文窗口当成模型记忆力指标。窗口只是可处理容量, 不等于稳定记忆与长期状态。

第二个误区, 是把 KV Cache 说成把之前答案缓存住。其实它缓存的是中间 attention 所需状态, 不是自然语言文本本身。

第三个误区，是认为 prompt 越长越充分。实际上很多任务在关键信息密度更高、噪声更低时效果更好。

第四个误区，是只看 token 费用，不看并发与显存。真正线上服务常常先被 KV Cache 和 TTFT 拖垮，而不是先被账单拖垮。

十一、截至 2026-04 的现代趋势

截至 2026 年，长上下文已经从模型参数表上的卖点变成应用系统的常规约束。OpenAI 模型页面已经把数十万到百万级 context window 作为新一代模型的常见能力，并提供 prompt caching 与 compaction 相关能力；Anthropic 不仅提供 prompt caching，还在模型能力描述里暴露 context management/compact 支持；Google Gemini 则同时把 long context、thinking、structured outputs、function calling、context caching 放进第一层开发文档中。行业在告诉你一件事：大窗口不是终点，围绕大窗口做状态治理、缓存复用、按需加载与压缩，才是现代 LLM/Agent 系统的日常。

十二、你应该记住的结论

如果这篇只记一句话，那就是：

长上下文解决的是放得下，而检索、重排、压缩、缓存解决的是用得好、用得省、用得快。

再补一句：

KV Cache 解决单次生成的重复计算，prompt/context caching 解决跨请求共享前缀的重复 prefill，它们是长上下文时代的核心工程武器。

资料来源与延伸阅读

官方与框架文档

1. OpenAI Models / Prompt Caching / Pricing / Changelog
<https://developers.openai.com/api/docs/models>
<https://developers.openai.com/api/docs/guides/prompt-caching/>
<https://developers.openai.com/api/docs/pricing/>
<https://developers.openai.com/api/docs/changelog/>
2. Anthropic Prompt Caching / Models Overview / List Models
<https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>

<https://docs.anthropic.com/en/docs/about-claude/models>

<https://docs.anthropic.com/en/api/models-list>

3. Gemini Long Context / Context Caching / Models / Changelog

<https://ai.google.dev/gemini-api/docs/long-context>

<https://ai.google.dev/gemini-api/docs/caching>

<https://ai.google.dev/gemini-api/docs/models>

<https://ai.google.dev/gemini-api/docs/changelog>

4. Vertex AI Context Cache

<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/context-cache/context-cache-overview>

5. vLLM Automatic Prefix Caching

https://docs.vllm.ai/en/latest/design/prefix_caching/

6. Hugging Face KV Cache / Cache Strategies

https://huggingface.co/docs/transformers/en/kv_cache

论文

7. Liu et al., *Lost in the Middle: How Language Models Use Long Contexts*

<https://arxiv.org/abs/2307.03172>

8. Google Cloud: Tiered KV Cache 工程文章

<https://cloud.google.com/blog/topics/developers-practitioners/boosting-llm-performance-with-tiered-kv-cache-on-google-kubernetes-engine>

04 幻觉、对齐与指令跟随

阅读目标：搞清楚模型为什么会“编”、为什么要做 alignment、为什么 instruction following 既是能力又可能带来副作用，以及如何用工程方法减轻幻觉。

一、幻觉就是面试官实际用模型遇到的烦恼

“幻觉”“对齐”“指令跟随”这三个词经常一起出现，但很多同学只会把它们当成安全八股去背。实际上，这一组问题在面试里经常用来区分“会用模型”和“理解模型边界”的人。

第一，面试官在看你是否知道：LLM 的目标函数从来不是“说真话”，而是“生成高概率的下一个 token”。只要你把这个根因讲明白，后面很多问题都会顺：为什么模型会编造来源、为什么缺证据时仍可能自信回答、为什么流畅性和真实性不是同一个维度。很多回答失败，就是因为上来把幻觉说成“训练数据不够”，这只说中了部分原因。

第二，面试官在看你对 alignment 的理解是不是停留在“让模型更安全”。更完整的定义应该是：对齐是让模型更符合人类或产品期望的过程，包含 helpfulness、harmlessness、honesty、instruction following、格式稳定性、工具使用边界等多个方面。换句话说，对齐不只是拒绝危险请求，也包括更好地遵循系统角色、遵守业务规则、承认不确定性。

第三，面试官会用这组题考你有没有产品视角。实际应用里，用户经常把“模型没答对”统称为幻觉，但工程上你必须区分：

- 真正的 factual hallucination；
- 上下文中明明有证据却没用好；
- 工具返回错了；
- 检索召回错了；
- 指令冲突导致模型选择了错误目标；
- 模型输出格式不符合要求；
- 模型其实不确定，但被 prompt 逼着必须回答。

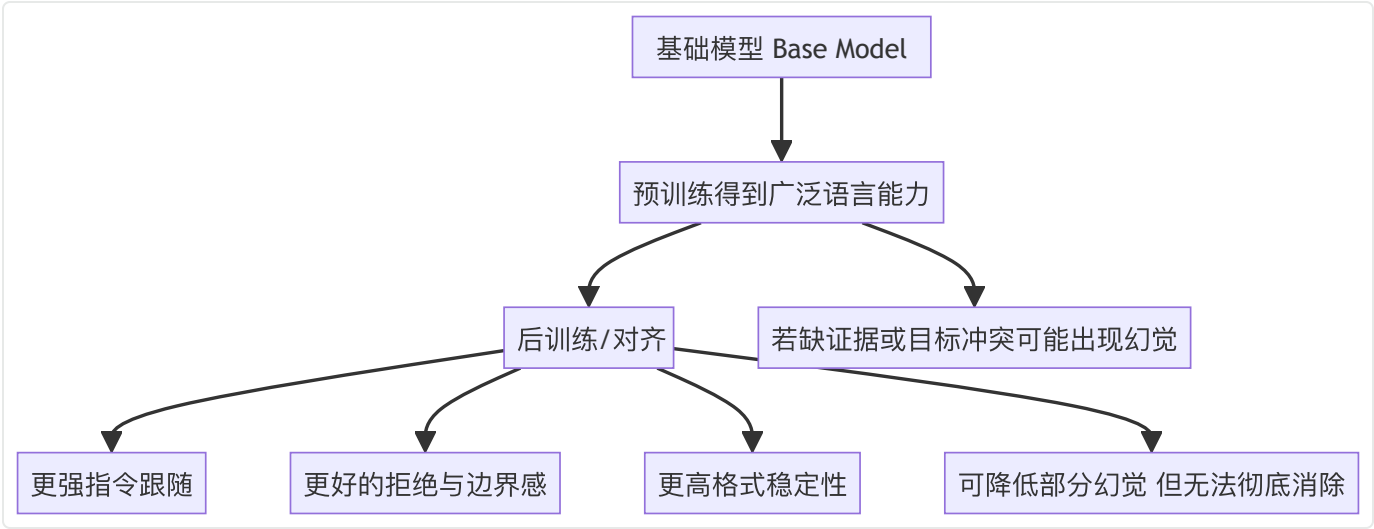
如果你分不清这些故障类型，后面就谈不上治理。

第四，这一题还常被用来引出安全与 Agent。Agent 一旦能调工具、发请求、执行操作，指令跟随能力越强，潜在副作用也越大；而如果 alignment 不够好，模型可能在被 prompt injection 或上下文污染后执行错误动作。所以这部分不只是“答得像论文”，而是后续系统设计与治理的基础。

对于这一章涉及到的问题，一个较成熟的项目表述是，展示了“故障分类 + 分层治理 + 可观测”三个真实工程能力：

“我们把线上错答分成四类：事实性幻觉、检索失配、工具误用、格式不一致，而不是一股脑都叫 hallucination。治理上先改任务定义，允许模型在无证据时返回 cannot_answer；再把关键证据抽成结构化片段而不是整段原文拼接；对高风险问题增加搜索或数据库校验；同时在输出 schema 里加 evidence_ids 和 uncertainty_note。这样做后，模型并不是‘不再犯错’，而是更少在没证据时硬编，错误也更容易被发现和拦截。”

二、怎么区分这三者



1. 幻觉是什么

严格说，幻觉（hallucination）指模型生成了流畅但不真实、无依据、与给定上下文不一致、或无法被外部证据支持的内容。它既可能是事实性错误，也可能是引用、出处、数字、代码 API、推理步骤、工具执行结果上的编造。

2. 对齐是什么

对齐（alignment）是让模型行为更符合人类意图与系统要求的过程。它的目标通常不止一个维度，常见包括：

- helpful：有帮助；
- harmless：降低伤害；
- honest / truthful：尽量承认不确定性，不乱编；

- steerable: 能稳定遵守角色与系统指令;
- reliable: 格式、工具调用、边界控制更稳。

3. 指令跟随是什么

指令跟随 (instruction following) 是 alignment 中一个非常核心的子能力, 指模型能不能正确理解并执行来自 system/developer/user 层的要求。它包括:

- 明确目标;
- 遵守优先级;
- 不被低优先级内容轻易带偏;
- 在冲突时做合理取舍;
- 能按照给定格式输出;
- 在信息不足时适当表达不确定, 而不是瞎补。

这三个概念的关系不是“并列概念”, 而是:

对齐是总框架, 指令跟随是对齐的一部分, 幻觉是模型在真实性/证据一致性上暴露出来的一类失败模式。

三、为什么模型会幻觉: 根因一定要答到底层

1. 目标函数不是事实核验器

LLM 在预训练阶段学的是语言分布, 而不是“真实世界校验规则”。它的本能是生成“看起来合理、接得下去”的 token 序列。于是当上下文缺证据、问题又逼着它必须给答案时, 它非常容易走向高流畅度、低真实性的输出。这是最根的原因。

2. 训练数据本身就有噪声、过时与冲突

语料中包含错误信息、过时知识、不同立场、不同版本 API、不同时间点的事实。模型把这些都压缩进参数后, 不会自动形成完全一致的真值库。于是你问一个细粒度事实、冷门版本问题、时间敏感问题时, 它很可能混用旧知识与近似模式。

3. 上下文证据不足或上下文组织不当

很多所谓“模型幻觉”, 其实根本原因是我们没给它足够证据, 或把证据塞得太乱。比如:

- 检索召回错了文档；
- 关键信息埋在长上下文中间；
- 多个版本的业务规则同时出现；
- 系统提示要求必须回答，禁止说“不确定”。

这种情况下，模型“编”出来的内容，某种意义上是系统设计在逼它编。

4. 解码随机性与长链误差积累

采样参数越发散，模型选择低概率 token 的机会越高；多步生成越长，局部小错越可能滚成整体大错。特别是长文本、复杂 SQL、长代码、工具串联计划，一步偏了，后面都跟着漂移。

5. 工具、检索和外部系统引入的新幻觉

到了 Agent 时代，幻觉不再只是“模型脑补”。还包括：

- 模型错误理解了工具返回；
- 工具本身返回脏数据；
- 模型声称工具执行成功，实际上并没有；
- 模型把未发生的外部动作说成已完成。

Anthropic 在发布 Sonnet 4.6 时特别提到“fewer false claims of success”，这正说明在 Agent/代码场景里，“虚报完成”已经成为一种非常实际的幻觉类型。

四、对齐为什么重要：不是锦上添花，而是可用性的基础

1. InstructGPT 给行业建立了一个分水岭

OpenAI 的 InstructGPT 工作一个非常重要的结论是：更大的 base model 不自动意味着更符合用户意图。一个较小但经过人类反馈对齐的模型，可能在人类偏好上优于更大的纯预训练模型。这个认识直接改变了行业路线：从“只追预训练规模”转向“预训练 + 后训练 + 偏好优化”的范式。

2. 对齐不仅是安全，也包括“可控”

从产品角度看，一个不对齐的强模型可能有很多能力，但不一定“好用”。比如：

- 不按格式输出；
- 听不懂系统角色；

- 拒绝边界混乱；
- 明明没证据还硬答；
- 不会承认不知道；
- 工具调错却继续装作没问题。

这些都是对齐不足，不只是“安全问题”。

3. RLHF、DPO、RLAIF、Constitutional AI 等都是手段，不是目的

你在面试里不一定要把所有后训练算法都展开，但至少知道方向：

- SFT：先让模型学会像样地完成指令；
- RLHF / 偏好优化：让模型更接近人类偏好；
- Constitutional AI / RLAIF：用规则与 AI 反馈帮助塑造更可控行为；
- 规则规格 / Model Spec：在产品层面明确优先级、边界与行为准则。

回答时最好不要把算法名念成口号，而是直接落到效果：“这些方法共同目标是让模型更 helpful、更稳定、更能遵守边界，并在不确定时更诚实。”

五、指令跟随：为什么它越强越需要优先级管理

1. 指令不是一层，而是多层

真实系统里的指令来源通常至少包括：

- 平台/模型层原则；
- system/developer 指令；
- 用户指令；
- 外部工具或检索内容；
- 历史对话中的旧约束。

这些指令可能冲突。如果模型没有优先级意识，就会被低优先级文本带偏。OpenAI 近年的 Model Spec 和相关工作一直强调 instruction hierarchy / chain of command，这本质上是在解决“听谁的”问题，而不是“能不能听见”问题。

2. 强指令跟随并不自动提升真实性

这是个很容易被忽视的点。模型如果非常听话，而你的提示词写着“给我一个明确答案，不要说不确定”，那它可能更积极地编。也就是说，指令跟随能力越强，越需要你写进“何时承认不知道、何时要求证据、何时调用工具验证”写进系统规则。

3. Agent 里更要控制 side effects

在普通问答里，错答是认知错误；在 Agent 里，错答可能变成错误操作。于是 instruction following 必须和权限、审批、可回滚、幂等、安全策略一起设计。否则强执行力会变成强风险。

六、怎么降低幻觉：面试要讲方法分层

第一层：任务定义层

最常见也最被忽略。

如果任务允许“不知道”，就明确允许；

如果需要证据，就要求“先找证据再回答”；

如果必须引用来源，就把 citation/evidence slot 放进输出 schema；

如果问题时间敏感，就要求调用搜索或检索。

你要让模型知道：没有证据时承认不确定，是被允许甚至被鼓励的。

第二层：上下文与检索层

降低幻觉最有效的方法之一，往往不是“改模型”，而是给更好的证据。包括：

- 提升检索召回与重排质量；
- 减少上下文噪声；
- 把关键证据放在更显著位置；
- 用结构化字段而不是大段原文堆叠；
- 对版本号、日期、实体名做显式突出。

Anthropic 的 reduce hallucinations 文档也强调让模型先提取证据、再作答的模式，这类“evidence-first”策略在长文档问答中很有用。

第三层：输出约束层

结构化输出并不会自动提高真实性，但可以强迫模型把“答案”和“依据”拆开。一个简单但很有效的 schema 设计是：

- answer;
- confidence / uncertainty note;
- evidence_ids;
- cannot_answer_reason。

当模型必须显式写出依据来源时，至少更容易被程序和评测发现它在编。

第四层：工具验证层

对时间敏感、事实敏感、高风险任务，不能只靠参数知识。应该：

- 调搜索；
- 查数据库；
- 走代码执行；
- 做二次校验；
- 必要时让 verifier model 或 rule engine 复核。

你在面试里如果能说“降低幻觉不是靠一个更聪明的模型，而是靠证据、验证和工作流设计”，会非常加分。

第五层：后评测与观测层

真正生产中，幻觉治理离不开数据闭环：

- 哪些问题类型最容易编；
- 哪些来源最不可靠；
- 哪些 prompt 促使模型过度自信；
- 哪些工具经常被误用；
- 哪些版本升级后出现回归。

也就是说，幻觉不是一次性解决，而是持续监控与回归测试对象。

七、Truthfulness 与 Helpfulness 的张力

很多校招面试会问一个很好的开放题：“为什么有的模型越 helpful，越容易编？”

这个问题的关键就在于张力。用户通常喜欢明确、流畅、完整的回答，但真实世界里很多问题本来就不确定、缺证据、有时间敏感性。模型如果一味追求“每个问题都给出像样答案”，就更容易输出似是而非的内容；如果一味保守，又会显得 evasive，体验很差。

Constitutional AI 相关工作的重要意义就在于，它尝试在 helpful 与 harmless 之间找到更好的平衡，同时减少那种“什么都拒绝”的无效安全。生产系统里也一样：好的 alignment 不是让模型只会拒绝，而是在该答时有证据地答，在该停时能停，在不确定时明确说不确定。

八、目前的前沿视角（截止至2026-04）

1. 厂商越来越把“诚实边界”做成产品能力

OpenAI 的 Model Spec 把 instruction hierarchy、underspecified instructions、agentic side effects 等问题上升为正式规范；Anthropic 的文档明确把 reduce hallucinations、mitigate jailbreaks、reduce prompt leak 放进 guardrails 体系；Sonnet 4.6 的发布说明中直接强调更少的 hallucinations 和 false claims of success；Google 的结构化输出、函数调用、long context 与 thinking 文档则不断把“证据、控制和可预测性”推到前台。行业已经不再满足于“模型更聪明”，而是要求它“更可控、更不乱说、更少瞎完成任务”。

2. 对齐正在从训练问题变成系统问题

以前大家把 alignment 主要看成模型训练问题；现在越来越明显，它同时也是系统设计问题。你怎么写 system prompt，怎么组织 context，怎么设计 tool schema，怎么定义失败时回退，怎么做审批和可观测，都会影响最终的“对齐感”。

所以面试里更高阶的回答，不会只停在 RLHF/DPO，而会主动补一句：“在应用侧，我们也要通过 prompt、context、tools、evaluation、guardrails 共同实现 alignment。”

九、高频面试题与答题要点

问题 1：什么是幻觉？

模型生成了缺乏证据支持、与事实或给定上下文不一致、但表面流畅合理的内容。

问题 2：幻觉的根因是什么？

最底层是训练目标不是事实核验，而是 next-token prediction；再叠加数据噪声、证据不足、任务约束不清、解码随机性和系统外部错误。

问题 3：对齐为什么重要？

因为大模型不只是一要“会”，还要“好用、可控、守边界、按格式、在不确定时诚实”。对齐提升的是可用性与可控性。

问题 4：指令跟随越强越好吗？

不绝对。强指令跟随需要正确的优先级和边界设计，否则可能更积极地执行错误或有害指令。

问题 5：为什么说“不要乱编”本身也是一种 prompt 设计？

因为你需要明确允许模型承认不知道，并把缺证据时的行为定义为合法输出，而不是逼它强答。

问题 6：如何降低幻觉？

从任务定义、证据检索、上下文组织、结构化输出、工具验证、评测监控六个层面共同治理，而不是只换模型。

问题 7：模型引用了来源就一定没幻觉吗？

不一定。它可能编造来源、错引段落、误解证据。引用只是辅助约束，不是事实保证。

问题 8：Agent 场景里的幻觉有什么新表现？

包括虚报任务完成、误报工具执行成功、误解工具返回、把计划当结果、把模拟输出当真实外部状态。

问题 9：为什么说 alignment 不只是安全问题？

因为格式稳定性、指令优先级、工具边界、诚实表达不确定性、对用户意图的可控遵循，都是 alignment。

问题 10：什么时候应该让模型拒绝回答？

当问题高风险、缺乏可靠证据、请求越权或有明显安全边界冲突时。拒绝本身也要设计成 informative refusal，而不是机械回避。

十、项目表达模板

十一、常见误区

第一个误区，是把所有错误都叫幻觉。其实检索错、工具错、路由错、格式错、指令冲突都需要分开治理。

第二个误区，是以为更强指令跟随天然会减少幻觉。错误目标下的强执行，反而可能放大幻觉。

第三个误区，是把 alignment 等同于“安全拒绝”。对齐同样包括 helpfulness、格式稳定、边界控制与诚实表达不确定性。

第四个误区，是只靠 prompt 解决所有幻觉。真正高风险任务必须引入外部证据和验证闭环。

十二、你应该记住的结论

如果只记一句话：

幻觉不是模型“坏习惯”，而是 next-token 目标、数据噪声、证据不足和系统设计共同作用下的自然结果。

再记一句：

alignment 不只是让模型更安全，更是让模型更像一个可控、诚实、能遵守优先级的系统组件。

资料来源与延伸阅读

论文

1. Ouyang et al., *Training language models to follow instructions with human feedback*
<https://arxiv.org/abs/2203.02155>
2. Bai et al., *Constitutional AI: Harmlessness from AI Feedback*
<https://arxiv.org/abs/2212.08073>
3. Lin et al., *TruthfulQA*
<https://arxiv.org/abs/2109.07958>
4. Huang et al., *A Survey on Hallucination in Large Language Models*
<https://arxiv.org/abs/2311.05232>

官方文档与说明

5. OpenAI Model Spec
<https://model-spec.openai.com/>

6. OpenAI 关于 Model Spec 的说明
<https://openai.com/index/our-approach-to-the-model-spec/>
7. Anthropic Reduce Hallucinations
<https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-hallucinations>
8. Anthropic Sonnet 4.6 发布说明
<https://www.anthropic.com/news/claude-sonnet-4-6>
9. Anthropic Models / Guardrails 文档
<https://docs.anthropic.com/en/docs/about-claude/models>
10. Gemini Structured Outputs / Function Calling (用于证据化与约束化输出)
<https://ai.google.dev/gemini-api/docs/structured-output>
<https://ai.google.dev/gemini-api/docs/function-calling>

05 Prompt Engineering

阅读目标：把 prompt engineering 从“会写几句提示词”提升到“能够稳定约束模型完成任务”的工程能力。

一、面试官在考什么

Prompt Engineering 是最容易被低估的一章。很多人觉得它“太浅”，结果一面试就暴露出明显问题：只会写一些自然语言描述，不会做任务建模，不会约束输出，不会排查失败原因，也分不清 prompt、context、tool schema 和 workflow 的边界。

第一，面试官会看你是不是把 prompt 理解成“和模型聊天的话术”。这是一个常见误区。真正的 prompt engineering，更像是给概率模型编写运行时程序：你要明确输入、任务目标、约束、输出结构、可用资源、失败边界和评价标准。只是这个程序是用自然语言、示例和 schema 写出来的。

第二，面试官会看你是否知道 prompt 的影响范围。一个好的 prompt 不只是让答案“更像人说的”，更重要的是：

- 减少歧义；
- 提高任务完成率；
- 提高格式稳定性；
- 提高工具调用准确率；
- 降低幻觉与越权行为；
- 降低后续解析和回滚成本。

如果你只从“文案更顺滑”去理解 prompt，那还是停留在使用者视角。

第三，Prompt Engineering 这章常被用来区分“会试”和“会做”。会试的人在 playground 里反复改文案；会做的人会先定义验收标准、先做失败分类、先区分不同任务阶段，再去迭代 prompt。真正线上系统里，prompt 改动本质上是一种产品逻辑改动，要能评估、回滚、对比，不是灵机一动的艺术创作。

第四，到 2025—2026 年，行业对 prompt 的理解也在变化。OpenAI 的新一代模型提示指导、Anthropic 的 prompt engineering 文档、以及 context engineering/harness engineering 的兴起，都在说明：prompt 仍然重要，但它已经不再只是“写一句更优雅的话”，而是模型接口设计的一部分。

你可以在面试的时候这样讲：

“我们把 prompt 当成应用逻辑的一部分管理。首先按任务类型拆成抽取、问答、工具执行三套模板；然后把规则、示例、输入和输出区分成独立区块；对证据敏感任务显式要求‘无证据返回 cannot_answer’，对结构化任务则强制 JSON schema。上线后不是凭感觉改 prompt，而是根据样本评测看格式合法率、幻觉率和任务完成率，再做小步迭代和版本回滚。”

二、什么是 Prompt Engineering

一个更工程化的定义是：

Prompt engineering 是围绕模型输入进行设计、约束、分层、示例化和迭代的过程，其目标是在给定模型与上下文预算下，稳定地实现特定任务结果。

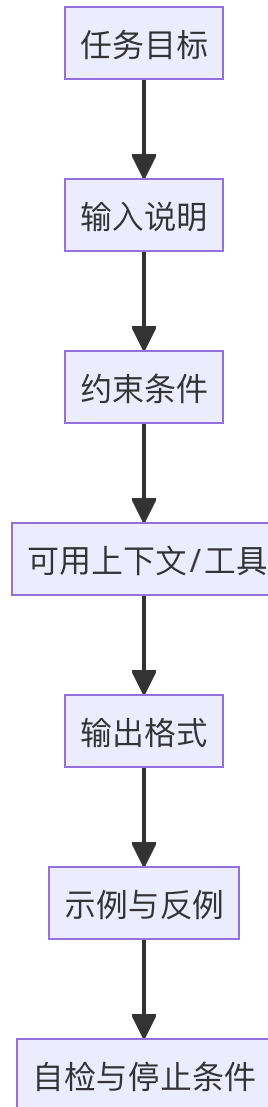
这一定义有三个重点。

第一，它强调“围绕模型输入”，说明 prompt 不只是用户输入，还可能包含 system 指令、developer 规则、few-shot 示例、输出 schema、工具说明、检索证据等。

第二，它强调“稳定实现”，说明 prompt 的目标不是偶尔跑出一个好答案，而是在真实流量里稳定、可复现、可评估地完成任务。

第三，它强调“给定模型与上下文预算”，说明 prompt 不能脱离模型家族、token 成本、上下文长度、结构化输出能力、工具协议单独讨论。一个在大模型上可行的 prompt，未必适合 mini/nano 模型；一个在短上下文下有效的 prompt，未必适合 Agent 多轮链路。

三、好的 prompt 不是长，而是清楚



一个强 prompt 通常至少包含六类信息。

1. 任务目标

模型到底要做什么。

是总结、抽取、分类、改写、SQL 生成、规划任务、写代码，还是从证据中回答问题？

目标必须可操作，不能只是“帮我处理一下”。

差的写法：

“请分析下面内容。”

好的写法：

“请从下面病历文本中抽取患者姓名、主诉、诊断、用药，并输出为 JSON。”

2. 输入边界

模型应该把哪部分内容当输入，哪部分当规则，哪部分是例子，哪部分是背景。

分隔符、标题、小节标签、XML/JSON 包裹、Markdown heading 等都属于输入边界设计。很多模型失败，不是不会做，而是没分清“正文”和“指令”。

3. 约束条件

包括：

- 不要输出什么；
- 不允许编造什么；
- 遇到信息不足怎么办；
- 是否允许调用工具；
- 是否必须严格按 schema 输出；
- 是否要先提取证据再回答。

约束越明确，模型越不容易自由发挥到错误方向。

4. 输出格式

这是最常被忽略，却最容易直接提升工程可用性的部分。

你要告诉模型输出是：

- 一段自然语言；
- Markdown 列表；
- 表格；
- JSON；
- 函数参数；
- 带字段的多段结构。

没有格式约束，后续解析和自动化都很难稳定。

5. 示例与反例

few-shot 示例不是“给模型看几个漂亮答案”，而是在告诉模型：

- 任务分布长什么样；
- 边界情况怎么处理；
- 输出风格应该对齐什么模板；

- 哪些答案算错。

尤其在小模型、抽取任务、分类任务、结构化输出任务里，示例常常比空泛指令更有用。

6. 自检与停止条件

在复杂任务里，你可以让模型在输出前做一轮检查，例如：

- 是否覆盖所有必填字段；
- 是否所有结论都有证据；
- 是否工具结果已回填；
- 是否最终输出是合法 JSON。

这类“轻量自检”在很多任务里比单纯写“请仔细思考”更有效。

四、Prompt 的常见模式：面试最爱问这些

1. Zero-shot

不给示例，只给任务说明。适合规则清楚、分布简单、模型本身能力足够强的任务。

面试里如果问 zero-shot 为什么越来越强，你可以回答：模型规模提升和 instruction tuning 让它在很多常见任务上不再严重依赖 few-shot。

2. Few-shot

给少量输入输出示例，帮助模型理解任务模式。

它尤其适合：

- label space 容易混淆的分类；
- 抽取字段边界不清的任务；
- 特定写作风格；
- 复杂 schema 生成；
- 小模型或成本敏感模型。

Few-shot 的关键不是越多越好，而是示例要覆盖真实边界。

3. Chain-of-thought / 分步提示

传统做法是显式让模型“逐步思考”。它在数学、逻辑、多步决策中可能有帮助，但 2025—2026 年的一个明显趋势是：对很多新一代 reasoning 模型，不必事无巨细规定中间步骤。更好的做法往往是明确任务、约束、验收标准，把内部推理留给模型。

所以面试里不要把“let’s think step by step”当成银弹，要强调：是否显式展示步骤，要看模型家族、任务类型、安全要求和成本。

4. ReAct / 工具调用提示

当模型需要“先思考、再决定是否调用工具、再根据工具结果继续”时，prompt 需要明确：

- 何时调用工具；
- 工具的用途和边界；
- 工具失败怎么办；
- 返回结果后怎么整合；
- 最终给用户什么格式。

这类 prompt 已经不是“问答文案”，而是执行逻辑说明。

5. Critique–Revise / Self–check

让模型先给草稿，再做自检和修订。

它适合：

- 长文生成；
- 复杂格式输出；
- 有多个验收维度的任务；
- 需要降低明显遗漏的场景。

不过注意，这种模式会增加 token 成本和延迟，不适合所有任务。

五、Prompt Engineering 的核心原则：校招回答要讲得像工程

原则 1：把目标写成可验收的任务

“写得更专业一点”“帮我优化一下”这种描述不够可操作。

更好的方式是把目标写成：

- 目标对象；
- 成功标准；

- 不允许事项；
- 输出格式。

原则 2：减少隐含假设

模型擅长补全，但不代表你的隐含假设都会被正确补全。

如果读者、语气、长度、字段、格式、失败边界都不说，模型只能猜。

Prompt 工程的本质之一，就是把你原本放在脑子里的假设尽量显式写出来。

原则 3：分离规则、数据和示例

强烈建议把 prompt 中不同性质的信息分块写：

- 规则区；
- 输入数据区；
- 示例区；
- 输出区；

这样不仅效果更稳，也更容易维护和测试。

原则 4：把“不知道怎么办”也写进去

这是降低幻觉和错误执行的关键。

你应该明确写：

- 若证据不足，返回 `cannot_answer`；
- 若字段缺失，返回 `null` 而不是编造；
- 若工具失败，输出 `error_reason`；
- 若请求越权，拒绝并说明原因。

好的 prompt 不是只定义成功路径，也定义失败路径。

原则 5：Prompt 是要测的，不是要信的

真正做过项目的人都知道，prompt 看起来很合理，不代表线上就稳定。

你需要用代表性样本去测：

- 任务成功率；
- 格式合法率；

- 幻觉率；
- 工具调用准确率；
- 平均 token 和延迟；
- 对边界输入的鲁棒性。

所以 prompt engineering 本质上是 eval-driven iteration。

六、2025—2026 的新变化：Prompt 还重要，但它不再是全部

1. OpenAI：更强调模型家族差异与提示模式迁移

OpenAI 的 reasoning best practices 和 GPT-5.4 prompt guidance 一再强调：不同模型家族行为差异明显。reasoning model 适合更明确目标与验收标准，而不是过细控制每一步；小模型则往往更需要显式约束与更具体提示。

这意味着“一个万能 prompt 走天下”的时代已经过去，prompt 设计必须与模型选型联动。

2. Anthropic：从 prompt engineering 走向 context engineering / harness

Anthropic 在 2025 年后连续发布 prompt engineering、effective context engineering、effective harnesses for long-running agents、advanced tool use 等文章，明显在传递一个信号：

单条 prompt 仍重要，但真正决定 Agent 表现的，是上下文怎么组织、工具怎么设计、长期任务怎么压缩状态。

所以今天谈 prompt，不能孤立谈“这句话怎么写”，而要连同上下文、工具和任务循环一起看。

3. Gemini：参数与提示经验开始出现家族差异

Gemini 3 文档多次强调 temperature 保持 1.0 是推荐设置，低温不一定更稳。这是一个很好的提醒：prompt 和参数经验都要看模型家族，不能机械套用旧经验。

对于多模态与结构化任务，Gemini 还把 structured output、function calling、files、long context 放在一个统一开发体系里，说明 prompt 早已不是孤立文本，而是 API 配置与上下文资产的一部分。

七、几个非常实用的 prompt 模板思路

1. 抽取型模板

适合信息抽取、简历解析、票据解析、工单解析。

关键要素：

- 字段列表；
- 字段含义；
- 缺失时输出 null；
- 不允许编造；
- 输出 JSON Schema。

这类任务通常温度更低、格式约束更强。

2. 证据问答模板

适合 RAG、文档问答、知识库问答。

关键要素：

- 只能依据给定证据回答；
- 先定位证据，再形成结论；
- 若证据不足明确返回无法回答；
- 最终附 evidence_ids / source list。

这类模板对减少幻觉特别有效。

3. 工具调用模板

适合 Agent、函数调用、自动化工作流。

关键要素：

- 工具说明；
- 调用时机；
- 输入参数约束；
- 工具失败处理；
- 禁止在未调用工具时假装已执行。

这类模板对于降低“虚报已完成”非常关键。

4. 评审/打分模板

适合 LLM-as-a-Judge、简历评分、内容审核。

关键要素：

- 评分维度；
- 评分等级定义；
- 先列观察，再给分；
- 输出 JSON 或表格；
- 禁止越过给定标准自由发挥。

评分任务最怕标准漂移，所以 prompt 里必须把 rubric 写清楚。

八、高频面试题与答题要点

问题 1: Prompt Engineering 到底是什么？

它不是“和模型聊天的技巧”，而是围绕模型输入进行任务定义、约束、示例化和迭代的工程过程。

问题 2: 好的 prompt 最重要的部分是什么？

最重要的是明确任务目标、约束边界和输出格式，而不是把 prompt 写得特别长或特别像自然对话。

问题 3: Few-shot 一定比 zero-shot 好吗？

不一定。强模型在常规任务上 zero-shot 已经很好；few-shot 更适合边界复杂、标签容易混淆、输出格式特殊、小模型场景。

问题 4: 为什么 prompt 要分块写？

因为规则、数据、示例、输出区分清楚后，模型更容易正确解释输入，你也更容易维护和调试。

问题 5: 如何降低 prompt 导致的幻觉？

显式写明证据边界、缺证据处理方式、字段缺失输出规则，并尽量提供结构化证据和输出 schema。

问题 6: 为什么不能只靠 prompt 解决一切？

因为模型能力、上下文质量、工具设计、缓存预算、评测闭环同样重要。prompt 是大杠杆，但不是唯一杠杆。

问题 7: reasoning model 的 prompt 和普通模型有什么不同?

复杂 reasoning model 往往更适合明确目标和验收标准，而不是过细 micromanage；小模型则更需要显式步骤和例子。

问题 8: prompt 改动为什么要配合评测?

因为它本质上会改变模型行为，需要用数据看成功率、鲁棒性、格式合法率和成本是否真的变好。

问题 9: Prompt Engineering 和 Context Engineering 的区别是什么?

前者更偏单次提示设计与任务表达，后者更偏整个运行时上下文的选择、压缩、排序和生命周期管理。

问题 10: 项目里 prompt 一般怎么迭代?

先定义数据集与指标，分析失败类型，再定向修改任务描述、约束、示例或结构化输出要求，而不是盲改措辞。

九、常见误区

第一个误区，是以为 prompt 越长越好。实际上很多长 prompt 只是把歧义放大了。

第二个误区，是只会改措辞，不会改任务结构。真正有效的优化往往是重新定义目标、加格式约束、补 few-shot、改失败边界。

第三个误区，是把 prompt 结果不好直接归咎于模型太弱。很多时候是任务没写清、上下文没分块、输出没约束。

第四个误区，是把 prompt 视为一次性工作。生产上 prompt 必须版本化、评测化、可回滚。

十、我的一些观点

Prompt engineering 的核心不是“写一句更聪明的话”，而是把任务、约束、证据和输出结构清楚地编排成模型能稳定执行的输入。

Prompt 是要靠评测驱动迭代的产品逻辑，不应该是黑箱。

资料来源与延伸阅读

官方文档

1. OpenAI Prompt Engineering
<https://developers.openai.com/api/docs/guides/prompt-engineering/>
2. OpenAI Prompt Guidance for GPT-5.4
<https://developers.openai.com/api/docs/guides/prompt-guidance/>
3. OpenAI Best Practices for Prompt Engineering
<https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api>
4. OpenAI Reasoning Best Practices
<https://developers.openai.com/api/docs/guides/reasoning-best-practices/>
5. Anthropic Prompt Engineering Overview
<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>
6. Gemini Prompting Strategies
<https://ai.google.dev/gemini-api/docs/prompting-strategies>
7. Gemini Text Generation / Gemini 3 Guide
<https://ai.google.dev/gemini-api/docs/text-generation>
<https://ai.google.dev/gemini-api/docs/gemini-3>

论文与补充

8. Brown et al., *Language Models are Few-Shot Learners*
<https://arxiv.org/abs/2005.14165>
9. OpenAI Cookbook: GPT-5 Prompting Guide
https://developers.openai.com/cookbook/examples/gpt-5/gpt-5_prompting_guide/

06 Context Engineering

阅读目标：理解为什么 2025—2026 年行业讨论已经从“怎么写 prompt”扩展到“怎么构造运行时上下文”，以及 context engineering 与 prompt engineering、RAG、memory 的边界关系。

一、context eng的演变

如果说 2023 年大家还在讨论“prompt engineering 有没有技巧”，那么到 2025—2026 年，越来越多团队开始把重点转向 context engineering。原因很现实：模型本身变强了，但真实系统失败越来越少是因为“不会写一句话”，越来越多是因为“上下文给错了、给多了、给乱了、给旧了、给得不合时机”。

第一，Agent 场景下，模型输入不再只是用户问题和一段 system prompt。它可能还包括：

- 角色与权限规则；
- 历史对话；
- 用户记忆；
- 检索证据；
- 工具 schema；
- 工具调用结果；
- 计划与待办状态；
- 文件路径、URL、数据库游标；
- 长任务中的阶段摘要。

这意味着“输入设计”从单条 prompt，升级成了一个多源上下文组装问题。

第二，大上下文不再昂贵到完全不能用，但也远没到可以随便乱塞的程度。上下文越大，延迟和成本越高；关键信息越分散，模型越可能找不到重点。所以真正决定效果的不是“有没有很多上下文”，而是“在当前这一轮，最应该让模型看到什么”。

第三，很多 Agent 失败都不是“不会推理”，而是“推理时看到了错误的东西”。比如：

- 历史任务状态过时；
- 工具结果没有清洗；
- 旧计划和新计划同时存在；
- 权限规则埋在很后面；
- 用户偏好与系统规则冲突；
- 大段无关日志稀释注意力。

这时候继续死磕 prompt wording 意义不大，必须上升到 context engineering。

第四，Anthropic 在 2025 年专门写了 *Effective context engineering for AI agents*，OpenAI 在新 Responses API 里强调 compaction，Anthropic models list 直接暴露 context management/compact 能力，Gemini 文档也在 Live/session 场景里讨论 summary 与 context reuse。行业已经用产品和 API 设计告诉你：上下文管理是第一等公民。

一个比较能打动面试官的表述是：

我们后来发现系统表现不稳定，主要不是 prompt wording 问题，而是上下文来源太多：系统规则、用户历史、检索证据、工具结果全混在一起。于是我们做了三件事：第一，把规则、证据、状态分区标注；第二，把长会话历史压成结构化 summary/state，而不是全量回灌；第三，对大文件和大工具集改成 just-in-time 加载，只在需要时注入。这样做以后，模型在复杂任务中的稳定性提升明显，token 成本和延迟也降了。”

二、先给一个工作定义

Context engineering 是围绕模型运行时输入进行选择、分层、排序、压缩、生命周期管理和验证的工程过程。

这个定义里有六个关键词。

- **选择**：哪些内容应该进当前轮上下文；
- **分层**：系统规则、任务目标、证据、示例、工具、状态要不要分区；
- **排序**：什么靠前，什么靠后；
- **压缩**：哪些历史要总结，哪些原文要裁剪；
- **生命周期管理**：哪些信息只在当前轮有效，哪些应跨轮保留；
- **验证**：输入给模型的上下文是否完整、冲突、过时或越权。

你会发现，prompt engineering 更像在优化“怎么说”，而 context engineering 更像在优化“让模型在这一刻看到什么、以什么结构看到”。

三、Prompt Engineering 与 Context Engineering 的关系

很多面试官会直接问：“它俩区别是什么？”

一个好回答不是非此即彼，而是分层。

1. Prompt Engineering：偏表达与约束

它更关注：

- 任务怎么描述；
- 规则怎么写；
- 输出格式怎么限定；
- few-shot 示例怎么给；
- 工具说明怎么表述。

你可以把它看成“单轮输入文本与 schema 的设计”。

2. Context Engineering：偏运行时状态组织

它更关注：

- 本轮需要哪些外部证据；
- 历史对话保留到什么程度；
- 用户记忆是否应该注入；
- 工具结果是原样放入还是摘要后放入；
- 哪些内容应该常驻前缀，哪些按需加载；
- 多轮长任务怎么做 compaction。

它是“整个运行时输入系统的设计”。

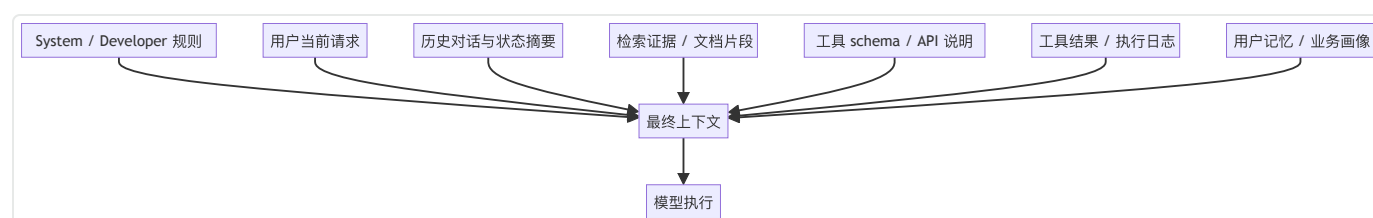
3. 两者不是替代关系

没有好的 prompt，context 再好也会执行歪；

没有好的 context，prompt 再漂亮也会在错误材料上发挥。

真正生产里，两者必须一起设计。你在面试里如果能说出这句，通常会被认为理解比较到位。

四、Context 到底由哪些东西组成



真正线上系统里的“上下文”，远比你在 playground 里看到的大。常见来源包括：

1. 规则上下文

system / developer prompt、产品规则、权限边界、品牌风格、合规要求、审批策略等。这些通常优先级最高，且适合做稳定前缀。

2. 任务上下文

用户当前请求、本轮输入数据、UI 状态、用户选中的文件、当前页面信息、表单字段等。这是最直接的任务驱动信息。

3. 知识上下文

检索召回的文档、FAQ、业务规则、数据库结果、网页搜索结果、代码片段等。它决定模型有没有事实依据。

4. 状态上下文

会话历史、计划进度、已完成步骤、待办事项、失败重试次数、上次工具返回、临时变量等。Agent 系统特别依赖这类上下文。

5. 记忆上下文

长期用户偏好、组织信息、项目约定、个人习惯、长期目标等。这类上下文不是每轮都变，但也不该每轮全部注入。

6. 工具上下文

工具名字、用途、参数 schema、权限边界、调用代价、返回格式、错误定义。工具越多，这部分越容易成为上下文膨胀源。

五、Context Engineering 的核心问题：不是收集，而是取舍

1. 相关性

只有和当前任务相关的内容才值得放进上下文。

“可能以后有用”通常不等于“这一轮值得占 token”。

一个很常见的失败是把所有历史工具结果都原样放进去，结果真正关键的最新状态反而被淹没。

2. 权威性

上下文里可能同时出现用户猜测、过时 FAQ、数据库实时结果、缓存旧状态、检索文档片段。它们的可信度不同。

Context engineering 不只是选“相关的”，还要选“优先相信谁的”。

这点在企业知识问答、配置规则执行、财务与运营类 Agent 场景尤其关键。

3. 新鲜度

时间敏感信息必须优先新鲜来源。

例如：

- 最新库存；
- 当前配置；
- 当日政策；
- 实时工单状态；
- 最新 commit。

把旧版本内容和新版本内容一起塞进去，模型大概率会混。

4. 冗余与冲突

重复信息会浪费预算，冲突信息会让模型摇摆。

上下文里最危险的不是“缺一点”，而是“多了两份互相打架的版本”。

5. 时机

Anthropic 的 context engineering 文章强调“just in time”思路：不要预先把所有可能有用的信息都塞进上下文，而是保留轻量引用（如文件路径、查询、URL、ID），等模型真正需要时再加载。这一思路对大型代码库、长文档、多工具 Agent 特别重要，因为它能显著降低上下文膨胀。

六、常见的上下文组织模式

模式 1：稳定前缀 + 动态后缀

把长期稳定的信息（角色、规则、工具说明）固化成前缀；把用户请求、实时数据、检索结果作为动态后缀插入。

这样既利于缓存命中，也更利于维护版本。

模式 2：计划状态与执行日志分离

很多 Agent 会把“计划”“中间观察”“工具日志”全混在一起，导致模型越跑越乱。

更好的方式是把：

- 当前计划；
- 已完成步骤；
- 当前可行动作；
- 最近一次关键观察；

提炼成结构化状态，而不是把完整执行日志原样喂回去。

模式 3：证据区与规则区分离

在 RAG/知识问答里，把“你必须遵守的规则”和“你可以参考的证据”混写，模型容易误读。

最好单独标注：

- 规则区：输出必须遵守；
- 证据区：回答只能依据；
- 无证据时：返回 `cannot_answer`。

这种设计对减少幻觉很有效。

模式 4：按需加载工具描述

工具很多时，不应每轮都把几十上百个工具 schema 全塞给模型。2025 年 Anthropic 提出的 tool search / programmatic tool calling，以及 MCP 生态的一些实践，本质都在解决“工具描述过大”的上下文负担。

真正要学会的是：

不是让模型从一开始就看到所有工具，而是让它在需要时发现、检索或装载工具。

七、Context Engineering 的关键手法

1. 选择：只保留当前决策所需最小充分信息

这是最核心的一条。上下文不是越多越完整，而是越接近“最小充分集”越好。

所谓最小充分，就是：

- 足够回答当前问题；

- 但不额外引入过多噪声与冲突。

2. 排序：关键信息位置要显眼

考虑到长上下文利用率问题，最关键的规则与证据最好靠前放置或显式标注。

“重要信息埋在长文档中间”是上下文设计的大忌。

3. 压缩：把历史从“原文”变成“状态”

尤其是长任务和多轮会话，不可能永久保留原始历史。更常见做法是把历史压成：

- summary;
- task state;
- user preference memory;
- unresolved issues。

OpenAI 的 `/responses/compact`、Anthropic 的 context management capability、Gemini Live 的 summary/resumption 思路都说明：压缩正在从应用技巧变成平台能力。

4. 标注：告诉模型每一块上下文是什么

你给模型一堆文本，它未必知道哪部分是规则，哪部分是证据，哪部分是示例。

强烈建议做明确区块标注，例如：

- `<rules>`
- `<task>`
- `<evidence>`
- `<memory>`
- `<tool_result>`
- `<output_schema>`

这既提升可读性，也提升模型解释输入的准确性。

5. 清理：删除失效内容

过期状态、旧版本规则、失败工具日志、重复检索片段都应该及时清掉。

一个成熟系统不会“越积越多”，而会“边跑边清”。

八、Context Engineering 与 RAG、Memory 的边界

1. RAG 不是全部 context engineering

RAG 更偏“如何拿到外部知识”。

Context engineering 则还包括：

- 是否需要这个知识；
- 用原文还是摘要；
- 放在上下文哪个位置；
- 和规则、历史、工具结果怎么拼；
- 是否只保留引用而不直接展开。

2. Memory 也不是全部 context engineering

Memory 关注跨轮持续信息，例如用户偏好、长期目标、项目背景。

Context engineering 则决定：

- 哪些记忆该在本轮注入；
- 以什么粒度注入；
- 何时不要注入；
- 记忆与实时输入冲突时谁优先。

3. Context Engineering 是上层调度框架

一个常见的好回答是：

RAG 负责“找”，memory 负责“记”，context engineering 负责“在这一轮怎么用”。

这句很适合面试。

九、调试上下文问题：真实系统里最值钱的能力

很多同学一出问题就改 prompt wording，但 context bugs 往往更致命。你应该学会这样排查：

1. 看遗漏

模型答错，是不是因为关键规则或关键证据根本没进上下文？

2. 看污染

是不是塞了太多不相关历史、重复片段、旧版本状态？

3. 看冲突

是不是两份互相矛盾的信息同时存在？

4. 看顺序

最关键的信息是不是位置不显著、被埋在中间？

5. 看粒度

工具结果是不是太原始，模型难以消费？
或者相反，摘要过头，把关键字段丢了？

6. 看生命周期

这段历史本轮还该不该保留？
用户偏好是不是在当前任务反而构成干扰？
一个成熟工程师不会只问“prompt 怎么改”，而会先问“这轮上下文是不是从根上组错了”。

十、高频面试题与答题要点

问题 1：什么是 context engineering？

是对模型运行时输入进行选择、排序、压缩、生命周期管理和验证的工程过程。

问题 2：它和 prompt engineering 的区别是什么？

Prompt 更偏单轮任务描述与约束表达；context engineering 更偏整个运行时上下文系统如何组装和管理。

问题 3：为什么现在越来越强调 context engineering？

因为模型和 Agent 的输入来源变多，失败往往来自错上下文、旧状态、噪声和冲突，而不仅是提示词措辞。

问题 4：上下文越多越好吗？

不一定。过多上下文会增加噪声、成本和延迟，还可能降低关键信息密度。

问题 5：如何提高上下文质量？

确保相关性、权威性、新鲜度，减少冲突和冗余，关键内容显式标注并放在更显著位置。

问题 6：RAG 和 context engineering 有什么关系？

RAG 提供知识来源，context engineering 决定这些知识是否该用、如何拼接、如何压缩、放在哪里。

问题 7：多轮 Agent 为什么需要状态压缩？

因为原始历史会快速膨胀并污染上下文，应该把历史转成结构化状态或 summary。

问题 8：什么是 just-in-time context？

先保留轻量引用，在模型真正需要时再加载详细内容，避免预先塞入大量可能无关的信息。

问题 9：为什么要分区标注上下文？

为了让模型区分规则、证据、示例、记忆和工具结果，降低误读输入结构的风险。

问题 10：怎样判断一个问题是 prompt bug 还是 context bug？

如果目标和约束表达已经清晰，但模型在错误材料上推理、混用旧状态或无视证据，通常更像 context bug。

十一、常见误区

第一个误区，是以为 context engineering 只是“多加一点背景资料”。其实它更强调取舍、排序和生命周期。

第二个误区，是把历史消息全量保留当成“信息更完整”。现实里这常常意味着更脏、更慢、更冲突。

第三个误区，是不区分规则与证据。模型会把它们混读，导致既不听规则，也不忠于证据。

第四个误区，是认为有了大上下文就不用做 context engineering。恰恰相反，窗口越大，选择和压缩越重要。

十二、我的一些思考

Context engineering 不是“给更多信息”，而是“在这一轮给最对的信息，以最合适的结构给出去”。

RAG 负责找信息，memory 负责留信息，context engineering 负责让模型在当前时刻真正用对信息。

资料来源与延伸阅读

官方与工程文章

1. Anthropic, *Effective context engineering for AI agents*
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
2. Anthropic, *Effective harnesses for long-running agents*
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
3. Anthropic Models List (context management capabilities)
<https://docs.anthropic.com/en/api/models-list>
4. Anthropic Prompt Engineering Overview
<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>

07 结构化输出与 Function Calling（重要）

阅读目标：搞清楚文本生成与可程序消费输出的差别，掌握 JSON Schema、Function Calling、工具执行循环与生产级防错方法。

一、它真的很重要

如果说 Prompt Engineering 是让模型更像会做题，那么结构化输出与 function calling 则是让模型真正接进系统里。校招 Agent 岗位里，这一题几乎可以说是绕不开的，因为只要模型要和外部程序、数据库、搜索、日历、审批流或前端界面协同，它就不能只输出一段好看的自然语言，而必须输出稳定、可验证、可执行的结构。

面试官问这一题，核心在看四件事。

第一，你是否知道输出合法 JSON 和输出符合 schema 的 JSON 不是一回事。历史上很多人用 JSON mode，以为模型只要输出 parseable JSON 就够了。但真实工程里，你需要的是字段名、类型、枚举值、必填项都稳定一致，否则程序照样会崩。OpenAI 在 2024 年推出 Structured Outputs 时就明确区分：JSON mode 只保证是有效 JSON，不保证符合你给定的 schema；Structured Outputs 才是让模型严格对齐 schema。到 2026 年，Anthropic 的 structured outputs 也已 GA，Gemini 也有正式的 structured outputs 文档。这说明把输出变成机器可消费资产已经成为平台级能力，而不再是技巧。

第二，面试官会看你是否理解 function calling 的本质。很多人会说 function calling 就是模型帮我调函数。这句话不够准确。更准确的说法是：模型输出一个**建议调用哪个工具、带什么参数**的结构；真正执行业务动作的是你的应用代码。也就是说，function calling 本质上是模型规划 + 程序执行的接口协议，不是模型自己直接跑外部世界。

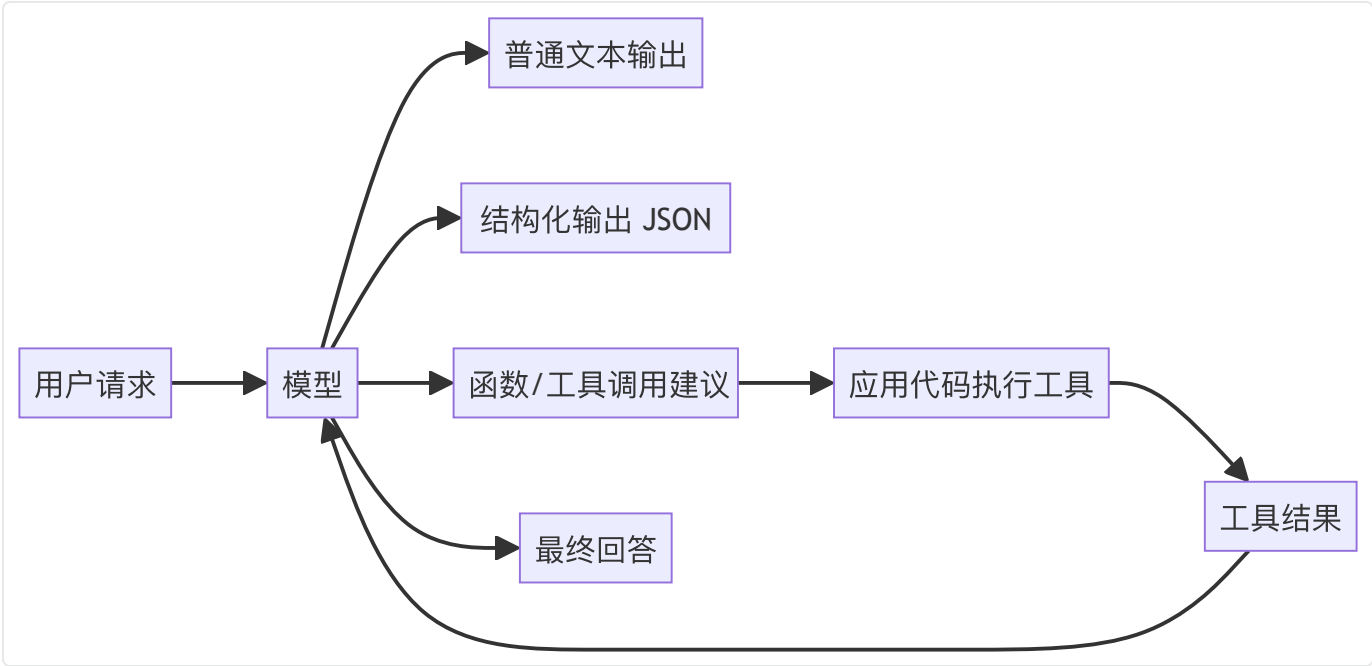
第三，面试官在看你有没有生产思维。真实系统里，工具调用最大的风险不是调不出来，而是调错了还继续往下跑。所以你要会说：

- schema 要严格；
- 参数要验证；
- 高风险动作要审批；
- 工具失败要回传；
- 外部副作用要幂等；
- 不允许模型在没执行时谎称执行成功。

一旦你能讲到这里，基本已经不再是懂 API，而是在讲 Agent 工程。

第四，这一题还经常连着问多工具、并行工具、工具 schema 设计、工具选择策略、structured outputs vs function calling 的区别。所以这里要学的不只是定义，而是一整套接口与 workflow 思维。

二、概念拆解与区分



1. 普通文本输出

模型输出自然语言，人读即可。适合聊天、写作、解释型内容，但不适合直接驱动程序。

2. 结构化输出

模型按预定义 schema 输出 JSON/枚举/对象等结构。它更适合：

- 信息抽取；
- 分类；
- 审核结果；
- 表单生成；
- UI 渲染；
- 下游程序消费。

这里的重点是结构稳定，不是看起来像 JSON。

3. Function Calling / Tool Calling

模型不直接给最终答案，而是输出：

- 要调用哪个工具；
- 工具参数是什么；

然后由应用侧代码执行，并把结果再喂给模型。

所以 function calling 更适合需要访问外部世界、实时数据或可执行动作的任务。

4. Structured Outputs 与 Function Calling 的关系

它们既相关又不同。

结构化输出关注最终输出长什么样；

function calling 关注模型如何以结构化方式提出动作请求。

很多平台把 tool schema 也建立在 JSON Schema 之上，因此两者底层思路越来越统一：都是用 schema 把模型自由文本空间收紧。

三、为什么结构化输出这么重要

1. 程序不消费差不多正确

人类读文本时能容忍字段名有点变体少个逗号我也懂这个枚举值写得像那个意思。程序不行。

只要字段缺失、类型错误、枚举漂移、层级不对，下游系统就会失败。

所以在应用系统里，可靠性来自可验证结构，不是可理解语义。

2. 结构化输出能把任务从生成变成判定

一旦你把任务转成 JSON schema，很多问题就更容易被评估了。

例如原来让模型总结用户问题意图，很难评；

改成：

- intent；
- urgency；
- required_action；
- confidence；

立刻就能算字段正确率、合法率、分布一致性。

这对迭代和回归测试非常有价值。

3. 结构化输出是 Agent 工作流的中间语言

很多 Agent 系统的本质，就是在多个步骤之间传递结构化状态：

- 计划；
- 子任务；
- 工具参数；
- 观察结果；
- 最终结论；
- 审批单。

如果这些都只是自然语言段落，系统就很难稳。

所以你可以把结构化输出看成 Agent 的中间表示。

四、什么是好的 schema：面试真正拉开差距的地方

1. 字段要贴近业务动作，而不是贴近原始文本

例如做工单路由，不要只定义一个 `summary` 字段；更好的 schema 可能是：

- `department`
- `priority`
- `category`
- `requires_human_review`
- `missing_information`
- `suggested_reply`

一个好的 schema 应该直接对应你的下游流程，而不是为了看起来完整。

2. required / optional / nullable 要分清

很多结构化失败来自这三者混淆。

- required：字段一定要出现；
- optional：字段可以不出现；
- nullable：字段必须出现，但值可以是 null。

校招面试如果你能主动说出这一点，会很加分，因为这体现了你考虑程序消费端的严谨性。

3. enum 比自由文本更稳

如果下游只有 5 个合法路由部门，就应该用 enum，而不是让模型自由写部门名。
能收紧的地方尽量收紧，模型自由度越大，漂移越大。

4. additionalProperties: false 很有价值

OpenAI 结构化输出、很多 JSON Schema 最佳实践都强调这一点：如果你不希望模型输出多余字段，就应显式禁止。否则模型可能在好心补充时把 schema 撑歪。

5. schema 不要为了通用而过度抽象

很多同学喜欢做一个超通用 schema，结果字段又多又虚，模型难学，下游也难用。
更好的做法往往是：一个任务一个清晰 schema，必要时版本化，而不是试图用一个万能 schema 解决所有业务。

五、Function Calling 的本质流程

OpenAI function calling 文档把流程概括得很清楚，核心一般分为五步：

1. 你把可用工具与 schema 发给模型；
2. 模型返回工具调用建议；
3. 应用侧代码执行工具；
4. 把工具结果回传给模型；
5. 模型给出最终答案，或者继续提出新的工具调用。

这五步里，真正容易出问题的不是第 2 步，而是第 3 步和第 4 步之间的衔接。因为一旦执行环境、权限系统、重试策略、幂等设计没做好，模型再聪明也救不了系统。

六、Function Calling 的关键工程点

1. 模型只决定建议调用，程序决定是否执行

这一句非常值得背。

模型可以建议调用 `transfer_money(amount=10000)`，但是否真的转账，必须由程序做权限校验、额度校验、审批校验，必要时还要二次向用户确认。

如果你在面试里能明确这句，基本就避开了 function calling 最大的认知坑。

2. 工具描述要清晰，但不要冗长

工具 schema 设计常见错误是：

- 工具名模糊，如 `process_data` ；
 - 描述太短，模型分不清和别的工具区别；
 - 描述太长，塞满实现细节，占上下文；
- 一个好的工具定义应该清楚说出用途、输入、边界和限制，而不是堆 SDK 文档。

3. 工具要做能力最小化

这和安全强相关。

如果一个高风险动作可以拆成查询和执行两个工具，就不要一上来暴露能直接执行副作用的接口。

例如：

- `get_balance`
- `create_transfer_draft`
- `confirm_transfer`

比单一 `transfer_money` 更安全、更好控。

4. 高风险动作必须有 approval / human-in-the-loop

Google 的 function calling 文档明确建议，对于有显著后果的函数调用，应在执行前做用户验证。

这是一个非常好的面试加分点：

function calling 的可靠性不只靠 schema，还靠审批流和 side-effect guardrail。

5. 工具失败必须结构化回传

很多系统工具一报错，就只返回自然语言出错了。

更好的方式是结构化返回：

- `error_code`；
- `retryable`；
- `human_message`；
- `raw_detail`（如有必要）；

这样模型才更容易决定是重试、降级、换工具还是请人工介入。

七、Structured Outputs vs JSON Mode vs Function Calling

这是校招面试里最容易被直接点名的问题。

1. JSON Mode

保证模型输出可以被解析为 JSON。

问题在于：它不保证字段齐全、不保证类型正确、不保证符合你的业务 schema。

所以它更像一个最低门槛的语法约束。

2. Structured Outputs

保证输出遵守你提供的 JSON Schema。

OpenAI 官方文档明确把它定义为确保输出匹配 schema；Anthropic 在 2026-01 已把 structured outputs GA；Gemini 也在 2026-02 发布了正式 structured outputs 文档。

这意味着如果你的任务本质是输出一个稳定对象，应优先考虑 structured outputs，而不是只开 JSON mode。

3. Function Calling

关注的是让模型以结构化方式提出外部动作请求。

它常用在需要实时信息、数据库、搜索、代码执行、系统操作时。

如果任务只是抽取字段，不一定需要 function calling；

如果任务需要查询订单、创建工单、调用搜索，就更适合 function calling。

4. 一个非常实用的经验

- 纯抽取/分类：优先 structured outputs；
- 有外部动作：优先 function calling；
- 最终结果还要给前端/程序消费：最终答案也尽量结构化。

也就是说，很多真实工作流会同时用到两者：

中间靠 function calling 调工具，最终靠 structured outputs 交付可消费结果。

八、多工具、并行工具与工具选择

1. 多工具场景常见难点

工具越多，模型越容易出现：

- 选错工具；
- 参数混淆；
- 工具 schema 占太多上下文；
- 明明需要组合工具却只调一个；
- 明明不该调工具却硬调。

所以多工具不是工具越全越好，而是工具集合是否最小、边界是否清楚、路由是否清晰。

2. 并行工具调用

OpenAI 和 Anthropic 都在工具使用文档/工程文章里涉及并行或程序化工具调用。它的价值在于：当多个工具彼此独立时，可以并发执行，缩短总时延。

但注意，并行调用只适合独立查询；有依赖关系的链式步骤不能强行并行。

3. 工具搜索与按需装载

当工具非常多时，把全部 schema 每轮放进上下文会非常重。Anthropic 在 2025 年提出 tool search / programmatic tool calling，就是在解决大工具集上下文膨胀问题。

这其实和 context engineering 是同一件事：不要在当前轮暴露不必要工具。

九、生产级防错：这部分面试最能拉差距

1. 校验先于执行

所有 function calling 输出都应该经过：

- schema 校验；
 - 类型校验；
 - 权限校验；
 - 业务规则校验；
- 通过后再执行。

2. 幂等与重试

如果工具调用可能重试，就必须考虑幂等。例如创建订单、发消息、扣费这些动作，不能因为模型重复调用或网络抖动导致副作用重复发生。

3. 显式记录 tool trace

要记录：

- 模型为什么选这个工具；
- 传了什么参数；
- 工具返回了什么；
- 最终答案如何利用了结果。

这不仅方便排障，也方便后续做回放和评测。

4. 不要把自由文本塞进高风险参数

能用 enum、ID、日期、受限字符串的地方，就不要让模型自由生成。

这是降低错误执行率最有效的方法之一。

5. 最终答案也要对齐工具真实结果

Agent 很容易犯的一个错是：工具失败了，但模型照样生成一个看似完成的自然语言答案。

这类false claim of success非常危险，所以最终回答必须与工具真实返回绑定，而不是只凭模型主观推断。

十、高频面试题与答题要点

问题 1：什么是 function calling?

模型根据工具 schema 输出一个调用建议，包括工具名和参数；应用程序执行后再把结果反馈给模型。

问题 2：function calling 和 structured outputs 的区别是什么?

前者主要解决如何发起外部动作/查询，后者主要解决如何稳定生成可程序消费的最终结构。

问题 3：JSON mode 和 structured outputs 有什么区别?

JSON mode 只保证合法 JSON，structured outputs 还保证符合你给定的 schema。

问题 4：模型会自己执行函数吗？

不会。真正执行函数的是你的应用代码，模型只负责输出调用意图和参数。

问题 5：为什么 schema 设计很重要？

因为它决定模型自由度、参数合法率、下游解析成本和系统安全边界。

问题 6：高风险工具如何设计？

应拆成查询、草稿、确认等多个阶段，并加审批或用户确认，避免模型直接触发不可逆副作用。

问题 7：什么时候不该用 function calling？

当任务只是简单分类、抽取、打标签或生成最终 JSON，而不需要访问外部系统时，直接 structured outputs 往往更简单。

问题 8：如何减少模型选错工具？

工具名和描述清晰、工具边界去重、减少同时暴露的工具数量、按需装载、提供高质量工具示例。

问题 9：工具失败了怎么办？

结构化回传错误码和可重试信息，让模型决定重试、降级、换工具或请人工介入，绝不能默认成功。

问题 10：并行工具调用的价值是什么？

当多个查询彼此独立时，可显著降低总时延；但要避免把有依赖关系的步骤错误并行。

最终答案与真实执行结果对齐。

十二、常见误区

第一个误区，是把 function calling 理解成模型直接调用外部 API。

真正执行的是应用层，不是模型本身。

第二个误区，是以为只要输出 JSON 就稳定。

没有 schema 约束的 JSON 很容易字段漂移。

第三个误区，是把所有业务都塞进一个巨型工具。

这会让工具选择困难、参数复杂、安全边界模糊。

第四个误区，是工具失败后只返回自由文本。

这让模型很难稳定处理错误分支。

十三、你应该记住的结论

如果只记一句话：

结构化输出解决的是模型结果怎么稳定接进程序，function calling 解决的是模型如何以受控方式请求外部能力。

再记一句：

模型负责提出动作意图，程序负责校验与执行；高风险副作用绝不能直接交给模型自由决定。

资料来源与延伸阅读

官方文档

1. OpenAI Structured Outputs
<https://developers.openai.com/api/docs/guides/structured-outputs/>
2. OpenAI Function Calling
<https://developers.openai.com/api/docs/guides/function-calling/>
3. OpenAI Responses API / 迁移指南
<https://developers.openai.com/api/docs/guides/migrate-to-responses/>
4. OpenAI 2024 Structured Outputs 发布说明
<https://openai.com/index/introducing-structured-outputs-in-the-api/>
5. Anthropic Release Notes (Structured Outputs GA)
<https://docs.anthropic.com/en/release-notes/overview>
6. Anthropic Tool Use / Prompt Engineering Overview
<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>
7. Gemini Structured Outputs
<https://ai.google.dev/gemini-api/docs/structured-output>
8. Gemini Function Calling

<https://ai.google.dev/gemini-api/docs/function-calling>

9. Vertex AI Function Calling / Structured Output

<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/function-calling>

<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/control-generated-output>

Cookbook 与示例

10. OpenAI Cookbook: Structured Outputs Intro

https://developers.openai.com/cookbook/examples/structured_outputs_intro/

11. OpenAI Cookbook: Reasoning + Function Calls

https://developers.openai.com/cookbook/examples/reasoning_function_calls/

08 模型选型与路由

阅读目标：知道如何在质量、延迟、成本、上下文、工具能力与稳定性之间做取舍，并能把选一个模型升级成构建一套路由策略。

一、模型选择的理解反映了你的技术敏感度

过去几年，大模型面试里常见问题是你用过哪些模型。到 2025—2026 年，这个问题已经明显升级成了你怎么选模型、怎么做模型路由、为什么不是所有请求都上最强模型。这背后反映的是行业成熟：模型越来越多、能力越来越分层、价格差距越来越大、接口越来越统一，真正重要的已经不是知道有哪些模型，而是知道如何做系统级决策。

第一，面试官会看你有没有摆脱排行榜思维。真实业务并不总是需要最强模型。分类、抽取、路由、意图识别、简单改写，往往小模型更划算；复杂规划、难代码、多步工具协作，才值得上 reasoning/frontier 模型。一个只会说我一般选最强的或者我一般选最便宜的同学，通常都没有做过生产权衡。

第二，面试官会看你是否知道模型能力不是单一维度。选型至少要看：

- 任务复杂度；
- 推理深度；
- 工具支持；
- 上下文长度；
- 结构化输出稳定性；
- 多模态能力；
- 延迟要求；
- 成本上限；
- 可用性与发布阶段（GA / preview）；
- 稳定性与回滚策略。

真正系统设计时，这些维度往往相互牵制。

第三，随着 OpenAI、Anthropic、Google 等平台都把模型家族做成大/中/小、reasoning/non-reasoning、多模态/轻量化分层，模型路由就成了自然选择。OpenAI 当前文档已经明确建议：复杂任务可以从旗舰模型开始，若追求延迟和成本则选择 mini/nano 级别；Anthropic 官方文档长期以 Opus / Sonnet / Haiku 的分层进行说明；Gemini 模型页面则频繁出现 preview、deprecated、shutdown 节

点，提醒开发者选型不仅看能力，也要看生命周期。

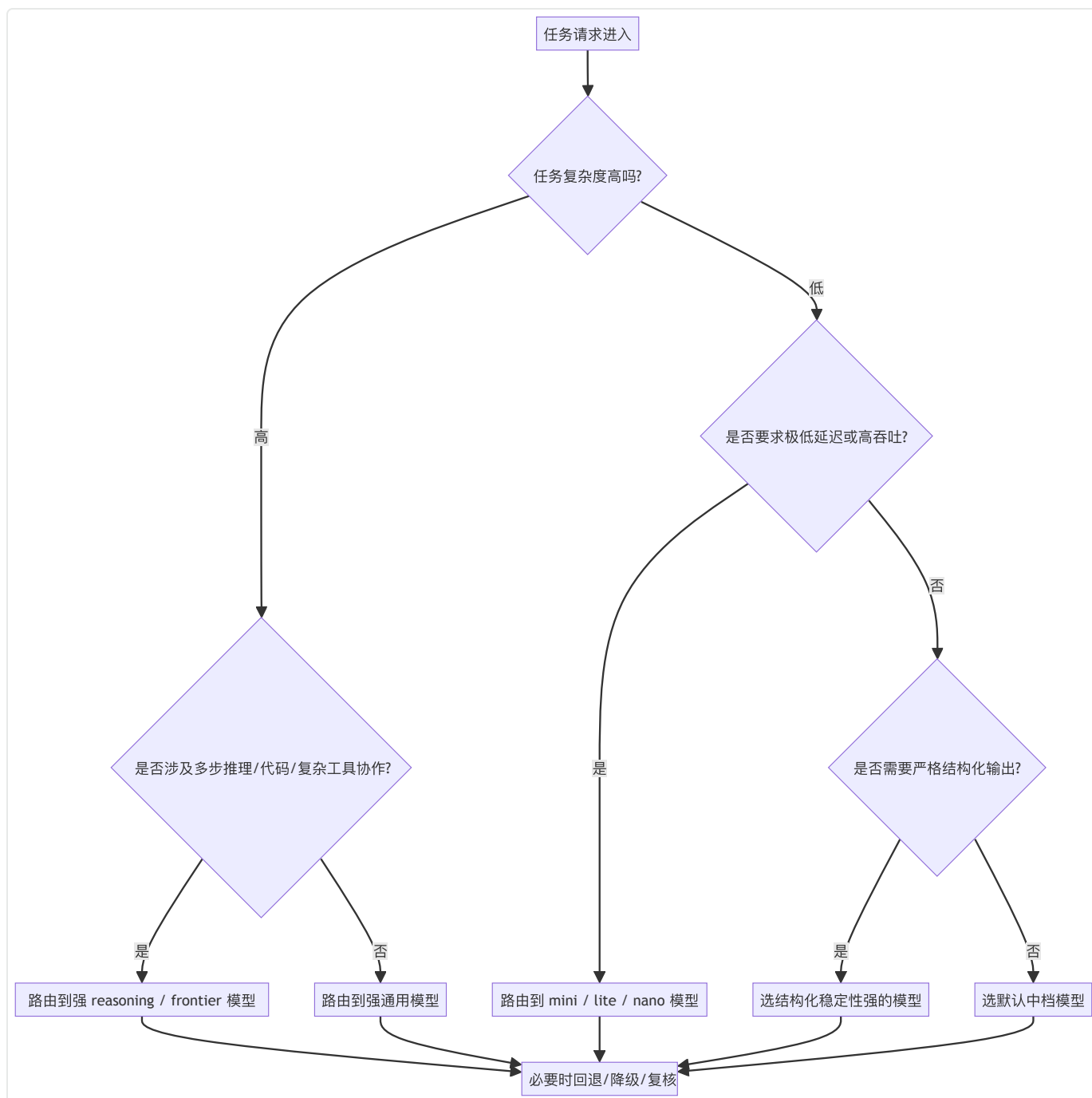
这就意味着：**选型是一个动态系统问题，不是一次拍板。**

第四，Agent 场景进一步放大了路由的价值。规划、检索、执行、校验、总结几个阶段不必用同一个模型。强模型做规划和裁决，小模型做并行子任务、抽取和格式整理，已经成为 2026 年很常见的工程思路。

这一章可以用这样的面试口语话术表述：

我们最初用单一大模型处理所有请求，效果还可以，但成本和延迟都偏高。后来把链路拆开做路由：复杂规划和最终审定仍交给强模型；文档抽取、证据整理、分类路由下沉到轻量模型；同时对系统 prompt 和工具 schema 做稳定前缀，提升缓存命中率。选型时我们不是看排行榜，而是根据任务完成率、格式合法率、平均延迟和单位请求成本做评测。最终整体质量基本持平，但成本显著下降，在线时延也更稳定。

二、先建立一个模型选型的基本框架



这个图最重要的意义，不是告诉你某个分支永远固定，而是提醒你：选型从来不是只看模型名，而是看任务维度。一个成熟系统往往从需求出发，而不是先爱上某家模型。

三、模型选型到底在看哪些维度

1. 任务类型

先问任务是什么。

- 问答/写作/总结：偏通用生成；

- 代码修复/多步规划：偏 reasoning；
- 分类/抽取/标签：偏结构化与低延迟；
- 实时语音/高频 Agent：偏低延迟；
- 图文理解/文档分析：偏多模态；
- 需要外部系统操作：偏工具使用可靠性。

如果任务分类都不清楚，后面谈选型基本没有意义。

2. 正确率要求

有些任务只要差不多能用，比如草稿生成；

有些任务必须尽量别错，比如合同审查摘要、报表分析、金融审批建议。

正确率要求越高，通常越值得上更强模型、更多验证链路或多模型复核。

3. 延迟要求

面向用户对话、实时语音、IDE 辅助、在线客服，延迟非常敏感。

如果业务要求首 token 极快，哪怕强模型质量更好，也可能不适合。

所以你在面试里应该明确区分离线批处理和在线交互，这是选型最基础的分层。

4. 成本与吞吐

同样 100 万次请求，选旗舰模型和选轻量模型，成本可能差一个数量级。

更重要的是，并发能力还会受上下文长度、KV Cache、缓存命中率和工具循环影响。

真实系统里经常是先用强模型打样，再用 eval 驱动看能否下沉到更便宜模型。

5. 上下文长度与上下文管理能力

长文档、多文件、多轮会话、大代码库、企业知识系统都会碰到上下文上限问题。

但你不能只看最大 context，还要看：

- 长上下文利用率；
- prompt caching/context caching 支持；
- compaction/support summary 能力；
- 多模态长上下文兼容性。

6. 工具与结构化能力

Agent 系统里，这一维经常比通用写作能力更重要。

你要看：

- function calling/tool use 是否成熟；
- structured outputs 是否严格；
- 多工具并行/程序化工具能力；
- 错误返回与 trace 可观测性；
- 是否支持 web/file/code/computer use 等原生能力。

7. 模型生命周期与发布阶段

这个点特别容易被忽略。

preview 模型可能能力很强，但：

- 行为变动快；
- 稳定性波动大；
- 参数建议变化快；
- 可能被较快弃用或替换。

Gemini 文档里的 deprecation/shutdown 提示是一个非常典型的提醒：

选型不只是看今天能不能跑，也要看未来三个月稳不稳。

四、截至 2026-04 的主流厂商分层认知，你应该怎么讲

1. OpenAI：从用哪个模型走向按任务选择 reasoning / GPT / mini / nano

截至 2026-04，OpenAI 模型文档已明确给出分层建议：复杂专业工作或复杂推理优先旗舰模型；追求延迟和成本时考虑 mini/nano；reasoning 模型与普通 GPT 模型在 prompting 和 usage 上有差异。模型页面还直接展示 context window、tool support、reasoning effort、价格层级等信息。

对面试来说，你不需要背所有型号，但最好知道一种说法：

OpenAI 家族已经把能力、速度、成本、工具支持显式产品化了，选型不能只看模型名，而要看家族定位。

2. Anthropic：Opus / Sonnet / Haiku 的工程分层非常清晰

Anthropic 长期用 Opus、Sonnet、Haiku 代表从最高能力到更高性价比的分层。到 2025—2026 年，Anthropic 文档又叠加了 structured outputs、prompt caching、extended/adaptive thinking、Agent

SDK、advanced tool use 等能力。

你在面试里可以这样概括：

- Opus：更适合极高复杂度与高质量要求；
- Sonnet：常作为默认生产主力，平衡质量、速度与成本；
- Haiku：适合高吞吐、低延迟和轻量任务。

如果再补一句Anthropic 近两年特别强调 long-running agents 的 harness/context/tool 设计，会更像跟进过最新生态。

3. Gemini：强多模态、长上下文、thinking 与预览节奏快

Gemini 家族到 2026 年的特点非常鲜明：

- 多模态与长上下文是强项；
- thinking 被当成正式能力暴露；
- 3.x / 2.5 等不同系列在温度建议、能力定位、预览状态上差异较大；
- 模型页面和 changelog 中常见 deprecation/shutdown 提示，说明选型要重视生命周期管理。

面试里如果你能提到Gemini 3 文档明确建议 temperature 默认 1.0，说明同样的调参经验不适用于所有家族，会体现你会看官方资料而不是只背旧经验。

五、为什么模型路由比选一个最好的模型更重要

1. 任务天然分层

一个典型 Agent 请求可能分为：

- 意图识别；
- 检索或搜索；
- 计划生成；
- 子任务执行；
- 结果总结；
- 结构化回填。

这些步骤对模型要求不同。

用一个最强模型串到底，常常既贵又慢，也未必最稳。

2. 路由可以做高低搭配

例如：

- 大模型做计划与最终审定；
- 小模型做文档抽取、标签整理、证据预处理；
- 超轻量模型做 spam/安全预检；
- 规则引擎先做确定性分流，模型只处理剩余复杂样本。

这种策略在 2026 年已经非常常见，尤其在代码 Agent、办公自动化、运营分析中。

3. 路由让你能用 eval 驱动降本

先用强模型建立 baseline，再用真实样本看看哪些任务可以安全下沉到更便宜模型，是非常成熟的策略。

如果你在面试里主动讲我会先建立质量基线，再评估哪些子任务能下放到 mini/haiku/flash-lite，这是很强的工程信号。

六、几种常见路由策略

策略 1：静态任务路由

按任务类型固定模型。例如：

- 分类 → 小模型；
- 长文分析 → 中大模型；
- 复杂代码 → reasoning/frontier。

优点是简单可控；缺点是同类任务内部复杂度差异可能被忽略。

策略 2：规则 + 阈值路由

根据输入长度、是否多文件、是否有工具、是否要求严格 JSON、是否涉及高风险动作等条件决定模型。

这是很多系统的第一代路由器，落地成本低。

策略 3：级联路由 (cascade)

先让便宜模型做，置信度不够或检测到复杂度过高时，再升级到更强模型。

它很适合抽取、分类、路由、评审这类可量化置信度的任务。

策略 4：多模型协同

一个模型负责规划，一个模型负责检索总结，一个模型负责最终裁决。
这适合复杂 Agent，但系统设计和观测成本也更高。

策略 5：回退与降级

当主模型超时、限流、preview 波动或某区域故障时，自动回退到兼容模型。
这个在高可用系统里非常重要，但很多校招同学完全不提。

七、模型选型的实践方法：面试里最好按这个顺序回答

步骤 1：定义任务与指标

先明确你要优化的是：

- 准确率；
- 格式合法率；
- 延迟；
- 成本；
- 工具成功率；
- 用户满意度；

没有指标，就没有选型。

步骤 2：用强模型建立上界

先用一个足够强的模型做 baseline，确定这个任务理论上能做多好。

步骤 3：做失败分类

错误是因为模型不够强、证据不足、prompt 不好、工具不稳，还是输出解析崩？
不同根因，对应不同选型决策。

步骤 4：尝试向下路由

把适合下沉的任务迁到更便宜更快的模型，观察质量损失是否可接受。
这是最实用的降本路径。

步骤 5：补缓存、压缩和工作流优化

很多时候不是非换模型不可，prompt caching、context engineering、structured outputs、tool schema 改善就能把模型表现拉上来。
选型不应该掩盖系统工程问题。

八、高频面试题与答题要点

问题 1：你怎么选模型？

先看任务类型与验收指标，再看复杂度、延迟、成本、上下文、工具/结构化支持，最后通过样本评测做决策。

问题 2：为什么不直接用最强模型？

因为不是所有请求都需要最高智能，最强模型往往更贵更慢；系统目标是综合最优而不是单点最强。

问题 3：什么时候应该上 reasoning 模型？

多步推理、复杂代码、复杂约束一致性、多轮工具协作、高风险分析场景更值得上 reasoning 或更强模型。

问题 4：什么时候适合小模型？

分类、抽取、路由、改写、模板填充、高频轻量 Agent 子任务，以及对延迟/成本极敏感的场景。

问题 5：模型路由是什么？

根据任务特征、复杂度、成本与 SLA，动态把请求分配给不同模型或模型组合的机制。

问题 6：preview 模型能不能直接上生产？

可以试，但要谨慎。要准备行为波动监控、回退策略、版本对比和 deprecation 预案。

问题 7：如何做降本？

先建立强模型 baseline，再通过 eval 发现可下沉任务，配合缓存、上下文压缩、结构化输出和更细粒度路由实现降本。

问题 8：长上下文模型就一定更适合长文档任务吗？

不一定。还要看长上下文利用率、缓存能力、证据组织方式和实际任务结构。

问题 9：Agent 系统为什么常用多模型？

因为规划、执行、抽取、总结等阶段需求不同，用单一模型串到底通常不经济。

问题 10：模型选型要不要考虑工具支持？

必须考虑。Agent 系统里，function calling、structured outputs、native tools、错误处理能力往往和通用写作能力同等重要。

十、常见误区

第一个误区，是把模型选型等同于挑一个最聪明的。

真实系统追求的是 Pareto 最优。

第二个误区，是忽视模型生命周期。

preview、deprecated、shutdown、region availability 都会影响生产可用性。

第三个误区，是只比较排行榜，不比较接口能力。

在 Agent 场景里，structured outputs、tool use、缓存、上下文管理能力可能比某个 benchmark 排名更关键。

第四个误区，是没有回退策略。

一旦主模型限流、波动或升级回归，没有 fallback 的系统很脆。

十一、截至 2026-04 的一个重要行业变化

2026 年一个非常明显的趋势是：

模型本身正在变成可编排资源，而不是单一核心依赖。

OpenAI 的最新模型体系把 reasoning、mini、nano、tool support、prompt caching、长上下文放在统一 API 体系里；Anthropic 从模型能力延伸到 Agent SDK、context engineering、advanced tool

use；Gemini 则在 models/changelog 中持续提醒开发者关注 preview 节奏与能力更新。
这意味着未来面试里，你会不会选模型其实越来越接近你会不会设计资源调度策略。

十二、你应该记住的结论

如果只记一句话：

模型选型的目标不是找到‘最好的模型’，而是为不同任务找到质量、延迟、成本和稳定性的最优组合。

再记一句：

路由不是可选优化，而是多模型时代的默认架构思维。

资料来源与延伸阅读

官方文档

1. OpenAI Models / Model Selection / Latest Model Guide
<https://developers.openai.com/api/docs/models>
<https://developers.openai.com/api/docs/guides/model-selection/>
<https://developers.openai.com/api/docs/guides/latest-model/>
2. OpenAI Reasoning Best Practices / Latency Optimization / Pricing
<https://developers.openai.com/api/docs/guides/reasoning-best-practices/>
<https://developers.openai.com/api/docs/guides/latency-optimization/>
<https://developers.openai.com/api/docs/pricing/>
3. Anthropic Models Overview / Release Notes / Pricing
<https://docs.anthropic.com/en/docs/about-claude/models>
<https://docs.anthropic.com/en/release-notes/overview>
<https://docs.anthropic.com/en/docs/about-claude/pricing>
4. Gemini Models / Changelog / Pricing / Gemini 3 Guide
<https://ai.google.dev/gemini-api/docs/models>
<https://ai.google.dev/gemini-api/docs/changelog>
<https://ai.google.dev/gemini-api/docs/pricing>
<https://ai.google.dev/gemini-api/docs/gemini-3>

补充阅读

5. OpenAI Cookbook: Model Selection Guide

https://developers.openai.com/cookbook/examples/partners/model_selection_guide/model_selection_guide/

6. Anthropic Engineering (advanced tool use / harness / context engineering)

<https://www.anthropic.com/engineering>

01 Agent定义、边界与适用场景

一、这一题在面试里到底考什么

什么是 Agent几乎是所有 Agent 面试的起手题，但它真正考察的绝不是百科式定义，而是候选人有没有建立清晰的边界意识。面试官通常借这个问题判断三件事：第一，你是否真的理解 agent 和 chatbot、workflow、tool-calling app 的区别；第二，你是否知道 agent 什么时候有必要、什么时候没有必要；第三，你是否具备将看起来很智能的概念落到可上线、可治理、可控风险的工程能力。

因为近两年大家都在讲智能体，很多简历也会写做过 agent，但实际做的可能只是：给模型挂了两个工具，用 function calling 调一下搜索，然后按固定顺序执行。这样的系统不一定错，很多时候还更实用，但它未必就是一个典型意义上的 agent。面试官之所以要追问定义，不是为了抠字眼，而是为了确认：你有没有把自治多步决策基于观察更新后续动作面向目标而非单次回答这些关键特征理解到位。

从校招生视角看，最容易犯的错误有两个。**第一个错误是把任何用了大模型和工具的系统都叫 agent，导致边界完全失焦。第二个错误是把 agent 神化，认为 agent 比 workflow 更高级、更智能、一定更值得做。**实际上，很多官方实践都反复强调：当任务流程清晰、约束强、结果要求稳定时，workflow 往往更合适；agent 只有在路径不确定、需要动态规划或环境交互时，才真正发挥价值。这意味着，面试官心里理想的回答，不是agent 是一种很强的 AI 系统，而是agent 是在一定自治程度下围绕目标进行多步决策与动作执行的系统，但是否值得引入，要看任务的不确定性和工程约束。

如果面试官问什么是 Agent？，可以这样答：

Agent 本质上是一个围绕目标运行的系统，而不只是一次输入输出的模型调用。它通常会在多步执行中基于当前观察结果动态决定下一步动作，并通过工具或外部环境持续推进任务完成。和普通 chatbot 相比，Agent 更强调目标驱动、多步决策、环境交互和状态持续；和 workflow 相比，Agent 的关键差异在于运行时有更高的决策自由度，而不是所有步骤都预先写死。

在工程上，我会把 Agent 看成由目标、策略/规划、工具、状态/记忆、控制约束和校验机制组成。是否应该用 Agent，要看任务路径是否不确定、是否需要多轮环境交互，以及引入自治后能否接受成本、延迟和风险。

二、什么是 Agent：从会回答到会完成任务

先给一个足够工程化的定义：**Agent 是一种围绕任务目标运行、能够在执行过程中根据中间观察结果决定后续动作，并借助工具或外部环境持续推进任务完成的系统。**

这个定义里有四个关键词，面试时最好展开：

1. 围绕目标运行

普通聊天模型更像给定输入，生成输出；agent 更像给定目标，持续推进直到结束。

比如把这份周报润色一下更接近一次性生成任务；而帮我收集竞品价格、汇总对比并发到指定群里就更接近 agent 任务，因为它天然包含多个步骤和中间状态。

2. 在执行过程中做决策

Agent 不是只在开始时被动生成一段答案，而是在每一步都可能根据环境反馈改变路线。

例如它先搜索资料，再发现资料不够，就继续检索；发现一个 API 返回错误，就换另一种查询方式；在工具结果与预期不符时，可能回退、重试或重新规划。

3. 借助外部能力推进任务

这通常表现为工具调用，但本质是 agent 不只依赖参数里的知识，还会主动使用模型外的能力：检索、数据库、网页、代码解释器、文件系统、企业系统 API、浏览器、手机、桌面环境等。

这也是为什么 agent 与纯对话系统的工程重心完全不同：它不只是生成文字，而是要和现实世界产生交互。

4. 持续运行直到终止条件满足

Agent 往往不是一问一答结束，而是存在一个运行循环：思考、选择动作、执行、观察、更新状态、再决策，直到任务完成、失败、超时、预算耗尽或被人工中断。

所以一旦系统进入 agent 范式，状态管理、终止条件、审计与回放能力就变得非常关键。

三、什么不算 Agent：边界比定义更重要

很多候选人会在这一题上吃亏，不是因为不会定义，而是因为不会说什么不算。下面给出面试中最常见的边界问题。

1. 普通 chatbot 不一定是 agent

如果系统只是把用户问题喂给模型，然后直接返回一段答案，没有多步决策，没有工具使用，也没有目标推进循环，那它通常只是 LLM application，不是典型 agent。

当然，如果这个 chatbot 内部会自动检索、调用工具、判断是否继续执行、持续处理上下文，那它就开始具有 agent 特征了。也就是说，界面像聊天不决定是不是 agent，关键看内部运行机制。

2. 固定流程的 workflow 不一定是 agent

假设有一个程序：先做意图分类，再检索文档，再让模型总结，再输出答案。每一步都是预定义的，没有动态决定下一步的能力，那么它更像 workflow。

很多面试官喜欢问：Workflow 算不算 agent？比较稳妥的答法是：

- 广义上，有些人会把带一点动态性的工作流也叫 agentic workflow；
 - 但严格区分时，workflow 侧重预先定义路径，agent 侧重运行时自主决策。
- 这种回答既承认现实里术语会混用，又保留了工程上的清晰边界。

3. 只有 function calling 不足以自动成为 agent

如果系统只是让模型根据 schema 选择一个函数，然后程序执行后直接结束，这更像 tool-augmented LLM。

只有当它具备多步循环、根据工具结果持续决策的能力，才更接近典型 agent。

换句话说：工具是 agent 的常见组成，不是充要条件。

4. 有记忆不等于 agent

有些应用会保存用户画像、会话历史、偏好设置，但如果没有自主行动和多步执行，仍然不一定是 agent。

Memory 解决的是状态延续问题，agent 解决的是目标推进问题。两者经常同时出现，但不是一回事。

四、为什么 Agent 会火：它到底解决了什么问题

理解 agent 的价值，不能只说更智能。更准确的说法是：它解决的是固定流程难以覆盖的开放式任务执行问题。

场景一：任务路径事先不确定

例如帮我调研最近一周行业新闻并写成分析，到底先查哪几个网站、要不要改查询词、哪些信息要交叉验证，这些步骤很难完全提前写死。

Agent 的价值在于让模型在运行过程中适配具体情况。

场景二：任务需要多轮环境交互

比如电脑操作、浏览网页、调用多个系统 API、管理文件、运行代码、读取执行结果、基于结果继续操作。

这种场景不是靠一次文本生成能完成的，而是需要动作—观察—再动作的闭环。

场景三：任务规模超过单轮上下文

例如复杂研究、长文档生成、跨多个数据源汇总、长期追踪式任务。

此时需要规划、拆解、记忆、阶段性产物存储，agent 由此变得更合适。

场景四：任务要求一定程度的自动化闭环

当系统需要从给建议升级为帮你完成，agent 的意义就上来了。

比如客服知识问答可能只需要 RAG；但如果要识别退款诉求—调订单系统—核验规则—发起审批—通知用户，那就明显更接近 agent。

五、Agent 的核心构成：面试一定要会讲的最小框架

真正回答什么是 agent，最稳妥的方法不是背论文，而是给出一个最小系统框架。一个生产可理解的 agent，通常至少包括下面这些部分：

1. Goal / Task

系统到底要完成什么目标。

目标可以来自用户，也可以来自上层程序分配。目标需要尽可能明确边界，例如成功标准、限制条件、预算、时限、权限范围。

2. Policy / Planner

由模型或规则决定下一步做什么。

有时是即时反应式决策（如 ReAct），有时是先生成计划再执行（如 Plan-and-Execute），有时还会混入程序化策略。

3. Tools / Environment

外部能力与外部世界接口。

包括搜索、数据库、代码执行、浏览器、办公系统、业务系统 API、设备控制等。没有外部环境交互，很多 agent 价值发挥不出来。

4. State / Memory

运行时状态、历史观测、阶段结果、用户偏好、任务上下文。

这部分决定 agent 是否能在多步执行中保持一致性，也决定是否支持长任务恢复。

5. Guardrails / Governance

权限、审批、参数校验、风险控制、日志审计、终止条件。

这是校园面试里最容易漏掉，但工程上最关键的部分。真正上线的 agent，不可能只靠模型自己判断。

6. Evaluator / Verifier

结果校验器或中间步骤检查机制。

例如检查是否回答完整、是否满足格式、是否存在自相矛盾、是否命中了业务规则。

很多 agent 失败，不是因为不会生成，而是因为没人验证。

把这六个部件讲清楚，面试官基本就能判断你是不是理解了 agent 的系统本质。

六、Agent 的自治程度：不要把全自动当成唯一答案

另一个高频追问是：Agent 一定要完全自治吗？

标准回答是：不一定。自治是连续谱，不是二元开关。

可以按自治程度粗略分成四类：

1. Assisted Agent

以人类为主，模型提供建议，工具执行仍然依赖人触发。

比如 IDE 里的代码建议、审批流中的推荐决策。

2. Semi-Autonomous Agent

大部分步骤可自动执行，但关键节点需要人工确认。

比如可自动检索和草拟邮件，但发送前需要审批；可自动起草工单处理方案，但涉及退款或删库操作必

须人工确认。

3. Autonomous Agent

在给定权限边界内可独立完成较长链路任务。

例如定期市场情报收集、自动化测试执行、基础级别的报表整理与推送。

4. Open-Ended Agent

目标开放、环境复杂、时间跨度长，可能长期运行。

这是研究和前沿系统更关注的方向，但在企业场景里通常最难治理。

面试里很加分的一句话是：自治程度越高，对权限、状态、审批、终止条件和评测体系的要求越高。

这样回答能体现你不是在崇拜自动化，而是在做系统权衡。

七、Agent 适合什么场景，不适合什么场景

适合的场景

1. 路径不固定、需要探索

例如复杂研究、网页操作、多源信息搜集、跨系统流程执行。

这类任务很难穷举所有路径，用 agent 能减少硬编码。

2. 明显依赖中间观察结果

工具返回什么，直接影响下一步怎么做。

比如网页某个按钮不存在、API 某个字段缺失、数据库返回空结果，都需要动态调整。

3. 任务可以被拆成阶段，但阶段之间仍需动态协调

比如先搜集信息，再筛选，再归纳，再输出报告，每个阶段都可能因中间结果而调整。

4. 任务价值高于控制成本

如果为了自动化一个小任务，引入 agent 会带来大量复杂性，那不值得。

Agent 更适合那些单次成功收益较高、人工成本较高、流程变体很多的任务。

不适合的场景

1. 路径清晰且稳定

例如固定报表生成、标准审批流、简单表单填写。

这些任务用 workflow 更简单、更稳、更便宜。

2. 错误代价极高但又难以完全约束

例如高风险金融交易、关键生产控制、无保护措施破坏性操作。

不是说绝对不能做，而是要极其谨慎，通常必须加强规则、审批与沙箱。

3. 数据与接口不稳定、但系统又不能容错

Agent 高度依赖工具和环境反馈，如果底层系统本身非常不稳定，agent 只会把问题放大。

4. 只是为了看起来高级

这是面试里经常会遇到的陷阱。如果一个需求明明两段规则 + 一次检索就能解决，却硬做成多 agent，那会显得你缺乏成本意识。

九、常见误区与纠偏

误区一：用了工具就是 agent

纠偏：工具增强型应用和 agent 的区别在于是否存在目标推进型运行循环，以及是否根据观察持续决策。

误区二：agent 一定比 workflow 高级

纠偏：工程上没有高级一说，只有是否合适。很多场景 workflow 更优。

误区三：agent 就是把 prompt 写复杂一点

纠偏：真正的 agent 设计重点不只在 prompt，而在状态、工具、约束、监控与恢复机制。

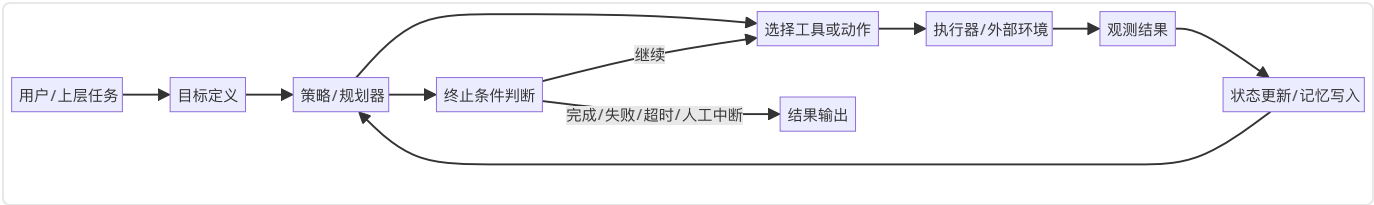
误区四：只要模型够强，就不需要系统设计

纠偏：模型再强，也无法替代权限管理、幂等控制、审批机制、日志审计和终止条件。

误区五：agent 的核心是思维链

纠偏：推理过程只是其中一部分。很多真实系统的关键瓶颈反而在工具可靠性、上下文管理和任务边界定义。

十、Agent 的最小运行闭环



十一、高频面试题与参考回答

题 1：什么是 Agent？和普通 LLM 应用有什么不同？

参考回答：

Agent 是围绕目标进行多步决策和动作执行的系统，不是单次文本生成。普通 LLM 应用更像输入问题，输出答案；Agent 更像给定目标，自动选择步骤、调用工具、根据观察继续执行，直到终止。

题 2：带 function calling 的聊天机器人算不算 agent？

参考回答：

不一定。如果只是一次性选择一个工具然后结束，更像 tool-augmented chatbot；如果系统会根据工具结果持续决策、多步推进任务，那就更接近 agent。

题 3：Workflow 和 Agent 的区别是什么？

参考回答：

Workflow 的路径更多是预定义的，强调稳定、可控、可回放；Agent 的路径在运行时动态决定，适合任务不确定性更高的场景。

题 4：Agent 一定需要工具吗？

参考回答：

从广义上说，不一定；但从高价值的工程应用看，几乎都需要外部能力。没有工具，agent 很难突破模型参数内知识的限制，也很难真正完成任务。

题 5：什么时候你不会建议上 Agent？

参考回答：

任务路径清晰、规则强、结果必须稳定可控、错误代价高且可通过规则系统解决时，我更倾向于 workflow 或纯程序逻辑。

题 6：为什么大家总说 agent 难做？

参考回答：

难点不在于让模型想一步，而在于如何在复杂环境里持续正确做事。包括工具可靠性、上下文污染、权限控制、长任务恢复、终止条件、评测与回放等。

题 7：Agent 的核心组件有哪些？

参考回答：

目标、策略/规划、工具、状态/记忆、控制约束、结果校验。这六块缺一不可，尤其在生产环境里。

题 8：Agent 和 RAG 的关系是什么？

参考回答：

RAG 是一种知识获取机制，解决从外部知识源取信息；Agent 是任务执行范式，解决如何基于目标多步推进。RAG 可以是 agent 的一个工具或子模块。

题 9：Agent 和多 agent 是一回事吗？

参考回答：

不是。多 agent 只是 agent 系统的一种组织方式。很多场景单 agent 足够，而且更稳。多 agent 只有在上下文隔离、角色专门化、并行化或团队分工价值明显时才值得上。

题 10：怎样判断一个需求是否应该 agent 化？

参考回答：

看三点：任务路径是否难以预先编码；是否需要多轮环境交互；自动化收益是否足以覆盖引入自治后的

复杂度和风险。

十二、面试速记版

- Agent 的关键不是能聊天，而是能围绕目标持续行动。
- Agent 的关键特征：目标驱动、多步决策、环境交互、状态持续。
- 工具不是充分条件；循环式决策才是核心。
- Workflow 强在稳定和可控，Agent 强在处理不确定路径。
- 真正的 Agent 设计必须补齐权限、审批、日志、恢复、终止条件。
- 不是所有任务都值得 agent 化，过度 agent 化是常见反模式。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》, 2024。适合用来界定 agent 与 workflow 的边界，强调能用简单工作流解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》, 2025。非常适合补充 agent 不只是 prompt engineering, 而是 context engineering 的近年观点。
3. Anthropic, 《How we built our multi-agent research system》, 2025。适合放在多 agent、任务拆解、subagent 协作章节中。
4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。
6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。
8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。
11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。

15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。

02 Workflow vs Agent

一、两者的比较体现了目前AI的热点发展

近两年的官方最佳实践里，一个被反复强调的观点是：**不要为了更智能而盲目把系统做成 agent**。这句话听起来像经验之谈，实际上是非常核心的系统设计原则。面试官爱问Workflow 和 Agent 有什么区别，什么时候用 workflow，什么时候用 agent，**本质上是在看你有没有工程判断力**。对校招生来说，这类问题尤其重要，因为它很能区分看过很多热词和具备系统取舍意识的候选人。

在大模型落地早期，很多团队的直觉是：既然模型变强了，就应该让它自由决定更多步骤。**但真实工程里，自由度越高，复杂度和风险往往也越高**。你会遇到结果不稳定、工具误用、不可回放、难调试、成本飙升、终止失控等一系列问题。于是行业很快形成共识：**先把确定的部分固化成 workflow，只把真正不确定、需要运行时决策的部分留给 agent**。也就是说，workflow 和 agent 不是低级 vs 高级的关系，而是确定性编排 vs 自主性决策的取舍关系。

这道题通常会有三层追问。第一层是概念层：两者分别是什么。第二层是设计层：为什么有时 workflow 更优。第三层是工程层：如何把一个系统拆成 workflow + agent 的混合结构。真正优秀的回答，应该能从任务特征、系统约束、评测方式、治理难度、可观测性、成本与延迟多个维度展开。

比如对于一个企业知识助手，在workflow和agent的取舍时，你可以这么回答

我不会把它简单做成纯 workflow 或纯 agent，而是会拆层。对于鉴权、文档权限过滤、路由、日志、配额、结果落库、审批这些确定性步骤，我会用 workflow 固化下来；对于开放式检索、跨文档对比、后续追问澄清、是否需要继续搜索等部分，我会用受约束的 agent。

如果业务只是 FAQ、制度问答、标准文档查询，那么 workflow + RAG 就足够；只有当需求升级为帮我跨系统搜集材料、整理观点、生成多阶段输出时，agent 的价值才会显著。

不要让 architecture 选择变成宗教问题，而是变成边界划分问题。

二、先给定义：Workflow 是什么，Agent 是什么

1. Workflow

Workflow 可以理解为：**系统执行路径主要由开发者预先定义的流程**。

它的特点是每个节点做什么、节点之间如何转移、在什么条件下走哪条分支，大多是程序提前写好的。

模型在其中可以承担某个步骤，例如抽取、分类、总结、改写，但整体控制权主要在程序手里。

典型例子包括：

- 先做意图分类，再走检索，再生成回答；
- 先 OCR，再表格解析，再字段校验，再入库；
- 客服问答中先识别是否需要人工，再选择 FAQ 检索或工单创建。

这些都可能用到大模型，但核心仍是 workflow。

2. Agent

Agent 可以理解为：系统在运行时拥有更高的决策自由度，可以根据中间观察结果动态决定下一步动作。

在 agent 系统中，程序提供目标、工具、约束、状态和运行边界，而接下来做什么更多地由模型或模型+规则共同决定。

比如面对帮我比较竞品定价并整理成报告，系统可能自主决定先搜索几个来源、提取什么字段、是否需要二次验证、是否调用代码工具做统计，以及在信息不足时是否继续探索。

3. 最关键的区别

面试时一定要把这句话说出来：

Workflow 的核心是路径基本预定义，Agent 的核心是路径在运行时动态形成。

这不是说 workflow 完全没有条件分支，也不是说 agent 没有任何规则约束，而是说：

- workflow 的主要价值来自确定性编排；
- agent 的主要价值来自面对不确定环境时的适应性。

三、为什么很多时候 Workflow 更好

很多候选人会在这里犯一个常见错误：一上来就说 agent 更灵活、更强大。没错，但面试官更想听的是：为什么 workflow 常常是更好的默认选项。

1. 更可预测

Workflow 最大的优势是稳定。

因为路径和节点事先定义好了，你更容易知道一个请求经过哪些步骤、调用哪些服务、在哪些条件下终止。这对线上系统尤其重要，因为稳定性和可预期性本身就是产品价值的一部分。

2. 更容易评测与回放

当流程固定时，你更容易为每个节点设计离线测试集、验收标准和回归测试。

例如检索召回率、抽取准确率、格式合规率，都能拆开测。

Agent 则容易出现明明任务完成了，但路径每次都不一样，这会让 debug 和回放变得困难。

3. 成本和延迟更可控

Workflow 通常只调用固定次数的模型和工具。

Agent 往往会有不确定的循环次数、额外思考 token、反复试错和更多上下文读写，所以成本和延迟上界更高，也更难估。

4. 更易治理

权限边界、审计、审批、风控，在 workflow 里更容易做。

因为每个动作和转移点都比较明确，你可以决定某些节点必须人工审批，某些动作只能由程序触发，某些接口必须经过白名单。

而 agent 如果工具开放太多、决策太自由，治理难度会显著上升。

5. 更容易让系统先上线

很多业务问题不需要很高的自主性。

如果可以先用 workflow 把 80% 常见场景解决，往往是更务实的路径。后续再把真正不确定的 20% 场景 agent 化，会更稳。

四、什么时候 Agent 才真正有价值

如果 workflow 已经这么好，为什么还需要 agent？答案是：**有些任务的路径没法被开发者提前穷举。**

1. 任务路径依赖环境反馈

例如浏览网页找信息。你不知道页面结构是不是变化了，也不知道第一轮搜索是否足够。

Agent 的强项是根据观察结果调整下一步。

2. 任务需要探索

比如开放式研究、复杂文件理解、多源对照、电脑操作、跨系统流程。
这些任务里接下来怎么做本身就是求解的一部分。

3. 任务步骤很多，且中间结果会改变计划

例如先收集候选方案，再筛选，再深入分析，再补齐数据，再输出结论。
如果每个阶段都可能推翻前面的假设，用固定 workflow 会变得又长又脆。

4. 任务规模和复杂性超过单轮输出

长上下文、分阶段交付、过程性校验、阶段性存储、阶段性纠错，这些都更适合 agent 系统。

五、一个高频面试点：并不是二选一，而是混合架构

真正的生产系统里，workflow 和 agent 往往不是对立关系，而是组合关系。
面试时如果你只说这个需求用 workflow 或者这个需求用 agent，答案通常不够立体。更好的说法是：**系统整体往往由 deterministic workflow 包住若干 agentic 子任务。**

举个常见的企业应用例子：员工发起帮我整理竞品分析报告并发给老板。
一个合理的系统可能这样设计：

1. 外层是 workflow

- 鉴权
 - 任务登记
 - 配额检查
 - 创建 trace 和任务 ID
 - 根据任务类型路由到研究子流程
 - 最终发送前做人类审批
- 这些部分最好都固定、可控、可审计。

2. 中间的研究子流程用 agent

- 自主决定搜索关键词
 - 阅读多个来源
 - 判断信息是否充分
 - 必要时重规划
 - 生成中间笔记与草稿
- 这一段需要更高的灵活性。

3. 最后收口仍回到 workflow

- 校验格式
- 落库存档
- 调用邮件系统
- 记录结果与耗时
- 通知发起人

这类回答很加分，因为它显示你理解自治应被包裹在确定性边界内。

六、从系统设计角度看，如何判断该选哪一种

面试里很实用的一种答法是给出比较维度。下面这些维度几乎都能用。

1. 任务路径可否预先编码

这是第一性问题。能预先编码，就优先 workflow；不能，才考虑 agent。

2. 结果容忍度

如果错误代价高、过程需要严格可控，workflow 通常更优。

如果允许一定探索和试错，agent 更有空间。

3. 环境不确定性

网页结构多变、工具返回复杂、数据源质量参差不齐时，agent 的适应能力更重要。

4. 可解释性与审计要求

越需要为什么这样做、走了哪一步、在哪一步失败，越倾向 workflow 或受约束 agent。

5. 成本与延迟预算

如果 SLA 很紧、预算很低，agent 需要谨慎，因为它的调用次数和上下文长度更不确定。

6. 迭代阶段

初期验证场景时，先 workflow 往往更容易看到可控收益。

当 workflow 的硬编码规则越来越复杂、维护成本过高时，再引入 agent 更合适。

七、Workflow 和 Agent 的典型架构对照

方案一：纯 Workflow

特点：固定节点，模型只是某些步骤的子能力。

适合：表单处理、意图路由、固定知识问答、结构化抽取、简单审批流。

优点：稳定、易测、成本可控。

缺点：面对未知情况容易僵住。

方案二：纯 Agent

特点：模型主导大部分下一步决策，程序只给工具、状态和边界。

适合：开放式研究、电脑操作、复杂调试、跨系统探索。

优点：灵活，覆盖长尾场景能力强。

缺点：不稳定、难治理、难预估成本。

方案三：Workflow 包裹 Agent

特点：外层确定、内层自主。

适合：企业落地中最常见。

优点：既保留灵活性，又保住治理与审计。

缺点：设计边界时需要经验，否则会出现两头都不纯的复杂系统。

方案四：Agent 生成 Workflow

特点：先由 agent 规划，再把计划编译成可执行 DAG 或任务图，由 workflow runtime 负责执行。

适合：复杂任务拆解、并行执行、长期任务。

优点：有机会兼顾规划灵活性与执行确定性。

缺点：系统复杂度高，对状态和任务表示要求高。

八、面试高频追问：为什么先用简单方案是正确原则

很多官方实践都强调先从简单可控的东西开始。这背后的逻辑非常朴素：

复杂度本身就是成本。

一个典型问题是：如果一个任务你既可以用 4 个固定步骤的 workflow，也可以上一个通用 agent，哪个更好？

标准答案通常不是看团队喜好，而是：

1. 先看任务是否真的需要运行时决策自由度；
2. 如果不需要，自由度只会增加不稳定性；
3. 在相同业务价值下，复杂度更低的系统更容易上线、评测、调试、回归和治理；
4. 只有当 workflow 的规则系统已经开始显著膨胀、难以维护，或者长尾场景太多时，agent 才显示优势。

这其实就是工程上的奥卡姆剃刀：**能写死的先写死，不能写死的再交给模型。**

九、常见反模式

反模式一：过度 agent 化

明明是一个稳定流程，却硬让模型决定每一步。

结果是成本更高、错误更多、回放更难。

反模式二：表面 workflow，实质乱跳

有些系统名义上叫 workflow，但中间塞了多个让模型自由决定接下来做什么的节点，导致流程图看似稳定，实际不可控。

这种设计最难调试，因为你以为自己做的是工作流，实际上已经进入半 agent 状态。

反模式三：agent 没有外部约束

自由度给了，但没有权限、审批、预算、停止条件。

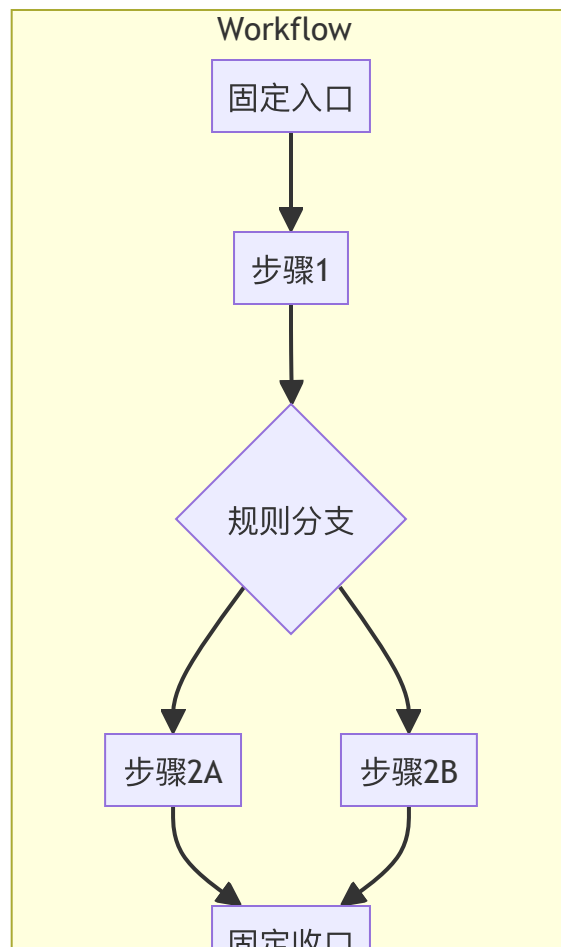
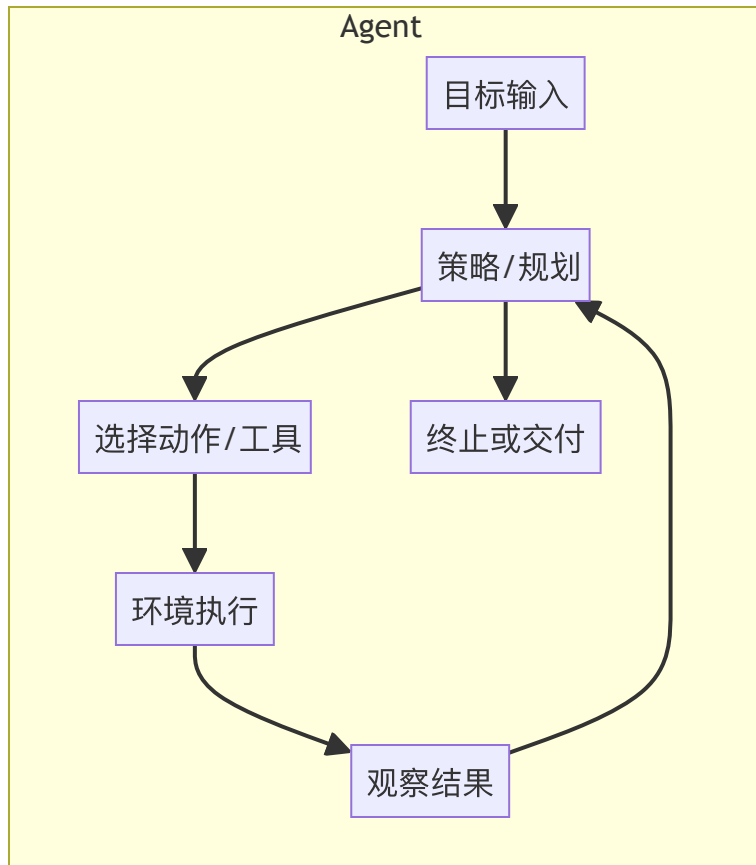
这会让系统在 demo 时显得很聪明，上线后则非常危险。

反模式四：把异常兜底也交给 agent

例如接口失败、权限不足、超时重试等问题完全交给模型决定。

好的系统会把基础设施问题尽量留给程序层解决，而不是让模型猜。

十、Workflow 与 Agent 的控制权差异



十二、高频面试题与参考回答

题 1: Workflow 和 Agent 最核心的区别是什么？

参考回答：

Workflow 的主要控制逻辑由开发者预先定义，路径基本可预测；Agent 的主要路径在运行时由模型根据环境反馈动态形成。

题 2: 为什么很多官方建议先从 workflow 开始？

参考回答：

因为 workflow 更稳定、易测、可控、可回放、成本更可预测。只有在任务不确定性高到固定流程难以覆盖时，agent 的额外复杂度才值得引入。

题 3: 是不是所有调用多个工具的系统都算 agent？

参考回答：

不是。如果工具调用顺序和条件都是程序写死的，那仍然更像 workflow。关键看运行时是否存在多步自主决策。

题 4: 一个系统可以同时有 workflow 和 agent 吗？

参考回答：

可以，而且这是非常常见、也很推荐的做法。外层 workflow 负责治理与流程稳定，内层 agent 负责处理开放式、不确定性高的子任务。

题 5: 什么时候 workflow 会开始不够用？

参考回答：

当规则越来越多、分支爆炸、长尾场景无法穷举、环境反馈强烈影响路径时，workflow 会变得臃肿和脆弱，此时 agent 的适应性更有价值。

题 6: 什么时候 agent 反而不如 workflow？

参考回答：

当任务步骤简单稳定、错误代价高、需要严格审计、预算和时延敏感时，agent 的不确定性通常得不偿失。

题 7：如何把一个 workflow 改造成更 agentic 的系统？

参考回答：

不是整体重写，而是识别那些路径无法预编码的环节，把这些环节抽成 agent 节点，并在外层保留 workflow 的边界控制。

题 8：Workflow 和 Agent 在评测上有什么区别？

参考回答：

Workflow 更适合分节点离线评测和回归测试；Agent 更需要过程级 trace、成功率、平均步数、工具错误率、人工接管率等综合指标。

题 9：设计题里怎么体现自己有取舍意识？

参考回答：

先说明需求中哪些部分确定、哪些部分不确定；再说明为什么确定的部分应该程序化，为什么不确定的部分才交给 agent；最后补上权限、审批、状态和终止条件。

题 10：一句话总结你的选择原则？

参考回答：

能写死的先写死，不能写死的再交给 agent；把自治限制在真正需要它的地方。

十三、面试速记版

- Workflow：预定义路径，强调稳定与可控。
- Agent：运行时动态决策，强调适应性与目标推进。
- 不是对立，而是常常混合。
- 默认先 workflow，再在必要处引入 agent。
- 控制权越放给模型，治理成本越高。
- 设计题里最加分的是边界划分，不是热词堆砌。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》, 2024。适合用来界定 agent 与 workflow 的边界, 强调能用简单工作流解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》, 2025。非常适合补充 agent 不只是 prompt engineering, 而是 context engineering 的近年观点。
3. Anthropic, 《How we built our multi-agent research system》, 2025。适合放在多 agent、任务拆解、subagent 协作章节中。
4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。
6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。
8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。
11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。
15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。

03 ReAct、Plan-and-Execute、CodeAct（重要）

一、这三者的概念与区别比较就是Agent面试的经典八股

如果说什么是 Agent考的是边界意识，那么 ReAct、Plan-and-Execute、CodeAct 有什么区别考的就是你对 agent 运行范式的理解深度。这类问题已经成为大模型应用开发面试里的标准题型，因为它直接触达一个核心问题：**当系统需要多步完成任务时，到底应该怎样组织思考—行动—观察—再思考的过程。**

很多候选人知道 ReAct，也听过 Plan-and-Execute，简历里甚至会提 CodeAct，但真正被问到工程取舍时就容易混乱。面试官通常不会满足于你只说ReAct 是边想边做，Plan-and-Execute 是先计划再执行，CodeAct 是用代码动作。他们往往会继续追问：

- ReAct 为什么简单但容易陷入局部最优？
- 先规划是不是一定更好？
- 计划过时了怎么办？
- 为什么代码执行在某些任务上会比自然语言动作更稳？
- 这三种模式能不能混合？
- 如果工具有副作用，比如发邮件、改数据库，哪种模式更安全？

所以，这一题真正考的是：你能否从运行控制结构的角度理解 agent，而不是只会背论文名字。

二、先把 ReAct 讲明白：最经典也最容易上手

1. ReAct 的核心思想

ReAct 可以概括为：**在每一步交替进行 Reasoning（推理）和 Acting（行动），并利用行动后的 Observation（观察）继续推进任务。**

它的经典形式就是：

1. 思考当前情况；
2. 决定下一步动作；
3. 执行动作；
4. 获得观察结果；

5. 基于新结果再次思考。

这比先想完整个答案再输出更适合工具使用，因为模型不必一次性把所有步骤都想清楚，而是可以边做边调整。

2. ReAct 为什么有效

因为很多真实任务不是静态问答，而是互动求解。

比如网页检索、数据库查询、代码调试、电脑操作，都需要先做一点，再看结果，再继续。

ReAct 的优势就在于它把模型从单轮生成器变成了交互式求解器。

3. ReAct 的优点

第一，简单直接，上手快。

第二，非常适合工具调用链路，容易集成到函数调用和工具执行器中。

第三，天然支持在每一步利用新观察修正方向。

第四，过程可 trace，便于调试 agent 做了哪些决策。

4. ReAct 的缺点

ReAct 最大的问题不是不会做，而是走一步看一步。

这会带来几个典型风险：

- 容易局部贪心：模型可能只根据当前观察做短视决策，而没有全局规划；
- 容易重复：如果没设计好终止条件，可能在两个工具之间来回跳；
- 上下文膨胀：每一步的思考、动作、观察都写进上下文，长任务会越来越重；
- 难以并行：它的默认节奏是串行循环，不适合天然可并行的任务。

5. ReAct 适合什么场景

- 问题中等复杂，但需要多轮环境交互；
- 不确定性来自环境反馈，而不是超长计划依赖；
- 需要快速构建一个可运行 agent 原型；
- 希望保留清晰的过程 trace。

三、Plan-and-Execute：先规划，再执行

1. 为什么需要先规划

ReAct 的问题在于每一步都只看眼前，所以当任务更长、更复杂时，很多研究和工程实践会加入 planning。Plan-and-Execute 的核心思想是：**先生成一个相对全局的计划，再按计划逐步执行，必要时再重规划。**

它常见的运行方式是：

1. Planner 根据目标生成步骤计划；
2. Executor 按步骤执行；
3. 每一步把结果反馈回来；
4. 如果执行受阻或计划失真，再触发 replanning。

2. 它解决了 ReAct 的什么问题

它试图解决的是长任务中的全局性缺失。

有了计划后，系统不容易每一步都只做局部最优选择，而是更容易对齐最终目标、识别依赖关系、减少无意义探索。

3. 它的优点

- 更有全局视角，尤其适合长链任务；
- 容易做任务拆解和职责分离；
- 可以为后续并行化提供基础，因为计划能暴露步骤依赖关系；
- 便于插入检查点、审批点和阶段性产物管理。

4. 它的缺点

- 计划可能从一开始就是错的，或者很快过时；
- 规划本身也要消耗 token 和延迟；
- 如果执行器死板地按照计划跑，系统会变得不灵活；
- 计划表达不好时，执行阶段反而会困惑。

5. 工程上常见的改造

真实系统很少做只计划一次然后死执行。更常见的是：

- 初始规划 + 分阶段重规划；

- 计划只是软约束，不是硬约束；
- 计划和执行之间有 verifier 检查；
- 关键路径固定，细节步骤 agent 自主化。

6. 适用场景

- 长链任务；
- 存在明显的子任务依赖关系；
- 需要阶段性交付；
- 希望在执行前先暴露总体路线以便审核或并行。

四、CodeAct：为什么代码动作会更强

1. CodeAct 的核心思想

CodeAct 关注的是：让模型通过生成可执行代码的方式表达动作，而不是只用自然语言描述下一步。它的直觉很强：对很多复杂任务，代码比自然语言更适合表示过程、控制流、变量引用、循环、条件判断和外部工具调用。

2. CodeAct 为什么会强

因为自然语言动作有两个问题：

一是模糊；二是不可复用。

比如去把 A 文件里的数据整理一下，然后统计平均值，再画图这种描述对人类好懂，但对执行系统来说不够精确。

而如果模型生成一段 Python：

- 读取文件；
 - 清洗字段；
 - 计算指标；
 - 输出可视化；
- 那么这个动作就更结构化、可执行、可复查、可复用。

3. 它适合什么任务

- 数据分析；

- 代码修复；
- 软件工程代理；
- 需要多步精确操作的任务；
- 可在沙箱环境中安全执行的任务。

4. 它的优势

- 表达力强，适合复杂控制流；
- 代码本身就是一种中间表示，可检查、可测试；
- 执行结果更客观，错误更容易定位；
- 适合和代码解释器、Notebook、软件工程环境结合。

5. 它的风险

- 代码执行带来安全问题；
- 需要沙箱、资源限制、文件权限控制；
- 模型可能生成高权限或破坏性动作；
- 错误分成代码语法错误运行时错误逻辑错误，调试链路更复杂。

6. 和 ReAct、Plan-and-Execute 的关系

CodeAct 不一定是 ReAct 的替代品，也不一定和 planning 冲突。

更准确地说，它是动作表达形式的升级：

- 你可以做 ReAct + CodeAct：每一步根据观察生成代码动作；
 - 也可以做 Plan-and-Execute + CodeAct：先规划，再让执行器把每一步翻译成代码或脚本执行。
- 所以面试时最好说：ReAct / Plan-and-Execute 更像运行控制结构，CodeAct 更像动作表示与执行方式。

五、三者如何比较：面试一定要会的对照表述

1. 控制结构维度

- ReAct：局部闭环、边做边想；
- Plan-and-Execute：全局先行、分步落地；

- CodeAct：不主要定义控制结构，而是让动作通过代码表达。

2. 全局性

- ReAct：弱；
- Plan-and-Execute：强；
- CodeAct：取决于外层是否有规划。

3. 灵活性

- ReAct：高；
- Plan-and-Execute：中，高度依赖是否支持 replanning；
- CodeAct：动作层灵活，但执行环境约束更强。

4. 并行化潜力

- ReAct：低，天然串行；
- Plan-and-Execute：高，因为计划可暴露并行子任务；
- CodeAct：如果代码层做任务编排，也能支持较强并行。

5. 上下文负担

- ReAct：高，trace 容易越来越长；
- Plan-and-Execute：中，可以把计划和阶段产物分开管理；
- CodeAct：如果用代码作为中间表示，有时反而能减少冗余自然语言。

6. 工程安全性

- ReAct：要防止错误动作和循环；
- Plan-and-Execute：要防止计划失真；
- CodeAct：要额外防止代码执行越权和资源滥用。

六、一个真正加分的点：三种模式不是互斥，而是可以组合

优秀候选人通常会补一句：**这三者经常组合使用。**

因为真实系统并不是教科书里的纯范式。

常见组合一：Plan-and-Execute + ReAct

先做粗规划，再让每个子任务内部用 ReAct。

适合研究任务、复杂检索、企业知识分析等场景。

常见组合二：ReAct + CodeAct

每一轮观察后生成一段代码或脚本作为动作。

适合数据分析、自动化测试、代码代理。

常见组合三：Planner 生成 DAG, Executor 用代码并行执行

这更接近 agent runtime 或 LLMCompiler 的思路。

适合明确存在并行子任务的复杂操作。

常见组合四：计划前置 + 关键动作审批

规划完成后先让人审核计划，再执行；涉及发送邮件、下单、改配置等高风险动作再做人类确认。

这对生产系统非常常见。

七、如何回答你会怎么选

面试里很常见的问题是：面对一个复杂任务，你会在 ReAct、Plan-and-Execute、CodeAct 里怎么选？

比较稳的答法是按任务特征分：

1. 如果任务短、探索性强、依赖实时反馈

优先 ReAct。

例如网页检索、简单工具链交互、即时问题求解。

2. 如果任务长、步骤依赖明显、需要全局路径

优先 Plan-and-Execute。

例如多阶段研究、复杂报告生成、长流程业务执行。

3. 如果动作本身需要精确操作、控制流或计算

优先引入 CodeAct。

例如数据处理、代码修复、自动化测试。

4. 如果既长又复杂，还需要精确执行

常见答案不是单选，而是Plan-and-Execute 外挂 CodeAct，并在子任务里保留 ReAct 式局部修正能力。

八、一个高频追问：为什么 ReAct 会容易循环，怎么治

这是特别容易被问到的工程问题。答案要从机制上讲，而不是只说加个上限。

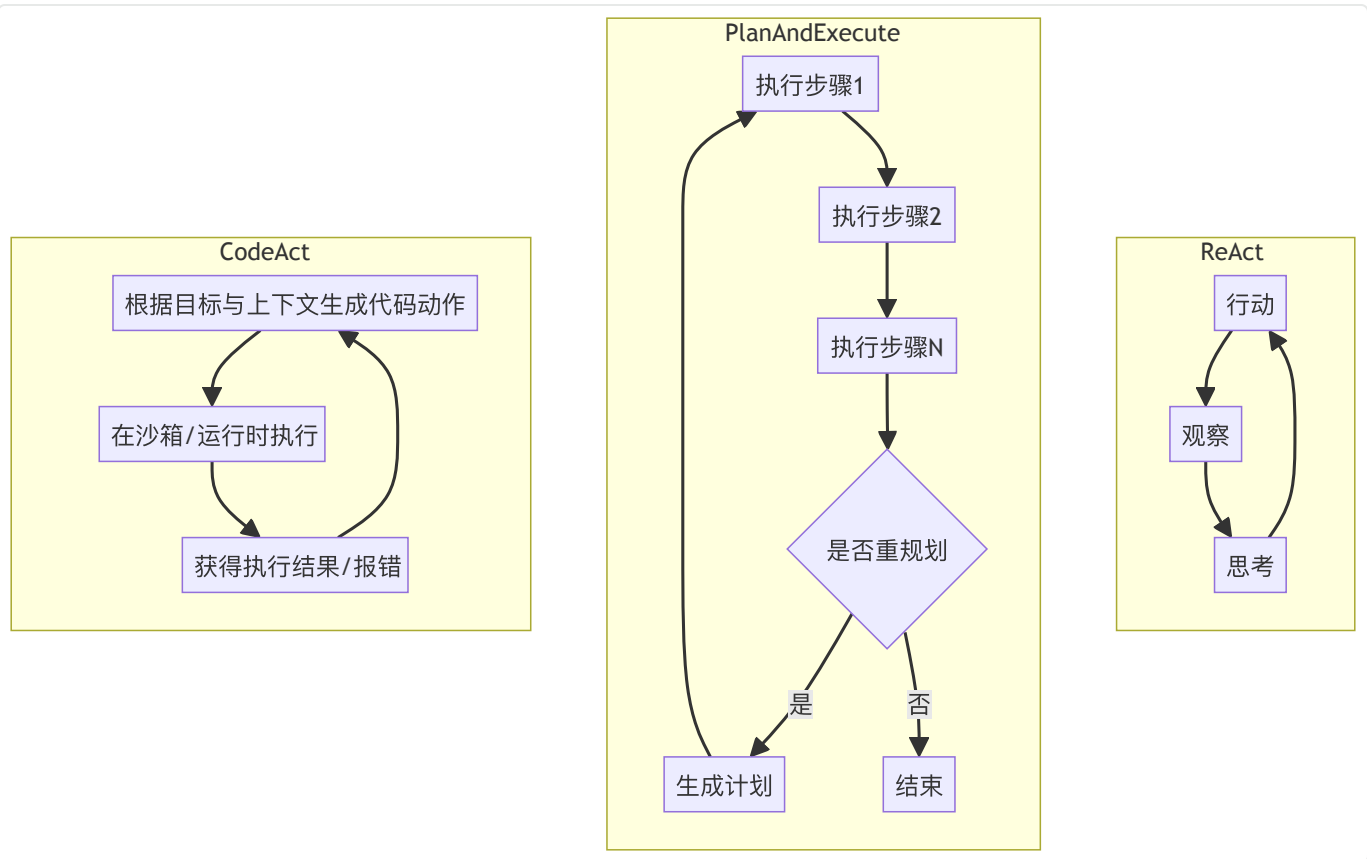
原因

- 没有清晰的成功判定；
- 工具结果不理想，模型不断尝试相似动作；
- 历史上下文里没有有效总结，导致反复探索同一无效路径；
- 缺乏失败记忆或去重机制；
- 模型没有预算感知。

治法

1. 设最大步数、时间预算、token 预算；
2. 对工具调用做去重检测；
3. 每轮总结已经试过什么、为什么失败；
4. 加 verifier，判断是不是应该停止或重规划；
5. 对高风险动作要求人工确认；
6. 用 planner 在较长任务前提供全局导向。

九、三种范式的控制结构



十、场景题：让 Agent 帮你修复一个线上报错，你会怎么设计

这是很适合用来展示你理解深度的题。一个较成熟的回答可以这样说：

如果只是根据日志做一次性分析，我会先用 ReAct，因为它需要边看错误、边查代码、边搜索信息。但如果任务升级成定位问题、修改代码、运行测试、生成修复说明、提交 PR 草稿，我会更倾向于 Plan-and-Execute + CodeAct。先让 planner 把任务拆成日志定位、相关文件检索、修复候选生成、测试验证、PR 总结几个阶段，再让执行器通过代码与命令行动作在沙箱里完成每一步。同时我会限制代码执行权限，要求高风险动作如 git push、修改生产配置必须人工确认，并为每阶段记录结构化产物和回滚点。

范式如何落到真实软件工程任务，才是我们在这类框架选择的场景题该回答的。

十一、高频面试题与参考回答

题 1：ReAct 的核心思想是什么？

参考回答：

在每一步交替进行推理与行动，依据行动后的观察结果持续更新后续决策，从而完成多步任务。

题 2：ReAct 最大的问题是什么？

参考回答：

缺乏全局规划，容易局部贪心、重复试错、上下文膨胀和循环调用。

题 3：Plan-and-Execute 比 ReAct 好在哪里？

参考回答：

它在长任务中更有全局视角，更适合任务拆解、依赖管理、阶段性交付和潜在并行化。

题 4：Plan-and-Execute 一定更强吗？

参考回答：

不一定。它也有规划成本，而且计划可能很快过时。若任务较短或环境变化快，过度规划反而浪费。

题 5：CodeAct 和 ReAct 是替代关系吗？

参考回答：

不是。ReAct 更多描述控制循环，CodeAct 描述动作表达方式。两者完全可以结合。

题 6：为什么代码动作有时比自然语言动作更好？

参考回答：

因为代码更精确、结构化、可执行、可复查，尤其适合计算、数据处理和软件工程任务。

题 7：什么时候你会优先用 ReAct？

参考回答：

任务不太长，但强依赖实时环境反馈，需要快速迭代试探时。

题 8：什么时候优先用 Plan-and-Execute？

参考回答：

任务较长、步骤依赖清晰、阶段目标明确、需要全局路线和中间产物管理时。

题 9：什么时候需要 CodeAct？

参考回答：

当动作本身涉及计算、条件控制、脚本执行、文件处理或代码修改时。

题 10：如果任务既长又复杂，你会怎么设计？

参考回答：

通常会采用混合模式：先规划，再执行；执行中保留 ReAct 式局部修正；对复杂计算或系统操作使用 CodeAct；对高风险动作做人类审批。

十二、面试速记版

- ReAct：边想边做，简单、灵活、适合交互式求解。
- Plan-and-Execute：先计划再执行，适合长链任务和阶段化管理。
- CodeAct：动作以代码表达，适合精确执行与软件/数据任务。
- ReAct 的痛点是局部贪心和循环。
- Plan-and-Execute 的痛点是计划失真和额外开销。
- CodeAct 的痛点是执行安全与运行环境治理。
- 工程里常见的是混合而不是单选。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》, 2024。适合用来界定 agent 与 workflow 的边界，强调能用简单工作流解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》, 2025。非常适合补充 agent 不只是 prompt engineering, 而是 context engineering 的近年观点。
3. Anthropic, 《How we built our multi-agent research system》, 2025。适合放在多 agent、任务拆解、subagent 协作章节中。
4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。
6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。

8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。
11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。
15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。

04 Task Decomposition 与 Planning

一、面试官为什么要单独问任务拆解与规划

很多候选人对 planning 的理解停留在先让模型列个步骤。这远远不够。真正的面试考点是：**你是否把 planning 当成一个结构化的系统能力，而不是一句提示词。**

当面试官问Agent 为什么需要 planning任务拆解怎么做什么时候需要 replanning计划和 workflow 是什么关系，他在考察的是你能不能把开放式任务变成可执行、可管理、可恢复的任务图。

在真实系统里，任务拆解和 planning 至少承担五个作用。第一，降低单步认知负担，把大问题分成更小、更稳定的子问题。第二，显式暴露依赖关系，让系统知道哪些步骤必须先做，哪些步骤可以并行。第三，给执行器提供方向，减少纯 ReAct 式局部贪心。第四，为监控、审批、回放和评测提供结构化抓手。第五，在长任务中形成阶段性产物，避免所有中间信息都堆在一个上下文里。

所以，这一题真正要回答的不是规划有没有用，而是规划是如何表示、如何执行、如何更新、如何失败、如何恢复。

一个推荐的面试话术表述：

我理解的任务拆解与 planning，不是让模型先列步骤，而是把开放目标转成可执行、可管理、可恢复的任务结构。它的关键点有三个：第一，拆解粒度要合适，过粗会失控，过细会脆弱；第二，计划最好是可消费的结构，而不是一段自然语言；第三，planning 是闭环能力，要支持检查、执行、观测和 replanning。真正好的 planning，不是写出一个好看的计划，而是让系统更稳地完成复杂任务。

二、为什么需要任务拆解：从大目标到可执行子任务

大模型擅长语言模式匹配，但面对复杂开放式目标时，如果不给它结构，它很容易出现三种问题：

1. 把问题看得过粗，一步跨太大，导致遗漏关键约束；
2. 把问题看得过细，在低价值小动作上浪费步数；
3. 在复杂任务中没有阶段概念，做着做着就迷失。

任务拆解的核心价值，就是把一个开放目标转成更稳定的执行单元。

例如帮我做一份某行业竞品研究报告这个目标，本身并不可直接执行。一个成熟的系统通常会拆成：

- 明确研究范围与评价维度；
- 搜集候选公司与来源；
- 提取价格、功能、市场策略等信息；

- 对数据做交叉验证；
- 归纳发现与差异；
- 输出结构化报告；
- 根据缺口做补充检索。

这意味着 planning 不是写出目录，而是让执行链路拥有结构。

三、任务拆解的常见粒度：拆得太粗和拆得太细都不行

面试里一个很好的回答点是：任务拆解的难点不只是会拆，而是确定合适粒度。

1. 拆得太粗的问题

如果子任务仍然很大，例如去研究竞品作为一个步骤，那执行器内部还是会重新面临复杂开放式问题，planning 形同虚设。

过粗的拆解会导致：

- 执行器难以明确成功标准；
- 无法并行；
- 无法复用中间结果；
- 无法在失败时精确回退。

2. 拆得太细的问题

如果把任务拆到诸如打开网页滚动页面点击某个按钮这种级别，规划会非常脆弱。

环境稍有变化，计划就过时。

过细的拆解还会导致：

- planning token 开销过大；
- 调度与状态管理负担上升；
- 高层目标被淹没在微动作里；
- 系统变得像脚本录制器而不是智能体。

3. 合适的粒度

一般来说，一个好的子任务应该满足三个条件：

- 有清晰的输入和输出；

- 有相对稳定的成功判定；
- 在必要时可由不同执行器完成。

比如从 5 个来源收集产品定价并标准化到统一表结构就是一个相对合适的子任务，因为它边界清晰、结果可校验、还可以考虑并行。

四、Planning 的表示形式：不是只有步骤列表

很多同学一想到计划，就是让模型输出一个 numbered list。但在工程上，计划可以有多种表示形式，不同形式决定了后续执行和治理能力。

1. 线性步骤列表

最简单，也最常见。

适合依赖关系基本是串行的任务。

优点是易于理解和实现，缺点是表达力有限，不适合并行和分支。

2. 树状计划

常见于从目标到子目标的层级分解。

例如总任务拆成几个阶段，每个阶段再拆子任务。

适合做分层规划，但执行时需要额外机制把树转换成运行顺序。

3. DAG（有向无环图）

当任务中存在依赖但也存在并行可能时，DAG 很合适。

例如先并行收集多个来源信息，再在下游统一汇总。

这类表示非常适合后续 runtime 做调度、并行执行和失败重试。

4. 状态空间 / 搜索树

在研究型 planning 里，有些方法更像搜索过程，比如 Tree of Thoughts。

系统会探索多个候选路径，对路径进行打分或剪枝。

这适合复杂推理问题，但在企业生产中要注意成本。

5. 程序化计划

有时计划不是自然语言，而是结构化 schema、JSON、DSL，甚至直接编译成执行图。

这种方式更适合和 agent runtime 集成，因为执行器不需要猜计划含义。

面试里很加分的一句话是：计划的价值不在于写下来，而在于能否被执行器、监控器和治理层真正消费。

五、Planning 的核心流程：不是一次性生成，而是闭环

一个成熟的 planning 链路，通常包含下面几个阶段：

1. 目标澄清

系统先把任务边界定义清楚，包括成功标准、范围、限制条件、预算与时间约束。

很多 planning 失败，根源其实不是不会规划，而是目标本身不够清楚。

2. 初始分解

把任务拆成候选子任务，形成初始计划。

这里可以要求模型输出：

- 子任务名称；
- 输入依赖；
- 预期输出；
- 成功判定；
- 需要的工具类型；
- 可否并行。

3. 计划检查

不是所有模型生成的计划都应该直接执行。

一个成熟系统往往会在执行前做静态检查：

- 是否遗漏关键约束；
- 是否存在循环依赖；
- 是否把不可执行动作写进计划；
- 是否越权；
- 是否有成本明显过高的路径。

4. 执行与观测

执行器按计划推进，每个步骤产生结果、日志和状态更新。

这时 planning 的职责不是消失，而是持续提供为什么做这一步的上层语义。

5. 进度评估与 replanning

当某一步失败、环境变化、信息不足、目标变化时，要决定是否重规划。

这一步非常关键，因为规划一次跑到底的系统在真实环境中通常不够鲁棒。

6. 收口与复盘

任务结束后，可以把过程产物存档，提炼可复用经验，例如下次类似任务的默认模板、失败模式、工具适配建议等。

六、静态规划与动态规划：面试最爱追问的点

1. 静态规划

开始时生成完整计划，然后按计划执行。

优点是全局性强、便于审批和分析；缺点是对环境变化适应差。

2. 动态规划

执行中不断调整计划，或者先做粗规划、局部细化。

优点是灵活，缺点是系统复杂度更高，也更难回放。

3. 工程上的折中

多数生产系统会选择折中：

- 先生成高层计划；
- 每个阶段开始前再做局部细化；
- 关键节点根据结果决定是否 replanning。

这种方式兼顾全局性与灵活性，是最实用的设计之一。

七、什么时候需要 Replanning

这是面试里非常高频的问题。一个优秀回答应该给出明确触发条件。

1. 执行失败且错误不可局部修复

比如某个数据源失效、页面结构变化、接口权限不足。

如果只是参数错误，可局部重试；如果是原路径整体失效，就应该 replanning。

2. 新观察推翻原先假设

例如原本以为可以从官网拿到所有数据，后来发现官网不完整，需要改成多源交叉验证。
这时继续按旧计划执行只会浪费步数。

3. 目标发生变化

用户中途增加限制，例如只看过去一个月的数据不要包含国外市场。
这通常需要重规划，而不仅是局部修改。

4. 成本或时间预算接近上限

即使还能继续跑，也可能需要规划一个更省资源的替代路径。

5. 出现更优路径

例如在早期步骤中发现一个更高质量的数据源，原来的多源抓取计划可以简化。

一个很加分的表达是：Replanning 不只是失败后补救，也是利用新信息重新优化路径。

八、Task Decomposition 与 Workflow 的关系

很多人会把任务拆解误当成 workflow。实际上两者有关，但不等价。

- Workflow 更强调预先编码的流程控制；
- Task decomposition 更强调把复杂目标拆成可执行单元；
- Planning 介于两者之间，它可能生成 workflow，也可能驱动更灵活的 agent 执行。

所以面试中可以这样说：

任务拆解是认知层面的结构化；workflow 是执行层面的确定性编排；planning 则是在两者之间把目标

翻译成可运行结构。

九、面试高频点：Planning 和 Prompt Engineering 有什么关系

有些候选人会把 planning 讲成写个 prompt，让模型先列计划。这是不够的。

更好的回答是：

1. Prompt engineering 可以影响 planning 的质量；
2. 但 planning 本身是系统能力，不只是提示词技巧；
3. 真正有效的 planning 需要配合结构化输出、状态管理、执行器、验证器、回滚和 replanning 机制；
4. 否则模型只是说了一个计划，系统却未必真的按这个计划可控地运行。

十、工程实践：如何让 planning 真的有用

1. 让计划结构化

尽量不要只输出自然语言。

推荐输出包括：

- step_id
- 描述
- 前置依赖
- 所需工具
- 输入来源
- 预期输出
- 成功判定
- 风险等级
- 是否允许并行

这样后续执行、监控、审批都更顺畅。

2. 区分高层计划和执行动作

高层计划不应直接细化到每个微动作，否则环境稍变就失效。

应当保持一定抽象度，把微动作留给执行器。

3. 给每一步定义完成条件

如果没有完成条件，执行器很难判断某一步是应该继续努力还是已经足够。这也是防止无穷探索的关键。

4. 保留阶段产物

不要让每一步结果只存在于上下文里。

最好把中间结果结构化存储，例如笔记、表格、候选结论、证据链。

这样 replanning 时就不会从零开始。

5. 加入 plan verifier

可以用规则、第二个模型或混合方式检查计划是否：

- 遗漏关键约束；
- 逻辑自相矛盾；
- 存在明显不可执行步骤；
- 风险动作没有审批。

6. 支持 partial completion

长任务中不要非黑即白。

即使计划未全部完成，也应允许阶段性输出，告诉用户：

- 已完成什么；
- 还缺什么；
- 为什么停止；
- 如果继续需要什么资源或审批。

十一、Planning 的典型失败模式

失败模式一：计划很好看，但不可执行

自然语言计划经常写得像报告，但执行器无法据此可靠行动。

失败模式二：子任务之间接口不清

例如上游输出不是下游真正需要的格式，导致中间产物无法复用。

失败模式三：过度规划

对简单任务先规划一大堆，结果规划成本比任务本身还高。

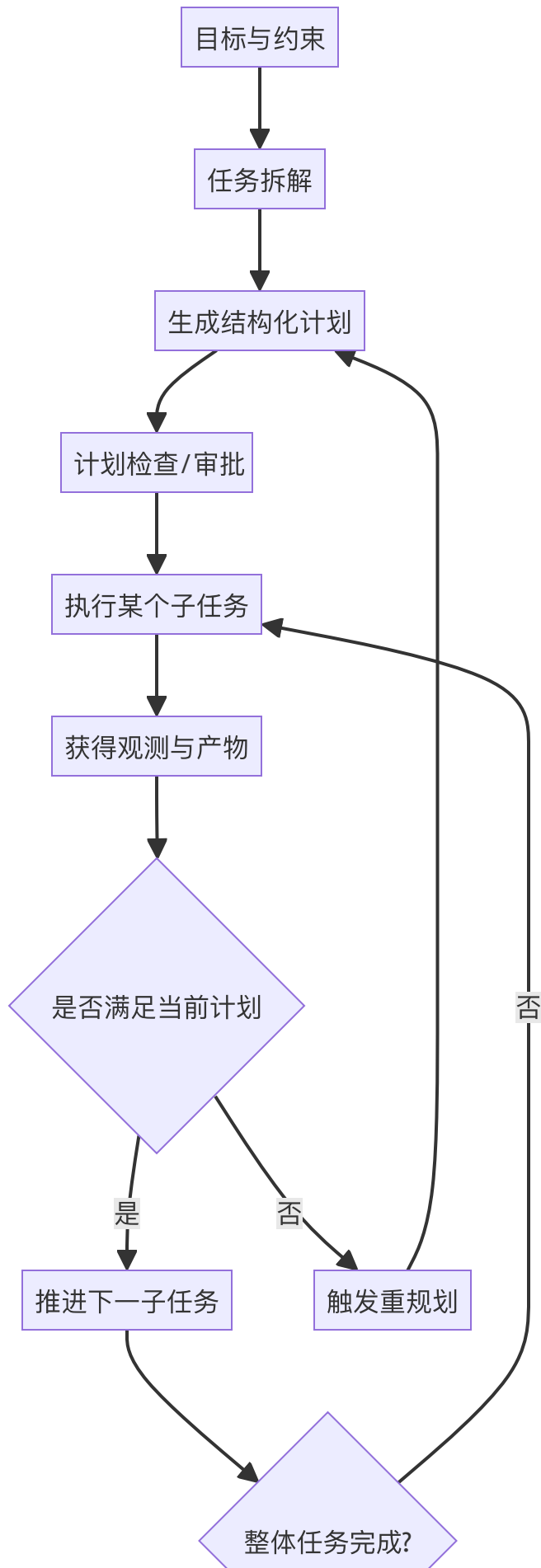
失败模式四：不重规划

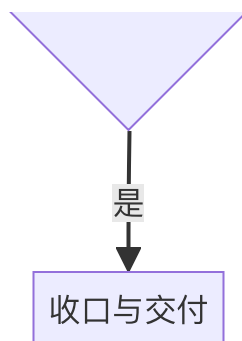
明明环境已经变化，系统仍死守旧计划。

失败模式五：把计划当真理

计划应当是指导而非神谕。现实世界的 agent 需要留出校正空间。

十二、Mermaid 图：Planning 闭环





十三、高频面试题与参考回答

题 1：为什么 Agent 需要 task decomposition?

参考回答：

因为复杂目标通常不能直接执行。拆解能降低单步复杂度、暴露依赖关系、支持并行与阶段性交付，并提升可控性与可评测性。

题 2：任务拆解时最重要的是什么？

参考回答：

不是拆得越多越好，而是拆到合适粒度。每个子任务都应有清晰输入输出、成功标准和相对稳定的执行边界。

题 3：Planning 和 Workflow 是一回事吗？

参考回答：

不是。Planning 是把目标转成执行结构，Workflow 是确定性编排。Planning 可以生成 workflow，也可以为 agent 提供运行指导。

题 4：什么时候需要 replanning?

参考回答：

当关键假设被新观察推翻、执行失败无法局部修复、目标变化、预算紧张或发现更优路径时。

题 5：计划应该用自然语言还是结构化表示？

参考回答：

工程上更推荐结构化表示，至少要包含步骤、依赖、输入输出、工具、成功判定等字段，便于执行与治

理。

题 6：为什么不能把计划拆得太细？

参考回答：

因为过细计划会脆弱、昂贵、难维护，而且容易把高层目标淹没在微动作里。

题 7：为什么也不能拆得太粗？

参考回答：

因为执行器仍然会重新面对一个复杂开放问题，计划失去作用，也难以评测和回滚。

题 8：计划失败最常见的原因是什么？

参考回答：

目标不清、步骤不可执行、依赖关系不明、没有完成条件、环境变化后不重规划。

题 9：如何让 planning 真正提高系统质量？

参考回答：

结构化计划、计划检查、阶段产物持久化、明确完成条件、支持重规划和部分完成交付。

题 10：一句话总结你对 planning 的理解？

参考回答：

Planning 不是让模型写一个好看的步骤列表，而是把开放任务转成可执行、可监控、可恢复的任务结构。

十四、面试速记版

- 任务拆解的目标是把开放问题变成可执行单元。
- 计划粒度要适中，过粗无用，过细脆弱。
- 计划可以是列表、树、DAG、结构化 schema，甚至编译成执行图。
- Planning 是闭环能力，不是一次性产物。
- Replanning 是常态，不是异常。
- 计划必须能被执行器、监控器和治理层消费。

- 真正好的 planning 会降低系统复杂度，而不是制造文档负担。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》, 2024。适合用来界定 agent 与 workflow 的边界, 强调能用简单 workflow 解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》, 2025。非常适合补充 agent 不只是 prompt engineering, 而是 context engineering 的近年观点。
3. Anthropic, 《How we built our multi-agent research system》, 2025。适合放在多 agent、任务拆解、subagent 协作章节中。
4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。
6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。
8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。
11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。
15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。

05 Tool Use 与 Action Execution

一、tool use就是agent loop的核心

很多人学 Agent 时最先关注的是 prompt、推理、planning，但面试官在真实工程岗位上往往更看重另一个层面：**Agent 是如何和外部世界发生可靠交互的**。这就是 Tool Use 与 Action Execution。

如果说 planning 解决的是应该做什么，那么 tool use 解决的是怎样真正做成。这也是为什么在企业里，一个会思考但不会稳定执行的 agent，往往价值远低于一个思考一般但动作可靠的系统。

从工程角度看，工具使用是 agent 失败最常见的地方之一。不是模型不会选工具，就是参数填错；不是工具返回异常，就是接口超时；不是动作有副作用无法幂等，就是结果校验不充分导致误操作。面试官问这部分，往往是在看你有没有从模型层切换到系统层的意识。你如果只回答让模型调函数就行，基本就是送分题没拿到。

真正成熟的回答应该覆盖完整链路：工具如何描述，模型如何选择，参数如何约束，执行器如何运行，结果如何验证，失败如何重试，副作用如何控制，权限如何收口，历史如何记录，何时并行，何时人工确认。

二、先把问题说清楚：Tool Use 到底是什么

Tool Use 可以理解为：让模型在文本生成之外，主动调用结构化外部能力，以便完成信息获取、计算、操作和控制类任务。

这些外部能力通常包括三大类：

1. Data tools

数据类工具，用于拿信息。

例如搜索、数据库查询、文档检索、向量检索、知识库、业务系统查询、网页读取等。

这类工具通常副作用较小，是最常见的 agent 工具类型。

2. Action tools

动作类工具，用于改世界。

例如发邮件、创建工单、更新 CRM、执行 SQL 写操作、修改文件、触发审批、调用第三方系统 API、提交代码、点击网页按钮等。

这类工具价值很大，但风险也高，因为它们有副作用。

3. Orchestration tools

编排类工具，用于组织其他能力。

例如创建子任务、唤起 sub-agent、暂停等待人工审批、提交到队列、触发 workflow 节点、切换角色等。

这类工具把做某件事提升成组织系统如何运行。

面试时很加分的一句话是：工具不只是查资料的能力，它也是模型与程序世界的接口层。

三、Tool Use 的标准链路：从选择到执行再到观察

一个完整的工具使用过程，至少包含下面几个环节：

1. Tool selection

模型判断当前目标最适合用哪个工具。

这一步的难点是：

- 工具太多时，选择空间会变大；
- 工具描述不清时，模型容易误用；
- 相近工具同时存在时，容易路由错误。

2. Argument grounding

模型根据当前上下文生成工具参数。

很多线上问题都出在这里。

例如用户说最近，到底是过去 7 天、30 天还是自然月；用户说帮我发给老板，系统能否正确解析联系人；搜索关键词如何构造；SQL 条件是否遗漏租户隔离。

3. Pre-execution validation

参数在真正执行前应由程序层校验。

这一步非常关键。不能把模型输出的参数直接视为可信输入。

典型校验包括：

- 类型检查；
- 必填字段检查；
- 枚举值检查；

- 权限范围检查；
- 跨字段逻辑检查；
- 风险级别判断。

4. Execution

执行器真正调用外部能力。

执行时可能涉及：

- 重试策略；
- 超时；
- 幂等键；
- 事务；
- 限流；
- 沙箱；
- 资源隔离；
- 审计日志。

5. Observation normalization

工具返回结果后，需要把原始响应转换成模型更好消费的形式。

不能一股脑把几百行 JSON 全喂回去。

通常要做：

- 关键字段提取；
- 状态码归一化；
- 错误类别映射；
- 证据摘要；
- 大结果分页或摘要。

6. Post-execution verification

执行完不代表任务就真的完成。

例如发邮件工具返回 success，不代表收件人正确；数据库更新执行成功，不代表业务逻辑正确；网页点击没有报错，不代表页面真的发生了预期变化。

所以很多系统会在执行后加入 verifier 或二次检查。

四、工具 schema 怎么设计：这是高频追问

面试官特别喜欢问：Function calling 的 schema 怎么设计？这题答得好非常加分。

1. 工具名称要明确

不要叫 `do_task`、`process_data` 这种含糊名字。

工具名应该让模型一眼知道边界，例如：

- `search_public_web`
- `query_customer_order`
- `create_refund_request`
- `send_email_draft`

2. 描述要强调边界和适用条件

工具描述不应只是用于搜索。更好的写法是：

- 什么时候该用；
- 什么时候不该用；
- 参数来源是什么；
- 是否有副作用；
- 是否需要审批。

3. 参数要尽量结构化

不要把一堆自由文本塞进一个 `input` 字段。

越结构化，模型越容易稳定生成，程序也越容易校验。

例如把邮件发送拆成：

- `to`
- `cc`
- `subject`
- `body`
- `require_approval`

而不是一个大字符串。

4. 把约束前置到 schema

如果某个字段只能取枚举值，就不要让模型自由发挥。

如果某字段必须是 ISO 日期格式，就明确约束。

如果某动作只允许当前租户资源，就在参数设计或执行层强约束，而不是靠提示词提醒。

5. 明确副作用和幂等要求

例如：

- 发送类动作是否支持 dry-run；
- 创建类动作是否要求 client_request_id；
- 更新类动作是否支持 version check；
- 删除类动作是否必须人工确认。

一个很成熟的回答是：schema 不是给模型看的说明书，它同时也是给执行器做安全控制和参数校验的合同。

五、工具选择为什么会错：根因分析

1. 工具集合过宽

什么都给模型，它反而更难选。

工程上应该按场景暴露最小必要工具集。

2. 工具定义重叠

两个工具都能查用户信息，但一个查 CRM，一个查订单系统。

如果描述不清，模型很容易混用。

3. 工具语义与用户语言错位

用户说帮我看看最近的销售情况，模型可能需要把自然语言映射成特定业务指标、时间粒度、区域过滤。

如果没有中间规范化层，参数很容易歪。

4. 缺乏反馈

如果工具调用失败后，系统只返回调用失败，模型很难知道错在哪里，也就难以修正。
工具返回需要携带足够的错误语义，例如权限不足、参数缺失、资源不存在、暂时超时等。

六、Action Execution：为什么执行器不是简单调个 API

工具设计得再好，没有执行器治理也不行。执行器本质上是 agent 和现实世界之间的安全门。

1. 幂等性

对有副作用的动作，比如发邮件、创建工单、下单、记账，必须考虑重复执行。

常见做法包括：

- client_request_id
- 幂等键
- 事务日志
- 写前去重
- 重试时只允许在幂等窗口内重放

2. 重试策略

不是所有错误都适合自动重试。

需要区分：

- 瞬时错误：如网络超时、限流，可重试；
- 参数错误：不应盲目重试，应修正参数；
- 权限错误：需要升级权限或人工介入；
- 业务规则错误：应停止并反馈。

3. 超时与取消

长动作如果没有超时和取消机制，agent 很容易被卡死。

特别是外部 API、浏览器、代码执行、文件读写，都需要时间边界。

4. 事务与回滚

对多步副作用动作，最好能设计事务边界。

比如创建草稿—上传附件—发送，如果中间某步失败，是否可回滚到草稿态？

如果完全不可回滚，就要在更早阶段做审批和 dry-run。

5. 沙箱与资源隔离

对代码执行、文件系统、浏览器操作，更要限制：

- 网络访问；
- 文件范围；
- CPU / 内存；
- 可调用命令；
- 持久化能力。

七、并行 Tool Use：收益很大，但不是免费午餐

很多候选人听到 parallel tool calls 会很兴奋，但面试官更想听到取舍。

并行的收益

- 降低总时延；
- 更适合独立数据源抓取；
- 提高整体吞吐；
- 适合 DAG 型任务执行。

并行的风险

- 参数依赖关系搞错会导致错误结果；
- 副作用动作并行可能引发一致性问题；
- 上下文中很难同时消费过多并发结果；
- 并发过高会引发限流、成本飙升和调试困难。

适合并行的场景

- 多来源检索；
- 多文档摘要；
- 独立 API 查询；

- 批量数据分析。

不适合并行的场景

- 有强顺序依赖的任务；
- 会修改共享状态的动作；
- 需要人工审批的关键路径；
- 高风险事务链路。

八、面试高频点：如何防止工具误调用和越权

这是几乎必追问的生产问题。

1. 最小权限原则

每次任务只暴露必要工具，不要把整个工具宇宙都交给模型。

2. 工具分级

可把工具分为：

- 只读低风险；
- 写操作中风险；
- 高风险破坏性操作。

不同级别对应不同审批和日志要求。

3. 参数白名单与租户隔离

工具不是只校验字段类型，还要校验是否属于用户有权访问的资源范围。

4. 审批前置

对于发送、支付、删除、发布等动作，最好提供 dry-run 和人工确认。

5. 二次确认与结果复核

某些操作即使执行成功，也要要求 verifier 再判断结果是否合理。

6. 审计记录

要能回答下面这些问题：

- 谁触发了任务；
- 模型选择了哪个工具；
- 参数是什么；
- 程序校验后变成什么；
- 最终执行结果怎样；
- 是否有人类审批；
- 是否发生重试或回滚。

九、Tool Use 与 Prompt / Context Engineering 的关系

这里非常容易答浅。成熟回答应该是：

1. Prompt 决定模型是否知道什么时候用工具；
2. Context 决定模型能不能拿到足够信息正确填参数；
3. Schema 决定模型输出是否稳定可校验；
4. Executor 决定动作能否可靠、安全地落地；
5. Verifier 决定系统是否知道这一步到底做成了没有。

所以 Tool Use 绝不是加一个 function calling 开关，而是从 prompt 到 runtime 的全链路设计。

十、典型失败模式

失败模式一：工具太多，模型不会选

解决：按任务裁剪工具集，做分层路由。

失败模式二：工具描述含糊，模型乱填参数

解决：重写 schema、增加示例、减少自由文本字段。

失败模式三：程序对参数完全信任

解决：执行前强校验，不可信输入全部经过正规化。

失败模式四：副作用动作缺少幂等

解决：引入 request_id、事务日志、状态检查。

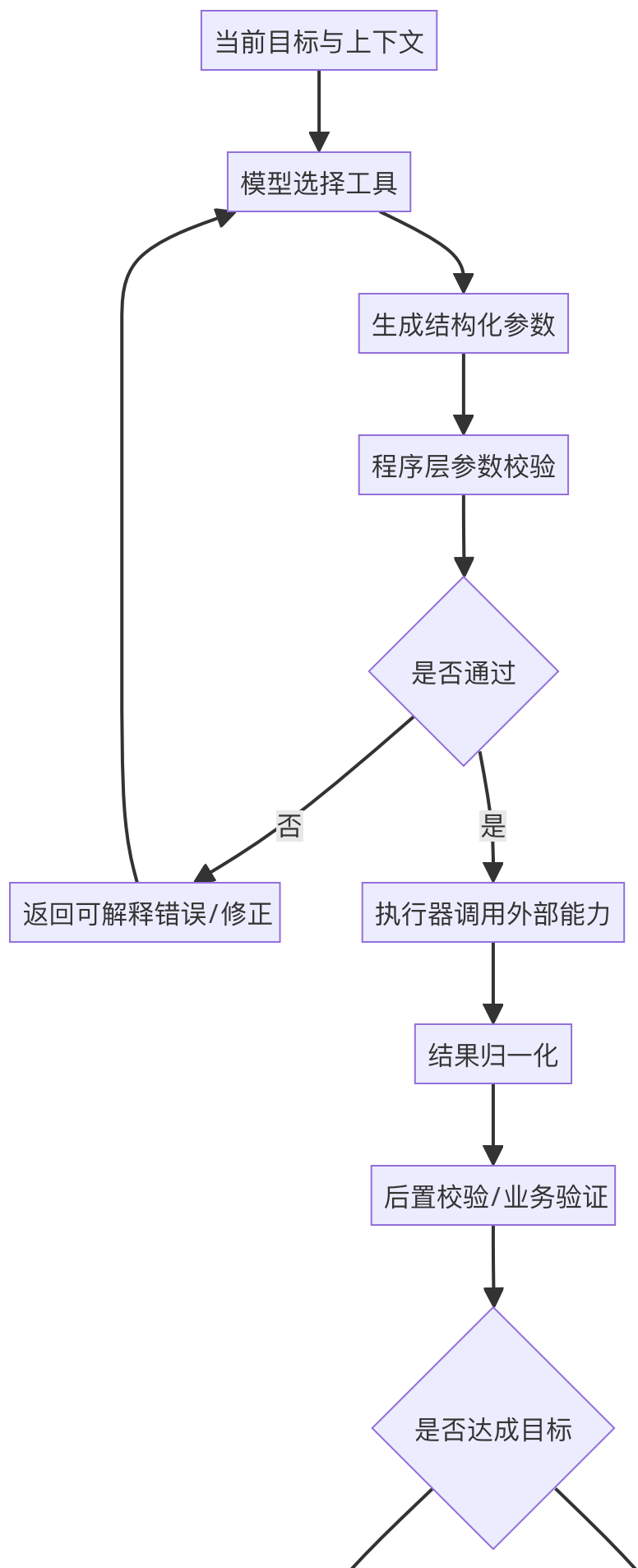
失败模式五：工具返回过于原始

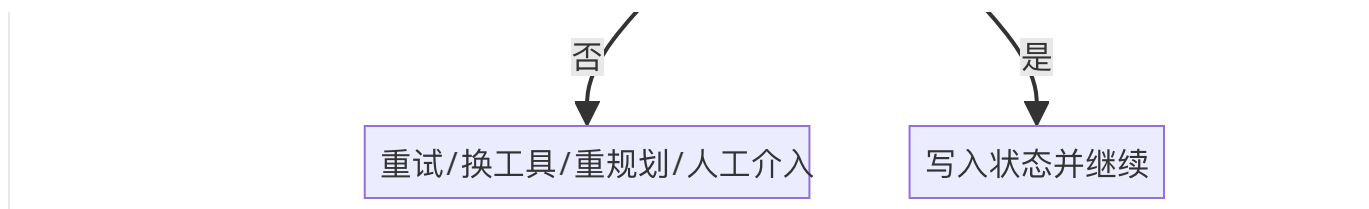
解决：结果归一化与摘要，让模型读得懂重点。

失败模式六：执行成功但业务失败

解决：增加 post-check，不只看 HTTP 200。

十一、Mermaid 图：工具使用执行链路





十二、高频面试题与参考回答

题 1：什么是 Tool Use？

参考回答：

让模型通过结构化接口调用模型外能力，完成信息获取、计算、操作和系统控制等任务。

题 2：为什么工具是 agent 的核心？

参考回答：

因为没有工具，模型大多只能生成文本；有了工具，agent 才能读取外部信息、执行动作并真正影响现实世界。

题 3：工具 schema 怎么设计更稳定？

参考回答：

名称清晰、边界明确、描述具体、参数结构化、约束前置、风险和副作用显式化，尽量减少模糊自由文本。

题 4：为什么不能直接执行模型生成的参数？

参考回答：

因为模型输出不应被视为可信输入，必须经过类型、权限、业务规则和租户边界等多层校验。

题 5：副作用动作最重要的工程点是什么？

参考回答：

幂等性、审批、审计、回滚设计和 post-check。仅仅调用成功不等于业务成功。

题 6：什么时候适合并行 tool call？

参考回答：

多个查询彼此独立、无共享副作用、结果可后续统一归并时。

题 7：什么时候不适合并行？

参考回答：

存在严格依赖关系、会修改共享状态、有高风险副作用或需要逐步验证时。

题 8：如果模型总是选错工具怎么办？

参考回答：

缩小工具集、重写描述、做上层路由、提供高质量示例，并分析错误是语义重叠还是参数映射问题。

题 9：如何防止越权操作？

参考回答：

最小权限、租户隔离、参数白名单、高风险审批、执行器强校验和完整审计。

题 10：一句话总结 Tool Use 的本质？

参考回答：

它是模型决策能力与程序执行能力之间的合同层和安全门。

十三、面试速记版

- Tool Use 不是会调函数，而是完整的选择—校验—执行—验证链路。
- 工具可分数据类、动作类、编排类。
- Schema 既是给模型看的，也是给执行器做安全控制的合同。
- 模型参数输出永远不是可信输入。
- 副作用动作必须考虑幂等、审批、回滚和审计。
- 并行调用有收益，但会带来一致性和调试成本。
- 工具层的可靠性，往往决定 agent 是否能上线。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》, 2024。适合用来界定 agent 与 workflow 的边界, 强调能用简单工作流解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》, 2025。非常适合补充agent 不只是 prompt engineering, 而是 context engineering的近年观点。
3. Anthropic, 《How we built our multi-agent research system》, 2025。适合放在多 agent、任务拆解、subagent 协作章节中。
4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。
6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。
8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。
11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。
15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。

06 Memory体系

一、为什么 Memory 是 Agent 面试里的高频误区题

Memory 几乎是所有 Agent 面试都会问到的话题，但它也是最容易被答错、答混和答空的部分。最常见的误区有三个：

第一，把 memory 直接等同于保存聊天记录；

第二，把 memory 和 RAG 混为一谈；

第三，把所有持久化信息都笼统叫长期记忆，却说不清它们的读写时机和使用边界。

所以面试官问Agent 的 memory 怎么设计memory 和 RAG 有什么区别长期记忆有什么风险，本质上是在看你是否理解 agent 的状态持续能力，而不是只会堆术语。

从系统角度看，memory 解决的是：系统如何在跨步骤、跨回合、跨任务甚至跨天的执行中，保留对后续决策真正有帮助的信息。

如果没有 memory，agent 每次都像刚被创建出来的新个体，只能依赖当前上下文；

如果 memory 设计得不好，agent 又会变得迟钝、污染、过时，甚至在错误经验上越走越偏。

因此，memory 的难点从来不是存起来，而是存什么、何时写、何时读、如何失效、怎样避免污染。

二、先把概念分清：Memory、Context、State、RAG 不是一回事

这是面试里必讲的边界。

1. Context

Context 是当前模型调用时实际送进 prompt 的那部分信息。

它是当下可见窗口，可能来自系统提示词、用户输入、工具结果、摘要、memory 检索结果等。

所以 context 是模型眼前看到的東西，不等于 memory 本身。

2. State

State 更偏运行时状态。

它包括当前任务 ID、阶段进度、已完成步骤、预算消耗、临时变量、中间产物、审批状态等。

State 不一定长期保存，也不一定适合被称为 memory。

3. Memory

Memory 更强调对未来决策有价值的可延续信息。

它可以来自历史对话、历史任务、用户偏好、成功经验、失败经验、总结笔记、长期知识片段等。
换句话说，memory 是可被未来再次利用的过去。

4. RAG

RAG 主要解决外部知识召回问题。

它关心的是：面对当前问题，从知识库或文档库里取回最相关的信息。

RAG 的对象通常是相对客观、共享的外部知识；

memory 的对象更偏个体化、任务化、经验化的信息。

一个非常稳妥的面试表达是：

RAG 面向外部知识找回来，memory 面向系统经历过的有用信息留得住。

三、为什么 Agent 需要 Memory

1. 长任务不可能全靠上下文硬扛

如果一个任务要持续几十步甚至几天，不可能把全部过程都一直塞进 prompt。
需要把阶段性结论和关键事实沉淀成 memory 或持久化状态。

2. 用户个体化体验需要持续

例如用户偏好什么格式、常用什么术语、是否偏好简洁回答、所在行业、常用数据源，这些都不应该每次重新问。

3. 经验会改变未来决策

如果 agent 之前试过某个工具在某类任务上经常失败，或者某种搜索策略更有效，这些经验理应被保留下来，服务未来决策。

4. 人类协同需要上下文延续

人类接管、审批、编辑之后，系统需要记住人类修正过什么，否则下次还会犯同样错误。

四、Memory 的经典分类：一定要会的层次化回答

面试中最推荐的回答方式是分层，而不是一股脑说短期记忆、长期记忆。

1. Working Memory（工作记忆）

也叫运行时记忆。

它服务于当前任务的短时推理与执行，例如：

- 当前已知事实；
- 最近几步工具结果；
- 当前计划和子任务状态；
- 暂时变量；
- 中间摘要。

它通常生命周期较短，和 session state 有很强重合。

2. Episodic Memory（情节记忆）

记录系统曾经经历过的具体事件。

例如：

- 某次任务的步骤与结果；
- 某次失败的原因；
- 某次人工审批的修改意见；
- 某个用户在某种任务上偏好的交付形式。

情节记忆的特点是有时间、有场景、有具体经过。

3. Semantic Memory（语义记忆）

更像从多次经验中抽象出来的稳定知识。

例如：

- 用户偏好 Markdown 表格；
- 某业务系统字段 `status=3` 表示已结案；
- 某类任务常用三个数据源更可靠。

它比情节记忆更抽象，也更适合作为长期可复用规则。

4. Procedural Memory（程序性记忆）

指怎么做的经验。

例如：

- 处理退款工单时优先检查订单状态再核对支付方式；
 - 调研类任务先搜英文资讯再补中文分析；
 - 修复代码 bug 时先跑最小复现。
- 它接近策略模板或技能，通常非常有价值。

5. User Memory（用户记忆）

围绕某个用户的稳定偏好和背景。

如岗位、团队、常用语言、输出格式偏好、时间区域、业务范围。

这是企业产品中最常见的一类 memory，但也最容易涉及隐私与过时问题。

五、一个非常重要的工程点：不是所有记住都应该叫 memory

很多线上系统会把以下东西都混在一起，这是危险的：

- 历史聊天记录；
- 中间推理链；
- 工具原始返回值；
- 用户画像；
- 长期知识笔记；
- 运行状态；
- 审批记录。

更稳妥的做法是把它们分成不同存储层和不同消费路径：

1. 会话状态层：服务当前任务运行；
2. 经验记忆层：保存可复用经验；
3. 用户画像层：保存用户长期偏好；
4. 知识检索层：服务外部知识 RAG；
5. 审计日志层：保留完整可追溯记录，但不直接喂给模型。

这类回答很加分，因为它体现你有信息分层治理的意识。

六、Memory 的读写策略：真正难的不是有，而是何时用

一）什么时候写入

1. 任务完成后写总结

这是最安全的方式。

任务结束后，把关键结论、有效策略、失败原因、人类修正点提炼成结构化记忆。

2. 在关键节点写 checkpoint

长任务中可以在阶段结束时写入阶段性记忆，例如已验证来源、已完成子任务、关键证据。

3. 遇到高价值事件写 episodic memory

例如某个工具反复失败、某次人类明确修正偏好、某次关键审批被拒绝并附理由。

4. 不建议无脑全量写

把每轮对话和每个工具返回都直接塞进长期记忆，会迅速导致污染和检索噪声。

二）什么时候读取

1. 任务初始化时读用户相关记忆

例如用户偏好、所在项目、常用格式。

2. 规划前读任务经验

如果这是一个以前做过很多次的任务，可以在 planning 前先取回以往成功经验。

3. 执行受阻时读失败案例

当某工具失败，可以查历史上类似失败是如何处理的。

4. 最终输出前读交付偏好

确保回答格式与用户历史偏好一致。

一个成熟的表达是：memory 应当按任务阶段分层读取，而不是每次调用把所有记忆一股脑塞进 prompt。

七、Memory 与 RAG 的区别：高频必答题

一个合格回答至少要包含下面这几条：

1. 数据对象不同

- RAG：客观文档、知识库、公共资料、业务资料；
- Memory：个体历史、任务经验、用户偏好、系统经历。

2. 更新机制不同

- RAG：通常由知识库构建、同步或文档更新驱动；
- Memory：更多由交互事件和任务执行过程驱动。

3. 使用目的不同

- RAG：补足模型不知道或不可靠的外部知识；
- Memory：让系统具备持续性和个体化。

4. 生命周期不同

- RAG 文档可能是长期共享的；
- Memory 可能是用户级、任务级、会话级，有明确作用范围。

5. 风险不同

- RAG 的风险更多是召回不准、版本过时、权限过滤错误；
- Memory 的风险更多是污染、过时偏好、错误经验固化、隐私问题。

一句话总结：RAG 更像查资料，Memory 更像有过经历。

八、Memory 的实现思路：不要只会说向量库

1. 不是所有 memory 都适合向量检索

用户偏好、角色信息、配额、当前审批状态，这些更适合结构化存储。

情节记忆和经验摘要则更适合文本检索或向量检索。

因此，memory 常常是混合存储，而不是单一向量库。感兴趣的可以看下nanobot

2. 记忆条目最好结构化

推荐包括：

- memory_id
- 类型 (user/episodic/semantic/procedural)
- 作用域 (session/user/project/org)
- 内容摘要
- 证据来源
- 置信度
- 更新时间
- 失效时间
- 可见权限
- 写入原因

3. 需要记忆压缩与摘要

长历史不能全量保留为原文。

应定期把冗长历史总结成高价值摘要，必要时保留指向原始日志的链接。

4. 需要评分与遗忘机制

并非越多越好。

可以按最近使用、成功贡献、人工确认程度、置信度进行排序或衰减。

九、Memory 的典型失败模式

失败模式一：记忆污染

系统把错误结论或幻觉写进长期记忆，下次反而更自信地重复错误。

解决：记忆写入前做验证；高价值记忆要求工具证据或人工确认。

失败模式二：记忆过时

用户偏好变了，业务规则更新了，但旧 memory 还在影响当前决策。

解决：时间戳、TTL、版本号、最后验证时间。

失败模式三：记忆检索噪声大

写了太多无关历史，检索时高价值内容淹没在噪声里。

解决：精写少写，分类型索引，按阶段选择性读取。

失败模式四：记忆泄露权限

把不该跨用户共享的 memory 当成公共知识召回。

解决：严格作用域、租户隔离、可见性策略。

失败模式五：把日志当 memory

日志适合审计，不适合直接进入模型上下文。

需要先提炼成可消费的记忆。

十、Memory 与 Human-in-the-Loop 的关系

当人类审批、编辑、驳回或接管时，系统其实获得了非常高质量的反馈信号。

这些反馈如果能沉淀为 memory，就能让 agent 逐渐知道这个组织怎样工作。

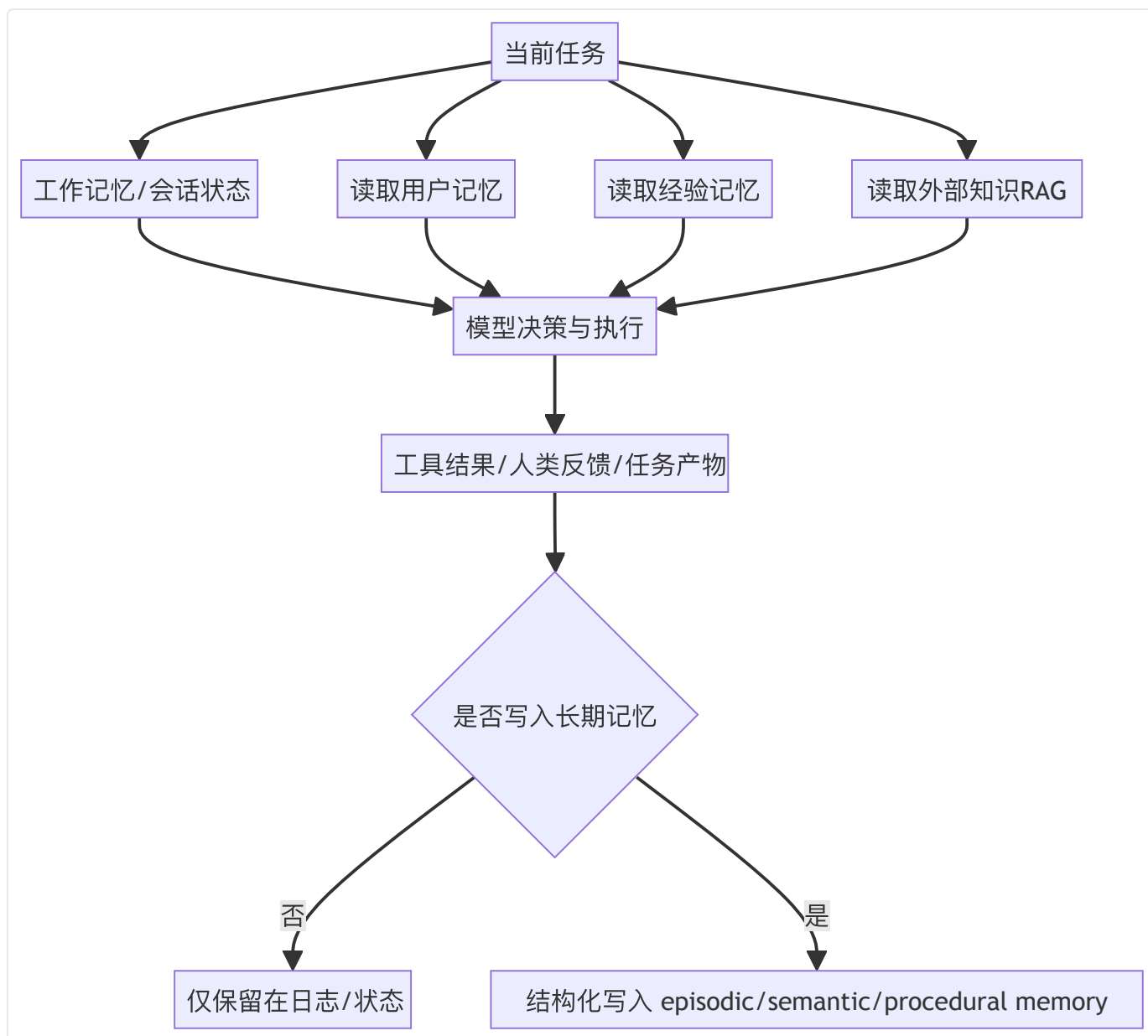
例如：

- 某类邮件正式程度要更高；
- 某位领导特别在意数据来源；
- 某类工单必须先核验身份再处理；

这些都不一定来自公开文档，却是极高价值的程序性或语义记忆。

当然，也要注意隐私、过时和组织边界，不是什么都该写。

十一、Memory 分层与读写路径



十二、高频面试题与参考回答

题 1: Agent 的 memory 是什么？

参考回答：

是对未来决策有价值、可跨步骤或跨任务持续利用的信息体系，而不只是历史聊天记录。

题 2: memory 和 context 有什么区别？

参考回答：

Context 是当前调用时送进模型窗口的信息；memory 是系统可供未来检索和利用的持久化信息源。

memory 只有被取回后才进入 context。

题 3: memory 和 RAG 有什么区别?

参考回答:

RAG 面向外部知识召回, memory 面向个体化经历和系统经验延续。一个解决找知识, 一个解决留经验。

题 4: 为什么不能把所有历史都直接存成长期记忆?

参考回答:

因为会产生污染、噪声和过时问题。memory 需要提炼、验证、分层和遗忘机制。

题 5: 长期记忆最重要的工程点是什么?

参考回答:

作用域、写入策略、读取策略、有效期、验证机制和权限隔离。

题 6: 什么是 episodic memory 和 semantic memory?

参考回答:

episodic 是具体事件经历, semantic 是从多次经历中抽象出的稳定知识或偏好。

题 7: 什么是 procedural memory?

参考回答:

是怎么做的经验或技能模板, 例如某类任务的优先步骤和稳定策略。

题 8: 人类反馈为什么适合写入 memory?

参考回答:

因为这是高质量监督信号, 能帮助 agent 学到组织偏好和有效操作模式, 但要注意作用域和过时性。

题 9: 如果 memory 用错了会有什么后果?

参考回答:

会导致错误经验固化、用户偏好过时、权限泄露、上下文噪声大和系统行为越来越偏。

题 10：一句话总结 memory 设计原则？

参考回答：

不是记得越多越好，而是要记真正有用、可验证、可控作用域、能在合适时机被正确取回的信息。

十三、面试速记版

- Memory 不是聊天记录，也不是所有持久化状态。
- 区分 context、state、memory、RAG。
- 常见类型：working、episodic、semantic、procedural、user memory。
- 写入要克制，读取要按阶段。
- 不是所有 memory 都该放向量库，很多应结构化存。
- 需要验证、TTL、作用域、遗忘和权限隔离。
- RAG 查知识，memory 留经历。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》, 2024。适合用来界定 agent 与 workflow 的边界，强调能用简单工作流解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》, 2025。非常适合补充agent 不只是 prompt engineering，而是 context engineering的近年观点。
3. Anthropic, 《How we built our multi-agent research system》, 2025。适合放在多 agent、任务拆解、subagent 协作章节中。
4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。
6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。
8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。

11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。
15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。

07 Multi-Agent 架构

一、Multi-Agent 既热门又危险

多 Agent 是近两年 Agent 面试里最容易被候选人讲得很热闹、却最容易被面试官一追问就露怯的话题。因为表面上看，多 agent 很符合直觉：人类团队靠分工协作完成复杂任务，那么智能体系统当然也可以拆成研究员、规划者、执行者、审稿人、工具专家、代码专家，各司其职。很多论文、框架和产品也都在强调角色分工、子 agent 协作、并行执行和群体智能。

但真实工程里的问题是：**多 agent 并不是更高级的单 agent**，它只是另一种组织复杂性的方式。

它可能带来更好的上下文隔离、更明确的角色职责、更容易并行的执行路径，也可能带来更高的通信成本、更难调试的错误链、更严重的上下文漂移、更多 token 和更长时延。

所以面试官问 multi-agent，不是在考你知不知道几个框架，而是在考：

1. 什么时候应该坚持单 agent；
2. 什么时候分 agent 才真正有收益；
3. 分完之后上下文、职责、权限、终止和治理怎么设计；
4. 多 agent 为什么常常比想象中更脆弱。

这也是为什么真正好的回答，一定不是一个主 agent 调多个子 agent 就行，而是要回答拆分的依据是什么，拆分后新引入了哪些问题。

二、为什么不直接用更强的单 Agent

这是多 agent 面试题里最重要的一问。正确的默认姿势通常是：**先把单 agent 做到位，再证明多 agent 有必要。**

为什么？因为单 agent 的优势非常明显：

1. 系统更简单

只有一个主要决策者，状态与 trace 更集中，调试链路更短。

2. 上下文共享天然完整

不需要在 agent 间传递信息，也不用担心信息丢失、误解和角色漂移。

3. 成本和时延通常更低

多 agent 需要额外的通信、角色提示词、任务分发、结果归并，token 消耗通常更高。

4. 评测更容易

当失败发生时，更容易定位是模型能力问题、工具问题还是 prompt 问题。

而多 agent 的失败常常是某个 agent 的中间输出误导了另一个 agent。

所以面试时非常加分的一句话是：多 agent 不是默认答案，它是一种在单 agent 开始出现明显瓶颈时才值得引入的架构。

三、那什么时候多 Agent 才值得做

多 agent 真正有价值，一般出现在下面几类情况。

1. 上下文太杂，需要隔离

单 agent 面对复杂任务时，往往需要同时记住任务目标、工具说明、用户偏好、多个中间结果、多个候选路径。

上下文一旦变得过于拥挤，模型决策质量就会下降。

这时把不同职责拆给不同 agent，可以降低单个 agent 的上下文负担。

例如：

- 研究 agent 专门负责搜集材料；
- 分析 agent 专门负责归纳比较；
- 写作 agent 专门负责成稿；
- 审核 agent 专门检查证据和格式。

每个 agent 只看到与自己任务强相关的信息，会更稳定。

2. 任务天然存在专业化角色

有些任务里，角色边界本身就很清晰。

例如软件工程任务里可以有：

- 问题定位 agent；
- 代码修改 agent；
- 测试验证 agent；

- PR 总结 agent。

这类专业化分工有助于不同子任务采用不同模型、不同工具、不同 prompt 和不同校验方式。

3. 任务可并行

如果任务中有多个独立信息源或子任务，拆成多个 sub-agent 并发执行有可能明显降低时延。

比如多来源检索、多文档分析、多仓库扫描、多网页观察。

4. 团队协作与模块化开发需求强

在大型工程团队里，多 agent 也有组织上的好处：不同模块可以独立开发、测试和维护。

此时多 agent 不只是推理上的好处，也是软件架构上的好处。

5. 需要不同能力边界和权限边界

有的 agent 只需要读权限，有的 agent 可以提交草稿，有的 agent 能操作代码沙箱。

通过 agent 分工，有时比在一个大 agent 内部做复杂权限表更清楚。

四、Multi-Agent 的几种典型架构（重要）

1. Manager-Worker（主管—执行者）

最经典。

一个 manager 负责理解总目标、拆解任务、分派给多个 worker，并聚合结果。

优点

- 管理结构清晰；
- 容易加入高层 planning；
- 便于统一收口和最终校验。

缺点

- manager 容易成为瓶颈；
- 如果 manager 拆解错了，整个系统都会偏；
- worker 之间协作通常较弱，容易信息孤岛。

适用场景

- 总目标明确；
- 子任务可清晰切分；
- 需要中心化控制。

2. Handoff（接力式切换）

当前 agent 在判断接下来应该由谁负责后，把控制权交给另一个 agent。这更像客服转接或角色接力。

优点

- 更符合某些业务流程；
- 每次只有一个当前主 agent，状态更容易理解；
- 适合阶段式任务。

缺点

- 接力边界设计不好会频繁来回切换；
- 上下文传递质量决定效果；
- 容易出现没人真正总览全局的问题。

适用场景

- 角色阶段性明显，例如售前 → 实施 → 售后；
- 每个阶段有主导角色。

3. Router + Specialist

先有一个路由器，根据任务类型把请求送到最合适的专家 agent。这更像多 agent 中的意图分发。

优点

- 结构简单；
- 易于扩展专家 agent；
- 适合任务类型分布清晰的场景。

缺点

- 主要解决路由，不一定解决复杂协作；
- 路由错了整个路径就偏。

适用场景

- 多模态、跨领域、多业务线系统。

4. Blackboard / Shared Workspace

多个 agent 围绕一个共享任务板、共享记忆区或共享文档协作。

例如每个 agent 可以往公共板上写结论、证据、待办，其他 agent 再消费。

优点

- 有利于并行协作；
- 中间产物可复用；
- 有机会减少点对点通信。

缺点

- 共享区容易污染；
- 谁负责最终裁决不清楚时会混乱；
- 对状态管理要求高。

5. Hierarchical Planning + Subagents

高层 planner 只做任务图和资源分配，具体子任务交给 specialized subagents。

这种模式常见于复杂研究或代码任务。

五、Multi-Agent 的核心设计点：拆解并不是那么简单的

1. 拆分依据必须明确

常见拆分维度有：

- 按任务阶段拆；

- 按专业能力拆；
- 按数据源拆；
- 按风险级别拆；
- 按权限边界拆。

如果拆分依据不清，多 agent 只会变成多几个 prompt。

2. Agent 间传什么，不传什么

这是最关键的工程问题之一。

如果把全部上下文都传给每个 agent，就失去了拆分意义；

如果传得太少，又会让下游 agent 决策失真。

更稳妥的做法是传结构化任务说明、必要约束、关键证据摘要和明确的 expected output，而不是原样转发全部历史。

3. 共享状态如何管理

系统需要明确：

- 哪些状态是全局共享；
- 哪些只属于某个 agent；
- 谁可以写共享 memory；
- 谁拥有最终裁决权。

4. 结果如何归并

多个 worker 返回的结果可能冲突、重复、质量不齐。

需要有 aggregator 或 reviewer 做统一整合、冲突解决和证据校验。

5. 出错时怎么定位

多 agent 常见难题是错误传递链：

A 拆错任务 → B 根据错任务输出 → C 又据此做总结，最后看起来像 C 答错了，但真正问题在 A。

所以 trace 必须能记录：

- 谁分派了什么；
- 传了哪些上下文；
- 下游基于什么证据做决定。

六、为什么 Multi-Agent 容易失败

这是很重要的面试加分点。不要只讲优点，一定要讲失败模式。

1. 通信损失

Agent 之间传递的信息往往是压缩后的，而压缩一定有损。
如果任务说明不完整，下游 agent 可能在错误前提上行动。

2. 角色漂移

一个 agent 本该只做事实收集，结果开始擅自做结论；
或者 reviewer 不只审核，还重新发明新方案。
职责不清会导致系统行为失控。

3. 错误放大

单 agent 出错通常是一处错误；多 agent 出错可能层层放大。
尤其是上游错误被下游当成事实时，问题更严重。

4. 成本高

每个 agent 都有自己的系统提示词、上下文和输出，通信越多，成本越高。

5. 调试困难

你要 debug 的不再是一个模型调用，而是一个网络。
问题可能来自拆分、传递、归并、顺序、权限、上下文摘要等多个环节。

6. 收益不一定显著

在很多任务上，一个更强的单 agent 外加好的工具、planning 和状态管理，效果可能比多 agent 更好、更便宜、更稳。
所以多 agent 必须证明自己的边际收益。

七、如何判断是否该上 Multi-Agent

面试里你可以给出一个判断框架

维度一：单 agent 是否已经出现明显瓶颈

- 上下文过载？
- 角色冲突？
- 难以并行？
- prompt 越写越长、越不稳定？

维度二：任务是否存在自然分工

- 不同步骤是否需要不同能力？
- 是否可以清晰定义输入输出？

维度三：拆分后的通信成本是否可接受

- 子任务结果能否结构化传递？
- 是否会产生大量往返？

维度四：是否有明确的治理需求

- 不同 agent 是否对应不同权限或审批要求？

维度五：收益是否大于复杂度

- 成功率是否显著提升？
- 时延是否下降？
- 可维护性是否变好？

八、多 Agent $\neq$ 人人都是完整 Agent

真实系统里，很多所谓multi-agent其实是不同等级的 agentic component：

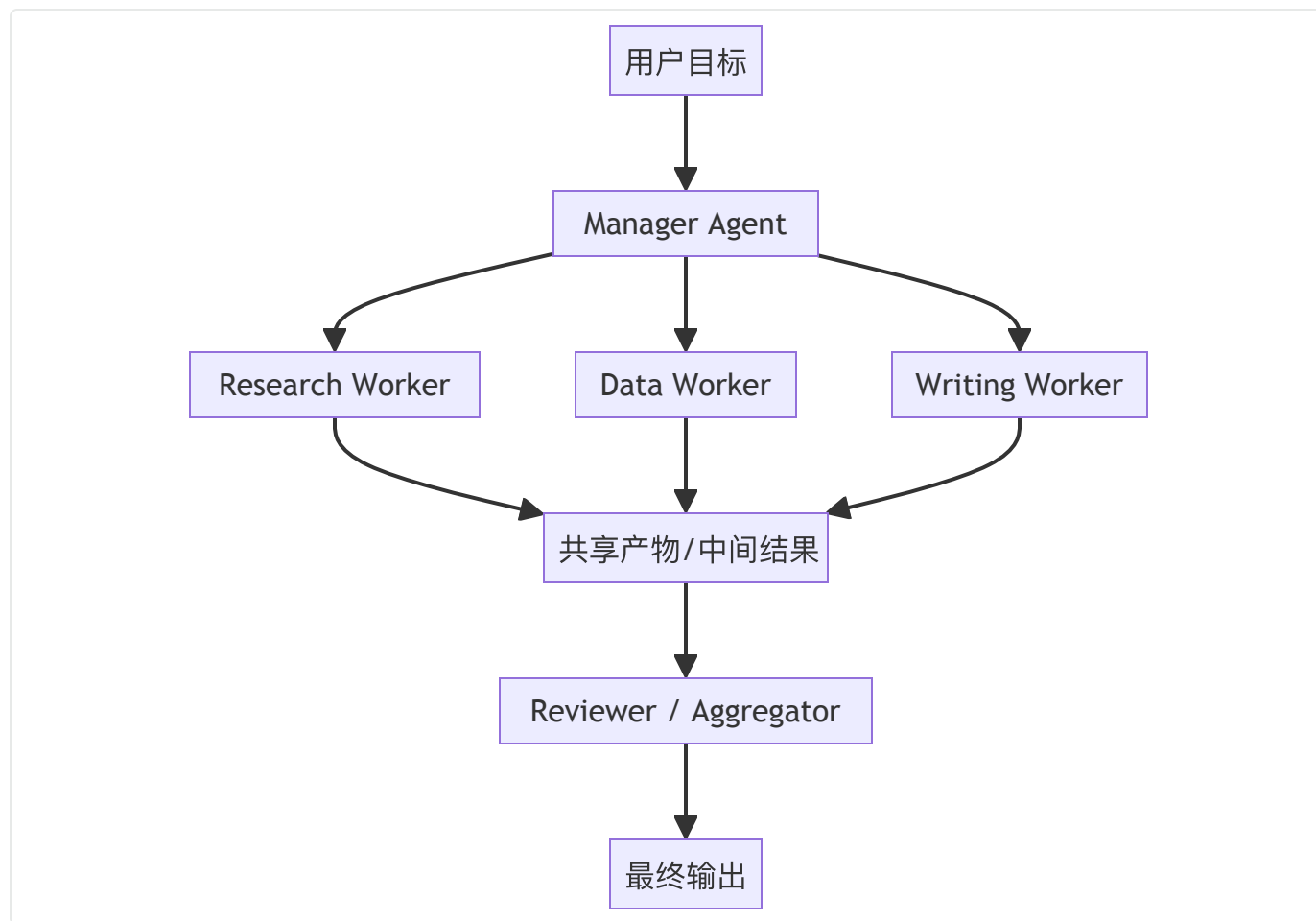
- 有的是完整自治 agent；
- 有的是带工具的小专家节点；
- 有的是只做分类或评审的 evaluator；

- 有的是 skill 或 subroutine。

所以你可以说：

多 agent 系统不一定由多个平权智能体构成，很多时候是中心化 runtime 管理若干不同自治程度的子单元。

九、典型的 Manager-Worker Multi-Agent



十、高频面试题与参考回答

题 1：为什么不直接用一个更强的单 Agent？

参考回答：

因为多 agent 会引入通信成本、调试难度和额外上下文开销。只有当单 agent 已经在上下文管理、角色分工或并行需求上出现明显瓶颈时，多 agent 才值得引入。

题 2：多 Agent 的主要收益是什么？

参考回答：

上下文隔离、专业化角色分工、并行执行、模块化开发，以及在某些任务上的更清晰责任边界。

题 3：Manager-Worker 和 Handoff 有什么区别？

参考回答：

Manager-Worker 是中心化分派和汇总；Handoff 是控制权在不同 agent 间接力式转移，通常每个阶段只有一个主导 agent。

题 4：多 Agent 最难的问题是什么？

参考回答：

上下文如何传递、共享状态如何管理、结果如何归并，以及错误发生时如何定位是哪一层出了问题。

题 5：什么时候多 Agent 反而会更差？

参考回答：

任务本身不复杂、通信成本高、上下文摘要有损、角色边界不清，或者一个强单 agent 已能胜任时。

题 6：如何设计 agent 之间的通信？

参考回答：

尽量结构化传递任务说明、关键约束和证据摘要，而不是原样搬运全部历史。让输入输出协议清晰、可验证、可追踪。

题 7：多 Agent 如何做权限控制？

参考回答：

按角色划分工具与权限，不同 agent 只暴露最小必要能力；高风险 agent 或动作要额外审批和审计。

题 8：多 Agent 如何做结果归并？

参考回答：

需要 reviewer 或 aggregator 对冲突、重复和证据质量做统一处理，而不是简单拼接。

题 9：怎样判断是否应该上多 Agent？

参考回答：

先看单 agent 是否已遇到上下文过载、专业化冲突或并行瓶颈；再看拆分后通信成本和治理复杂度是否可接受。

题 10：一句话总结你对 Multi-Agent 的态度？

参考回答：

它是解决复杂性的一种架构手段，但只有在分工收益明显大于通信与治理成本时才值得做。

十一、面试速记版

- 多 agent 不是默认答案，先把单 agent 做好。
- 价值点：上下文隔离、专业化、并行、模块化。
- 主要模式：manager-worker、handoff、router+specialist、shared workspace。
- 真正难点：通信、共享状态、归并、调试。
- 常见失败：角色漂移、错误放大、成本高、收益不足。
- 不是拆得越多越好，拆分必须有明确依据。
- 多 agent 的引入，需要证明边际收益。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》, 2024。适合用来界定 agent 与 workflow 的边界，强调能用简单工作流解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》, 2025。非常适合补充 agent 不只是 prompt engineering, 而是 context engineering 的近年观点。
3. Anthropic, 《How we built our multi-agent research system》, 2025。适合放在多 agent、任务拆解、subagent 协作章节中。
4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。
6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。

8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。
11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。
15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。

08 Human-in-the-Loop

一、为什么 Human-in-the-Loop 不是最后点一下确认

很多候选人一听到 Human-in-the-Loop，第一反应就是高风险动作前让用户确认一下。这当然没错，但如果面试里只答到这里，通常不够。因为在真正的 agent 系统里，Human-in-the-Loop (HITL) 不是一个孤立按钮，而是一套把人类判断力、责任边界和系统自治能力结合起来的设计机制。

面试官问这部分，通常是在考几个层面：

1. 你是否理解 agent 不应在所有场景都全自动；
2. 你是否知道人类应在什么节点介入，而不是只会说最后确认；
3. 你是否能把人工审批、编辑、接管、纠错、恢复运行这些过程设计成系统机制；
4. 你是否能考虑审计、责任、用户体验与效率之间的平衡。

对于企业级 Agent，这一题非常关键。原因很简单：真正高价值的动作往往带有风险，例如发邮件、批量更新数据、审批报销、触发退款、执行脚本、修改代码、发布内容、调用内部系统。纯自治当然很酷，但真正能上线的系统，通常需要在关键节点让人类保有最终决定权，或者至少保有纠偏权。

二、Human-in-the-Loop 到底是什么

Human-in-the-Loop 可以理解为：在 agent 的运行过程中，让人类在关键节点对计划、动作、状态或结果进行审批、编辑、驳回、澄清或接管。

这里有两个关键词很重要：

1. 关键节点

不是每一步都让人看，否则自动化就没意义；

也不是完全不让人看，否则风险不可控。

所以核心问题不是要不要有 HITL，而是插在哪些节点。

2. 介入形式

人类介入不只有 approve。常见形式还包括：

- reject：拒绝执行；
- edit：修改计划、参数、结果；

- clarify: 补充信息;
- takeover: 直接接管;
- override: 覆盖系统结论;
- defer: 延后、挂起、等待更多信息。

三、为什么 Agent 系统需要 HITL

1. 风险控制

对有副作用、高成本、不可逆的动作，人类审批是最直接的安全阀。
例如发正式邮件、执行退款、删除数据、提交代码、修改生产配置。

2. 责任与合规

在很多企业场景里，决策责任不能完全交给模型。
审批、审计、授权链路都要求有人类责任主体。

3. 信息不完备

有些时候 agent 缺少上下文，而人类知道关键隐含信息。
例如这个客户虽然满足规则，但最近在谈续约，先不要发强硬催款邮件。

4. 组织偏好与风格校正

模型能生成正确的东西，不代表符合组织风格。
人类反馈可以把组织规范逐渐显式化。

5. 逐步建立信任

很多系统上线初期会采用更强 HITL，等积累足够评测数据和信任后，再逐步放宽自治范围。
所以 HITL 也是一种渐进式发布策略。

四、Human-in-the-Loop 的几种典型介入点

这是面试中最应该展开的部分。

1. 计划审批 (Plan Approval)

在执行前先让人看计划是否合理。

适合：

- 长任务；
- 高成本任务；
- 涉及多个系统或多个角色的任务；
- 任务目标模糊，容易跑偏。

优势是错得早发现，而不是等动作已经做出去再后悔。

2. 工具动作审批 (Action Approval)

对高风险工具调用，在执行前让人审批。

例如：

- 发送邮件；
- 修改数据库；
- 创建或关闭工单；
- 发布内容；
- 执行高权限命令。

这是最常见、也最容易理解的一类 HITL。

3. 中间结果审核 (Checkpoint Review)

长任务执行到阶段节点时，让人检查当前方向是否正确。

适合复杂研究、长报告、多阶段项目任务。

4. 最终结果确认 (Final Sign-off)

输出前让人确认内容质量与业务风险。

适合对外发送、正式入库、提交给上级的内容。

5. 异常升级 (Exception Escalation)

当系统遇到未知错误、权限不足、工具反复失败、冲突证据时，自动升级给人类。

这类 HITL 的意义在于：人类不是一直盯着，而是在系统真正需要时介入。

6. 交互式澄清 (Clarification Loop)

有些介入不是审批，而是补上下文。

例如 agent 发现最近销售情况这个说法太模糊，于是主动向人类询问时间范围、指标口径或输出格式。

五、Approve / Edit / Reject：为什么三种都要支持

很多系统只做 approve/reject，这其实不够好。

因为在真实场景里，人类最常做的往往不是同意或完全拒绝，而是改一下。

1. Approve

直接通过。

适合低风险或结果已经很接近要求的情况。

2. Edit

人类修改计划、参数或输出，然后系统继续执行。

这是非常高价值的模式，因为它把人类经验转化成机器后续可执行的输入。

例如把邮件语气改得更正式，或把查询时间范围从最近改成过去 30 天。

3. Reject

彻底驳回该动作或该方案。

系统此时应支持：

- 停止；
- 返回上一步；
- 触发 replanning；
- 要求人类补充理由。

优秀系统通常还会把 reject 的原因沉淀为经验信号，帮助未来少犯类似错误。

六、Human-in-the-Loop 不只是 UI 功能，它和 Runtime 深度耦合

面试里很加分的一点是：HITL 的本质不是一个前端弹窗，而是 runtime 的中断—等待—恢复能力。

一个真正可用的 HITL 系统至少需要支持：

1. 中断 (Interrupt)

当触发审批点时，agent 执行必须暂停，而不是继续偷偷往下跑。

2. 状态持久化

暂停时必须保存：

- 当前任务状态；
- 即将执行的动作；
- 上下文快照；
- trace；
- 等待哪个人或角色审批。

3. 恢复 (Resume)

审批通过、编辑或拒绝后，系统需要从正确位置恢复，而不是重头再跑。

这通常要求 durable execution 或检查点机制。

4. 分叉处理

人类 edit 后，系统应能基于新输入继续，而 reject 则可能需要回退或重规划。

这意味着状态机里要有明确分支，而不是简单把审批结果当文本拼回 prompt。

七、如何决定哪些节点要插 HITL

这是一个很典型的设计题。比较成熟的回答通常会从风险—成本—不确定性三维来讲。

1. 风险高就插

动作不可逆、对外可见、涉及金钱、隐私、权限、生产环境时，应强制审批。

2. 不确定性高就插

例如信息不足、证据冲突、任务目标模糊、模型置信度低、历史上失败率高。
这类场景不一定危险，但很容易跑偏。

3. 成本高就插

某些动作一旦执行会带来较大资源消耗，如批量运行测试、调用昂贵服务、长时间浏览器自动化。
即使无风险，也可能需要人确认是否值得继续。

4. 组织要求就插

有些组织流程天然要求审批。
不要试图让 agent 绕开业务制度。

八、Human-in-the-Loop 与用户体验的平衡

一个成熟回答不能只讲安全，还要讲体验。
因为审批太多，系统就变成人工流水线；审批太少，又失去治理。

1. 不要把低风险动作全部拦住

例如普通检索、只读查询、草稿生成，通常无需审批。

2. 批处理审批

多个相似动作可以合并审批，减少打扰。

3. 提供充分上下文

审批界面应展示：

- 为什么建议执行这个动作；
- 参数是什么；
- 风险是什么；
- 预期结果是什么；
- 可选修改点是什么。

否则人类也很难判断。

4. 支持异步审批

不是所有审批都要实时。

长任务和跨团队流程往往需要挂起等待。

5. 支持角色审批

不同动作可能需要不同审批人，而不一定是最终用户本人。

九、HITL 与评测、学习闭环的关系

这是很多候选人不会主动提，但非常加分的点。

人类审批、编辑、驳回，实际上构成了非常高质量的监督数据：

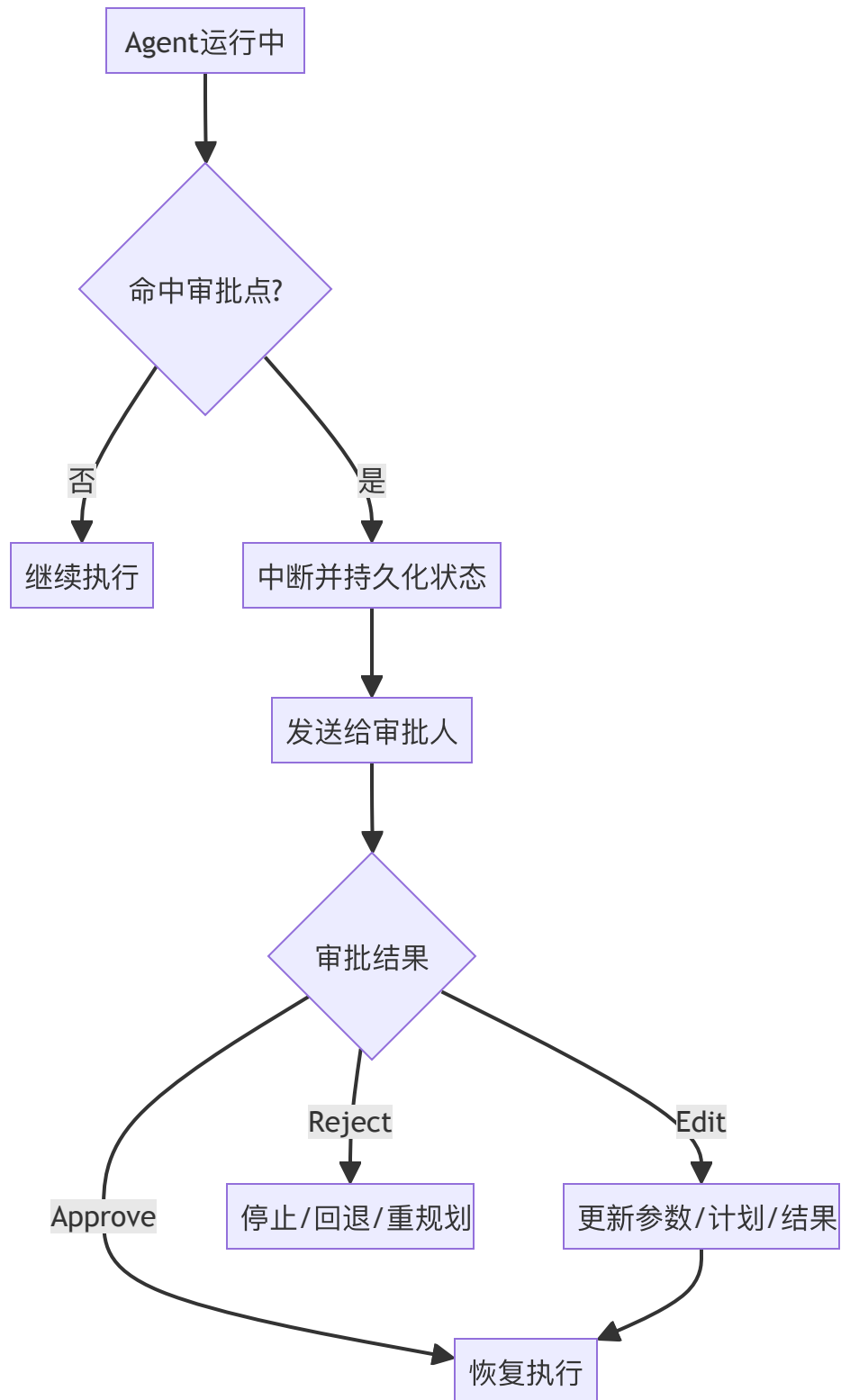
- 哪些动作经常被驳回；
- 哪些计划总被改；
- 哪些参数经常填错；
- 哪类结果人类总要补证据。

这些数据既可以用于：

- 优化 prompt；
- 改进 schema；
- 调整审批点；
- 训练或微调评分器；
- 更新 memory。

所以 HITL 不只是让系统更安全，也能让系统越来越懂组织习惯。

十、HITL 中断与恢复链路



十一、高频面试题与参考回答

题 1: 什么是 Human-in-the-Loop?

参考回答：

是在 agent 运行过程中引入人类对计划、动作、状态或结果的审批、编辑、驳回、澄清或接管，从而在保留自动化收益的同时控制风险和责任边界。

题 2：为什么 HITL 不是只做最后确认？

参考回答：

因为很多错误应该在计划阶段或动作阶段更早暴露；有些场景需要中间 checkpoint 或异常升级；只在最后确认可能已经太晚。

题 3：哪些动作最适合强制审批？

参考回答：

对外发送、支付退款、删除修改数据、执行高权限命令、发布内容、提交代码、修改生产配置等高风险副作用动作。

题 4：为什么要支持 approve/edit/reject 三种模式？

参考回答：

因为真实业务里人类最常做的是改一下，而不是纯通过或纯拒绝。edit 能把人类经验转成系统可继续执行的输入。

题 5：HITL 对 runtime 有什么要求？

参考回答：

必须支持中断、状态持久化、等待审批、基于审批结果恢复执行，以及 reject 后的回退或重规划。

题 6：如何决定哪些节点插入 HITL？

参考回答：

看风险、成本、不确定性和组织制度要求。不是所有节点都要插，也不是只看最终输出。

题 7：HITL 会不会损失自动化效率？

参考回答：

会，所以要精确地把它插在真正关键的节点，并对低风险动作减少打扰。好的设计是在安全和效率间做平衡，而不是一味加审批。

题 8：HITL 数据有什么额外价值？

参考回答：

审批、编辑、驳回数据是高质量监督信号，可用于优化 prompt、工具 schema、风险规则、memory 和评测体系。

题 9：如果审批人迟迟不处理怎么办？

参考回答：

系统应支持挂起、超时、提醒、转派和部分完成状态，而不是无限等待或默认继续执行。

题 10：一句话总结你对 HITL 的理解？

参考回答：

它是把人类判断力嵌入 agent runtime 的机制，不是一个简单的确认按钮。

十二、面试速记版

- HITL 的本质是把人类判断嵌入 agent 运行闭环。
- 介入形式不只有 approve，还有 edit、reject、clarify、takeover。
- 介入点包括计划、动作、中间检查点、最终结果和异常升级。
- HITL 依赖 runtime 的中断、持久化和恢复能力。
- 不是审批越多越好，要平衡安全与效率。
- 人类反馈是高质量监督数据。
- 企业级 agent 几乎都离不开 HITL。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》，2024。适合用来界定 agent 与 workflow 的边界，强调能用简单工作流解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》，2025。非常适合补充 agent 不只是 prompt engineering，而是 context engineering 的近年观点。
3. Anthropic, 《How we built our multi-agent research system》，2025。适合放在多 agent、任务拆解、subagent 协作章节中。

4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。
6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。
8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。
11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。
15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。

09 Agent 状态机与终止条件

一、状态机与终止条件是最能区分工程深度的话题

很多校招生在 Agent 面试里会把注意力集中在 prompt、规划和工具上，但真正懂工程的面试官通常还会问一个看似不起眼、实则非常关键的问题：你的 agent 怎么知道什么时候该停？任务中断后怎么恢复？为什么它不会死循环？

这就是状态机与终止条件。

从系统视角看，一个 agent 只要进入多步运行，就不再是一次函数调用，而是一个持续演化的状态系统。只要是状态系统，就必然有：

- 当前处于什么阶段；
- 下一步允许去哪里；
- 哪些条件下停止；
- 出错后如何回退或挂起；
- 被人工中断后怎样恢复；
- 失败和成功如何判定。

如果这些东西不显式建模，系统初期可能还能跑 demo，但一旦接入真实工具、异步任务、审批流、长任务和多 agent，很快就会出现下面这些问题：

- 在两个工具之间反复跳；
- 因一次暂时失败而无休止重试；
- 中途崩掉后无法从上一步继续；
- 人类审批后不知道恢复到哪一步；
- 完成标准模糊，明明该停了却还在探索；
- 状态混乱，日志里看不出任务到底发生了什么。

所以这道题真正考的是：你能不能把 agent 当成一个可控的运行系统而不是一个会自己想的 prompt。

二、为什么 Agent 一定要有显式状态

1. 多步任务天然有阶段

例如一个企业知识 agent 可能经历：

- 初始化；
- 澄清需求；
- 规划；
- 检索；
- 工具执行；
- 验证；
- 等待审批；
- 汇总输出；
- 结束。

如果没有显式状态，系统就很难判断当前应该做什么、能做什么、不能做什么。

2. 状态决定可恢复性

当一个长任务执行到一半失败，如果你不知道已经完成了哪些步骤、哪些产物已经落库、哪些审批已通过，就无法安全恢复。

3. 状态是治理的抓手

权限、预算、超时、审批、人工介入、日志审计，都需要以状态为依附。

例如等待审批状态就天然意味着不能继续自动执行高风险动作。

4. 状态是防循环和防越界的重要手段

系统不能只靠模型自己判断该停了。

程序层必须有明确边界：某些状态下只允许某些动作，某些条件下必须终止或升级。

三、什么是 Agent 状态机

一个简洁但很有力的定义是：

Agent 状态机就是把 agent 运行过程表示为一组离散状态以及状态之间在特定条件下的转移规则。

这不是说 agent 要做成非常僵硬的传统有限状态机，而是说：

即使内部某些步骤由模型自由决策，系统外层仍应有明确的阶段与转移控制。

这就是局部自治，外层受控。

典型状态示例

- **INIT**：初始化与鉴权；
- **CLARIFYING**：向用户补充询问；
- **PLANNING**：生成或更新计划；
- **EXECUTING**：调用工具或执行子任务；
- **WAITING_HUMAN**：等待审批或输入；
- **VERIFYING**：检查结果与约束；
- **REPLANNING**：原路径失效后的重规划；
- **COMPLETED**：成功结束；
- **FAILED**：失败终止；
- **CANCELLED**：被用户或系统取消；
- **TIMEOUT**：超时收束。

这些状态不一定每个系统都要有，但你最好能说明：状态是服务于控制目标的，不是为了画图好看。

四、终止条件为什么不能只写max_steps

很多初学者会说给个最大步数就好了。当然最大步数是必要的，但远远不够。因为真正健康的 agent 终止条件应该是**多维的**。

1. 成功终止

最理想的终止条件是任务已经满足成功标准。

例如：

- 所需字段已全部收集；
- 验证器确认答案完整；
- 高风险动作已审批并成功执行；
- 结果已落库并通知成功。

2. 失败终止

当任务已不可继续推进，应明确失败。

例如：

- 必要资源不可用；

- 权限不足且无法升级；
- 多次修复无效；
- 输入本身不合法。

3. 预算终止

包括最大步数、时间预算、token 预算、费用预算。
这些都是防止无限探索和资源失控的硬边界。

4. 风险终止

当系统判断继续执行风险过高，如证据冲突、工具异常、副作用动作缺审批、越权可能性存在时，应停止或升级给人类。

5. 人工终止

用户取消、审批驳回、管理员中断。
这类终止必须是一级公民，而不是靠把一段话放进上下文让模型自己理解。

6. 无进展终止

如果连续若干步没有实质进展，例如重复查询、重复失败、重复在同一页面兜圈子，也应停止或重规划。
这是防循环的关键。

五、如何定义完成：这是面试里的隐藏难点

很多 agent 死循环，不是因为工具有 bug，而是因为完成这个概念没有被定义清楚。
例如让 agent 帮我调研一下，如果没有明确：

- 调研哪些维度；
 - 需要多少个来源；
 - 结果输出到什么粒度；
 - 是否必须交叉验证；
- 那系统就很难知道什么时候算够了。

所以一个成熟设计会在任务入口就定义 success criteria，例如：

- 至少 3 个可信来源；
- 覆盖价格、功能、市场定位三类字段；
- 输出一份表格和一段总结；
- 若有冲突信息必须标记不确定。

面试时非常加分的一句话是：

终止条件的前提是成功标准显式化，否则 agent 只会不断探索而不知道何时足够。

六、状态机设计中的几个关键实践

1. 状态与动作解耦

状态表示系统此刻处在哪个阶段，动作表示当前阶段可以采取什么操作。

不要把状态和动作混成一锅，否则图会非常乱。

2. 状态转移最好显式化

即使内部由模型决定某些分支，最终也最好映射到显式事件，例如：

- `tool_call_succeeded`
- `tool_call_failed`
- `approval_granted`
- `approval_rejected`
- `budget_exceeded`
- `goal_satisfied`

这样才能做可靠控制和回放。

3. 给每个状态定义不变量

例如：

- 在 `WAITING_HUMAN` 状态下，不允许继续执行高风险动作；
- 在 `COMPLETED` 状态下，只允许查询日志，不允许修改结果；
- 在 `FAILED` 状态下，如果要重试，必须经过新事件触发。

不变量是治理层的重要抓手。

4. 保存检查点

在状态迁移关键点保存检查点，尤其是：

- 规划完成后；
- 高风险动作前后；
- 审批前后；
- 阶段产物落库后。

这样系统崩溃或被中断后才能恢复。

5. 支持挂起与恢复

长任务、异步任务、人类审批，都意味着不是一路跑到结束。

状态机必须支持挂起，并在恢复时知道从哪里继续。

七、如何防止 Agent 死循环

这是非常高频的工程追问，必须答透。

1. 最大步数只是底线，不是全部

必须有，但不能只依赖它。

2. 检测重复动作

如果连续出现相同工具、相似参数、相同失败结果，要触发去重或升级。

3. 跟踪进展指标

例如：

- 新获取了多少有效证据；
- 是否推进了任务图；
- 是否减少了不确定性；
- 是否完成了新的子任务。

如果若干轮都没有进展，就停止或 replanning。

4. 摘要失败历史

让模型显式知道已经尝试过什么，为什么无效，可以减少重复犯错。

5. 引入 verifier

由规则或第二模型判断：当前是否应该结束、重试、换路、还是交给别人。

6. 对高风险动作引入审批或 dry-run

这能避免 agent 因为不确定而反复尝试真实动作。

八、异步任务、长任务与 Durable Execution

很多真实任务不是秒级完成的，例如长时间网页操作、数据管道、审批流等待、批量测试。

这时状态机必须和 durable execution 结合。

1. 什么是 durable execution

可以理解为：任务在中断、进程重启、等待外部事件后，仍能从保存的状态继续执行，而不是从头开始。

2. 为什么需要它

- 等待人类审批；
- 等待外部系统回调；
- 长耗时工具运行；
- 服务重启或容器迁移；
- 需要跨天执行。

3. 需要保存什么

- 当前状态；
- 历史事件；
- 中间产物引用；
- 已执行副作用动作记录；

- 预算消耗；
- 待恢复入口。

4. 恢复时要防什么

最重要的是防止副作用重复执行。

如果一个邮件已经发出，恢复时不能再发一次；

如果一个工单已经创建，也不能因为状态丢失再创建一遍。

所以 durable execution 必须和幂等机制结合。

九、状态机与 Human-in-the-Loop 的关系

这是很强的结合点。

HITL 本质上会引入一个非常重要的状态： `WAITING_HUMAN` 。

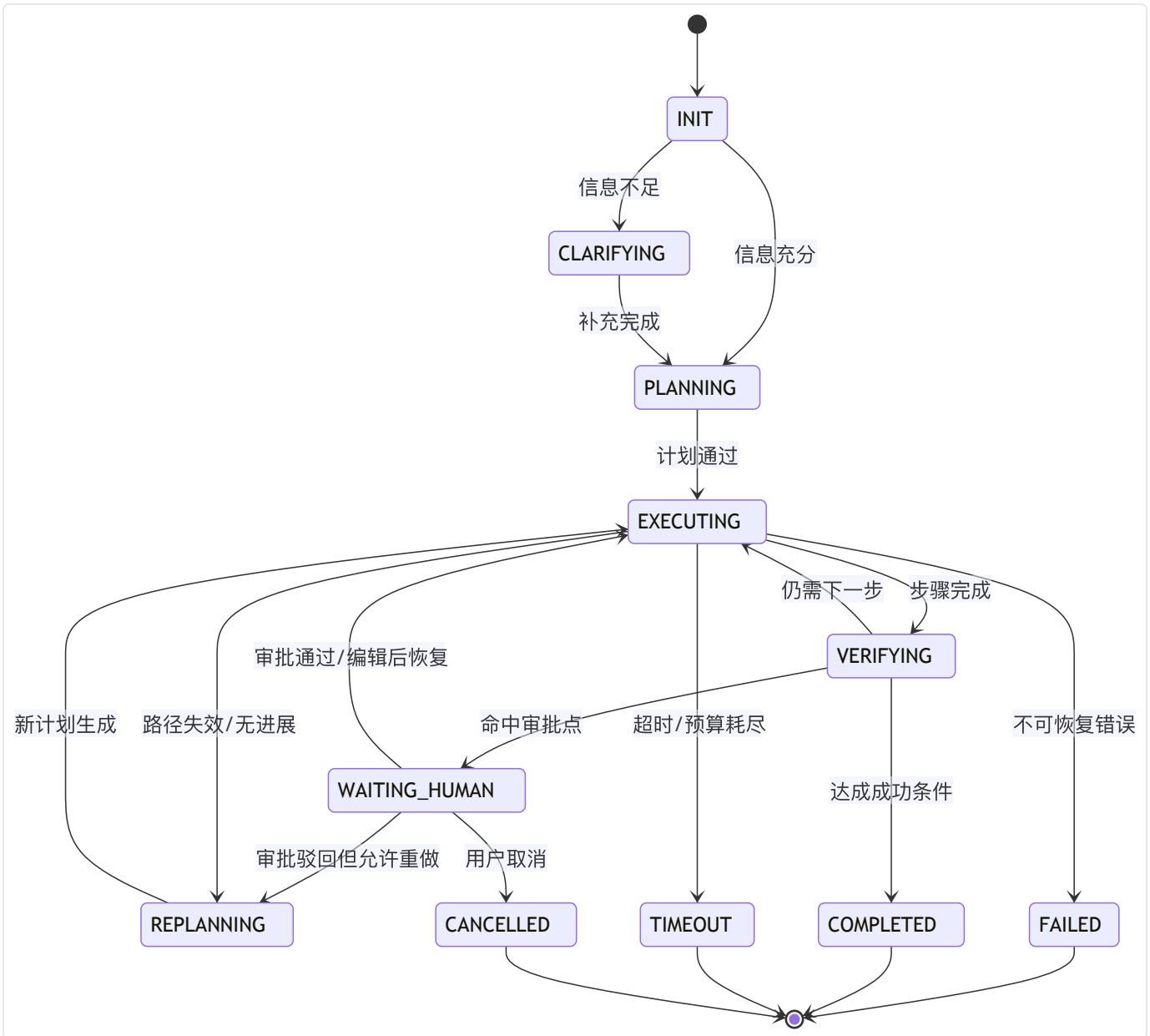
它说明系统当前不能继续自动推进，而必须等待某个事件。

审批通过会触发恢复，驳回会触发取消或重规划，编辑会触发状态更新后继续。

所以你可以说：

没有状态机，HITL 很容易沦为前端弹个框；有了状态机，HITL 才能成为 runtime 的一等机制。

十、Mermaid 图：状态机与终止路径



十一、高频面试题与参考回答

题 1：为什么 Agent 需要状态机？

参考回答：

因为多步运行意味着系统有阶段、有转移、有暂停与恢复需求。状态机能把这些阶段和允许的转移显式化，从而支持控制、治理、恢复和调试。

题 2：终止条件只设置 max_steps 够吗？

参考回答：

不够。还需要成功判定、失败判定、预算限制、风险终止、人工终止和无进展终止等多维条件。

题 3：如何定义任务完成？

参考回答：

要在任务入口显式定义 success criteria，例如覆盖范围、最少证据数、结果格式、校验要求，而不是让模型自己猜什么时候算够。

题 4：Agent 为什么会死循环？

参考回答：

常见原因包括完成标准不清、缺乏进展感知、重复工具调用、错误历史没有被总结、没有无进展终止和重规划机制。

题 5：如何防止死循环？

参考回答：

最大步数、时间和费用预算只是底线；还要检测重复动作、跟踪进展、总结失败历史、加入 verifier，并在必要时重规划或升级人工。

题 6：状态机和工具调用有什么关系？

参考回答：

状态机决定在什么状态下允许什么工具动作，以及动作成功或失败后状态如何转移。工具调用不是散点事件，而应纳入状态演化框架。

题 7：什么是 durable execution？

参考回答：

是在长任务、中断、等待审批或系统重启后，能基于持久化状态从正确位置继续执行的能力，而不是从头重跑。

题 8：恢复执行时最容易出什么问题？

参考回答：

副作用动作重复执行，所以必须结合检查点、幂等键和已执行动作日志。

题 9: Human-in-the-Loop 为什么需要状态机支持?

参考回答:

因为审批意味着系统进入等待状态, 审批结果会触发恢复、回退或取消。没有状态机就很难可靠处理中断和恢复。

题 10: 一句话总结你对状态机与终止条件的理解?

参考回答:

它们是把 agent 从会跑的 demo变成可控的生产系统的关键基础设施。

十二、面试速记版

- Agent 一旦多步运行, 就必须显式管理状态。
- 状态机解决阶段、转移、挂起、恢复、治理和回放问题。
- 终止条件应是多维的, 不只是 max_steps。
- 成功标准不清是死循环的重要根源。
- Durable execution 让长任务和审批流可恢复。
- 状态机是 HITL、重试、回滚、审批和幂等的共同基础。
- 这是区分会写 prompt和懂系统工程的关键题。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》, 2024。适合用来界定 agent 与 workflow 的边界, 强调能用简单工作流解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》, 2025。非常适合补充agent 不只是 prompt engineering, 而是 context engineering的近年观点。
3. Anthropic, 《How we built our multi-agent research system》, 2025。适合放在多 agent、任务拆解、subagent 协作章节中。
4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。

6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。
8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。
11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。
15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。

00-高频面试题（汇总）

一般都考察什么

对于知识/记忆/检索的考察表面上在问 RAG、Memory、检索优化、GraphRAG，实际上在考你有没有把一个知识系统拆成下面几层：

1. 知识从哪里来：外部知识库、数据库、搜索、代码仓库、用户记忆、会话状态分别是什么。
2. 知识怎么进系统：解析、切块、索引、权限绑定、增量更新做得怎么样。
3. 知识怎么被取出来：query understanding、hybrid retrieval、rerank、上下文组装是否合理。
4. 模型怎么被证据约束：有没有引用、拒答、groundedness 和 trace。
5. 系统怎么上线：有没有多租户隔离、审计、删除、回放、回归测试。

高频题速览

RAG 基础

- 什么是 RAG？为什么它比直接让模型回答更适合企业知识问答？
- RAG 和微调有什么区别？什么时候用哪个？
- RAG 和长上下文模型谁更好？
- RAG 的核心链路是什么？
- 为什么说 RAG 不是一个“向量库项目”？
- 什么时候不应该上 RAG？
- RAG 的核心指标有哪些？
- 为什么很多 RAG demo 能跑，线上却不好用？

检索优化

- 为什么说 chunking 决定了检索系统的上限？
- chunk size 应该怎么定？
- overlap 有什么作用，为什么不能太大？
- embedding 模型怎么选？

- 为什么 dense retrieval 很强了，生产里还要 sparse?
- Hybrid Retrieval 的本质是什么?
- 为什么很多系统要加 rerank?
- Query Rewrite 的目的是什么? 风险是什么?

Memory 与长期记忆

- RAG 和 Memory 最本质的区别是什么?
- 为什么会话历史不等于长期记忆?
- session state、working memory、long-term memory、cache 分别是什么?
- 为什么说 memory 需要写入策略?
- semantic、episodic、procedural memory 分别是什么?
- 长期记忆和用户画像有什么区别?
- 为什么用户记忆不能只靠向量检索?
- 长期记忆怎么做更新、遗忘和删除?

Agentic RAG / GraphRAG

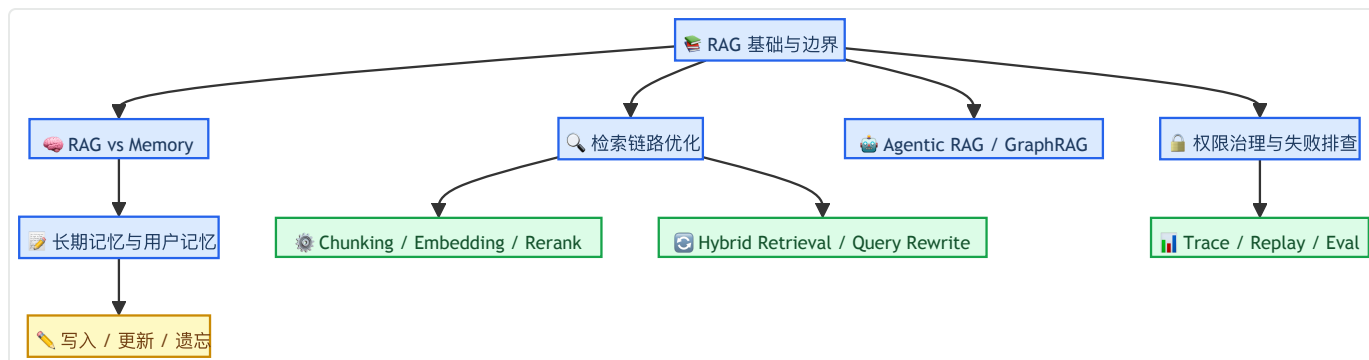
- 什么是 Agentic RAG?
- 什么情况下普通 RAG 不够用?
- 如何证明你的 Agentic RAG 不是“套个 Agent 壳”?
- GraphRAG 和知识图谱问答是一回事吗?
- GraphRAG 的 local search、global search、DRIFT search 分别适合什么问题?
- GraphRAG 一定比 vanilla RAG 强吗?

失败模式 / 治理

- RAG 最常见的失败模式有哪些?
- 为什么说很多 RAG 问题不是模型问题?
- 如何区分召回失败和排序失败?
- 什么是 lost-in-the-middle?
- Prompt Injection 在 RAG 里为什么危险?
- 为什么说 RAG 系统里最严重的问题可能不是幻觉，而是越权?

- 多租户隔离有哪些常见方案？
- 为什么需要 trace、回放和回归测试？

思维导图



一、RAG 基础与边界

Q1: 什么是 RAG? 为什么它比直接让模型回答更适合企业知识问答?

A:

RAG 是 **Retrieval-Augmented Generation**，也就是先从外部知识源中检索证据，再基于证据生成答案。面试里不要只说“向量检索 + Prompt 拼接”，更高分的讲法应该是：**RAG 是推理时访问外部知识，并用证据约束生成的一整条工程链路。**

它比直接让模型回答更适合企业知识问答，主要有四个原因：

1. **知识可更新**：制度、商品、价格、代码、工单状态都在变，不能每次都靠重新训练模型。
2. **私域知识可接入**：企业 wiki、CRM、工单、代码仓库本来就不在底座模型参数里。
3. **答案可追溯**：企业场景通常要求引用来源，而不是只给一个“像是真的”的答案。
4. **治理成本更低**：相比把所有知识塞进上下文或做频繁微调，RAG 更便于更新、审计和权限控制。

面试里最后一定要补一句边界：**RAG 能降低无依据胡说的概率，但不能自动保证正确，仍然需要权限、引用、评测和拒答机制。**

追问1: 怎么解释 parametric memory 和 non-parametric memory?

参数记忆是模型训练进参数里的知识，优点是调用方便，缺点是更新成本高、不可追溯；非参数记忆是外部知识索引，优点是可热更新、可审计。RAG 的核心就是把二者结合起来。

Q2: RAG 和微调有什么区别？什么时候用哪个？

A:

RAG 主要解决的是**推理时访问外部知识**，微调主要解决的是**改模型行为和输出风格**。如果知识变化很快、需要来源追溯、属于私域数据，优先用 RAG；如果问题是输出格式不稳定、领域表达不一致、工具调用习惯不好，优先考虑微调。

最稳妥的表述是：**RAG 补知识，微调改行为**。生产里两者常常一起用，而不是二选一。

Q3: RAG 和长上下文模型谁更好？

A:

不能绝对比较。长上下文解决的是**容量问题**，RAG 解决的是**从大规模候选中选出真正该看的那一小部分**。在知识库较小、权限简单、整库放进上下文更省事的时候，直接 long-context 往往更简单；在知识源很大、更新频繁、权限复杂、成本敏感时，RAG 更合适。

更成熟的工程说法不是“谁替代谁”，而是**路由**：小库直接 long-context，大库走 RAG，复杂任务走“检索 + 长上下文综合推理”。

Q4: RAG 的核心链路是什么？

A:

面试里最好把 RAG 拆成离线链路和在线链路。

离线链路：数据接入 -> 解析与标准化 -> 切块 -> 索引构建 -> metadata 和 ACL 绑定。

在线链路：查询理解 -> 召回 -> 重排 -> 上下文组装 -> 生成答案 -> 返回引用 -> trace 和回放。

离线链路决定知识能否被正确表征，在线链路决定证据能否被正确使用。

Q5: 为什么说 RAG 不是一个“向量库项目”？

A:

因为向量索引只是召回层的一种实现，真实生产系统还包含：

1. 文档解析与清洗。
2. 切块和 metadata 设计。
3. dense / sparse / hybrid 召回。
4. rerank 和上下文组装。
5. 引用、拒答、权限、审计、评测和回归。

所以更准确的说法是：**RAG 是知识访问、证据约束、系统治理和成本控制的组合方案，不是单点向量检索技巧。**

Q6: 什么时候不应该上 RAG?

A:

至少有三种典型情况：

1. 知识库很小，直接放进上下文更简单。
2. 任务本质不是补知识，而是改行为，比如格式、风格、分类边界不稳，这更像微调问题。
3. 团队还没有解析、评测和治理能力，这时先上复杂 RAG 很容易把系统做得又慢又不稳。

一句话总结就是：当“选信息”的难度很低时，不一定要上 RAG；当“改模型行为”的需求更强时，也不应该硬上 RAG。

Q7: RAG 的核心指标有哪些?

A:

一定不要只说“看回答准不准”。更完整的讲法是拆成三层：

1. **检索指标**：Recall@k、MRR、nDCG、Hit Rate。
2. **生成指标**：正确率、faithfulness、citation accuracy、拒答准确率、hallucination rate。
3. **系统指标**：P95 延迟、单问成本、索引更新时间、权限错误率、失败重试率。

高分表达是：**RAG 的问题定位必须按链路拆指标，否则你根本不知道问题在召回、排序还是生成。**

Q8: 为什么很多 RAG demo 看起来能跑，线上却不好用？

A:

因为 demo 只验证单轮 happy path，而线上会遇到脏文档、表格、代码块、权限、长尾术语、版本冲突、查询分布漂移和性能约束。很多 demo 连引用、回放、ACL、增量更新和回归测试都没有，所以只能证明“能跑”，不能证明“能上线”。

面试里可以用一句话收尾：**demo 验证的是可行性，生产系统考验的是稳定性、可追溯性和治理能力。**

二、检索链路优化

Q9: 为什么说 chunking 决定了检索系统的上限？

A:

因为召回只能在已有 chunk 上做选择。如果切块时把关键定义、限制条件、表格列、代码函数边界切坏了，后面的 embedding 和 rerank 再强，也只能在坏候选里挑“相对没那么坏”的。

所以 chunk 的本质不是按长度裁文本，而是定义一个可被索引、可被召回、可被引用、可被组装的最小语义单元。

Q10: chunk size 应该怎么定？

A:

不要背固定参数。正确答法是：**根据文档类型、查询类型、上下文预算和评测结果联调。**

1. FAQ、短问答更适合细粒度。
2. 制度文档、长流程说明更适合保留结构边界。
3. 表格、代码、合同条款往往需要特殊解析。
4. 起步可以用中等长度 + 适度 overlap，再根据 recall@k、citation precision 和人工评审来调。

面试里要体现的是 trade-off，而不是死记一个 512 或 1024。

Q11: overlap 有什么作用，为什么不能太大？

A:

overlap 是为了减少边界截断问题，防止定义和限制条件恰好被切在两个 chunk 里。但它不是越大越好，因为 overlap 太大会导致索引膨胀、重复召回、rerank 浪费和上下文冗余。

可以把它理解成一种**边界保险机制**，而不是默认拉满的参数。

Q12: embedding 模型怎么选？

A:

不要只看榜单。至少要看下面几个维度：

1. 领域适配。
2. 语言覆盖。
3. query / document 是否区分编码。
4. 向量维度和存储成本。
5. 推理延迟和吞吐。
6. 是否适配你的向量库和索引框架。

更成熟的工程路径是：**先用成熟通用模型起步，再用自己的评测集验证是否值得替换。**

Q13: 为什么 dense retrieval 很强了，生产里还要 sparse？

A:

dense 擅长语义相似，但对编号、版本号、函数名、报错码、产品代号、长尾缩写这类**词面强约束**不一定稳。企业知识库里恰好充满这类信号，所以 sparse 仍然很重要。

一句话记忆：**dense 保语义，sparse 保词面，hybrid 保稳。**

追问1: Hybrid Retrieval 的本质是什么？

本质是用多路相关性信号覆盖不同错误类型，让 sparse 负责精确词面，让 dense 负责语义相似，再通过融合和 rerank 得到更稳的候选池。

追问2: RRF 为什么常被用来做融合?

因为不同检索器的原始分数往往不可直接比较, 而 RRF 只依赖排序位置, 稳健、实现简单, 对异构检索器友好。

Q14: 为什么很多系统要加 rerank?

A:

首阶段召回追求的是别漏掉好候选, 所以偏 recall; rerank 追求的是把最像答案证据的候选排到前面, 所以偏 precision。很多系统“能召回来, 但用户还是觉得不准”, 根因就在 top-k 排序质量不够。

所以可以这样说: hybrid 扩大并稳定候选池, rerank 决定最终 front page 的质量。

追问: cross-encoder rerank 和向量召回的关系是什么?

向量召回适合在大规模文档里快速粗筛; cross-encoder 更适合对少量候选做精排序。前者负责“不漏”, 后者负责“更准”。

Q15: Query Rewrite 的目的是什么? 风险又是什么?

A:

Query Rewrite 的目标不是润色自然语言, 而是把原始问题改造成更适合被检索系统命中的表达。它常见的作用包括: 补齐上下文、消歧、显式化约束、改善词项覆盖、拆解复杂问题。

它的最大风险是查询漂移。如果把实体词改偏了, 或者错误继承多轮对话里的历史语境, 就会把正确意图带偏。所以高风险场景里最好保留原 query, 采用“原 query + rewrite query”双路检索。

追问1: rewrite 和 expansion 有什么区别?

rewrite 更强调保留原意并重述; expansion 更强调增加同义词、别名、相关词, 扩大召回覆盖面。

追问2: HyDE 和 step-back 各适合什么场景?

HyDE 适合原始 query 很短、语义稀疏时, 先生成一段“假想答案文档”作为语义锚点; step-back 更适合问题过于具体、词面不稳定时, 先上抽象层找基础原理, 再回到具体问题。

Q16：元数据为什么对检索效果重要？

A：

很多业务问题不仅依赖正文，还依赖标题层级、版本号、更新时间、页码、产品线、项目标签、权限标签。没有 metadata，就很难做过滤、排序、引用和 ACL 控制。

所以一个成熟的答法是：正文决定语义，metadata 决定可检索性、可治理性和可引用性。

三、RAG vs Memory 与上下文分层

Q17：RAG 和 Memory 最本质的区别是什么？

A：

RAG 主要解决外部知识访问与证据提供，Memory 主要解决个体化持续信息管理。前者面向世界和业务知识，后者面向用户、任务和经历。

更高分的表达是：二者最大的区别不在“是不是都能检索”，而在于服务对象、更新规律、权限边界和评测方式不同。

Q18：为什么会话历史不等于长期记忆？

A：

会话历史是原始日志，会话摘要是为了继续推理做的压缩，而长期记忆是经过筛选后留下、未来仍然有复用价值的信息。三者生命周期和用途完全不同。

所以面试里一定要说明：长期记忆不是把聊天记录全存起来，而是对高价值信息做抽取、压缩、写入、检索和淘汰。

Q19：session state、working memory、long-term memory、cache 分别是什么？

A:

这是非常高频的边界题，可以按四层来答：

1. **session state**：当前流程运行状态，比如 step、工具参数、中间结果。
2. **working memory**：当前会话中临时有用的信息，比如对话摘要、当前任务约束。
3. **long-term memory**：跨会话复用的信息，比如用户偏好、稳定背景、历史决策。
4. **cache**：为了省成本和延迟复用结果，不代表系统真正“记住了什么”。

一句话记忆：**state 管流程，memory 管认知，cache 管成本。**

Q20：为什么说 memory 需要写入策略？

A:

因为不是所有交互都值得存。如果每轮都写，就会出现三类问题：噪声堆积、错误沉淀、隐私风险。
memory 的关键不只是怎么查，更是什么时候写、写什么、写到哪一层、是否允许覆盖旧值。

工程上常见做法是：先让模型抽取候选记忆，再通过显著性、稳定性、来源可信度和作用域来筛选是否落库。

Q21：semantic、episodic、procedural memory 分别是什么？

A:

1. **semantic memory**：稳定事实和偏好，比如用户喜欢简洁回答、所在行业、岗位方向。
2. **episodic memory**：过去经历和事件，比如上次尝试过某方案但失败了。
3. **procedural memory**：做事方式或规则，比如某类任务的固定 SOP、回复模板、审批路径。

如果面试官继续追问，你还可以补充 **working memory**，表示当前任务中短期有效的临时信息。

Q22：什么时候优先走 RAG，什么时候优先查 memory？

A:

共享知识、客观资料、可被外部验证的证据，优先走 RAG；与某个用户或某个任务绑定的长期信息，优先查 memory。

举例：

1. “公司报销制度怎么走” -> RAG。
2. “我以后都喜欢你先给结论” -> user memory。
3. “这次审批走到哪一步了” -> session state / working memory。

面试里最关键的是把知识、记忆、状态、缓存这几个边界讲清楚。

四、长期记忆与用户记忆

Q23：长期记忆和用户画像有什么区别？

A:

用户画像通常只是长期记忆里最稳定、最结构化的一部分，比如偏好、行业、岗位方向；长期记忆还包含经历、计划、失败经验、共享上下文和历史决策。

所以你可以说：**profile** 是长期记忆的一部分，但长期记忆明显比 **profile** 更宽。

Q24：长期记忆不能每轮都写，那到底该记什么？

A:

长期记忆更适合记录五类东西：

1. 稳定偏好。
2. 显式用户事实。
3. 高价值经历和失败经验。
4. 跨会话计划与承诺。
5. 多 Agent / 多角色共享的项目背景。

判断标准是：未来是否高概率复用、是否足够稳定、是否来源可信、是否确实影响后续决策。

Q25: 哪些信息更适合做常驻记忆, 哪些更适合做可检索记忆?

A:

高频、短小、稳定、几乎每次都会影响回答的信息, 适合常驻记忆; 长文本、事件型、低频但重要的经历, 更适合做按需检索的长期记忆。

例如:

1. 回答风格偏好、语言偏好、岗位意向 -> 常驻。
2. 历史项目背景、上次为什么否决某个方案 -> 可检索。

这就是常说的resident memory vs retrievable memory 分层。

Q26: 显式触发写入和自动提炼写入怎么取舍?

A:

显式触发精度高但覆盖低, 自动提炼覆盖高但风险大。工程上通常是两者结合: 显式输入优先级最高, 自动提炼只作为候选, 再经过规则、阈值、确认或后台审查决定是否落库。

如果面试官问你偏向哪种, 你可以回答: 高风险字段偏显式, 高价值低风险字段可以自动提炼。

Q27: 如果用户偏好发生变化, 你怎么处理?

A:

不能简单追加一条新记忆, 而是要识别冲突字段, 更新旧值, 并保留时间戳、来源、版本历史和是否用户显式确认过。

一个成熟的 memory schema 至少应包含:

1. memory_type。
2. key / value。
3. source 和 confidence。
4. created_at / updated_at。
5. status (active / superseded / deleted) 。

面试里主动提到 versioning 和 conflict resolution, 分会更高。

Q28: 为什么用户记忆不能只靠向量检索?

A:

因为很多高价值记忆本质上是结构化事实, 比如 `response_style = concise`、`job_target = backend`。这类信息直接按 key 读取更稳。向量检索更适合召回经历型、长文本型、事件型记忆。

所以成熟系统往往是: KV / JSON profile + 文档存储 + 向量检索 的组合, 而不是一个向量库包打天下。

Q29: 长期记忆怎么做遗忘、删除和隐私治理?

A:

这是长期记忆最容易被忽略、但实际最重要的问题之一。常见策略包括:

1. TTL 或生命周期分层。
2. 最近使用频率衰减。
3. 冲突覆盖与旧值失活。
4. 用户显式删除。
5. 敏感字段暂停写入和作用域收缩。

一句话总结: 长期记忆不是“多存”, 而是“有选择地存、可纠错地改、可删除地管”。

Q30: 怎么评估长期记忆系统好不好?

A:

长期记忆比 RAG 更难评测，但至少可以从六个维度看：

1. 写入精度。
2. 召回相关性。
3. 个性化收益。
4. 冲突更新正确率。
5. 删除生效率。
6. 用户满意度。

高分表达不是说“记住得越多越好”，而是强调记忆精度、治理能力和长期一致性。

五、Agentic RAG 与 GraphRAG

Q31: 什么是 Agentic RAG?

A:

Agentic RAG 不是“给 RAG 套一层 Agent 壳”，而是让检索进入 计划 -> 行动 -> 观察 -> 反思 的闭环。也就是说，系统会根据中间结果动态决定：要不要继续查、怎么改 query、是否拆子问题、要不要切换知识源。

所以它的关键不在 Agent 三个字，而在于检索是否变成了一个有决策的迭代过程。

Q32: 什么情况下普通 RAG 不够用?

A:

当问题需要多步取证、跨工具或跨知识源、先查 A 再查 B、或者必须边查边调整策略时，单次 top-k 检索通常不够。

比如：

1. 先查投诉数据，再查 roadmap，再看是否有重合。
2. 先查文档，再查数据库，再回到文档做解释。
3. 先定位问题实体，再围绕实体继续扩展检索。

这种场景本质上已经不是单跳问答，而是**检索驱动的任务求解**。

Q33：怎么证明你的 Agentic RAG 不是“套个 Agent 壳”？

A:

要说明它的检索策略会随着中间结果变化，而不是固定调用一次 retriever。比如：

1. 先判断问题复杂度。
2. 对复杂问题做 decomposition。
3. 针对不同子问题路由不同知识源。
4. 如果证据不足，再改 query 或继续追检。
5. 最后对证据链做校验。

只要你讲出了“**中间结果会反过来影响后续检索动作**”，这就不是简单套壳。

Q34：GraphRAG 和知识图谱问答是一回事吗？

A:

不完全是。GraphRAG 更强调从原始文本中自动抽取实体、关系和 claims，构建图结构，做 community detection，再基于社区报告和图搜索来回答问题。

所以它不是简单地“把知识图谱接到 RAG 前面”，而是在**知识表示层做结构化增强**，特别适合多跳关系和全局主题问题。

Q35：GraphRAG 的 local search、global search、DRIFT search 分别适合什么问题？

A:

1. **local search**: 适合围绕某个实体或局部主题做细粒度关系推理。
2. **global search**: 适合整库主题概览、宏观总结、跨社区综合。
3. **DRIFT search**: 先从全局社区信息出发，再逐步收缩到局部细节，适合既要背景又要落地细节的问题。

面试里如果能把这三个模式讲清楚，通常说明你不是只听过 GraphRAG 这个词。

Q36: GraphRAG 一定比 vanilla RAG 强吗？

A:

不一定。只有在任务确实依赖实体关系、多跳推理、跨文档关系网络或全局主题总结时，GraphRAG 才更可能带来收益。对简单定位题、FAQ、条款检索题，GraphRAG 可能更慢、更贵，还可能引入图噪声。

所以更成熟的说法是：**GraphRAG 是条件性增强，不是默认替代方案。**

Q37: GraphRAG 的主要成本在哪？

A:

主要有五类成本：

1. 实体和关系抽取。
2. 实体归一化。
3. 图构建和社区检测。
4. 社区报告生成。
5. 增量更新和线上查询复杂度。

所以图方案的真正门槛不在“能不能搭”，而在“图构建质量和维护成本是否值得”。

Q38: Agentic RAG 和 GraphRAG 是替代关系吗？

A:

不是。Agentic RAG 解决的是**检索过程层**的问题，GraphRAG 解决的是**知识表示层**的问题。一个系统完全可以同时是 Agentic 的，也是 Graph-based 的。

比如 planner 先判断这是一个关系密集问题，于是路由到 GraphRAG 的 local search；如果还需要原文验证，再补一次普通文档检索。这才是更接近真实生产系统的组合方式。

六、失败模式、安全与治理

Q39: RAG 最常见的失败模式有哪些？

A:

不要只答“幻觉”，要按链路拆：

1. 解析失败。
2. 切块失败。
3. 召回漏失。
4. 排序错误。
5. 上下文组装失败。
6. 生成不受证据约束。
7. 知识过期和版本冲突。
8. 权限误配置。
9. Prompt Injection / 检索投毒。
10. 评测盲区。

面试里最关键的是体现你的**分层排障意识**。

Q40: 为什么说很多 RAG 问题不是模型问题？

A:

因为很多错误在模型回答之前就已经发生了：文档没解析对、chunk 切坏了、gold evidence 没召回、rerank 没排上来、上下文组装把关键证据淹没了。模型只是最后一个暴露问题的环节。

所以排障时一定要先问：证据在不在、证据有没有被召回、证据有没有被模型真正读到。

Q41：如何区分召回失败和排序失败？

A:

看正确证据是否出现在更大的候选集合里：

1. 如果 top-50 里有、top-5 没有，通常是排序问题。
2. 如果 top-50 都没有，通常是召回问题。

这是非常经典的面试追问，答对说明你会做链路定位，而不是只会说“继续调模型”。

Q42：什么是 lost-in-the-middle？为什么加入更多 chunk 不一定更好？

A:

lost-in-the-middle 指的是关键信息虽然在上下文里，但被放在长上下文中部，模型没有有效利用。加入更多 chunk 不一定更好，因为它会带来：

1. 噪声变多。
2. 重复内容变多。
3. 关键证据被稀释。
4. 成本和延迟上升。

所以不是“塞得越多越好”，而是怎么把最关键的证据放到最有效的位置。

Q43：知识过期和版本冲突怎么处理？

A:

要在索引层和生成层同时处理：

1. 文档写入版本字段和更新时间。
2. 默认偏向最新文档。
3. 旧版本显式失活。
4. 建立增量更新和缓存失效机制。
5. 关键场景下在答案里展示生效日期。

如果不做这些，RAG 的“时效性优势”就会变成“把过期知识说得更像真的”。

Q44: Prompt Injection 在 RAG 里为什么危险？

A:

因为恶意文档可能被检索进上下文，再借助模型对指令的服从性来操纵后续行为，或者污染答案。RAG 把外部内容拉进 prompt，本身就扩大了攻击面。

最基本的防护思路包括：

1. 来源可信度分层。
2. 内容清洗与安全过滤。
3. 生成阶段显式忽略文档中的操作性指令。
4. 高风险场景做 groundedness 检查和人工审批。

Q45: 为什么需要 trace、回放和回归测试？

A:

因为没有中间过程可见性，就无法知道问题到底出在哪一层，也无法稳定复现修复效果。RAG 不是普通黑盒问答系统，它天然需要链路可观测。

一套成熟的 trace 至少要记录：

1. 原始 query 和 rewrite query。
2. 过滤条件和 ACL。
3. 召回候选和 rerank 结果。
4. 最终注入上下文。
5. 生成答案和引用。

回归测试则要按错误类型建桶，覆盖解析、召回、排序、权限、安全和最终回答。

Q46：为什么说 RAG 系统里最严重的问题可能不是幻觉，而是越权？

A:

因为幻觉大多是质量问题，而越权是安全和合规问题。一次错误暴露私域信息，系统可能直接无法上线。

所以企业知识 Agent 的底线通常不是“必须百分百答对”，而是绝不能把不该看的东西检出来、拼进去、生成出来。

Q47：多租户隔离有哪些常见方案？

A:

常见有三种：

1. 独立库 / 独立集合 / 独立索引：隔离最强，运维成本更高。
2. namespace / shard / partition 隔离：隔离和效率之间的折中。
3. 单集合 + tenant_id / payload filter：资源利用率高，但对查询过滤和审计要求更高。

面试里一定要体现：没有唯一最佳方案，关键看租户规模、隔离强度和运维成本。

追问1：为什么不建议只做后过滤？

因为后过滤意味着越权结果可能已经被召回、进入日志、缓存或 rerank，同时还会损失 top-k 质量。更安全的方式是 tenant 和 ACL 在检索前就下推过滤。

追问2：ACL 应该挂在哪里？

至少要能从 chunk 追溯到源文档，并在查询时基于文档级或 chunk 级 ACL 做过滤。权限设计必须从 ingest 阶段就跟随 chunk 一起下沉。

七、面试里的项目表达模板

项目表达模板：

我们没有把系统简单做成“向量库 + 大模型”，而是把它拆成四层：当前流程状态放 session state，当前会话的临时结论放 working memory，用户偏好和跨会话信息放 long-term memory，企业制度和文档走 RAG。检索链路上我们先做结构化解析和 metadata 保留，再用 hybrid retrieval 保 recall，用 rerank 提 top-k 质量，最后在生成阶段加引用和拒答约束。上线前我们重点补了 ACL、trace、回放和回归测试，因为企业场景里最危险的不是答错，而是越权。

01-RAG基础

一、RAG的考察内容

RAG 基础题表面上在问什么是 RAG，本质上在考你是否理解大模型应用为什么需要把模型参数里的知识和外部知识源拆开管理。校招生最容易把 RAG 讲成把文档切块后丢进向量库，再把检索结果拼到 Prompt 里，这只能拿到及格分。更高分的表达必须说明：**RAG 解决的是知识时效性、领域覆盖、可追溯性、成本与可维护性之间的工程平衡**；它不是单一算法，而是一整条以检索为核心、以生成为出口、以评测和治理为约束的数据通路。

因此，这篇题真正考的是你能不能把为什么要检索、检索什么、什么时候检索、检索到之后如何约束生成、RAG 与长上下文/微调/记忆分别是什么关系说清楚。只要你把这些关系讲明白，面试官通常会认为你具备了进入 RAG 细化题、Agent 检索题、评测题和系统设计题的基础。

面试官在这一题里，通常不是只想听一个名词解释，而是想顺着这个主题判断你是否具备下面这些能力：

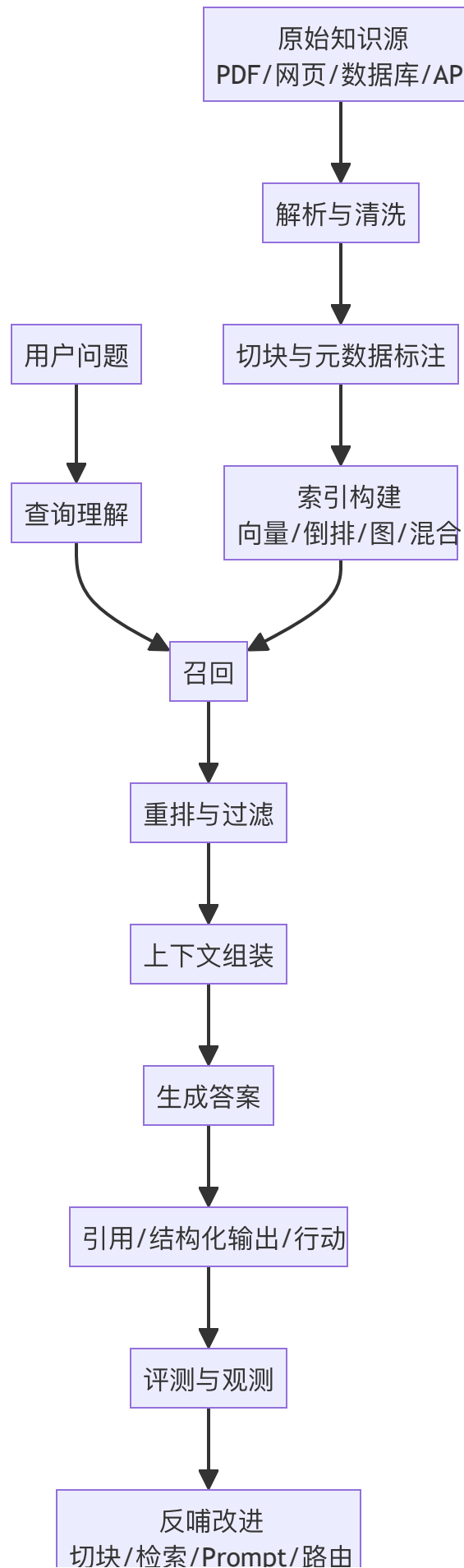
- 是否能准确区分参数知识、外部知识、会话上下文、用户记忆这四类信息载体。
- 是否理解 RAG 的核心链路：数据接入、解析、切分、索引、召回、重排、上下文组装、答案生成、引用与回传。
- 是否知道 RAG 的收益和边界：能降低幻觉、提升时效与可解释性，但不能自动保证正确，更不能替代权限控制与评测。
- 是否能说出 RAG 与微调、长上下文、缓存、知识图谱、Agent 的关系，而不是把这些概念混成一团。
- 是否具备工程 trade-off 意识，能从质量、成本、延迟、维护、权限、安全这几个维度比较方案。

二、一些重要结论

- RAG 的本质是检索驱动的受控生成，不是向量库 + Prompt 拼接的机械套路。
- RAG 价值最大的场景是：知识频繁变化、知识属于私域、需要来源追溯、上下文很长但真正有用的信息只占一小部分。
- RAG 的效果由两段决定：一段是能不能把对的证据召回来，另一段是模型会不会老老实实依据证据作答。
- 做不好 RAG 的根源往往不在模型，而在解析、切块、召回、重排、权限、上下文打包与评测闭环。
- 长上下文模型变强以后，生产上越来越常见的做法不是永远用 RAG，而是按查询类型在 RAG 与长

上下文之间做路由。

三、先有一张总图



四、系统化八股知识

RAG 到底是什么：一句定义，三层含义

RAG 是 Retrieval-Augmented Generation，也就是检索增强生成。最朴素的理解是：先从外部知识源中找到与问题最相关的证据，再把证据和问题一起交给模型作答。这个说法没有错，但还不够完整。面试里最好把它展开成三层。

第一层，**RAG 是信息访问机制**。模型自身参数里的知识是静态的、训练期固化的，而外部知识可以是实时更新的数据库、文档库、企业知识库、搜索引擎结果、图数据库甚至业务系统 API。RAG 的作用就是在推理时把这些外部知识安全、经济地接进来。Lewis 等人在 2020 年提出的经典 RAG 工作，把它表述为把参数化记忆和非参数化记忆结合起来的生成范式，这也是今天绝大多数面试官默认的理论起点。

第二层，**RAG 是证据约束机制**。它不是让模型凭感觉回答，而是尽量让模型依据检索到的证据回答。这就是为什么生产系统往往要求返回引用片段、文档标题、页码、URL 或 chunk id。只要没有证据，你就应该尽量让系统拒答、澄清或继续检索，而不是胡编。

第三层，**RAG 是工程系统而不是单点算法**。从数据接入、解析、切块、索引，到检索、重排、上下文压缩、答案生成、引用回传、权限过滤、评测观测，任何一个环节出问题，最后用户都会感知为 RAG 不准。所以真正做过的人，讲 RAG 时一定会强调全链路，而不只盯着 embedding 模型。

为什么今天还需要 RAG：不是模型不够强，而是系统边界不同

很多同学会说因为大模型会幻觉，所以需要 RAG。这个说法对，但太浅。更完整的回答应该至少包含四个动机。

第一，**知识时效性**。模型预训练截止之后发生的事情，它不知道；企业内部昨天刚更新的制度、今天刚上的商品、刚变更的价格、刚修复的 bug，它也不知道。RAG 可以在不重新训练模型的前提下接入最新知识。

第二，**知识所有权与私域性**。公司内部 wiki、CRM、工单、合同、代码仓库、会议纪要，根本就不可能全部纳入底座模型预训练。RAG 是把私域知识接给模型最自然的方式。

第三，**可追溯与可治理**。在企业环境里，答案像不像真的远远不够，通常还要能追溯答案是根据哪份文档、哪条记录、哪一次更新给出的。没有来源，你很难审计、纠错、追责，也很难让用户信任系统。

第四，**成本与效率**。即使长上下文模型已经很强，直接把整库内容塞进上下文仍然很贵。Google DeepMind 在 2024 年的对比研究里发现，资源充足时长上下文模型常常能优于 RAG，但 RAG 仍有显

著的成本优势，因此更合理的工程思路是路由，而不是二选一。换句话说，RAG 并不是旧时代妥协方案，而是在成本、权限、更新频率、可解释性约束下依然有竞争力的系统设计。

面试官爱追问的点通常是：既然长上下文越来越强，RAG 会不会过时？你的回答应该是：**不会过时，但会从默认方案变成按任务选择的方案**。当知识库小于一个上下文窗口且权限简单时，直接 long-context 很可能更省事；当知识源很大、更新频繁、权限复杂、证据追溯重要时，RAG 依旧是主流。Anthropic 在 Contextual Retrieval 文里甚至明确提到，如果你的知识库小于约 20 万 token，直接把整库放进提示里可能比上 RAG 更简单；这反而说明成熟团队已经开始按规模和任务做分层选择。

RAG 的标准链路：从离线索引到在线回答

面试里最好把 RAG 拆成离线索引链路和在线查询链路，这样会显得你对系统边界很清楚。

离线索引链路通常包含：

1. 数据接入：接文件、网页、数据库、消息流、代码仓库、SaaS API。
2. 解析与标准化：抽取正文、标题、层级、表格、图片说明、作者、更新时间、权限标签。
3. 切块：按段落、章节、语义、表格、代码函数等切成适合检索的最小单元。
4. 索引构建：建立向量索引、BM25 倒排索引、知识图谱或混合索引。
5. 元数据与权限绑定：记录 doc_id、chunk_id、tenant_id、acl、版本号、时间戳、语言、业务域等。

在线查询链路通常包含：

1. 查询理解：识别问题意图、语言、是否需要检索、是否需要改写。
2. 召回：用 dense、sparse、hybrid、graph 或 agentic retrieval 拿回候选证据。
3. 重排：用 cross-encoder、late interaction 或更强的 reranker 对候选重新排序。
4. 上下文组装：去重、按信息密度和 token budget 选择片段，必要时做压缩或摘要。
5. 生成答案：提示模型基于证据作答、不要越界推断、无证据就说不知道。
6. 返回结果：答案、引用、置信线索、后续动作建议。
7. 观测与回放：记录 query、retrieved docs、rerank scores、latency、cost、用户反馈。

为什么要这么拆？因为**离线链路决定知识能否被正确表征，在线链路决定证据能否被正确使用**。很多面试题其实就是在问你哪一段出了问题。例如召回率低怎么办是**在线问题**，表格总是答错往往是**解析和切块问题**，跨租户泄漏是**权限绑定问题**。你一旦会这样拆，后续所有 RAG 细题都容易讲。

RAG 与微调、长上下文、缓存、搜索的关系

这部分是面试里非常高频的概念边界题。回答时建议用解决什么问题来区分，而不是按技术实现来背诵。

RAG vs 微调：RAG 主要解决让模型在推理时访问外部知识的问题，适合知识更新快、证据需要追溯的场景；微调主要解决让模型学会特定输出风格、领域表达、工具调用习惯、分类边界的问题。知识更新如果天天变，更适合 RAG；回答格式总是不稳，更适合微调。很多生产系统同时使用二者：底座经过指令/格式微调，知识通过 RAG 接入。

RAG vs 长上下文：长上下文是给模型更多上下文容量，RAG 是从大量候选信息里挑出应该放进去的那一小部分。前者解决容量，后者解决选择。容量大不等于选择好。现实里常见的是两者结合：先检索，再把选中的证据交给长上下文模型做综合推理。

RAG vs 缓存：缓存是复用过去已经算过的结果，比如相同问题的答案缓存、embedding 缓存、query rewrite 缓存。它主要优化成本和延迟；RAG 主要优化知识获取和回答正确性。缓存不能替代检索，只能加速检索后的部分流程。

RAG vs 搜索：传统搜索通常输出文档列表，而 RAG 目标是把搜索结果转化成可消费的模型上下文并完成答案生成。所以你可以把 RAG 理解为搜索 + 证据编排 + 生成。如果只会搜不会用证据，还是做不好 RAG。

什么是好 RAG：不要只说准确率，要看一整组指标

很多校招生答 RAG 题时只会说看回答准不准。面试官如果听到这里，通常会继续追问：那怎么知道是检索错了，还是生成错了？这时你必须能把指标拆开。

第一类是**检索指标**，比如 Recall@k、MRR、nDCG、Hit Rate。它衡量正确证据是否进入候选集。如果 gold evidence 根本没被召回，再好的生成模型都救不回来。

第二类是**重排与上下文打包指标**。候选里有对的文档，不代表最终拼进 Prompt 的就是对的文档。你需要关注 rerank 之后 top-N 的质量、去重率、token 利用率、不同来源覆盖率，以及是否把关键证据淹没在一堆边缘片段里。

第三类是**生成指标**，比如答案正确率、faithfulness、citation accuracy、refusal accuracy、hallucination rate。也就是说，不仅看答案像不像，还看它是否忠于给定证据、引用是否对得上、该拒答时能不能拒答。

第四类是**系统指标**，包括 P95 延迟、吞吐、单问成本、索引更新时间、失败重试率、权限过滤命中率。这些指标决定了你能不能把 RAG 放进生产环境，而不是只停留在 demo。

真正成熟的回答应该是：一个高质量 RAG 系统，至少要同时优化召回、重排、证据打包、生成忠实度和系统成本。你甚至可以补一句：RAG 系统的问题定位通常要通过 tracing 把 query rewrite、retrieval、rerank、prompt assembly、generation 这些环节全部打点，否则改了半天也不知道改对了没有。

Naive RAG、Advanced RAG、Modular RAG：面试里怎么讲层次

面试里经常会问你理解的 RAG 演进阶段是什么。比较稳妥的讲法是三层。

Naive RAG：文档切块、做 embedding、向量检索、把 top-k 拼给模型回答。这种方式容易上手，但对关键词问题、长文结构问题、多跳问题、权限问题都比较脆弱。

Advanced RAG：开始引入混合检索、query rewrite、rerank、metadata filter、chunk contextualization、query routing、citation、拒答机制。也就是说，不再把向量检索当成唯一武器，而是把 retrieval 做成多阶段系统。Anthropic 的 Contextual Retrieval、Pinecone/Weaviate/Qdrant 的 hybrid + rerank 官方实践，都是这个阶段的典型代表。

Modular / Production RAG：进一步把检索、重排、压缩、权限、评测、观测做成可插拔模块，并与缓存、异步索引、灰度发布、回归测试、成本治理整合。到这一步，RAG 已经不是一个小 demo，而是平台能力了。

如果面试官继续追问 Agentic RAG 算哪一层，你可以回答：它通常建立在 Advanced/Modular RAG 之上，把检不检、检几次、用什么工具检、何时停止检索交给 Agent 动态决策。它不只是 RAG 的一个小优化，而是检索控制权的升级。

校招生最该避免的两个误区：把问题讲小，把边界讲乱

第一个误区是把 RAG 讲成解决幻觉的银弹。实际上，RAG 只能降低无依据胡说的概率，但不能保证答案一定正确。因为它可能检错、排错、拼错，也可能模型拿到证据后仍然过度总结、错误归纳、断章取义。

第二个误区是把 RAG、记忆、搜索、长上下文、微调混成一锅。面试官非常介意这一点。你必须清楚：企业文档属于外部知识检索，用户偏好属于记忆，系统提示属于程序性约束，长上下文属于容量，微调属于行为塑形。边界讲不清，通常说明你没有真正做过系统设计。

一个很实用的答题口诀是：先讲为什么需要外部知识，再讲 RAG 链路，再讲与其他方案的边界，最后讲质量与成本 trade-off。只要沿着这条线说，基本不会乱。

五、目前RAG的演进趋势

1. 长上下文与 RAG 的关系正在从替代关系变成路由关系。Google DeepMind 的研究表明，在资源足够时，长上下文模型平均表现往往优于经典 RAG，但 RAG 依旧有显著成本优势，所以生产系统更像是在做 query routing，而不是宣布 RAG 失效。
2. 向量检索 alone 越来越不够。Anthropic 的 Contextual Retrieval 提醒大家，真正的生产级 RAG 常

常是更好的 chunk 表征 + 稀疏检索 + rerank + 合理 top-k，而不是单靠一个 embedding 模型。

3. RAG 的关注点从能不能搭出来转向能不能治理。权限、来源追踪、评测回归、灰度发布、延迟成本控制，正在成为面试里更常见的加分点。
4. RAG 与 Agent 的边界也在前移。原来的两段式 RAG 更像固定 workflow；今天越来越多系统会让 Agent 决定是否继续检索、是否切换知识源、是否二次提问，这就是 Agentic RAG 的入口。

六、常考面试题与参考答法

1. 什么是 RAG？为什么它比直接让模型回答更适合企业知识问答？

- 参考回答：先给定义：推理时从外部知识源检索证据，再基于证据生成答案。然后补三点：知识可更新、私域知识可接入、答案可追溯。最后说边界：RAG 不能天然保证正确，还需要评测、权限和拒答机制。

2. RAG 和微调有什么区别？什么时候用哪个？

- 参考回答：微调主要改模型行为和输出风格，RAG 主要补外部知识。知识频繁变化选 RAG，输出格式/行业表达不稳可用微调，生产里常常二者结合。

3. RAG 和长上下文模型谁更好？

- 参考回答：不能绝对比较。长上下文在资源足够时可能有更强整体理解，但 RAG 在大知识库、复杂权限、成本敏感场景更有优势。更成熟的做法是路由：小库直接 long-context，大库走 RAG。

4. 一个 RAG 系统为什么会答错？

- 参考回答：至少四类原因：解析切块错、召回错、重排/上下文组装错、生成越界。面试里要体现你会分层定位，不要笼统说模型幻觉。

5. 为什么说 RAG 不是向量库项目？

- 参考回答：因为向量索引只是召回的一种实现。真实系统还包含解析、元数据、权限、混合检索、重排、引用、拒答、评测、观测等模块。

6. 什么时候不应该上 RAG？

- 参考回答：当知识库很小、权限简单、整库放进上下文更省事时；或者任务本质不是查知识而是改行为时。Anthropic 甚至建议小于约 20 万 token 的知识库可以直接塞进上下文。

7. RAG 的核心指标有哪些？

- 参考回答：至少分三段：检索指标 Recall@k / MRR / nDCG，生成指标正确率/faithfulness/citation accuracy，系统指标延迟/成本/更新时延/权限错误率。

8. 你会怎么向面试官解释 parametric memory 和 non-parametric memory？

- 参考回答：参数记忆是模型训练进参数里的知识，更新成本高、不可追溯；非参数记忆是外部

知识索引，可热更新、可审计。RAG 的核心就是把二者结合。

9. 为什么很多 RAG demo 看起来能跑，线上却不好用？

- 参考回答：因为 demo 只验证单轮 happy path，线上会遇到脏文档、表格、权限、长尾词、更新延迟、查询分布漂移、性能约束和评测闭环缺失。

10. 如何一句话概括成熟团队的 RAG 观？

- 参考回答：RAG 不是一个检索技巧，而是知识访问、证据约束、成本治理和系统可追溯性的组合方案。

七、容易踩坑的表达

- 只背检索增强生成六个字，不解释为什么增强、增强的是什么、增强在哪个阶段发生。
- 把向量检索当成 RAG 的全部，不提 BM25、hybrid、rerank、metadata filter、权限过滤。
- 把 RAG、记忆、微调、长上下文混用，导致概念边界模糊。
- 说 RAG 能彻底消灭幻觉，这会显得没有做过线上系统。
- 只讲技术栈，不讲用户价值、业务约束和评测方法。：

九、参考资料

- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks —— 经典 RAG 起点，定义参数记忆与非参数记忆结合
- Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach —— RAG 与长上下文的系统对比
- Contextual Retrieval in AI Systems —— 官方工程文章，讨论何时直接 long-context、何时改进 RAG
- Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers —— 2025 年较新的 RAG 总体综述

02 Chunking-Embedding-Rerank

一、当我们在谈论这三者时，我们在聊什么

表面在考RAG 细节优化，本质上在考你是否理解检索系统里最常见的质量瓶颈根本不在大模型本身，而在索引前后的表征与筛选。校招生最容易犯的错是把 chunking、embedding、rerank 当成三个互不相关的小点背诵：切块大小 512/1024，embedding 选某个模型，rerank 用 cross-encoder。这样的回答太碎，也无法支撑深入追问。

更成熟的表达应该是：切块决定可被召回的最小语义单元，embedding 决定召回阶段如何测相似，rerank 决定候选集合如何被重新排序。这三者共同影响召回率、精确率、上下文冗余、延迟和成本。面试官真正想听的是你能否把这条链路串成一个工程优化闭环：先定义查询类型，再确定切块策略，再选 embedding 表征，再选首阶段召回与二阶段重排，最后通过离线评测和线上观测做调整。

面试官在这一题里是想顺着这个主题判断你是否具备下面这些能力：

- 是否理解 chunk 不是越小越好，也不是越大越好，而是在语义完整性、召回粒度、上下文预算之间做平衡。
- 是否能区分首阶段召回与二阶段重排各自负责什么，以及为什么很多系统先粗召回、再精排序。
- 是否了解 embedding 模型选择不仅看排行榜，还要看领域适配、语言覆盖、维度、延迟、价格和向量库存储成本。
- 是否知道元数据、标题、段落结构、表格边界、页码等信息会直接影响检索质量，而不是只有正文文本重要。
- 是否能讲清 late chunking、contextual retrieval、ColBERT/late interaction 这类 2024–2026 被频繁提到的新趋势。

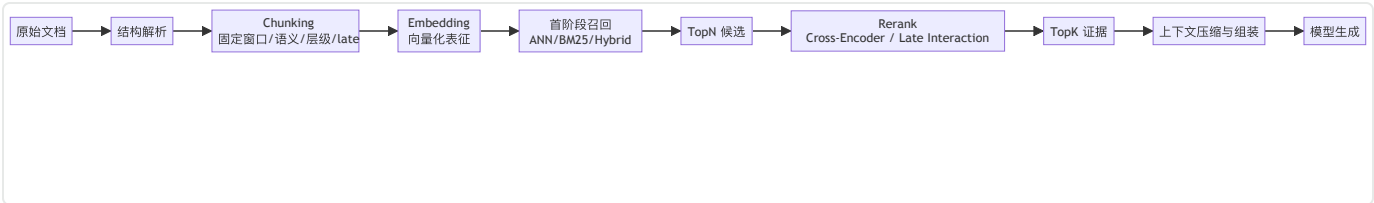
如果你在项目里确实做过这一块，可以按下面这个顺序回答：

我们的知识库里有 FAQ，也有制度文档和长流程说明，所以没有采用全库统一切块。先做结构解析，保留标题层级、页码、更新时间、权限标签等 metadata；FAQ 采用较细粒度 chunk，制度文档按章节和段落边界切，表格单独抽取。首阶段不是只用向量，而是 hybrid retrieval，先保证别漏；二阶段加 rerank，解决 top-k 里相关但不够直接的问题。我们把 recall@20、top-5 引用命中率、首屏正确率作为关键指标，发现相比只调 embedding，先把切块和 rerank 做好，体感提升更明显。后面针对上下文依赖强的长文档，还尝试了 contextual retrieval 思路，用自动补充的上下文说明改善 chunk 脱离原文后的信息贫血问题。

二、先把这几个结论记住

- 切块决定候选池上限，embedding 决定召回空间，rerank 决定最终排序，三者必须联调。
- 生产环境里最常见的组合不是只靠向量，而是结构化切块 + dense/sparse 召回 + rerank。
- 如果 chunk 本身破坏了语义边界，后面的 embedding 和 rerank 往往只能补救一部分，不能完全挽回。
- rerank 很少提升 recall，但能显著提升 top-k 质量，因此常常是用户体感提升最大的步骤之一。
- 近年的趋势不是单一固定切块，而是按文档类型和查询类型采用层级切块、语义切块、late chunking 或 contextual retrieval。

三、先有一张总图



四、系统化八股知识

为什么这一题经常被追问

很多候选人能够流畅地讲出 RAG 流程，但一旦面试官追问你们为什么把 chunk size 设成 512 而不是 1024为什么要加 rerankembedding 模型为什么换代，就会露出没有做过系统优化的问题。因为真正影响效果的，不是会不会背 RAG 定义，而是你有没有把召回质量当成一个可拆解、可测量、可迭代的工程问题。

在多数企业场景中，RAG 出现答非所问引用看起来相关但并不关键明明知识库里有答案却检不出来的根因，常常不是 LLM 太弱，而是 chunk 破坏了信息边界，或者 embedding 无法表达领域概念，又或者首阶段召回把高质量候选压在了很后面。这就是为什么 chunking、embedding、rerank 会成为面试的高频追问点。

Chunking：切块到底在切什么

切块的本质不是把长文本裁成固定长度，而是为检索系统定义一个可被索引、可被召回、可被引用、可被拼装的最小语义单元。一个好的 chunk 至少要满足四件事：第一，包含相对完整的语义；第二，不要

引入太多无关内容；第三，能挂载足够多的元数据；第四，长度要适合后续 embedding、召回和上下文预算。

常见切块策略可以分成四类。第一类是**固定窗口切块**，例如按 token 或字符数切成 256/512/800/1024 大小并加入 overlap。这种方式简单、吞吐高、可控性强，因此是许多生产系统的起点。OpenAI 官方 file search 在默认策略下就采用固定大小 chunk 和重叠窗口，默认 chunk size 为 800 tokens、overlap 为 400 tokens，这说明即便是成熟平台，也认为稳定、可控的基础切块仍然是默认选项之一。第二类是**基于结构的切块**，例如按标题、章节、段落、表格、代码块、问答对、页面块来切，这通常比纯固定窗口更能保留语义边界。第三类是**语义切块**，通过 embedding 相似度、语义边界检测或句间断点识别来决定分块边界。第四类是**层级切块**，同时保留 coarse chunk 和 fine chunk，用于先粗定位，再细取证。

面试时你最好强调：切块不是越细越好。chunk 太小会导致上下文不完整，尤其是在制度文档、表格说明、长代码函数、合同条款这类依赖前后文的场景；chunk 太大又会把噪声带进来，导致向量平均化、召回不准、top-k 上下文挤占。一个常见经验是：问答型知识更适合细粒度，流程型/制度型文档更适合保留结构边界，代码和表格往往需要特殊解析而不能直接混同普通段落。

Overlap、标题增强与元数据为什么重要

很多人只谈 chunk size，却忽略 overlap 和元数据。Overlap 的作用是防止重要信息恰好被切在边界上。典型例子是某个条款的定义在上一段、限制条件在下一段，如果不做重叠，两个 chunk 都是不完整的。Overlap 不是越大越好，因为过大意味着索引膨胀、重复召回、重排浪费和上下文冗余。你需要把 overlap 看作一种针对边界损失的保险机制，而不是默认拉满的参数。

更重要的是**标题增强与元数据保留**。很多业务文档的正文语义高度依赖标题层级、产品名、业务线、版本号、日期、作者、权限标签、表格列名、页码等信息。如果切块时丢掉这些上下文，召回质量会明显下降。现实里常见的优化是：把章节标题、子标题、文档标题、所属目录、最近更新时间拼到 chunk 前缀里，或者作为 metadata 参与过滤和排序。这样做的效果往往不亚于更换 embedding 模型。

这也是为什么你在面试里最好少说切块就是按 500 字切，而应该说切块时要保留结构信息、来源信息和权限信息。因为对企业知识库来说，能被正确引用、能做权限过滤、能回溯原始文档的位置，和能否召回同样重要。

Embedding：召回阶段到底在表示什么

Embedding 模型的任务，是把文本映射到一个向量空间，使得语义相近的内容在空间里距离更近。很多候选人会把 embedding 讲成把文字转成向量，这句话太浅。面试更看重你是否理解：embedding 其实在决定首阶段召回时什么被视为相似。

为什么同一个知识库换 embedding 模型，效果会差很多？因为不同模型对语言、领域术语、长文本、数字表达、代码片段、表格描述、跨语种术语映射的能力不同。举例来说，面向通用搜索训练的 embedding 对 FAQ、百科、网页摘要表现通常不错，但对金融术语、法律条款、企业内部缩写、代码 API 名称不一定敏感。再比如，一些模型擅长多语言，但对中文企业文档里的中英混排、产品代号、内部简称并不稳。

模型选择不能只看榜单。你至少要从这些维度比较：一，领域适配；二，语言覆盖；三，是否区分 query/document instruction；四，维度大小和存储成本；五，推理吞吐和延迟；六，是否支持批量离线重建索引；七，向量库和检索框架是否原生支持。生产上常见做法是先用成熟通用 embedding 起步，再用自己的评测集比较候选模型，而不是一开始就追求最强榜单模型。

Instruction-aware Embedding、Query/Document 区分与领域适配

近几年 embedding 的一个明显趋势是对 query 和 document 进行不同格式的编码，或者在输入里加 instruction。原因很简单：用户问题往往是意图表达，而文档 chunk 往往是知识陈述，两者分布并不一致。如果模型训练时显式区分 query 和 document，它在检索时往往更稳。

你在面试里可以补一句：embedding 并不是文档和查询各自转向量然后算距离这么简单。好的 embedding 设计会关注**查询分布**。如果你的业务里用户问题普遍比较短、带很多别名、夹杂拼音缩写和产品代号，那么 query encoding 的适配性会直接决定召回效果。另外，领域适配并不一定要重训 embedding，有时只需要在 chunk 前补充结构化标签、在 query 侧做 rewrite 或 expansion，效果就能显著改善。

Rerank：为什么大多数成熟系统都有二阶段排序

首阶段召回通常追求的是别漏掉好候选，因此它会偏 recall；二阶段 rerank 追求的是把最相关的候选放到前面，因此它偏 precision。向量检索和 BM25 这类首阶段方法本质上都属于高吞吐、近似筛选，很难在 top-10 或 top-20 上做到特别精细的相关性判断，于是 rerank 成了把候选集合精修一遍的关键步骤。

常见的 rerank 方式包括 cross-encoder 和 late interaction 两大类。Cross-encoder 让查询和候选文档在同一模型里联合编码，通常相关性判断更准，但成本较高；ColBERT 这类 late interaction 方案把文档 token 表征保留下来，在匹配时做更细粒度的 token-level 交互，在效果和效率之间提供了一个有代表性的折中。对于企业知识库问答而言，rerank 往往能显著减少看上去相关但其实不是直接答案的 chunk 排到前面的情况。

面试里可以这样讲：rerank 本身不创造新信息，它做的是把第一阶段粗召回得到的候选按照更像答案证据的标准重新排序。它未必提升召回率，但常常能显著提升 top-k 质量，进而改善最终答案。

Late Chunking、Late Interaction 与 Contextual Retrieval：为什么它们常被放在一起讨论

这几个概念经常一起出现，但不是一回事。**Late chunking** 的核心思想是先在更大范围内编码长文上下文，再在后处理中得到 chunk 级表示，目的是减少先切再编码造成的上下文割裂。**Late interaction** 典型代表是 ColBERT，它不是简单把整段文档压成一个向量，而是保留 token 级表征，在匹配时做更细粒度相似性计算。**Contextual Retrieval** 则更偏工程方案，Anthropic 给出的思路是先为 chunk 补充上下文说明，再配合 contextual embeddings 和 contextual BM25 做召回，还建议在此基础上加 rerank；其公开材料中提到，相比传统检索，其 failed retrievals 可显著下降，加入 rerank 后还会进一步降低。

三者共同回应的是同一个问题：传统固定切块 + 单向量表征会丢失上下文。只是它们的切入点不同。Late chunking 关注编码阶段如何保留长程上下文；late interaction 关注匹配阶段如何保留细粒度信号；Contextual Retrieval 则关注如何显式把 chunk 所在上下文补写进 chunk。本质上，它们都是在对抗 chunk 独立化导致的语义贫血。

怎么选：一个工程上好用的决策框架

如果你在面试里被问那到底怎么选 chunking、embedding 和 rerank，不要给出一个固定参数，而应该给出决策顺序。

第一步，看文档类型。FAQ、操作手册、制度规范、会议纪要、合同、代码仓库、表格报表，它们最适合的切块策略不同。第二步，看问题类型。是事实型查找、流程型解释、跨段归纳、多跳推理还是精确定位？第三步，看目标指标。你更在乎 recall、precision、citation accuracy、延迟还是成本？第四步，再决定方案组合。一个常见起点是：结构化切块 + dense/sparse hybrid + rerank；如果文档长且上下文依赖强，再考虑层级切块、late chunking、contextual retrieval；如果预算紧张，就尽量把复杂度留在离线索引而不是在线推理。

还要补一句：所有选择都应该由评测集驱动。因为没有离线 benchmark，只靠主观提问，很容易把偶然答对当成真实提升。

评测怎么做：不要只看最终答案

这一篇最容易被忽略的一点是评测。你不能只看模型最后答对没，而要把链路拆开。对 chunking 来说，要看 gold evidence 是否被切到同一个或相邻 chunk、是否保留关键字段、chunk 覆盖率如何。对 embedding/召回来说，要看 recall@k、MRR、nDCG 等。对 rerank 来说，要看 top-1 / top-3 是否更靠前，是否把真正证据提到了模型能读到的位置。对最终问答来说，再看 groundedness、citation precision、答案完整性和拒答质量。

只有把链路拆开，你才知道问题出在哪。否则会出现一种很常见的误判：实际上是解析把表格读坏了，但团队却在不停换 embedding 模型；或者其实是 rerank 没加，但大家一直在调 chunk size。

项目里怎么讲出做过的感觉

讲项目时不要说我们把文档切块后存到向量库。更好的是说：我们先按文档结构解析，保留标题层级、页码、权限标签等 metadata；基础策略采用中等长度 chunk 和适度 overlap，针对表格与代码走单独解析链；首阶段使用 hybrid retrieval 保证召回，二阶段使用 rerank 提升 top-k 质量；所有变更用一套标注好的问答集和引用评测集做对比，最终观察到 recall@20、citation precision、首屏正确率等指标提升。

五、值得跟面试官分享的新动态

这一题近两年有几个更新点值得记住。第一，固定切块仍然是很多系统的起点，但越来越多团队会按文档类型做结构化切块，而不是全库一个参数。第二，late chunking 被越来越多地当成减少上下文割裂的思路来讨论，尤其适合长段文本和多句子依赖强的资料。第三，Anthropic 推出的 Contextual Retrieval 让给 chunk 自动补上下文说明成为热门实践，其公开案例提到 failed retrievals 降低 49%，加入 rerank 后可降低 67%，这使得 chunk 本身的信息贫血被更系统地讨论。第四，rerank 已经从可选优化逐渐变成很多高质量 RAG 系统的默认组件，尤其是在企业知识问答、客服、合规与代码检索场景里。第五，ColBERT 这类 late interaction 思路虽然不是新概念，但在效果与成本折中的讨论里依然经常被拿来作为代表方案。

六、常考面试题与参考答法

1. 为什么说 chunking 决定了检索系统的上限？

- 参考回答：因为首阶段召回只能在已有 chunk 上做选择。如果关键语义在切块时被拆散、表格被读坏、标题丢失或限制条件和结论被分到不相关的块里，后续 embedding 和 rerank 再强也只能在坏候选里挑相对没那么坏的，无法凭空恢复被破坏的结构。

2. chunk size 应该怎么定？

- 参考回答：不要给固定答案。应该根据文档类型、查询类型、上下文预算和评测结果决定。FAQ 或短问答适合较小粒度；制度规范、长流程说明、代码函数说明需要更完整的上下文；表格和代码常常要走特殊解析。起步可以用中等长度配合 overlap，再结合 recall@k、citation precision 和人工评审调参。

3. overlap 有什么作用，为什么不能太大？

- 参考回答：作用是减少边界截断带来的信息损失，防止定义和限制条件被切开。不能太大是因为会导致索引膨胀、重复召回、rerank 浪费和上下文冗余。它本质上是一种边界保险，而不是默认越大越好。

4. embedding 模型怎么选？

- 参考回答：要看领域、语言、查询分布、维度、延迟、成本和向量库支持。不要只看榜单。一般先用成熟通用模型起步，再用自己的评测集比较候选模型，重点看 recall、top-k 质量和线上稳定性。

5. 为什么很多系统要加 rerank？

- 参考回答：因为首阶段召回追求高吞吐和高 recall，相关性判断不够精细，容易把有点相关但不是答案证据的 chunk 排得很前。rerank 用更细粒度的相关性模型重新排序，可以显著提升 top-k 质量，从而提高最终回答质量。

6. cross-encoder rerank 和向量召回的关系是什么？

- 参考回答：向量召回适合在大规模候选中快速找一批可能相关的文档；cross-encoder 更适合对少量候选做精排序。前者负责不漏，后者负责更准。

7. late chunking 解决什么问题？

- 参考回答：它试图缓解先切后编码造成的上下文割裂。先在更长范围内建模，再得到 chunk 级表征，可以让每个 chunk 的表示包含更多全局语境。

8. Contextual Retrieval 的核心思想是什么？

- 参考回答：核心是给 chunk 自动补充它所在文档与段落的上下文说明，再结合 contextual embeddings 和 contextual BM25 进行检索，并建议用 rerank 进一步精排。它针对的是 chunk 脱离原文后信息贫血的问题。

9. 元数据为什么对检索效果重要？

- 参考回答：因为很多业务问题依赖来源、时间、版本、产品线、文档层级、权限等信息。没有这些 metadata，就很难过滤、排序、引用和做权限控制。

10. 如果预算有限，你优先优化哪一项？

- 参考回答：通常先优化解析和切块，其次引入 hybrid retrieval，再视情况加 rerank。因为数据入口质量和候选质量往往比单纯换更贵的模型更影响系统上限。

11. 如何证明 rerank 值得加？

- 参考回答：用同一套评测集对比无 rerank 与有 rerank 的 top-k 质量、引用正确率、首屏正确率和人工偏好，同时观察延迟和成本增量。如果 precision 提升明显且延迟可接受，就值得加。

12. 为什么很多项目中换 embedding 没有预期效果？

- 参考回答：常见原因是问题不在 embedding，而在文档解析坏了、chunk 边界不对、query 表达不稳定、权限过滤放错位置，或者最终生成阶段没有严格基于证据作答。

七、容易踩坑的表达

- 把 chunking 当成固定参数背诵，不说明它与文档类型、查询类型和评测指标的关系。
- 把 embedding 模型选择讲成排行榜比较，而不谈领域适配、向量维度、成本和线上吞吐。
- 把 rerank 说成提高 recall，这是不准确的。它主要提高候选排序质量。
- 忽视 metadata、标题层级、页码、表格边界、权限标签这些对企业检索系统极其关键的信息。
- 把 late chunking、late interaction、contextual retrieval 混为一谈。

九、参考资料

- Pinecone Rerank Results —— 官方 rerank 指南
- Pinecone Hybrid Search —— 官方 hybrid retrieval 指南
- Weaviate Hybrid Search —— 官方 hybrid search 文档
- Qdrant Hybrid Queries —— 官方 RRF 与 hybrid 检索文档
- ColBERT —— late interaction 代表工作
- Late Chunking for Long-Context Embedding Models —— late chunking 代表论文
- Anthropic: Introducing Contextual Retrieval —— 官方工程文章

03 Hybrid-Retrieval与Query-Rewrite（了解）

一、这篇在考察什么

这一篇不是单纯考两个技术点，而是在考你是否理解用户原始问题往往并不是检索系统最适合处理的查询表达。现实中的问题经常带有简称、歧义、隐含约束、多轮上下文、省略指代，或者同时需要词面匹配和语义匹配。只靠单一路径的 dense retrieval 往往会漏掉关键词极强的文档，只靠 BM25 又容易漏掉语义相近但词面不一致的证据。因此，Hybrid Retrieval 和 Query Rewrite 经常一起出现，因为它们分别在召回通道与查询表达两个层面补洞。

面试官真正想考的是：你是否知道为什么要把 sparse 与 dense 结合；你是否理解 query rewrite、decomposition、expansion、HyDE、step-back 这些方法各自想解决什么；你是否能说明什么时候 rewrite 会帮忙，什么时候会害人；以及你是否具备把检索链路做成多策略召回 + 查询理解 + 二阶段排序的系统意识。

面试官在这一题里，通常不是只想听一个名词解释，而是想顺着这个主题判断你是否具备下面这些能力：

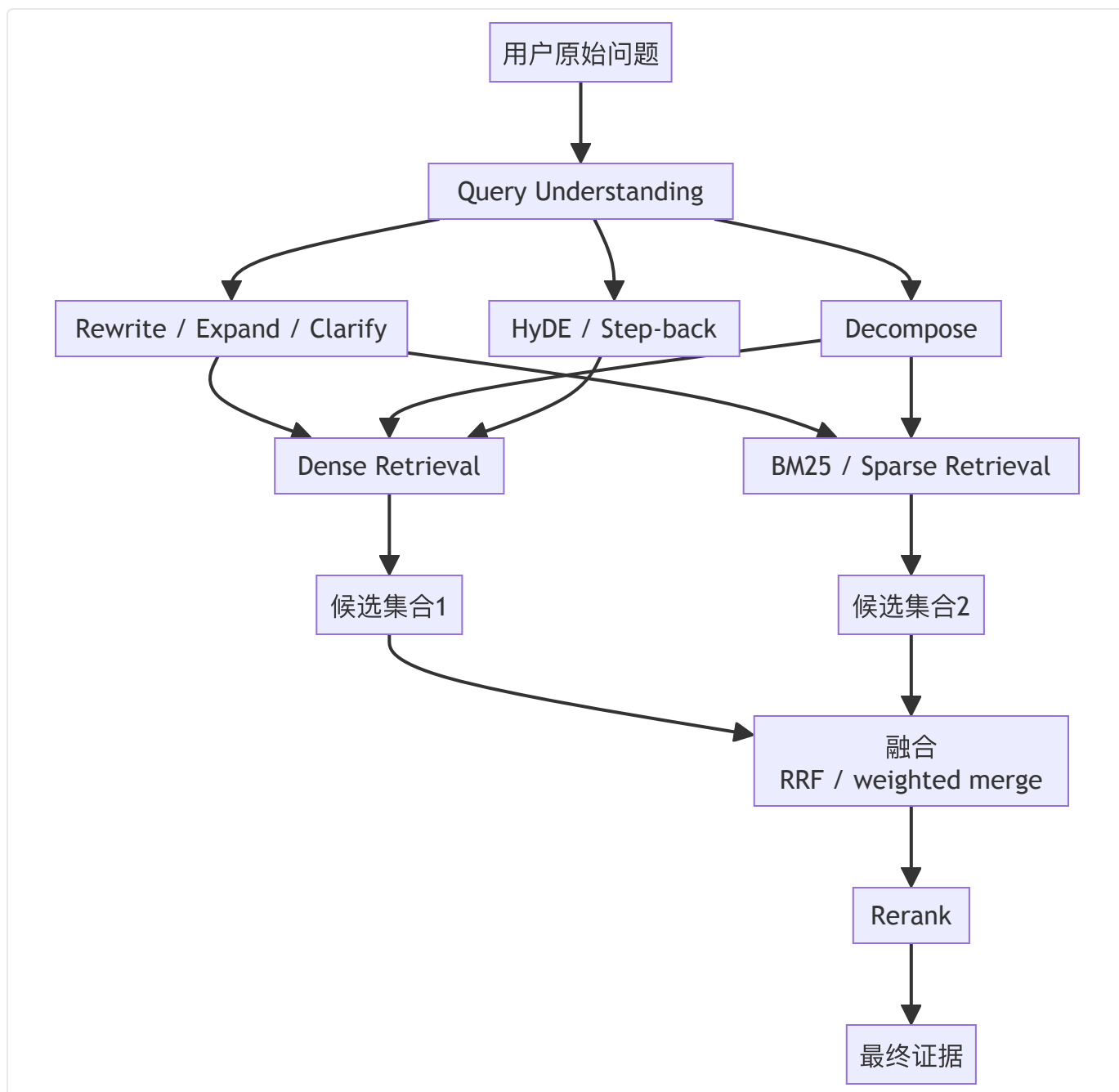
- 是否理解 sparse retrieval 与 dense retrieval 分别擅长什么，以及两者为什么经常组合而不是互相替代。
- 是否知道 hybrid search 的常见融合方式，如加权融合、分数归一化、RRF，以及它们的优缺点。
- 是否能解释 query rewrite 的必要性，包括消歧、补全上下文、显式约束、拆解多跳问题、生成假设文档等。
- 是否理解 rewrite 不是越激进越好，因为改写漂移会把正确意图带偏。
- 是否知道多轮对话检索、企业搜索和 Agent 检索里，query understanding 往往比单次相似度计算更关键。

如果你有做过类似项目，可以借鉴下面的项目表达组织：

我们知识库里有大量产品代号、版本号和内部缩写，所以只用 dense 检索会漏掉一些必须词面命中的文档。后来我们把首阶段改成 sparse+dense 并行召回，再通过 RRF 合并候选，最后让 reranker 做精排。与此同时，我们发现多轮对话里很多 query 本身不完整，于是加了一层 query understanding：先补齐会话指代，再对原问题做轻量 rewrite，但保留原 query 同时检索，防止 rewrite 漂移。离线评测里，我们把问题按精确术语型、语义改写型、多轮指代型分桶，发现 hybrid 对前两类都有帮助，而 rewrite 主要改善多轮和口语化问题。

二、先来些总的结论

- Hybrid Retrieval 的本质是用不同信号源覆盖不同类型的相关性，常见是 sparse 保词面、dense 保语义。
- Dense 很强，但对精确术语、错误拼写、版本号、产品代号、长尾别名不一定稳定，因此 sparse 往往仍然必要。
- Query Rewrite 的目的不是让问题更像自然语言，而是让问题更适合被检索系统命中。
- Rewrite、decompose、expand、HyDE、step-back 是不同策略，不要混成一个概念。
- 最稳的工程路径通常是：先做 query understanding，再做 hybrid retrieval，再用 rerank 精排，并通过评测约束 rewrite 漂移。



三、系统化八股知识

为什么 Hybrid Retrieval 成了主流工程解法

只要你的知识库不是完全干净、同质、标准化的百科数据，单一路径检索就很难稳定。企业知识库充满了缩写、产品代号、版本号、工单号、法条编号、SQL 字段名、函数名、团队黑话和中英混排。这类信息往往是词面强信号，BM25 或 sparse retrieval 很擅长；但用户也会问怎么开通新员工报销权限这种和文档词面不完全一致的问题，这时 dense retrieval 更容易抓住语义相近关系。

所以 Hybrid Retrieval 的核心不是为了炫技把两种检索拼起来，而是承认不同相关性信号各有盲区。Sparse 检索擅长精确关键词、长尾标识符和词面约束，dense 检索擅长语义近义、改写表达和上下文相似。把两者结合，可以显著减少精确词没命中和语义像但没有关键词这两类漏召。

Sparse、Dense 各自擅长什么

Sparse retrieval 典型代表是 BM25 或学习式稀疏检索。它依赖倒排索引，对关键词匹配、实体名、版本号、编号、拼写接近的词面线索特别有效，延迟低、可解释性强，也更容易做布尔过滤和权限预筛。Dense retrieval 则把查询和文档映射到向量空间，更适合近义表达、paraphrase、长文本语义匹配、多语言表达等场景。

但两者都有天然弱点。Sparse 容易错过语义相近但词面差异大的表达，尤其是中文口语化问题对制度性书面文档的检索；dense 则可能在编号、专有名词、产品代号、细粒度术语上不稳，还可能被高频泛化语义吸走，出现看上去相关但不够精准的问题。因此混合召回几乎成了生产系统的常见配置。

Hybrid Retrieval 怎么融合

Hybrid 不是简单把两个 top-k 拼在一起。你需要考虑不同检索器打分空间不一致的问题。常见融合方式有三类。

第一类是**线性加权**。对 sparse 分数和 dense 分数做归一化后加权，适合你能稳定估计两边权重、分数分布也比较平稳的场景。第二类是**排序级融合**，代表是 RRF (Reciprocal Rank Fusion)。Qdrant 等官方文档都明确给出了这种公式：最终得分与文档在各路结果中的 rank 倒数相关，而不是依赖原始分数的绝对值。RRF 的好处是稳健，不需要强依赖不同模型的分数校准。第三类是**两阶段并联 + rerank**。先分别召回，再把候选合并后让 reranker 判断最终排序。很多工程团队更喜欢这种方式，因为它对底层检索器的异构性更包容。

Weaviate 官方文档就强调 hybrid search 会并行执行向量搜索和 BM25，再对结果做融合；Pinecone 文档也反复强调 hybrid 往往要配合 rerank 使用。这些官方实践本身就说明：Hybrid 的关键不是用两种索引，而是**怎么把多路信号整合成最终候选排序**。

什么时候一定要加 sparse

如果你的问题里经常出现下面这些东西，就几乎一定要把 sparse 纳入方案：产品代号、政策编号、合同条款号、报错码、函数名、类名、SQL 字段、工单号、药品名、证券代码、SKU、型号、精确版本号、团队内部黑话。这些词一旦没被精确匹配到，dense 往往没有足够信心把对应文档排到前面。

另外，多租户系统里经常有大量模板化文档，内容语义相似但只差一些关键字段。如果只靠 dense，容易把邻近模板混起来；加 sparse 之后，很多必须命中的词面约束才有稳定保障。

Query Rewrite：为什么原始问题经常不适合直接检索

用户说的话通常是面向人的，不是面向检索器的。比如我想知道新同学入职以后办公设备怎么领，真正适合检索的形式可能是新员工 入职 办公设备 领取 流程或者IT 资产申请 新员工 办公设备 配置。再比如多轮对话里，用户第二轮问那审批人是谁，如果你不把上一轮里的出差申请补进查询，检索器拿到的 query 基本不可用。

所以 Query Rewrite 的目标不是润色语言，而是**重建检索意图**。它通常包含四种动作：补全被省略的上下文、显式化隐含约束、消歧、把复杂问题变成更容易召回的子问题。你要把 rewrite 看作 retrieval engineering，而不是 prompt polish。

Rewrite、Expansion、Decomposition、HyDE、Step-back 分别是什么

这几个概念面试里很容易混。

Rewrite 通常指把用户原问题改写成更适合检索的查询，例如补齐主语、消除代词、省略和口语。

Expansion 通常指扩展关键词、同义词、实体别名、相关术语，让 query 覆盖更多可能的表达。

Decomposition 指把一个复杂问题拆成多个子问题，适合多跳、组合约束或先查 A 再查 B 的检索。

HyDE (Hypothetical Document Embeddings) 指先让模型根据问题生成一段假想答案文档，再用这段文档做向量检索，适合原始问题过短、语义稀疏时补足语义锚点。

Step-back prompting 则更偏抽象化，把问题提升到更一般的层级，以便检索更基础的原理或背景知识，再回到具体问题。

你可以把它们看作对 query 的不同改造方式：rewrite 是让问题更清楚，expansion 是让问题覆盖更广，decomposition 是把问题拆细，HyDE 是让问题更像答案片段，step-back 是让问题站高一点看。

Rewrite 什么时候会害人

这很多候选人把 rewrite 说成纯正向优化，但实际上它非常容易引入**查询漂移**。比如用户想问离职员工的账号多久自动停用，模型 rewrite 成账号权限回收流程，听起来相关，但如果企业内部把离职停用和权限回收分属不同流程，就可能把真正证据带偏。又比如多轮对话中，模型错误继承上一轮语境，会把当前问题强行绑定到错误主题上。

因此 rewrite 要有边界。第一，尽量保留原 query 的关键信号，不随意替换实体词。第二，对高风险实体可以采用原 query + rewrite query 并行检索，而不是只信改写后的版本。第三，评测时要单独统计 rewrite 之后 recall 变化和错误类型，不要只看最终答案。第四，必要时让模型输出 rewrite rationale 或结构化字段，便于观测。

对话检索与 Agent 检索里的 query understanding

一旦进入多轮对话和 Agent 系统，query understanding 的重要性会明显上升。因为用户的表达经常是不完整的，甚至是任务级指令而不是信息级问题。例如帮我看上周那个会议里谁负责跟进预算把刚刚那个接口报错的排查方式找一下，这种问题如果不做会话状态补齐，检索器根本不知道上周那个会议和刚刚那个接口具体指什么。

在 Agent 系统里，查询甚至可能不是用户直接说的，而是 planner 生成的 tool query。这时 query rewrite 不再只是一个小优化，而是 agent reasoning 与 retrieval 之间的接口层。你要让面试官感觉到你知道：检索质量不只是底层索引能力，还是上游 query formulation 能力。

如何做评测：把每种查询拆开看

Hybrid 与 rewrite 的评测不能只看混合后的最终问答。更好的做法是把问题分成若干类型：精确术语型、语义近义型、长尾别名型、多轮指代型、多跳型、组合约束型，然后分别看 sparse、dense、hybrid、rewrite+hybrid 的表现。这样你才能知道某个优化到底是在哪一类问题上真正有效。

对于 rewrite，还应单独记录：原始 query 的 recall@k，rewrite 后的 recall@k，原始+rewrite 双路并发后的 recall@k，以及 rewrite 失败案例的比例。只有这样，面试里你才能有理有据地说我们不是盲目加 rewrite，而是知道它在什么问题上的提升最大、在什么问题上的风险最大。

工程上的推荐默认配置

如果你需要给出一个务实的默认配置，可以这么说：对通用企业知识问答，默认先做结构化 query understanding，保留原 query；并行做 sparse + dense 召回；融合时优先用更稳的 RRF 或 merge 后 rerank；对多轮场景加入会话补齐；对复杂问题视情况启用 decomposition 或 agentic retrieval。这样一套组合，通常比只靠一个 embedding 模型检索 top-k 更稳。

面试官一般不要求你给出唯一正确方案，但会很看重你是否知道每一步是在对抗什么错误类型。

四、一些值得跟面试官分享的变化

近两年的明显变化有三点。第一，Hybrid Retrieval 已经从可选增强越来越接近生产默认，因为 dense 与 sparse 的互补性在企业场景里非常明显，多个主流向量数据库和检索框架都把 hybrid search 做成了一等能力。第二，RRF 这类排序级融合因为稳健、无需分数严格对齐，在工程上越来越常见。第三，query rewrite 不再只是简单同义词扩展，而是逐渐演化出针对多轮对话、复杂推理、Agent 工具调用的 query understanding 管线，包含 rewrite、decomposition、HyDE、step-back、clarification 等多种策略。学术上也出现了像 RQ-RAG、GuideCQR 等更系统地研究查询重写与检索质量关系的工作。

五、常考面试题与参考答法

1. 为什么 dense retrieval 这么强了，生产里还要 sparse?

- 参考回答：因为 dense 擅长语义相似，但对编号、缩写、型号、产品代号、函数名、版本号等词面强约束不一定稳定。企业知识库恰好充满这类长尾标识符，所以 sparse 仍然非常重要。

2. Hybrid Retrieval 的本质是什么?

- 参考回答：本质是用多路不同的相关性信号覆盖不同错误类型，让 sparse 负责词面、dense 负责语义，再通过融合和 rerank 得到更稳的候选集合。

3. RRF 为什么常被用来做融合?

- 参考回答：因为不同检索器的原始分数往往不可直接比较，RRF 只依赖排序位置，稳健、实现简单，对异构检索器友好。

4. 什么时候线性加权比分数级融合更合适?

- 参考回答：当你对不同通道的分数分布和贡献比较清楚，并且能通过验证集稳定调出权重时，线性加权能更细致地利用分数信息。否则排序级融合通常更稳。

5. Query Rewrite 的目的是什么?

- 参考回答：不是润色自然语言，而是把原始问题改造成更适合被检索系统命中的表达，包括补全上下文、消歧、显式化约束和改善词项覆盖。

6. rewrite 和 expansion 有什么区别?

- 参考回答：rewrite 更强调保留原意并重述；expansion 更强调增加同义词、别名、相关词，扩大召回覆盖面。

7. HyDE 适合什么场景?

- 参考回答：适合原始问题很短、语义稀疏或者问法过于抽象时。先生成一段假想答案文档再做向量检索，可以给 retrieval 一个更丰富的语义锚点。

8. step-back prompting 为什么可能帮助检索?

- 参考回答：因为用户问题有时过于具体而词面不稳定，先上抽象层找到通用原理或上位概念，再回到具体问题，可以让检索更容易命中基础背景材料。

9. query rewrite 有什么风险?

- 参考回答：最大的风险是查询漂移，把问题改偏，尤其是在多轮对话、实体词替换和隐式约束补全时。高风险场景最好保留原 query 并行检索。

10. 你会怎么设计多轮对话检索的 query understanding?

- 参考回答：先从会话状态里补齐指代和省略，再抽取显式实体、时间和约束，必要时保留原 query 与 rewrite query 双路检索，对复杂问题做 decomposition，最后统一 rerank。

11. Hybrid Retrieval 和 rerank 的关系是什么?

- 参考回答：Hybrid 扩大并稳定候选池，rerank 在候选池上做精排序。前者解决别漏，后者解决谁排前。

12. 项目里怎么证明 hybrid 比单路检索好?

- 参考回答：按问题类型分桶对比 sparse、dense、hybrid 的 recall@k、nDCG、top-5 引用命中率和人工偏好，尤其看长尾实体和语义改写类问题上的增益。

六、容易踩坑的表达

- 把 hybrid 讲成向量库支持稀疏和稠密所以我们就开了而不说明它解决什么错误类型。
- 把 query rewrite 说成纯正向能力，不提查询漂移和双路保底。
- 分不清 rewrite、expansion、decomposition、HyDE、step-back。
- 只看最终答案不看分桶评测，导致无法判断 hybrid 或 rewrite 具体在哪类问题上有效。
- 忽略多轮对话场景中的会话补齐和指代消解。

七、参考资料

- Pinecone Hybrid Search —— 官方 hybrid 检索文档
- Weaviate Hybrid Search —— 官方 hybrid 搜索文档
- Qdrant Hybrid Queries —— 官方 RRF / hybrid 查询文档
- Hypothetical Document Embeddings —— HyDE 论文
- Take a Step Back: Evoking Reasoning via Abstraction —— step-back prompting 论文
- RQ-RAG: Learning to Refine Queries for Retrieval Augmented Generation —— query rewrite for RAG
- GuideCQR —— 对话检索中的查询重写研究
- LangChain / LangGraph retrieval & query analysis docs —— 工程化检索与 query analysis 实践

04 RAG-vs-Memory（重要）

一、这篇在考察什么

主要考察你是否能把知识系统中的不同信息载体区分清楚：什么是知识库，什么是会话上下文，什么是长期记忆，什么是用户画像，什么又只是缓存或者状态。很多候选人会把所有额外给模型的信息都统称为 context，这在工程上是不够精确的。

更高质量的回答必须说明：RAG 的核心对象通常是外部知识证据，偏向从库里查资料；Memory 的核心对象通常是与用户、任务、经历、偏好、状态相关、需要跨轮或跨会话延续的信息，偏向系统对人和任务的持续认知。二者在读写时机、存储形态、更新方式、权限边界、评测方式上都不同。面试官真正想看的是你能不能设计一个既有知识检索、又有用户记忆、还有 session state 的 Agent 系统，而不是把它们混成一锅。

面试官在这一题里，通常不是只想听一个名词解释，而是想顺着这个主题判断你是否具备下面这些能力：

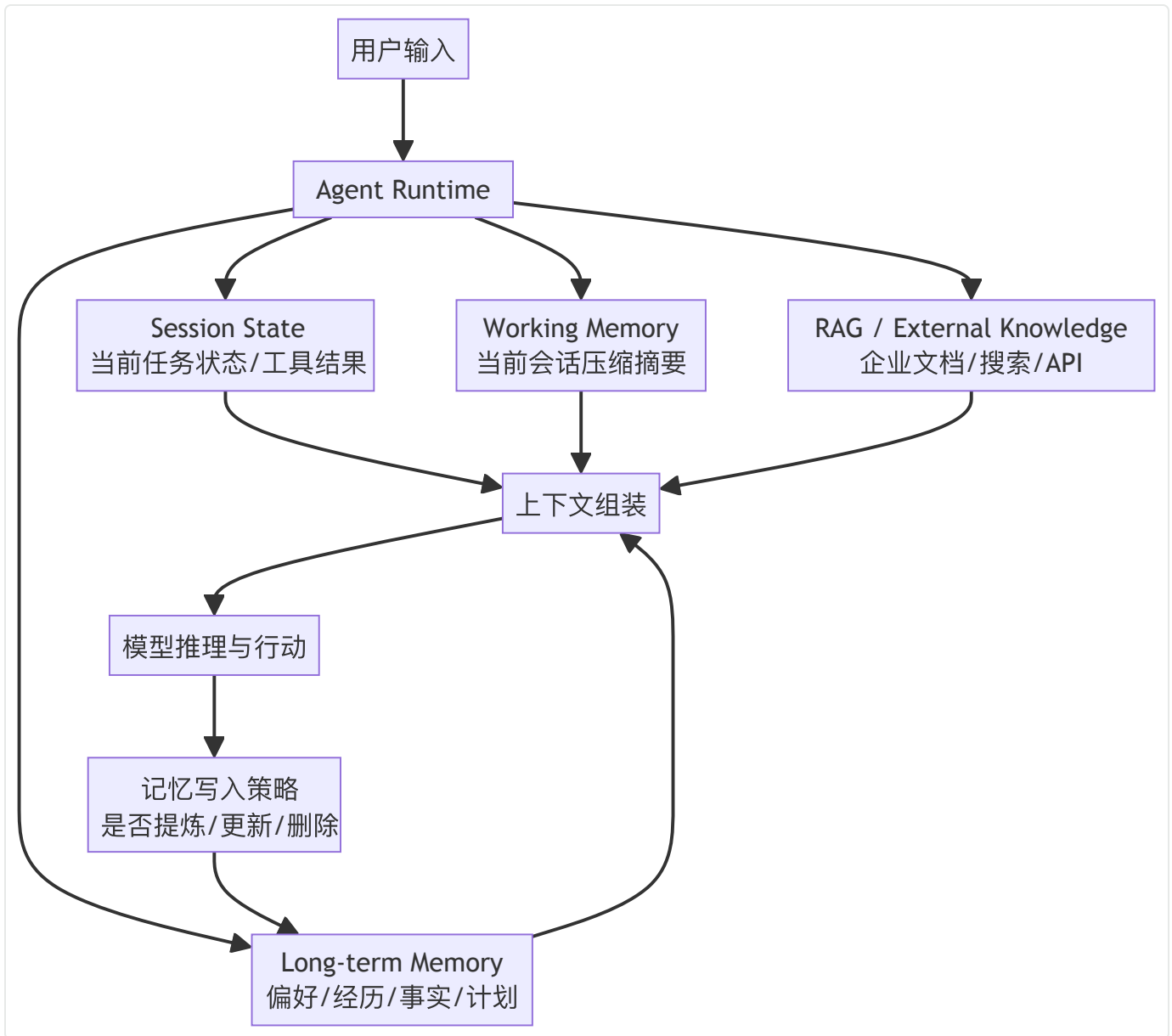
- 是否能准确区分知识库检索、用户长期记忆、会话工作记忆、缓存、状态存储这几类概念。
- 是否知道 RAG 更偏外部证据访问，Memory 更偏个体化持续信息管理，两者服务对象不同。
- 是否理解 memory 不是把聊天记录全塞进上下文，而是要经过抽取、压缩、写入、检索和淘汰。
- 是否能说明 RAG 与 memory 在 read/write pattern、schema 设计、权限与删除要求上的差异。
- 是否了解 LangGraph、Letta 等框架在记忆体系上的官方设计思路，以及近年的 agent memory 研究方向。

可以通过引入一个项目来把这件事在面试中讲的更加清楚：

我们把系统分成四层：当前流程状态放 session state，当前会话的压缩摘要和中间结论放 working memory，用户偏好和可跨会话复用的信息放 long-term memory，企业制度和文档走 RAG。这样做是为了避免把所有信息都塞进一个库里。比如用户明确说以后回答简洁一些，我们会写入用户作用域的 semantic memory；如果只是当前任务里确认了某个参数，就只写 session state。真正需要引用出处的制度和产品知识不放 memory，而是走知识检索。这样系统既能个性化，又不会把用户偏好和客观知识混掉。

二、一些总的结论

- RAG 查的是‘世界和业务知识’，Memory 记的是‘这个用户、这个任务、这段经历’。
- RAG 通常读多写少，memory 通常既要读也要写，而且写入要有策略，不是每轮都存。
- 会话上下文不等于长期记忆，缓存也不等于记忆，状态机字段更不等于知识库。
- 一个成熟 Agent 往往同时有：session state、working memory、long-term memory、external knowledge 四层。
- 面试里只要你能把对象、写入时机、检索方式和治理要求讲清，基本就能把这题讲透。



三、系统化八股知识

先把几个概念分开：知识、记忆、上下文、状态、缓存

面试里最容易失分的地方，就是把这些概念全部混成给模型的上下文。其实它们不是一回事。

知识库 / RAG 数据源：偏向相对客观、可共享、可追溯的外部知识，比如公司制度、产品文档、数据库记录、工单系统、网页搜索结果。这类信息通常和某个用户无关，重点在可检索、可更新、可引用。

Memory：偏向某个用户、某个会话群体、某个 Agent 或某段任务执行过程中沉淀下来的持续信息，比如用户偏好、常用表达、过去做过的决定、已完成的子任务、显式提供的背景资料。

Session State：偏向当前流程运行所需的结构化状态，比如当前 step、已调用过的工具、上一步结果、待确认参数、审批单号。

Context Window：是模型这一轮真正能看到的输入载荷，不等于你的全部知识系统。

Cache：通常是为了节省成本和延迟而复用先前结果，例如 prompt cache、embedding cache、工具结果缓存，不代表系统理解并记住了什么。

只要你先把这几个层次讲清楚，RAG vs Memory 这道题基本就有了框架。

RAG 与 Memory 的核心区别：服务对象不同

RAG 的第一职责是回答世界中有什么信息。例如公司报销制度、某个 API 文档、某个工单状态、某篇文章内容。它强调证据来源、覆盖范围、可更新性和可追溯性。Memory 的第一职责则是回答系统应该持续记住关于这个用户、这个任务、这个 Agent 的什么信息。例如用户喜欢简洁回答、用户正在找后端岗位、这个任务上一次已经完成了需求澄清、该客户偏好邮件沟通、这个 Agent 曾经尝试过 A 方案失败。

所以两者的根本区别不是一个放向量库，一个放数据库，而是**服务对象与更新规律不同**。知识库通常是共享资源，很多用户共用；memory 通常是个体化资源，要考虑作用域和隐私。知识库常常由外部文档变更驱动，memory 常常由交互与经历驱动。

读写模式不同：RAG 多读少写，Memory 读写并重

这是工程上非常关键的区别。RAG 系统当然也会写，比如导入新文档、增量更新索引，但对单次用户交互来说，它更像一个以读取为主的知识访问层。Memory 则不同，它除了读，还要决定**什么时候写、写什么、写到哪里、是否覆盖旧记忆、是否需要用户确认**。

如果把每轮对话都原样写进 memory，会产生两类问题：一是噪声爆炸，二是错误沉淀。更合理的做法是把 memory 写入当作一个提炼过程，只在满足显著性条件时写入。例如用户明确表达长期偏好、披露稳定背景、做出持续性决定、要求记住某事，或者某段任务经验对未来有复用价值时，才写入长期记忆。LangGraph 官方文档就把长期记忆区分为 semantic、episodic、procedural 等类型，并强调可以在热路径或后台异步更新；这反映的正是 memory 是主动管理，不是日志备份。

Memory 的典型分类：working / episodic / semantic / procedural

如果面试官继续追问那 memory 有哪些类型，你可以给出一个四分法，这也是近年很多 Agent memory 讨论里较常见的结构。

Working memory：当前任务或当前会话中临时有用的信息，类似短时工作记忆，比如这轮已经确认的参数、上一步工具结果、临时中间结论。它生命周期短，通常跟 session 绑定。

Episodic memory：关于过去经历和事件的记忆，例如上次给这个用户推荐过 X 方案但他不满意上周这个任务调用过某工具失败。

Semantic memory：关于用户或世界的稳定事实，例如用户偏好、常用单位、所在行业、岗位意向、团队术语释义。

Procedural memory：关于做事方式或规则的记忆，例如某类任务的固定 SOP、代码生成规范、审批路径、回复风格模板。

LangGraph 的记忆文档直接采用了 semantic、episodic、procedural 这种划分；这在面试里是一个很好的既学术又工程的回答方式。

Letta 和 LangGraph 的设计思路能说明什么

你可以借助官方框架的设计来讲出 memory 不是玄学。LangGraph 官方把长期记忆设计成命名空间中的 JSON 文档，由 store 持久化，强调长期记忆是跨线程/跨会话存在的，并可根据语义、情节、程序三类进行管理。Letta 则把记忆分成 memory blocks 和 archival memory：前者是始终位于 agent 上下文中的结构化记忆块，更像高优先级、常驻、显式可编辑的记忆；后者则是按需检索的长期档案记忆，更像一个语义可搜索的长期存储层。

这两个设计恰好能帮你讲清一个关键点：并不是所有 memory 都应该放进当前上下文窗口。有些记忆是**常驻显式信息**，比如用户名字、回答偏好、任务约束；有些记忆更适合**按需检索**，比如用户几个月前某次复杂讨论的详细过程。前者要短小、稳定、结构清晰；后者要可搜索、可压缩、可版本化。

RAG 与 Memory 在检索逻辑上的不同

虽然两者都可能使用相似度检索，但查询逻辑不一样。RAG 的查询通常围绕当前问题的知识意图展开，例如报销流程某个 API 的鉴权方式合同违约条款。Memory 的查询往往围绕个体化相关性展开，例如这个用户偏好什么这个任务之前做到哪一步了我们之前为什么放弃过方案 A。

这意味着 memory 检索要更关注作用域和时间性。比如同一句预算是多少，对知识库来说可能去检索制度或报表；对 memory 来说可能要先看看当前任务是否已经讨论过预算。一个成熟 Agent 的上下文组装通常不会只做一次统一检索，而是按槽位拉取：先取 session state，再取 working memory，再视情况查询 long-term memory，最后再去外部知识库找证据。

什么时候该用 RAG，什么时候该用 Memory

可以记一个简单判断：如果信息本质上属于共享知识、客观资料、可外部验证证据，优先走 RAG；如果信息本质上属于用户偏好、个体背景、长期任务状态、历史经历，优先归入 memory。

例如Java 里 HashMap 的扩容机制是什么显然是外部知识；这个用户更喜欢代码示例还是概念解释是 memory；当前工单已经审批到哪一步更像 session state 或工具状态；上次我们为什么选择 A 框架而不是 B 如果未来可能复用，可沉淀为 episodic memory。用这个区分法，面试官通常会觉得你理解了系统边界。

为什么很多人误把会话摘要当长期记忆

因为会话摘要很直观：把长对话压缩一下，下轮继续喂给模型，看起来就像记住了。但会话摘要更多是上下文管理手段，不等于真正的长期记忆。摘要的目标是让模型在 token 预算内继续理解当前对话，而长期记忆的目标是跨会话、跨时间地保存未来仍有价值的信息。两者的写入标准、生命周期和治理要求都不同。

摘要更偏为了当下推理方便，长期记忆更偏为了未来复用与个性化。如果你把摘要当长期记忆，容易把大量临时噪声、错误猜测和不再适用的信息永久留下。

删除、纠错与权限：Memory 比 RAG 更敏感

因为 memory 往往与具体用户绑定，所以删除、修改、撤回和可见性控制比知识库更敏感。用户可能要求不要再记住这个偏好把我之前的个人信息删掉这段对话不要用于后续个性化。这些要求在知识库层也存在，但在 memory 层尤为突出。换句话说，memory 设计天然要面对隐私和治理，而不能只从效果出发。

你在面试里可以补充：memory 需要明确 schema、来源、更新时间、置信度、作用域、是否用户确认过、删除接口和审计记录。只有这样，系统才能既个性化又可控。

一个好用的系统分层方式

如果你被问那你会怎么设计，推荐给出一个四层模型：

第一层是 **session state**，保存当前流程状态、工具参数、中间产物；

第二层是 **working memory**，保存当前会话压缩摘要和临时任务信息；

第三层是 **long-term memory**，保存用户偏好、稳定背景、任务经历和高价值经验；

第四层是 **external knowledge / RAG**，访问企业知识库、搜索、数据库和 API。

模型每轮并不是盲目读全量，而是按需拉取这些层的数据拼装上下文。这样回答通常既清晰又工程化。

四、值得在面试里分享的演进点

最近两年的一个明显变化，是大家不再把记忆简单理解为聊天历史，而是更系统地讨论 Agent memory 的分类、写入策略和生命周期。LangGraph 官方文档明确区分长期记忆与短期线程状态，并把长期记忆分为 semantic、episodic、procedural。Letta 官方则把常驻 memory blocks 与可按需检索的 archival memory 分开。学术上，《Memory in the Age of AI Agents》等工作也开始从 token-level、parametric、latent 等视角讨论更广义的记忆形态。这说明业内正在把记忆从一个产品体验词，逐渐沉淀为一个可建模、可治理、可评测的系统层。

五、常考面试题与参考答法

1. RAG 和 Memory 最本质的区别是什么？

- 参考回答：RAG 主要解决外部知识访问与证据提供，Memory 主要解决个体化持续信息管理。前者面向世界和业务知识，后者面向用户、任务和经历。

2. 为什么会话历史不等于长期记忆？

- 参考回答：会话历史是原始交互日志，会话摘要是为了继续推理做的压缩，长期记忆则是经过筛选后留下、可跨会话复用的高价值信息。三者生命周期和用途不同。

3. 缓存是不是一种记忆？

- 参考回答：通常不是。缓存的目标是复用结果、降低成本或延迟，不代表系统理解了信息的重要性，也不一定有长期语义价值。

4. session state 和 memory 有什么区别？

- 参考回答：session state 更偏当前流程运行状态和结构化字段，如当前 step、工具结果、参数；memory 更偏可复用认知，如偏好、经历、稳定事实。

5. 为什么说 memory 需要写入策略？

- 参考回答：因为不是所有交互都值得存。没有写入策略会造成噪声堆积、错误沉淀和隐私风险。

6. semantic、episodic、procedural memory 分别是什么？

- 参考回答：semantic 是稳定事实，episodic 是过去经历事件，procedural 是做事方式或规则。

7. 什么时候优先走 RAG，什么时候优先查 memory？

- 参考回答：共享知识、可验证资料优先走 RAG；与某个用户或任务绑定的长期信息优先查 memory。

8. 为什么 Memory 比 RAG 更强调删除和权限？

- 参考回答：因为 memory 往往包含用户偏好和个人信息，作用域更敏感，必须支持纠错、删

除、确认和审计。

9. Letta 的 memory blocks 和 archival memory 有什么区别？

- 参考回答：memory blocks 是常驻上下文中的显式结构化记忆，archival memory 是按需检索的长期档案记忆。

10. LangGraph 里的长期记忆是什么形态？

- 参考回答：官方设计中长期记忆通常以命名空间下的 JSON 文档形式持久化，通过 store 跨线程/跨会话访问。

11. 如果项目里用户说‘记住我以后都用简短回答’，你会怎么处理？

- 参考回答：这是典型的长期偏好，应该写入用户作用域的 semantic memory，并在后续上下文组装时以高优先级常驻或显式注入。

12. 如果用户说‘上一轮那个预算先不用了’，你会更新哪层？

- 参考回答：优先更新当前任务相关的 working memory / session state；如果这是长期决策且未来会复用，再视情况同步到长期记忆。

六、容易踩坑的表达

- 把一切额外输入都称为 context，不区分知识、记忆、状态、缓存。
- 把聊天记录原样长期保存，误以为这就是 memory。
- 讨论 memory 时只谈存储，不谈写入标准、删除机制和权限边界。
- 把 RAG 和 memory 的检索逻辑混为一谈，不说明它们服务对象不同。
- 忽略 session state 在 Agent runtime 中的重要性。

七、参考资料

- LangGraph Memory Overview —— 官方记忆体系说明
- LangGraph Long-term Memory —— 跨线程长期记忆示例
- Letta Memory Blocks —— 官方常驻记忆块文档
- Letta Archival Memory —— 官方长期档案记忆文档
- Memory in the Age of AI Agents —— 关于 agent memory 的较新综述
- Anthropic / OpenAI / LangGraph agent memory related docs —— 工程实践背景

05 长期记忆与用户记忆（重要）

一、这篇在考察什么

这一篇是在上一题RAG vs Memory基础上的深挖题。面试官不再满足于你会区分概念，而是要看你能不能把长期记忆和用户记忆真的设计出来。也就是说，不只是知道记住用户偏好这句话，而是要说明：记什么、何时写、怎么更新、怎么删、如何召回、如何评测、怎样避免记错和过度个性化。

校招生经常会把用户记忆理解成保存用户资料卡，这只覆盖了最浅的一层。真正可用的长期记忆系统，至少要覆盖稳定偏好、显式用户事实、跨会话任务进度、历史经历、失败经验、常用格式、共享记忆、忘却与纠错机制。近两年 Agent memory 的研究和框架实践也都在朝这个方向发展：把记忆做成独立层，而不是聊天记录附件。

面试官在这一题里，通常不是只想听一个名词解释，而是想顺着这个主题判断你是否具备下面这些能力：

- 是否知道用户记忆不是简单 profile，而是包含偏好、事实、经历、计划、共享上下文等多个层次。
- 是否理解长期记忆必须有写入策略、更新策略、遗忘策略和用户可控策略。
- 是否能设计 memory schema，包括来源、置信度、时间戳、作用域、冲突处理与可删除性。
- 是否了解 Letta memory blocks / archival memory、LangGraph long-term memory、MemoryOS、A-MEM / Agentic Memory 等近年代表方向。
- 是否能从产品、工程、隐私和评测四个维度讲长期记忆，而不只是谈个性化体验。

项目里可以这么讲：

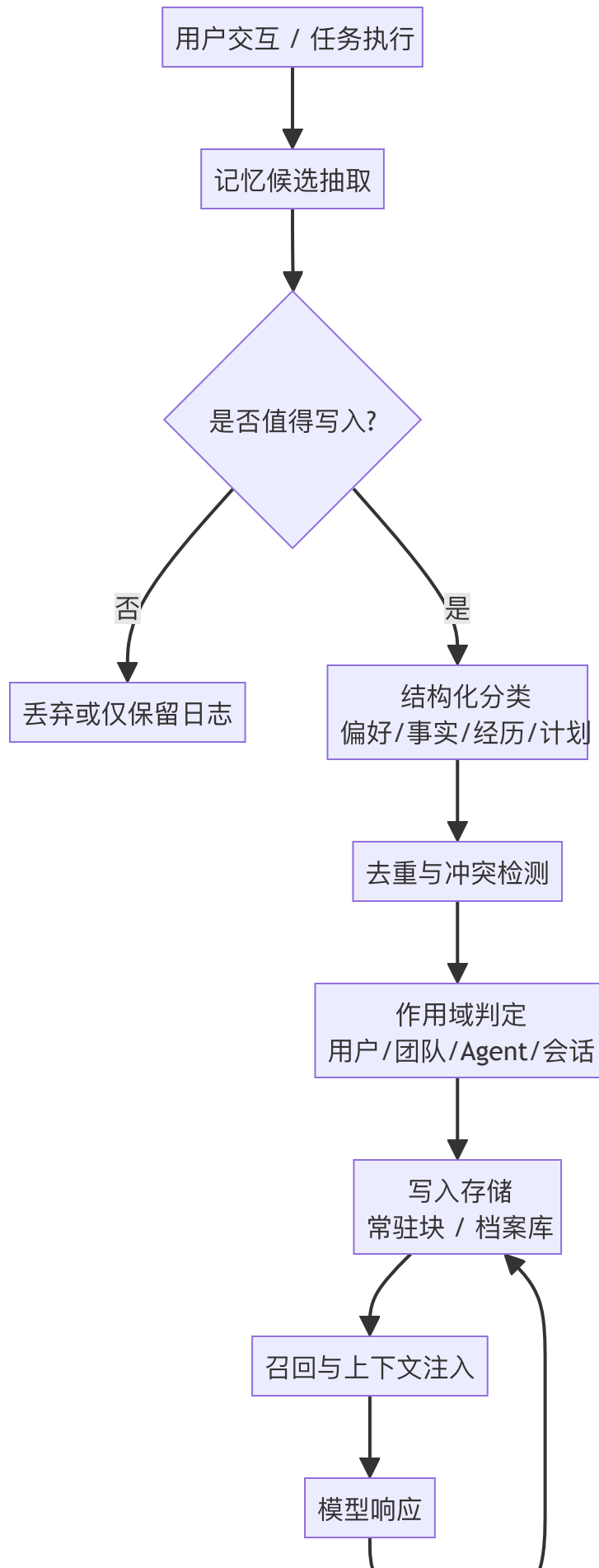
我们把用户长期记忆拆成两层：一层是结构化 profile，记录回答偏好、岗位方向、长期项目目标等稳定字段；另一层是 episodic memory，保存重要经历和历史决策。写入上不是每轮都存，而是先抽取候选记忆，再按显著性、稳定性和来源可信度筛选。高频稳定的信息做成常驻记忆，长事件按需检索。记忆记录里会带来来源、时间戳、置信度和作用域，冲突时优先显式用户输入，并保留版本。这样做比单纯保存聊天历史更稳，也方便用户查看、修改和删除。

二、核心结论

- 用户记忆的核心不是‘多存’，而是‘有选择地存、可验证地用、可纠错地改、可删除地管’。
- 长期记忆应区分稳定偏好、显式事实、经历事件、任务计划、共享记忆，不应混成一个自由文本字

段。

- 高质量 memory 写入依赖 salience 判断、结构化抽取、去重合并与冲突更新。
- 常驻记忆和可检索记忆要分层：重要短事实常驻，长经历按需取。
- 没有治理的长期记忆会迅速变成噪声和风险源，尤其是在隐私与错误沉淀方面。





反馈校验
用户修正/显式删除/衰减

三、系统化八股知识

长期记忆到底要记什么

这是设计长期记忆的第一问。如果你的答案只有记住用户偏好，面试官大概率会继续追问，因为这太窄了。一个更完整的分类可以分成五类。

第一类是**稳定偏好**，例如回答风格、格式偏好、常用语言、是否喜欢表格、是否希望先给结论后给细节。

第二类是**显式用户事实**，例如岗位意向、所在行业、是否是应届生、使用的技术栈、所在时区、长期项目背景。

第三类是**经历与事件**，例如某次任务失败原因、之前尝试过的方案、用户明确否决过的选项。

第四类是**计划与承诺**，例如下周继续完善知识库第 04 章这轮面试准备重点攻克 Agent Runtime。

第五类是**共享记忆**，例如同一个团队多个 Agent 共享的项目背景、当前目标、标准约束。

这五类信息的共同点是：它们在未来还有复用价值，而且与某个用户或任务作用域相关。反过来说，一次性的寒暄、暂时性的猜测、模型自己的臆测，都不应该轻易写入长期记忆。

为什么‘原样保存聊天历史’不是长期记忆系统

原样保存聊天历史最多只是有了日志。日志当然重要，但它不是长期记忆本身。长期记忆系统的难点在于**抽取、压缩、结构化和持续演化**。因为未来真正需要被召回的，往往不是几十轮聊天原文，而是从里面提炼出来的稳定信息。比如用户连续三次都要求回答尽量简短，你需要沉淀的是偏好：简洁回答，而不是三段原始对话全文。

此外，原始聊天历史天然带噪声：用户会临时改变想法，会说错，会反悔，模型也可能误解。如果不做结构化处理直接写长期记忆，系统很快就会被噪声和错误污染。近年的 agent memory 研究，包括 A-MEM、Agentic Memory、MemoryOS 等，都在强调记忆管理不是简单存储，而是组织、更新和检索的问题。

写入策略：什么时候值得写入

长期记忆最核心的不是怎么查，而是什么时候写。因为写错一次，后面可能每轮都被错误强化。一个实用的写入标准可以从四个角度判断。

第一是**显著性**：这条信息未来复用概率高不高？第二是**稳定性**：它是长期偏好还是临时想法？第三是**验证性**：这是用户明确说的，还是模型猜的？第四是**作用域价值**：它影响的是当前会话、这个用户的未来对话，还是团队/多个 agent 的共享任务？

工程上常见的策略有三类。

其一，**显式触发**：用户说请记住.....以后都这样.....，可信度最高。

其二，**规则触发**：如偏好、个人资料、计划节点、长期项目背景等，满足模式就候选写入。

其三，**模型提炼 + 审核**：模型提取候选记忆，再通过规则、置信度阈值、用户确认或后台审查决定是否落库。

你在面试里要明确表达：长期记忆不是每轮写，而是候选生成后经过筛选才写。

更新、冲突与遗忘：记忆不是只增不减

这是高分点。很多人会谈写入，却不谈更新和删除。事实上长期记忆如果只有追加、没有演化，很快就会变脏。比如用户一开始说我偏好详细解释，后来又明确说以后尽量简短，这不是再加一条新记忆就完了，而是要识别出冲突、更新旧值，甚至保留版本和更新时间。

一个可靠的 memory schema 至少应包含：字段类型、值、来源、时间戳、置信度、作用域、是否用户确认过、是否允许自动覆盖、历史版本链接。对于显式用户事实和偏好，通常更适合用结构化 key-value 或 JSON 文档，而不是一长段自由文本。对于经历型记忆，则可以是事件记录或 note 形式。

遗忘机制也很重要。不是所有记忆都值得永久存在。你可以引入 TTL、最近使用衰减、显著性重估、用户显式删除等策略。MemoryOS 等工作就把短期、中期、长期做成分层存储，本质上也是在承认：不同记忆应该有不同生命周期。

常驻记忆 vs 可检索记忆：为什么要分层

让模型记住一切最直接的方法，是把所有重要信息都塞进 prompt。但这会很快撞到上下文预算，而且噪声会越来越大。所以长期记忆系统几乎都需要分层。

一种实用分法是：**常驻记忆**只保留极少量、对几乎所有未来交互都高价值的信息，如用户名字、回答风格偏好、岗位意向、禁忌话题、系统高优先级约束；**可检索记忆**则保存详细的经历、项目历史、复杂事件、共享背景，需要时再检索注入。Letta 的 memory blocks 与 archival memory 之分，本质上就是这种分层：前者常驻，后者按需检索。

这背后的原则很简单：高频、短小、稳定的信息常驻；低频、长文本、事件型的信息检索。你这样说，面试官通常会认为你理解了 context budget 与记忆层之间的关系。

用户记忆的 schema 应该怎么设计

这一题如果能落到 schema，面试官会很容易判断你是不是做过。一个较实用的用户记忆 schema 可以包含这些字段：

- memory_id: 唯一标识
- user_id / tenant_id: 作用域
- memory_type: preference / fact / episode / plan / shared_context
- key: 结构化字段名，如 response_style、job_target
- value: 值
- evidence: 来源片段或对话引用
- source_type: user_explicit / system_extracted / imported_profile
- confidence: 置信度
- created_at / updated_at
- expires_at: 可选过期时间
- status: active / superseded / deleted
- version_of: 若是更新版本则关联旧记录
- privacy_level: private / team_shared / org_shared

有了这些字段，后续才谈得上冲突合并、删除审计和精细化召回。

如何检索用户记忆：不是只靠向量相似度

长期记忆当然可以用向量检索，但不能只靠向量。因为很多用户记忆其实是结构化事实，直接按 key 读取更稳。例如用户偏好简短回答这种信息，最好的方式往往是直接查 profile，而不是做语义召回。向量检索更适合回忆经历和长文本事件，比如我们之前为什么否决过方案 B。

因此，一个成熟的用户记忆读取流程通常是分层的：先读取高置信结构化 profile，再根据当前任务是否需要去 episodic memory 里召回相关经历，必要时按时间、项目、主题做过滤。这也是为什么 memory 系统常常是 KV + document store + vector search 的组合，而不是单一向量库。

共享记忆与多 Agent 场景

一旦进入多 Agent 场景，记忆不再只有用户专属这一种。多个 Agent 可能需要共享项目背景、约束、进度和标准操作。Letta 官方就支持 shared memory blocks，让多个 agent 读写同一块共享记忆，并立即看到更新。这类共享记忆更像团队知识与任务状态的结合体，作用域介于 session state 和公共知识库之间。

你在面试里可以指出：共享记忆要特别注意来源可信度、写入权限和冲突解决。因为多个 agent 同时写，错误传播速度会更快。如果没有 versioning 和审计，问题会非常难排查。

长期记忆的评测：比 RAG 更难

RAG 的评测还能围绕证据召回与答案正确性展开，长期记忆的评测更偏用户体验和长期一致性。你至少要看这些维度：记忆写入 precision（有没有记错、乱记）、召回相关性（该想起的时候能不能想起）、个性化收益（是否真的让回答更贴合用户）、冲突更新正确率、删除生效率、用户满意度。

近年的 MemoryCD 等基准就试图从跨领域用户历史中评估记忆方法是否能更好筛选和压缩长期信息，但总体上这个方向仍然远未成熟。你在面试里承认评测比 RAG 更难，但可以分层评通常是加分项。

隐私、治理与可解释性：长期记忆最大的产品风险

长期记忆能显著提升个性化体验，但也天然带来风险。用户可能不知道系统记住了什么，也不知道为什么后续回答会越来越懂他。如果系统没有提供查看、修改、删除、暂停记忆、限制作用域的能力，体验很容易从‘贴心’滑向‘越界’。

因此，产品和工程都需要为长期记忆提供治理能力：可查看、可纠错、可删除、可导出、可暂停、可设置敏感级别。对企业场景还要加入租户隔离和审计。这一点在校招面试里很容易被忽视，但一旦你主动提到，通常会显得你不是只会做 demo。

四、值得和面试官分享的演进细节

这部分近两年的变化很明显。第一，主流 Agent 框架开始把长期记忆做成独立能力：LangGraph 提供跨线程持久化与命名空间 memory；Letta 明确区分 memory blocks 与 archival memory。第二，学术上对 Agent memory 的研究从让模型记住更多历史转向如何组织、更新、遗忘与评测，例如 A-MEM 强调动态记忆组织，MemoryOS 强调短中长期分层，Agentic Memory/ AgeMem 试图统一长期与短期记忆管理，MemoryCD 则开始关注跨域用户历史下的记忆评测。第三，业内越来越承认：长期记忆首先是治理问题，其次才是召回问题。

五、常考面试题与参考答法

1. 长期记忆和用户画像有什么区别？

- 参考回答：用户画像通常只是长期记忆里最稳定、最结构化的一部分，比如偏好和背景；长期记忆还包括经历、计划、失败经验、共享上下文等更丰富的信息。

2. 为什么长期记忆不能每轮都写？

- 参考回答：因为会产生大量噪声和错误沉淀。长期记忆应该只保存未来复用价值高、稳定性较强、来源可信的信息。

3. 你会把哪些信息写成常驻记忆？

- 参考回答：高频、短小、稳定、跨多数对话都有效的信息，比如回答风格偏好、名字、岗位意向、长期项目目标。

4. 哪些信息更适合做可检索记忆？

- 参考回答：复杂经历、历史讨论过程、失败案例、较长的项目背景和事件记录。

5. 显式触发和自动提炼写入怎么取舍？

- 参考回答：显式触发精度高但覆盖低；自动提炼覆盖高但风险大。工程上通常是两者结合：显式优先，自动提炼作为候选并经过规则或确认。

6. 如果用户偏好发生变化，你如何处理？

- 参考回答：识别冲突字段，更新旧值而不是简单追加，并保留时间戳、来源和版本历史，以便回溯。

7. 为什么用户记忆不能只靠向量检索？

- 参考回答：因为很多高价值记忆是结构化事实，直接按 key 读取更稳；向量检索更适合召回经历型长文本。

8. 共享记忆和个人记忆的区别是什么？

- 参考回答：区别在作用域和权限。共享记忆供多个 agent 或团队成员使用，个人记忆只对特定用户或 agent 生效。

9. 长期记忆怎么做遗忘？

- 参考回答：可以结合 TTL、使用频率、显著性衰减、用户显式删除和冲突覆盖等策略。

10. 怎么评估长期记忆系统好不好？

- 参考回答：看写入精度、召回相关性、个性化收益、冲突更新正确率、删除生效率和用户满意度。

11. Letta 的 memory blocks 为什么值得提？

- 参考回答：因为它把高价值短记忆显式放在上下文常驻位置，非常适合解释‘常驻记忆 vs 按需检索记忆’这个分层思路。

12. 如果用户说‘别再记住我的私人信息’，系统要做什么？

- 参考回答：立即停止相关类型记忆写入，删除或失活既有敏感记忆，并提供可审计的反馈与确认。

六、容易踩坑的表达

- 把长期记忆理解成 profile 表，不谈经历、计划、共享记忆和更新机制。
- 只谈怎么检索，不谈什么时候写、什么时候删。
- 把原始聊天历史直接当长期记忆使用。

- 忽略隐私治理、用户可控和删除能力。
- 只用向量库做全部用户记忆，不区分结构化 profile 与经历型记忆。

七、参考资料

- LangGraph Memory Overview —— 官方长期/短期记忆说明
- Letta Memory Blocks —— 官方常驻记忆块
- Letta Memory / Archival Memory —— 官方长期档案记忆
- MemoryOS: A Hierarchical Memory Management System for LLM Agents —— 分层记忆系统
- A-MEM: Agentic Memory for LLM Agents —— 动态记忆组织
- Agentic Memory / AgeMem —— 长期与短期记忆统一管理
- MemoryCD —— 用户中心长期记忆基准

06 知识库权限与多租户隔离（了解）

一、这篇在考察什么

很多校招生可以把 RAG 跑通，但一旦面试官问企业知识库怎么做权限控制不同租户怎么隔离能不能保证不会串数据，就暴露出没有做过生产化设计。因为在真实企业场景里，检索效果很重要，但权限与隔离往往更重要：一次错误召回、一次越权引用，就足以让系统无法上线。

面试官在这一题里真正考察的，是你是否理解知识库不是一个公共语料池，而是一个受组织结构、角色权限、数据分区、合规要求、审计要求约束的资源系统。你不仅要知道 tenant、namespace、collection、partition、payload filter 这些词，更要知道它们解决的是不同层次的问题：物理隔离、逻辑隔离、检索过滤、权限映射、生命周期管理与成本控制。

面试官在这一题里，通常不是只想听一个名词解释，而是想顺着这个主题判断你是否具备下面这些能力：

- 是否理解知识库权限控制不能只在应用层做字符串判断，而要贯穿索引、检索、引用和审计全链路。
- 是否能区分多租户隔离的几种典型方案：独立库/独立集合、namespace/shard、payload 过滤，以及它们的 trade-off。
- 是否知道检索前过滤、检索中隔离、检索后裁剪的不同风险，尤其是后过滤可能导致的泄漏和召回损失。
- 是否了解 Pinecone、Weaviate、Qdrant、Milvus 等主流向量数据库在多租户上的官方建议差异。
- 是否能从权限模型、索引设计、运维成本、查询性能、审计与删除角度谈企业知识库隔离。

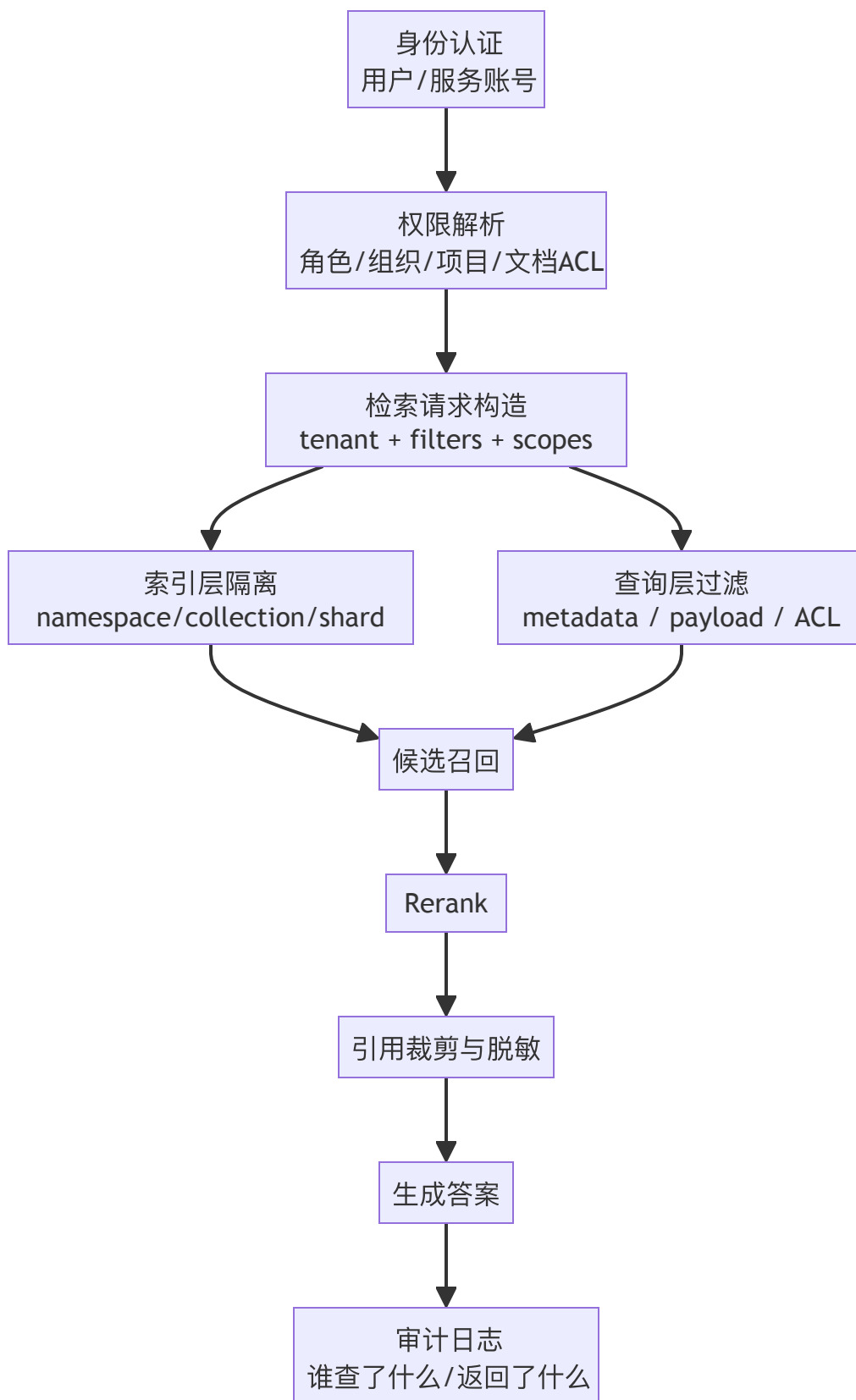
对于在面试时候你可以这么讲你的设计：

我们把权限设计成两层：tenant 级隔离和文档级 ACL。tenant 级在索引层做隔离，比如 namespace / shard，确保检索不会跨租户；tenant 内部再通过项目、部门、角色等 metadata 做预过滤。文档 ingest 时就把 ACL 下沉到 chunk，查询执行时先带 tenant 和 ACL 过滤，再召回和 rerank，避免先全局召回再后过滤。输出层还会对引用链接和敏感字段做脱敏，并且全链路记录审计日志。这样既能保证检索质量，也能保证系统不会串数据。

二、一些总的结论

- RAG 系统最可怕的不是答错，而是越权答对；因此权限与隔离是上线底线。

- 多租户隔离没有唯一方案，关键在租户规模、隔离强度、运维成本和跨租户检索需求之间做权衡。
- 预过滤优先于后过滤，因为后过滤可能先把越权内容召回来，再在结果阶段裁掉，既有泄漏风险又可能损失 top-k。
- 结构化 ACL 与 metadata 设计必须从 ingest 阶段就进入，而不是上线前临时补。
- 对企业系统来说，知识库权限设计至少要回答：谁能写、谁能读、读到哪一层、引用能否暴露、日志如何审计、删除如何生效。



三、系统化八股知识

权限问题就是生产落地分水岭

如果你的系统面向企业内部知识、客户数据、合同文件、工单系统或代码仓库，权限问题就不是可选增强，而是系统能否上线的前提。面试官会追问这题，是因为很多 demo 只证明技术能跑，却没有证明系统不会泄密。而在真实公司里，一次跨部门、跨客户、跨项目的错误召回，风险远大于一般性的幻觉。

所以你在回答时要先把基调立住：检索系统本质上是信息访问系统，只要涉及私域知识，就必须把权限和隔离设计进内核，而不是上线前补个过滤 if-else。

权限控制至少有哪几层

一个成熟的知识库权限系统，至少有四层。

第一层是身份认证，也就是调用方是谁：员工、管理员、机器人账号、集成服务，属于哪个租户、组织、部门、项目。

第二层是授权与作用域解析，把身份映射成可访问范围，例如哪些空间、哪些知识库、哪些文档标签、哪些对象级 ACL。

第三层是检索层约束，把权限信息下推到索引和查询，确保只在允许范围内检索。

第四层是输出层约束，即便检索结果本身可见，也可能还要做字段级脱敏、引用裁剪、日志审计与审批。

只要你把这四层说出来，面试官会感觉你知道权限不是一个布尔字段。

多租户隔离的几种主流方案

多租户隔离没有单一答案，常见方案大致有三种。

方案一：每个租户独立索引/独立集合/独立数据库。

优点是隔离最强，权限边界清晰，便于删除与迁移，也更容易做强监管；缺点是租户多时运维复杂，资源利用率可能不高。Milvus 官方文档就把 collection-level multi-tenancy 视为强物理隔离方案之一，并强调其 RBAC 支持较好。

方案二：共享底层集群，但按 namespace / shard / partition 隔离。

这是很多向量数据库推荐的折中方案。Pinecone 官方明确建议一个 tenant 一个 namespace，并强调 serverless 下 namespace 带来物理隔离与成本收益；Weaviate 则把 multi-tenancy 建模为每个 tenant 一个独立 shard；Milvus 也有 partition / partition-key 多种层级。

方案三：单集合 + payload/metadata 过滤。

Qdrant 官方比较强调这一思路，尤其在 tenant 数量很大时，不建议创建成百上千个 collection，而是更推荐单 collection 配合 tenant_id 之类的 payload 分区与索引。这种方案扩展性好，但隔离强度更多依赖查询过滤与系统正确性。

所以这题不是背哪家文档，而是看你是否知道：隔离强度和运维复杂度通常是此消彼长的。

namespace、collection、partition、payload filter 各解决什么问题

这些词面试里经常被混用。更稳妥的说法是：

- **collection / index / database** 更偏粗粒度资源边界，适合强隔离或差异化索引策略。
- **namespace / shard / partition** 更偏中粒度隔离，兼顾一定物理分离和管理效率。
- **payload / metadata filter** 更偏查询时逻辑过滤，适合大规模 tenant、细粒度 ACL、文档级或段落级权限控制。

不要把它们当同义词。它们分别作用在资源组织、物理布局和查询执行三个层面。很多成熟方案其实是组合使用：例如 tenant 级别走 namespace，文档或项目级别再走 metadata ACL 过滤。

为什么预过滤比后过滤重要

这是最值得强调的工程点之一。所谓后过滤，是先在全局或大范围里召回，再把不该看的结果裁掉。这样做有两个问题。第一，越权内容可能已经进入中间候选集、日志、缓存或 rerank 输入，存在泄漏面。第二，裁掉之后 top-k 可能不够了，导致系统看起来检索不到，其实是权限裁掉了高分候选。

更安全的方式是**预过滤或检索中隔离**：在候选生成前就把 tenant_id、project_id、ACL 标签等过滤条件下推到索引或查询执行层。这样召回本身就在允许范围内进行，既安全，也更容易保证 top-k 质量。面试里你只要主动提到避免 post-filter only，通常会显得很有生产经验。

ACL 应该挂在文档还是 chunk 上

标准答案是：两层都要考虑。很多权限以文档为边界，例如整份合同、整个项目文档库只对某角色可见；但切块以后，系统实际检索和引用的是 chunk，因此 chunk 也需要继承足够的 ACL / metadata，至少能追溯到源文档并在查询时做过滤。

如果文档内部存在更细粒度权限，比如同一份文档不同段落可见范围不同，那就必须支持 chunk 或片段级 ACL。否则你要么过度暴露，要么过度屏蔽。这里没有绝对统一答案，但你至少要表达：ACL 不是 ingest 之后再补的，而是 ingest 时就应随着 chunk 一起下沉。

主流向量数据库官方建议有什么差异

这一段可以帮助你在面试里显得不是闭门造车。Pinecone 的多租户官方建议非常鲜明：一个 tenant 一个 namespace，并强调 namespace 带来的物理隔离、无 noisy neighbors、删除方便和成本效率。Weaviate 的 multi-tenancy 文档则强调每个 tenant 一个 shard，属于在集合内部提供 tenant 级隔

离。Qdrant 更强调单 **collection + payload-based partitioning**，甚至明确提醒不要随意创建成百上千个 collection；其文档还提供了 tenant 字段 payload index 和 shard key 的做法。Milvus 则系统性给出了 database/collection/partition/partition-key 多层多租户模式，并分析了隔离强度、RBAC 支持与规模上限的权衡。

你不需要背每家的参数上限，但要能讲出：不同数据库对多租户推荐方案不同，背后原因是它们的存储组织和执行模型不同。

权限不仅是检索问题，还是引用与生成问题

即使召回阶段做了权限过滤，生成阶段仍然可能带来风险。例如模型综合多个 chunk 作答时，是否可能通过描述性语言泄露敏感字段？引用是否会暴露文档标题或路径？如果用户无权查看原文，系统能不能只给结论不给引用链接？某些场景里，甚至答案本身也需要审批。

因此，权限控制不应止于检索到了没。你还要考虑字段级脱敏、链接隐藏、引用省略、审批流与审计日志。这一点在合规、法务、金融、医疗等领域尤其关键。

删除、迁移与审计：多租户系统的运维视角

生产系统不仅要能查，还要能删、能迁、能审计。租户退租时是否能快速清理数据？用户要求删除个人信息时是否能级联到 chunk 和索引？跨区域迁移时如何保证权限映射不丢？审计里能否回答谁在什么时间检索了什么范围、返回了哪些 chunk、生成了什么答案？

Pinecone 官方把 tenant offboarding 时删除 namespace 当成一项优势，正说明了资源边界清晰对运维的价值。你在面试里提到删除与审计，通常会让回答明显超出 demo 水平。

一个可落地的默认设计

如果被问你会怎么设计企业知识 Agent 的知识库权限，一个稳妥回答是：

登录后先解析用户身份、组织、角色、项目组 and 租户；检索请求必须携带 tenant scope；索引层按 tenant 做中粒度隔离，例如 namespace 或 shard；文档与 chunk 同时带 ACL metadata，包括项目、部门、可见角色、敏感级别；查询执行时先做 tenant 与 ACL 预过滤，再召回与 rerank；输出层对引用做脱敏和可见性控制；全链路记录审计日志，支持删除与追溯。

这类回答兼顾了架构、查询和治理，很适合校招系统设计题。

四、一些值得分享的演进点

这一题近两年的一个明显变化，是主流向量数据库都开始更明确地给出多租户推荐模式，而不是只讲可以加一个 `tenant_id` 过滤。Pinecone 强调 `namespace-per-tenant`；Weaviate 强调 `tenant shard`；Qdrant 更强调单 `collection + payload` 分区与 `tenant index`；Milvus 则系统化给出 `collection/partition/partition-key` 多级方案。这说明行业已经从能否支持多租户转向不同规模和隔离要求下哪种模式更合适。另外，越来越多团队也开始重视查询前过滤、字段级脱敏和审计，而不仅仅是向量检索本身。

五、常考面试题与参考答法

1. 为什么说 RAG 系统里最严重的问题可能不是幻觉，而是越权？

- 参考回答：因为幻觉通常是质量问题，越权是安全与合规问题。一次错误暴露私域信息就可能让系统无法上线。

2. 多租户隔离有哪些常见方案？

- 参考回答：独立库/独立集合、`namespace`或`shard`隔离、单集合配合 `payload/metadata` 过滤。这几种方案在隔离强度、成本和管理复杂度上各有取舍。

3. 什么时候适合每个租户独立索引？

- 参考回答：当租户数量不大、隔离要求极高、需要独立生命周期管理或差异化配置时更适合。

4. 什么时候适合单集合 + `tenant_id` 过滤？

- 参考回答：当 `tenant` 数量很大、需要更高资源利用率和统一运维时更适合，但必须把过滤和审计做得非常扎实。

5. 为什么不建议只做后过滤？

- 参考回答：因为越权结果可能已经被召回、进入日志或 `rerank`，存在泄漏面；同时后过滤还会损失 `top-k` 质量。

6. ACL 应该挂在哪里？

- 参考回答：至少要能从 `chunk` 追溯到源文档，并在查询时基于文档级或 `chunk` 级 ACL 做过滤。权限设计应从 `ingest` 阶段就跟随 `chunk` 一起下沉。

7. `namespace` 和 `metadata filter` 是替代关系吗？

- 参考回答：不一定。很多系统会 `tenant` 级别用 `namespace` 隔离，再在 `tenant` 内用 `metadata filter` 做项目级或文档级权限控制。

8. 为什么说向量数据库的多租户最佳实践不一样？

- 参考回答：因为不同产品的存储布局、分片模型、权限机制和上限不同，所以推荐模式也不同。

9. 知识库权限为什么不仅是检索问题？

- 参考回答：因为生成答案和引用链接也可能泄露信息，还要考虑字段级脱敏、链接可见性和审计。

10. 租户删除时你最关注什么？

- 参考回答：数据能否快速、彻底、可审计地删除，包括原文、chunk、索引和缓存。

11. 怎么做跨项目细粒度权限控制？

- 参考回答：在 tenant 之下继续给文档和 chunk 打项目、部门、角色等 ACL metadata，并在查询前下推过滤条件。

12. 如果两个租户都要访问一份共享知识怎么办？

- 参考回答：可以放在公共作用域的共享知识库，显式区分 public/shared/private 范围，避免通过复制数据制造权限复杂度。

六、容易踩坑的表达

- 只会说给向量加 tenant_id，不讨论检索前过滤、审计和引用脱敏。
- 把 namespace、collection、partition、metadata filter 当同义词使用。
- 忽略删除、退租、迁移、缓存污染等运维问题。
- 假设应用层过滤就足够，不把权限下推到检索与索引层。
- 不区分租户级隔离和文档级 ACL。

七、参考资料

- Pinecone Multitenancy —— 官方 namespace-per-tenant 建议
- Weaviate Multi-tenancy —— 官方 tenant shard 文档
- Weaviate RBAC / permissions —— 集合与 tenant 权限
- Qdrant Data Partitioning —— payload 分区与多租户建议
- Qdrant Collections and Multitenancy —— collection 数量与设计建议
- Milvus Multi-tenancy —— collection/partition/partition-key 多级隔离

07 Agentic-RAG与GraphRAG（重要）

一、这篇在考察什么

本质上在考你是否理解检索不再只是一次 top-k 查询，而是可以成为一个有计划、有工具调用、有多步推理、甚至有图结构支持的动态过程。很多候选人会把 Agentic RAG 讲成让 Agent 调用一下检索工具，把 GraphRAG 讲成在 RAG 前面加个知识图谱，这样的回答都太浅。

更成熟的回答应该说明：Agentic RAG 强调让检索成为推理与行动循环中的一部分，能够根据中间结果继续搜索、改写 query、分解子问题、路由数据源；GraphRAG 强调把非结构化文本转化为图结构与社区报告，支持多跳、全局主题、关系推理和不同粒度的信息导航。但它们都不是银弹：Agentic RAG 更复杂、成本更高；GraphRAG 在很多真实任务上并不一定优于 vanilla RAG，图构建质量和成本也是大问题。面试官想看的是你能不能讲清楚适用场景与代价，而不是只会追热点名词。

面试官在这一题里，通常不是只想听一个名词解释，而是想顺着这个主题判断你是否具备下面这些能力：

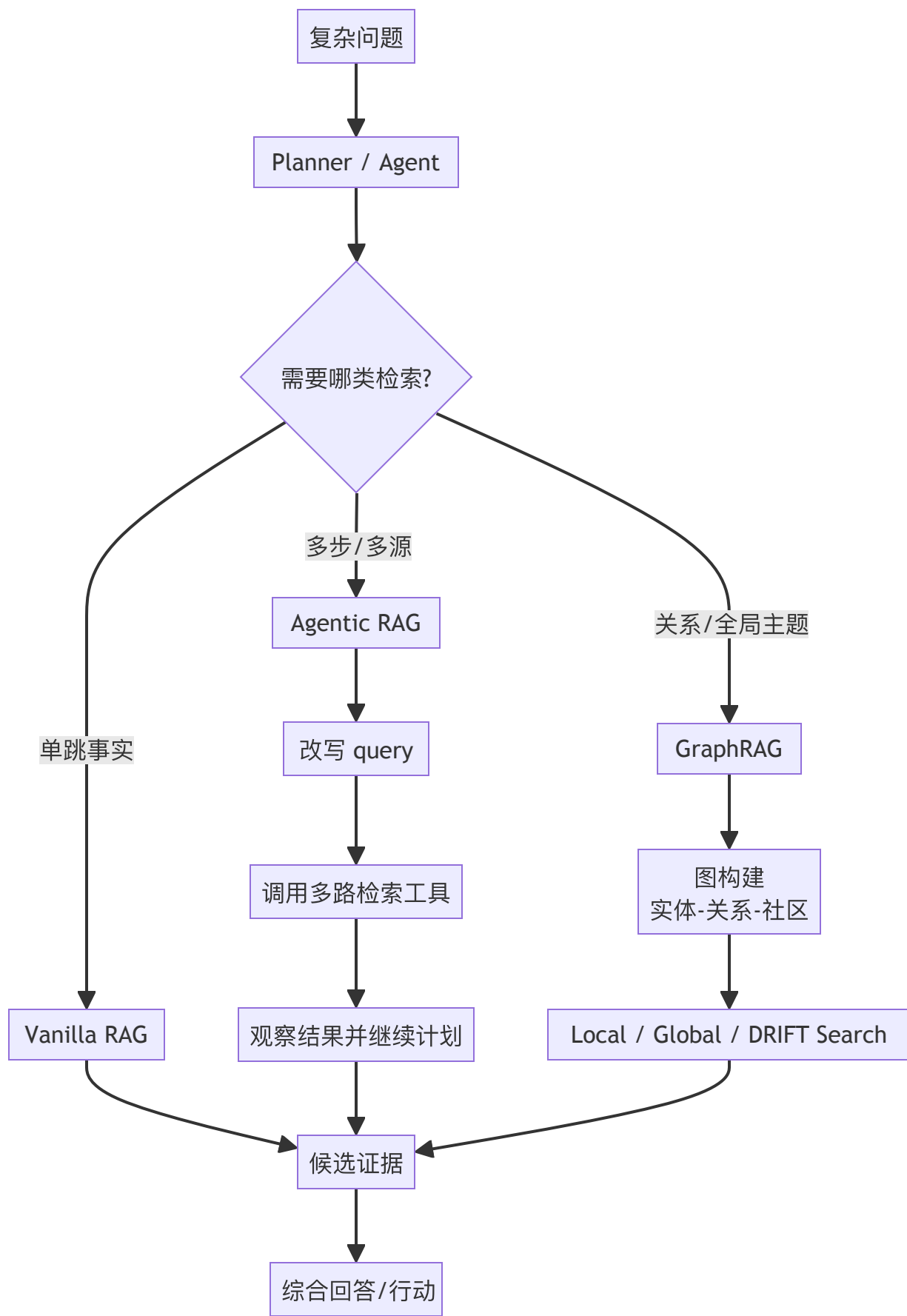
- 是否理解 Agentic RAG 的核心是‘迭代式、有决策的检索’，而不是简单多查几次。
- 是否知道 GraphRAG 的主要流程：实体/关系抽取、图构建、社区层次、社区报告、local/global/DRIFT search。
- 是否能区分 vanilla RAG、Agentic RAG、GraphRAG 分别适合哪类问题。
- 是否理解多跳推理、跨文档关系、全局主题总结是 GraphRAG 常见强项，但图构建与维护成本也很高。
- 是否知道近期研究对 GraphRAG 的态度并不是一边倒看好，而是更强调在什么条件下它值得使用。

项目里可以这样讲：

我们把复杂问答分成三档：单跳事实问题走普通 RAG；需要多步取证和跨系统查询的问题走 Agentic RAG，由 planner 拆分子问题并路由不同检索工具；涉及实体关系梳理和全局主题总结的问题，尝试走 GraphRAG 路径。GraphRAG 部分不是所有问题都用，而是只在确实需要关系网络和跨文档主题聚合时启用。我们离线对比过 vanilla RAG 和图增强方案，发现图方案在多跳和全局分析类问题上更好，但在简单定位题上反而更慢且收益有限，所以最后做成了路由而不是一刀切。

二、一些关键结论

- Agentic RAG 的关键不在 ‘Agent’ 三个字，而在检索是否进入了计划—行动—反思闭环。
- GraphRAG 的关键不只是‘有图’，而是是否真的需要实体关系和社区层级来支持问题求解。
- vanilla RAG 适合大多数单跳证据问答；Agentic RAG 适合复杂、多步、跨工具检索；GraphRAG 适合关系密集、多跳和全局主题问题。
- GraphRAG 并非总比普通 RAG 强，图构建质量、图噪声、更新成本和查询类型匹配度都非常关键。



三、系统化八股知识

从 Vanilla RAG 到 Agentic RAG：升级的根因是什么

普通 RAG 默认假设：给定一个问题，只要构造一个 query，从一个知识源召回 top-k 证据，再交给模型生成即可。这在单跳事实问答、FAQ、制度查询、局部流程说明等场景里通常足够。但一旦问题变成先查 A，再根据 A 查 B，再综合两个来源做判断，单次检索就容易不够。

Agentic RAG 的出现，本质上是为了处理这些**需要检索决策和中间推理**的问题。比如用户问这个客户上季度投诉最多的问题和当前产品 roadmap 中正在修复的问题有没有重合，你可能要先查投诉数据，再查 roadmap，再做字段映射和总结。这时检索不再是一条固定流水线，而是一个被 planner 或 agent 驱动的循环过程。

什么叫 Agentic RAG

Agentic RAG 可以理解为：让检索成为 Agent reasoning loop 的一部分，而不是一次性预处理。它通常具备下面几个能力：第一，**query routing**，知道去哪个知识源查；第二，**query rewriting/decomposition**，知道怎么拆问题、改 query；第三，**iterative retrieval**，知道看完第一批结果后是否继续查；第四，**tool composition**，知道何时从文档检索切换到数据库、搜索引擎或代码库；第五，**reflection / verification**，知道当前证据是否足够支撑回答。

如果只是用户问题来了，调用一次 retriever 工具，再返回 top-k，这还只是把 RAG 包装成 tool use，不算强意义上的 Agentic RAG。高分回答一定要体现检索策略会根据中间观察动态变化。

Agentic RAG 常见 workflow

一个典型的 Agentic RAG 流程通常是：先判断问题复杂度与数据源类型；如果是单跳问题，直接普通检索；如果是复杂问题，先做 decomposition，把问题拆成若干子问题；然后为每个子问题选择合适的数据库和检索工具；拿到结果后做中间总结或结构化抽取；如果发现证据不足，再次检索或改写 query；最后综合多轮证据作答，并在必要时附上引用链路。

这类流程最适合多源、多步、跨文档和需要工具联动的任务，也是许多 agent 平台把 retrieval 做成 tool + planner 的原因。

GraphRAG 到底是什么

GraphRAG 不等于把知识图谱接到 RAG 前面。更准确地说，微软提出的 GraphRAG 是一套把原始文本加工为**实体-关系图 + 社区层次 + 社区报告**，再围绕这些结构执行查询的 RAG 方法。它的流程一般包

括：从文本中抽取实体、关系和 claims；构建图；对图做 community detection；为社区生成摘要报告；在查询时根据需要选择 local search、global search 或 DRIFT search。

这套设计的核心价值在于：普通 chunk 检索擅长找局部相似文本，但对跨文档关系网络全局主题总结某群体内的共性模式并不天然友好。GraphRAG 通过图和社区报告，给模型一个更适合多跳和整体视角的问题表达。

Local Search、Global Search、DRIFT Search 分别在干什么

这是 GraphRAG 面试里最容易被问到的细节之一。

Local Search 更适合围绕特定实体或局部主题做关系推理。微软官方文档描述中，会把实体、关系、文本单元和社区报告一起作为候选，让模型围绕某个实体邻域做更细致的分析。

Global Search 更适合整个语料的大主题总结这类问题。官方做法通常是对社区报告进行 map-reduce 式处理，先对各社区回答，再做全局汇总。

DRIFT Search 是后来加入的思路，结合 global 和 local 的特点：先从社区层信息出发扩大搜索起点，再通过后续跟进收缩到局部，更适合那种既需要全局背景、又要最终落到细节的问题。

你如果能把这三个模式说清楚，面试官通常会认为你不是只听过 GraphRAG 这个词。

GraphRAG 适合什么，不适合什么

GraphRAG 更适合下面这些任务：多跳关系推理、跨文档实体网络分析、全局主题概览、人物/组织/事件之间的关联梳理、复杂报告生成、知识探索型问答。比如这家公司在过去三年里与哪些合作伙伴形成了最关键的战略联盟，并分别影响了哪些业务线，这类问题就比纯相似度检索更适合图结构和社区报告。

但 GraphRAG 不一定适合简单事实查找、精确条款定位、局部流程问答、纯关键词约束问题。因为这些问题 vanilla RAG 往往更直接、更便宜，图构建反而可能引入噪声和额外延迟。

近期研究对 GraphRAG 的真实态度：值得用，但不是银弹

这是这一题里最值得主动补充的部分。微软及相关研究展示了 GraphRAG 在多跳和某些复杂推理任务上的优势；例如在部分 MultiHop-RAG 设置下，图增强方法相较无图基线取得了更高分。但同时，较新的研究也指出 GraphRAG 在很多真实任务上会落后于 vanilla RAG，尤其当任务本身并不需要关系推理、图构建质量一般、图社区摘要不够准时，GraphRAG 反而可能放大噪声并增加成本。

所以面试里的高分说法应该是：GraphRAG 更像针对特定问题分布的结构化增强方案，而不是默认替代普通 RAG。是否值得引入，取决于关系密度、多跳需求、图构建质量与预算。

Agentic RAG 与 GraphRAG 的关系

这两个概念不是对立关系。GraphRAG 可以被 Agent 调用，因此完全可以出现在 Agentic RAG 体系里。换句话说，Agentic RAG 关注的是检索过程是否有计划和决策，GraphRAG 关注的是底层知识组织是否用图和社区层次。前者更像 runtime / workflow 层增强，后者更像 knowledge representation 层增强。

所以一个系统完全可能同时是 Agentic 的，也是 Graph-based 的。比如 planner 先判断问题需要实体关系推理，于是路由到 GraphRAG local search；如果需要更广视角，再调 global search；拿到结果后继续普通文档检索验证原文。这种组合比单独背两个词更像真实系统。

什么时候不该上 Agentic RAG 或 GraphRAG

如果问题绝大多数都是单跳事实问答、知识更新频繁、预算敏感、延迟敏感，而且团队还没有稳定的评测集，那就不应该一上来就做复杂的 Agentic RAG 或 GraphRAG。先把解析、切块、混合召回、rerank、权限和评测做好，往往收益更大。

面试官其实很喜欢听到这种知道什么时候不用的回答。因为这说明你不是见新词就上，而是会做成本收益判断。

一个实用的决策框架

你可以记住这样一个判断顺序：

如果是单文档或单证据可回答的问题，先用 vanilla RAG。

如果需要多步查询、跨系统取证、动态拆解、验证与反思，引入 Agentic RAG。

如果问题依赖实体关系网络、跨文档多跳和全局主题汇总，再考虑 GraphRAG。

如果三者都可能，优先做 query routing，由 planner 选择路径，而不是所有问题都走最重方案。

这个回答方式非常适合系统设计追问。

四、一些重要趋势

这一题最近的两个重要趋势必须记住。第一，Agentic RAG 正在从会调 retriever 工具升级为基于 planner 的迭代检索与工具路由，很多框架与官方示例都开始强调自适应检索而不是固定 top-k。第二，GraphRAG 热度很高，但最新研究并没有给出‘GraphRAG 全面优于 RAG’的结论。微软官方文档持续强化 local/global/DRIFT 三种搜索模式，而一些新论文则指出 GraphRAG 在不少现实任务中会落后于

vanilla RAG，只有在关系密集、多跳和全局主题任务上才更有优势。这种‘更强，但有条件’正是面试里应该讲出的判断。

五、常考面试题与参考答法

1. 什么是 Agentic RAG?

- 参考回答：让检索进入 agent 的计划—行动—观察—反思闭环，能够根据中间结果继续改写 query、拆分问题、路由数据源和验证证据。

2. 什么情况下普通 RAG 不够用?

- 参考回答：当问题需要多步取证、跨工具或跨数据源、先查 A 再查 B、或者必须边查边调整策略时，单次 top-k 检索往往不够。

3. GraphRAG 和知识图谱问答是一回事吗?

- 参考回答：不完全是。GraphRAG 更强调从原始文本自动构建实体关系图、社区层次和社区报告，再把这些结构用于检索与总结。

4. GraphRAG 的 local search 适合什么问题?

- 参考回答：适合围绕特定实体或局部主题做关系推理和细粒度分析。

5. global search 适合什么问题?

- 参考回答：适合整个语料的大主题概览、宏观总结和跨社区综合。

6. DRIFT search 想解决什么?

- 参考回答：它试图结合 global 和 local 的优势，先从更全局的社区信息出发，再通过后续跟进定位到局部细节。

7. GraphRAG 一定比 vanilla RAG 强吗?

- 参考回答：不一定。只有在任务确实需要关系网络、多跳推理或全局主题总结，且图构建质量较高时，GraphRAG 才更可能带来收益。

8. GraphRAG 的主要成本在哪?

- 参考回答：实体关系抽取、图构建、社区检测、社区报告生成、增量更新和查询复杂度都带来额外成本。

9. Agentic RAG 和 GraphRAG 是替代关系吗?

- 参考回答：不是。前者是检索过程层增强，后者是知识表示层增强，可以组合使用。

10. 什么时候不建议做 Agentic RAG?

- 参考回答：当问题大多是简单单跳问答、延迟和成本敏感、评测体系还不成熟时，不建议一开始就上复杂方案。

11. 如何向面试官解释你项目中的 Agentic RAG 不是‘套个 Agent 壳’?

- 参考回答：要说明检索会根据中间结果动态调整，例如分解子问题、路由不同数据源、验证证据是否足够，而不是固定一次检索。

12. GraphRAG 项目里最容易踩的坑是什么？

- 参考回答：图构建质量不稳、实体归一化差、社区报告噪声大、更新成本高，以及把不需要图的问题也硬塞给图检索。

六、容易踩坑的表达

- 把 Agentic RAG 讲成‘调用了一次 retriever 工具’。
- 把 GraphRAG 讲成‘用知识图谱检索’而不提社区层次和多种搜索模式。
- 把 GraphRAG 神化为对所有任务更强，不提任务分布和构图成本。
- 分不清 runtime 增强（Agentic）与表示增强（Graph）两个层次。
- 忽略增量更新与图维护成本。

七、参考资料

- Microsoft GraphRAG —— 官方文档
- GraphRAG Query: Local Search —— 官方 local search
- GraphRAG Query: Global Search —— 官方 global search
- GraphRAG Query: DRIFT Search —— 官方 DRIFT search
- From Local to Global: A Graph RAG Approach to Query-Focused Summarization —— GraphRAG 代表论文
- Rethinking GraphRAG: Conditions When GraphRAG Outperforms Vanilla RAG —— GraphRAG 条件性优势研究
- MultiHop-RAG benchmark related GraphRAG results —— 图增强在多跳任务上的表现
- LangGraph Custom RAG Agent —— Agentic RAG 工程实践

08 RAG常见失败模式

一、失败模式其实就是在考察你的system design

面试官问RAG 常见失败模式并不是让你列几个名词，而是在看你能否像工程师一样定位问题：到底是解析坏了、切块错了、召回漏了、重排歪了、上下文拼坏了、模型没按证据答、权限过滤裁掉了、还是知识本身过期了。很多候选人会把所有错误都归为‘幻觉’，这会显得你没有排障方法论。

更成熟的回答应该呈现一条分层故障树：从数据入口、索引构建、检索召回、排序、上下文组装、生成、引用、权限、安全、评测和线上观测逐层排查。近两年的研究也开始把 RAG 失败系统化拆解，例如 Document-Level Retrieval Mismatch、CRUD-RAG、对抗性证据、检索投毒等，都说明失败模式并不只是模型编造，而是整条链路都可能出问题。

面试官在这一题里，通常不是只想听一个名词解释，而是想顺着这个主题判断你是否具备下面这些能力：

- 是否能从系统链路而不是从单个模型角度分析 RAG 失败原因。
- 是否知道常见失败模式的分类：解析问题、切块问题、召回问题、排序问题、上下文问题、生成问题、权限与安全问题。
- 是否能举出具体排查指标和定位手段，而不是只会说‘调 prompt’。
- 是否了解近年的 RAG failure/robustness 研究，例如 CRUD-RAG、document-level mismatch、adversarial evidence、poisoning。
- 是否具备从错误类型反推架构优化的能力。

怎么在项目里描述？

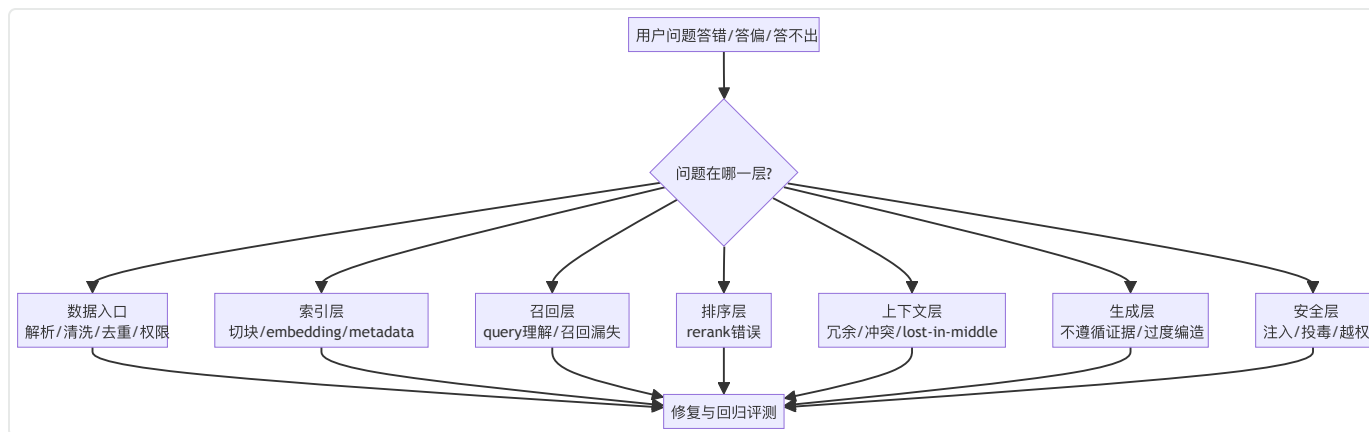
我们把 RAG 错误拆成数据、召回、排序、组装、生成和权限六层来排。先看正确证据有没有被解析和入库，再看 recall@k；如果 top-50 有但 top-5 没有，就优先查 rerank 和上下文打包；如果证据已经进入上下文但答案仍然错，就看 groundedness、引用和拒答策略。线上每次回答都记录检索 trace、过滤条件、候选 chunk、最终注入上下文和引用结果，方便回放。对高风险知识源还会做 prompt injection 清洗和来源可信度分层。这套方法比单纯说‘模型幻觉’更能真正定位问题。

二、先把这几个结论记住

- RAG 失败不等于幻觉，很多失败发生在模型回答之前。

- 最有效的排障方式是把链路拆开看：数据、索引、召回、重排、上下文、生成、输出、权限、安全、评测。
- 上线后没有 trace、引用、重放和错误分桶，RAG 基本无法稳定迭代。
- 同一个‘答错了’背后可能是完全不同的问题，必须用指标和样例定位。
- 安全失败、权限失败和投毒失败在生产上往往比普通准确率下降更严重。

三、先有一张总图



四、系统化八股知识

先建立一个正确心智：RAG 失败是链路失败，不只是模型失败

很多候选人一看到 RAG 出错就说‘模型 hallucination 了’，这太粗糙。因为模型最后编错当然是现象，但根因可能在更早阶段。知识库其实有正确答案，但 PDF 解析把表格读坏了；或者切块把定义和限制条件切散了；或者 query rewrite 把实体改偏了；或者权限预过滤太严把正确证据挡掉了；或者 rerank 把真正答案压在 top-10 之外；或者上下文打包时重复 chunk 太多，关键证据被挤到中间位置。

所以这道题最好的回答方式不是背诵召回失败、生成失败，而是给出一条完整故障树：数据入口、索引、召回、排序、上下文、生成、引用、安全与治理。

失败模式一：数据入口与解析失败

这是最常被低估的一层。很多业务文档不是干净纯文本，而是 PDF、扫描件、表格、代码仓库、网页、PPT、数据库导出、图片混排。只要解析阶段把表格列错位、标题层级丢失、代码缩进破坏、页眉页脚混进正文，后续检索就会建立在错误文本上。最终用户只会觉得系统答错了，但本质上是知识入口已经坏了。

排查方法包括：抽样回看原文与解析文本对齐度；重点检查表格、代码块、列表、标题层级、页码和附件；建立解析质量监控；对高价值文档保留 source-to-chunk 可追溯链路。现实里很多所谓的 RAG 优化，收益最大的是换解析器而不是换模型。

失败模式二：切块失败与语义边界损坏

切块失败常见于三类情况。第一类是 chunk 太小，导致上下文不完整，例如定义在上一段、例外条件在下一段；第二类是 chunk 太大，噪声太多，相关信息被稀释；第三类是边界不对，比如按固定长度把表格行、代码函数、条例条款硬切开。

这一类问题的表征通常是：系统检到了一堆‘有点相关’的 chunk，但真正能直接回答问题的证据不在其中，或者明明同一段信息就在原文里，检索结果却总是缺半句。解决方向往往是结构化切块、保留标题层级、针对表格/代码做特殊处理、引入 overlap 或层级切块。

失败模式三：召回漏失

召回漏失是最直观的一类问题：正确答案明明在库里，但 top-k 里没有。原因可能有很多：embedding 选型不匹配、dense 对精确术语不敏感、sparse 没有启用、query rewrite 改偏、多轮对话没有补齐上下文、过滤条件过严、向量索引参数不合适、top-k 太小、文档更新后索引没刷新。

排查时要先问三个问题：一，正确证据是否真的入库并可检索；二，原始 query、rewrite query、hybrid query 各自能否召回；三，问题集中在哪类 query 上。只有先确认是召不回来，才能决定是调 embedding、加 sparse、做 rewrite 还是放宽过滤。

失败模式四：排序错误与看似相关的伪证据

很多时候召回其实成功了，真正证据已经在 top-50 里，但因为排序不好，前面挤满了看似相关的噪声 chunk，模型根本读不到真正答案。这类问题在 dense only 系统里很常见，尤其是很多模板文档彼此非常像时。二阶段 rerank 正是为了解决这个问题。

一个典型信号是：增大 top-k 后偶尔能答对，但延迟和成本明显上升；加入 rerank 后首屏正确率提升明显。这通常说明问题不在 recall，而在 ranking quality。

失败模式五：上下文组装失败

就算候选排序没问题，上下文组装仍然可能失败。最常见的形式有：重复 chunk 太多，浪费上下文；多个来源互相冲突，没做冲突标记；关键 chunk 被塞在上下文中部，触发 lost-in-the-middle；引用信息

和正文分离，模型难以判断来源；太多无关上下文把真正答案稀释掉。

这一层的修复常常包括：去重、压缩、摘要化、基于来源多样性选 chunk、把最强证据前置、按问题类型动态控制 top-k 和上下文预算。很多团队只优化召回，不优化组装，最终让前面的工作白费。

失败模式六：生成阶段没有被证据约束

即便证据已经进了上下文，模型也可能没有老老实实根据证据回答。表现形式包括：把证据中的概率性表达说成确定事实；把多个 chunk 的信息拼接成文档中并不存在的新结论；忽略证据中的限制条件；当证据不足时仍然硬答。用户视角看还是‘幻觉’，但更准确地说是 grounded generation 失败。

这时需要从 prompt 约束、答案结构、引用机制、拒答策略和模型选择上修复。例如要求答案按结论-证据-引用输出，证据不足时明确说明；或者把答案生成和引用抽取拆成两个步骤。重要的是，你要在面试里强调：RAG 只能提供证据，不自动保证模型会守规矩。

失败模式七：知识过期、版本冲突与源文档不一致

RAG 的核心价值之一是时效性，但前提是知识库真的及时更新。如果旧文档没下线、新旧版本同时可见、索引增量没完成、缓存没失效，就会出现版本冲突。系统可能召回过期规则，却给出看似很自信的答案。企业内部制度、产品文档、接口说明、活动规则都很容易出现这种问题。

修复思路包括：版本字段入索引、更新时间入 metadata、默认偏向最新文档、旧版本显式失活、回答时展示生效日期、对关键知识源建立增量更新监控。

失败模式八：权限过滤与多租户误配置

用户会把‘我明明有权限却查不到’和‘系统答不上来’归为一类，但工程上这往往是权限问题。ACL 过严会导致正确答案永远召不回来；ACL 错配更严重，可能让模型拿到不该看的文档。由于权限通常和检索、缓存、引用、审计纠缠在一起，这类问题如果没有 trace 很难定位。

所以生产上一定要把 tenant、ACL、scope 信息写进检索 trace，能够回放这个用户的这次查询为什么看到这些 chunk，为什么没看到另外那些。否则排障会非常痛苦。

失败模式九：Prompt Injection、检索投毒与对抗性证据

近年的研究越来越强调：RAG 不只是质量系统，也是攻击面。攻击者可以在知识源里植入带有恶意指令或误导性内容的文档，让系统在检索到这些内容后被劫持；也可能通过检索投毒，把伪造证据排到前

面。对抗性证据研究表明，RAG 模型面对有害或误导性支持文档时可能明显退化，尤其在高风险领域更值得注意。

防护上至少要做三件事：来源信任分层、内容清洗与安全过滤、生成阶段显式忽略文档中的操作性指令（除非来源可信且在允许范围内）。同时，对高风险领域要做更严格的 groundedness 检查和人工审批。

失败模式十：评测盲区与基准偏差

如果评测集不覆盖真实问题分布，系统就会出现‘离线挺好，线上很差’。比如 benchmark 多是短事实问答，但线上问题很多是多轮、省略、跨文档、权限受限、版本敏感问题；或者离线评测只看答案正确率，不看引用是否对、检索是否越权、拒答是否合理。这样即便分数高，也不意味着能上线。

CRUD-RAG 等基准开始尝试系统地覆盖 RAG 各组成部分的能力，但生产评测依然需要自己分桶。你在面试里主动提到‘按错误类型建回归集’会很加分。

如何建立一套排障方法论

一个可操作的方法是四步法。

第一步，看证据是否存在：正确答案是否真的在库里，是否解析正确。

第二步，看证据是否召回：gold chunk 是否进入 top-k。

第三步，看证据是否被模型读到：排序、去重、组装是否把证据放在有效位置。

第四步，看模型是否依据证据作答：是否有引用、是否存在无依据补全。

配套指标至少包括：解析质量抽检、入库覆盖率、recall@k、nDCG、top-k 引用命中率、citation precision、groundedness、拒答率、越权率、线上 trace 重放。只要你能把这套框架讲顺，这道题就很稳。

五、2025–2026 值得补进脑子的更新点

这一题近两年的新意主要来自RAG robustness研究的系统化。除了传统的召回与生成错误，研究开始专门讨论 Document-Level Retrieval Mismatch（找到了相关段落却来自错误文档）、CRUD-RAG 这类覆盖检索链路不同环节的基准、以及对抗性证据和检索投毒对系统鲁棒性的影响。这些工作说明，RAG 失败不再只被看成‘模型幻觉’，而是一个包含数据质量、安全攻击、检索偏差和评测盲区的综合系统问题。

六、常考面试题与参考答法

1. RAG 最常见的失败模式有哪些？

- 参考回答：可以从链路看：解析失败、切块失败、召回漏失、排序错误、上下文组装失败、生成不受证据约束、知识过期、权限误配置、安全攻击和评测盲区。

2. 为什么说很多 RAG 问题不是模型问题？

- 参考回答：因为正确证据可能根本没被解析、没入索引、没被召回、没排到前面或没被正确组装，模型只是最后一个暴露问题的环节。

3. 如何区分召回失败和排序失败？

- 参考回答：看 gold evidence 是否出现在更大的候选集合里。如果 top-50 有但 top-5 没有，通常是排序问题；如果 top-50 都没有，通常是召回问题。

4. 什么是 lost-in-the-middle？

- 参考回答：关键信息被放在长上下文中间位置，模型注意力不足，导致虽然信息在上下文里但没被有效利用。

5. 为什么加入更多 chunk 不一定更好？

- 参考回答：因为会引入噪声、重复和上下文稀释，可能让关键信息更难被模型抓住。

6. 知识过期问题怎么处理？

- 参考回答：要做版本字段、更新时间、旧版本失活、增量索引监控和缓存失效策略，回答时必须展示生效日期。

7. Prompt Injection 在 RAG 里为什么危险？

- 参考回答：因为恶意文档可能被检索进上下文，借此操纵模型的后续行为或污染答案。

8. Document-Level Retrieval Mismatch 是什么？

- 参考回答：段落表面相关，但来自错误来源文档，导致模型引用了不该引用的证据或在文档级别出现事实错配。

9. 如果用户有权限但系统总查不到正确文档，你会怎么排查？

- 参考回答：先查 ACL 和过滤条件是否正确下推，再查索引是否包含文档、query 是否匹配、召回与 rerank 是否把结果压掉。

10. 为什么需要 trace 和回放？

- 参考回答：因为没有中间过程可见性，就无法知道错误发生在哪一层，也无法稳定回归修复。

11. CRUD-RAG 这类基准说明了什么？

- 参考回答：说明 RAG 需要分组件评估，不能只看最终答案，还要评估检索链路不同能力。

12. 项目里如何建立 RAG 回归测试？

- 参考回答：按错误类型分桶建设数据集，覆盖解析、召回、排序、权限、安全、引用和最终答案，

并对每次改动做回放与指标对比。

七、容易踩坑的表达

- 把所有错误都叫幻觉，不做链路拆分。
- 只调 prompt，不查解析、切块、召回和上下文组装。
- 离线只看答案正确率，不看引用、越权和拒答质量。
- 没有 trace 与重放机制，导致问题复现困难。
- 忽视安全型失败，如 prompt injection 和检索投毒。

九、参考资料

- CRUD-RAG —— 覆盖 RAG 组件能力的基准
- Document-Level Retrieval Mismatch —— 文档级错配问题
- Adversarial Evidence in Retrieval-Augmented LLMs —— 对抗性证据研究
- A Comprehensive Study of RAG Robustness —— RAG 架构、增强与鲁棒性综述
- PoisonedRAG / retrieval poisoning related works —— 检索投毒与安全
- RAG Failure Modes analyses —— 失败模式综述类参考

00 协议-runtime-工程化高频面试题（汇总）

这篇在考察什么

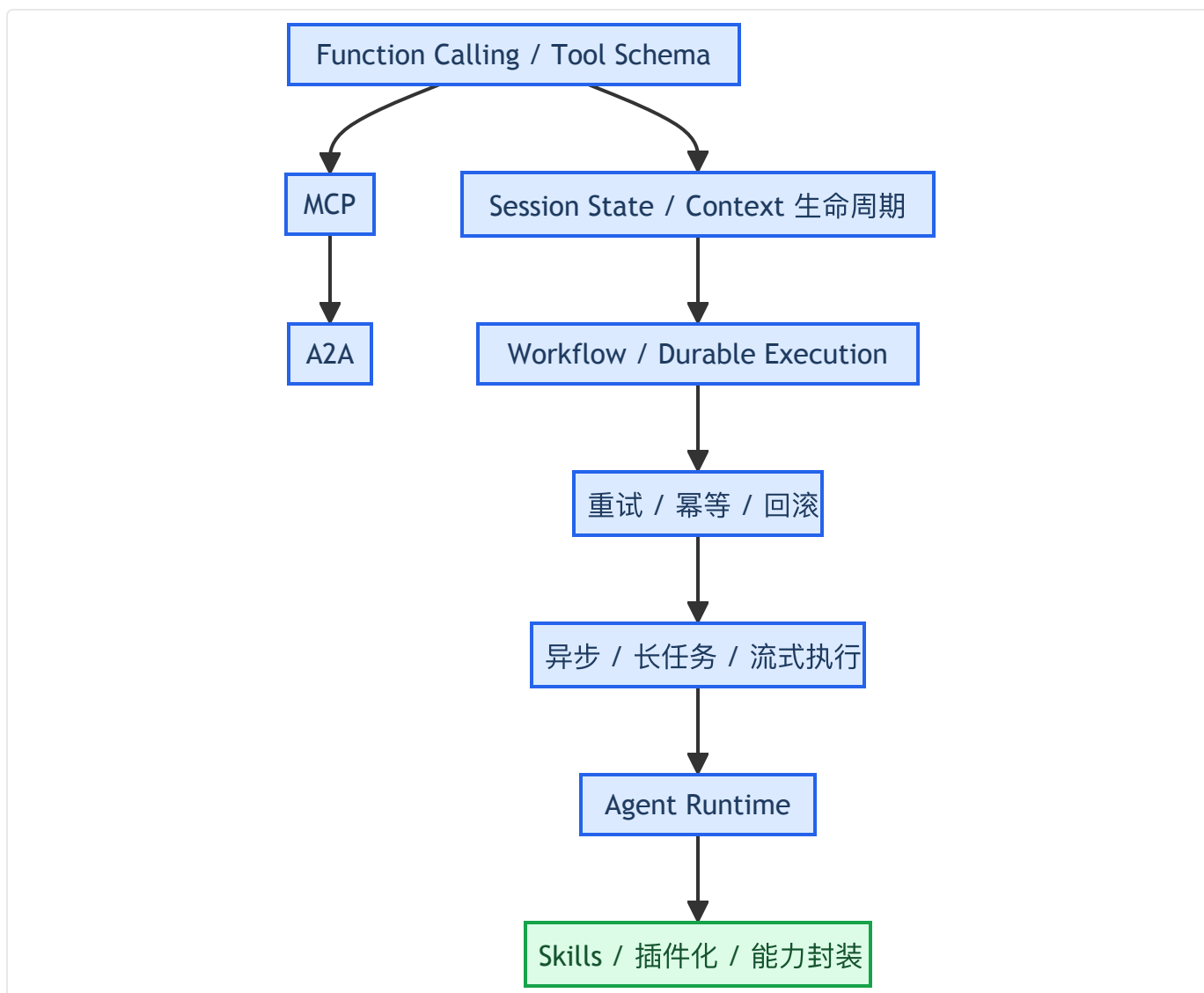
关于协议和runtime本质上在考你是否具备Agent 平台工程视角：

1. 模型怎么把动作表达出来。
2. 能力怎么接入、注册、发现和治理。
3. 状态怎么管理、长任务怎么恢复。
4. 工具、远端 Agent、Skills、Workflow 怎么协同。
5. 整个 Runtime 如何支撑安全、可靠、可观测和多租户。

高频题速览

- 01 Function Calling 与 Tool Schema
- 02 MCP 原理与实践
- 03 A2A 与 Agent 互操作
- 04 Session-State 与 Context 生命周期
- 05 Workflow-Orchestration 与 Durable Execution
- 06 任务调度、重试、幂等、回滚
- 07 异步任务、长任务、流式执行
- 08 Agent Runtime 设计
- 09 Skills、插件化与能力封装

思维导图



一、Function Calling 与 Tool Schema

Q1: Function Calling 和普通 prompt 输出 JSON 有什么区别?

A:

普通 prompt 输出 JSON 依然是自由生成文本，系统还要自己解析和容错；Function Calling 则把模型输出约束到工具名和参数 schema 上，更适合承载外部动作。

一句话区分：前者更像格式约束，后者更像动作契约。

Q2: Structured Outputs 和 Function Calling 怎么选?

A:

如果我要的是最终答案的结构，用 Structured Outputs；如果我要的是让模型触发外部能力，用 Function Calling。前者约束输出，后者组织执行。

Q3：为什么副作用型工具要更严格？

A:

查询类工具最多影响结果正确性，副作用型工具会改变外部世界，一旦重复执行、越权执行或顺序错误，代价更大。所以副作用型工具要默认串行、默认幂等、默认可审计，必要时走审批。

二、MCP 原理与实践

Q4：MCP 和普通 HTTP API 的核心差异是什么？

A:

HTTP API 是通用服务接口，MCP 是面向 AI 应用能力接入的标准协议。MCP 不只关心请求和响应，还关心能力发现、上下文资源、提示模板、生命周期协商、认证和生态互操作。

追问1：为什么 MCP 要区分 tools、resources、prompts？

因为做动作、给上下文、给可参数化提示模板是三类不同能力。把 resources 都包装成 tools，会让只读上下文获取变得别扭，也不利于订阅、浏览和标准化治理。

追问2：为什么企业里常走网关 / 代理模式，而不是 client 直连每个 MCP server？

因为企业更关心统一认证、审计、配额、策略、注册发现和多租户治理。直连当然能跑，但很快会在权限和运维上失控。

Q5：MCP 和 Function Calling 是什么关系？

A:

Function Calling 更偏模型如何表达动作意图，MCP 更偏外部能力如何被标准化接入。前者是模型接口，后者是能力接入协议，它们经常协同使用。

Q6: 什么时候用 stdio，什么时候用 HTTP?

A:

本地开发者工具、IDE 场景更适合 stdio；远程服务、多租户、企业治理和统一认证更适合 HTTP。别把这个答成“只是传输方式不同”，面试官更想听你讲部署边界和治理边界。

三、A2A 与 Agent 互操作

Q7: 为什么 A2A 不能简单用 Tool Calling 代替?

A:

Tool Calling 更适合短、同步、边界清晰的动作；A2A 面向的是完整 Agent 协作，它需要任务状态、artifact、多轮继续、异步更新和能力发现，这些都不是函数调用能自然表达的。

Q8: A2A 和 MCP 的区别是什么?

A:

MCP 解决的是 agent-to-tool / app-to-context 的标准接入问题，A2A 解决的是 agent-to-agent 的标准协作问题。前者接能力，后者协同工作。

Q9: 为什么 Task 比 Message 更重要?

A:

Message 是一次通信，Task 是一件持续存在的工作。跨 Agent 协作的关键不是发出一句话，而是把任务在多个状态之间推进并跟踪 artifact 和进度。

追问1: Agent Card 的作用是什么?

它是 Agent 的能力名片, 帮助调用方在调用前知道对方是谁、会什么、支持什么交互能力, 从而做发现、选择和策略判断。

追问2: 为什么 A2A 强调 Artifact?

因为协作结果经常不是一句文本, 而是报告、代码、表格、文档等工作产物。只有把 Artifact 抽象成一等对象, 才能支持持续更新、版本和后续协作。

追问3: 为什么 A2A 需要 streaming / polling / webhook?

因为远端 Agent 不一定同步返回结果, 很多任务会持续运行、阶段性产出中间结果, 甚至需要继续补输入。所以异步交互模式是 A2A 的自然要求, 而不是附加功能。

四、Session-State 与 Context 生命周期

Q10: Session、Conversation、Thread 有什么区别?

A:

Session 更偏应用或 SDK 层的会话容器; Conversation 更偏平台持久化的对话实体; Thread 更偏工作流或图执行中的一条执行脉络。三者都和连续性有关, 但语义层次不同。

Q11: 为什么要把持久状态和模型上下文分开?

A:

因为很多信息需要系统记住, 但不需要每轮都给模型看。把两者混在一起会导致上下文膨胀、成本上升、噪声变多和安全风险变大。

Q12: 为什么 Compaction 很重要?

A:

真实对话和任务会越来越长，不能无限保留原始历史。Compaction 的价值是保留未来推理真正需要的信息，同时减少 token 成本和噪声。

五、Workflow-Orchestration 与 Durable Execution

Q13: Workflow 和 Agent 是什么关系？

A:

Workflow 管步骤之间的确定性与状态转移，Agent 管步骤内部的不确定性与推理。生产系统里常见的是 Workflow 包裹 Agent，而不是二选一。

Q14: 什么是 Durable Execution？

A:

它让流程在崩溃、重启、长时间等待或人工中断后，仍能从已知正确状态继续，而不是从头执行。重点是持久化进度和恢复语义，不只是失败重试。

Q15: Durable Execution 和 Retry 的区别是什么？

A:

Retry 关注某个动作失败后再试一次，Durable Execution 关注整个过程断了以后能否从正确位置继续。前者是局部容错，后者是流程级可靠性。

六、任务调度、重试、幂等、回滚

Q16: 为什么重试一定要和幂等一起设计？

A:

因为只要允许重试，同一动作就可能执行多次。没有幂等保护，就会产生重复副作用，比如重复发邮件、重复扣费、重复建工单。

Q17: 什么错误适合重试，什么错误不适合？

A:

网络抖动、短时超时、临时限流、下游 5xx 更适合重试；参数非法、权限不足、资源不存在、业务规则不满足通常不适合自动重试。

Q18: 为什么很多时候要补偿而不是回滚？

A:

因为外部副作用一旦发生，通常无法像数据库事务一样直接回退。更现实的做法是执行一个语义上的反向动作，把系统恢复到可接受状态。

七、异步任务、长任务、流式执行

Q19: 为什么 Agent 特别适合异步执行？

A:

因为 Agent 任务经常是多步、多工具、可能很长，还可能等待外部系统或人工输入。同步请求-响应模式很难承载这类长生命周期任务。

Q20: 流式执行和异步执行是什么关系？

A:

异步是执行方式，流式是反馈方式。一个任务可以在后台异步跑，同时把阶段性进度流式推给前端。

Q21: Cancel / Resume 最重要的设计点是什么？

A:

把取消和恢复看成状态转移，而不是简单杀进程或重开一个请求。必须保存 task_id、当前阶段、已完成步骤、已生成 artifact、等待输入状态和取消标记。

八、Agent Runtime 设计

Q22: Agent Runtime 至少包含哪些核心组件？

A:

至少包括编排核心、状态存储、能力注册、模型路由、执行器 / 隔离层、调度与恢复、权限审批、观测与回放。

追问1: 为什么 Runtime 不能被理解成 Workflow 引擎？

因为 Runtime 不只管流程推进，还管上下文装配、模型路由、能力发现、执行隔离、权限审批、artifact、trace 和多租户。Workflow 引擎通常只是它的一部分底座。

Q23: 为什么 Runtime 要把执行与模型解耦？

A:

模型只负责表达意图，不应该直接触达外部副作用。Runtime 通过执行层和策略层把模型自由输出变成受控动作，才能保证安全、可靠和可审计。

追问2: 控制面 / 数据面视角怎么讲更像做过系统？

控制面负责策略、路由、权限、审批、版本、灰度；数据面负责实际执行，包括上下文装配、模型调用、工具执行、artifact 读写。用这套视角回答，能把 Runtime 从“大杂烩模块列表”讲成有层次的系统。

追问3：为什么要把 capability registry、state store、artifact store 分开？

因为它们解决的是不同问题：capability registry 管能力发现与授权，state store 管运行中的会话与状态，artifact store 管大对象和中间产物。全塞一个库，短期省事，长期一定难治理。

Q24：多租户 Runtime 最危险的问题是什么？

A：

串租户状态和串租户权限。只要 session、memory、artifact、缓存或凭据没有 tenant-aware，后果通常比普通系统更严重。

九、Skills、插件化与能力封装

Q25：Tool 和 Skill 的区别是什么？

A：

Tool 是单次动作接口，Skill 是围绕某类任务的一整套方法封装，通常包含指令、示例、资源和脚本。前者暴露“能做什么”，后者沉淀“该怎么做”。

Q26：为什么 Agent 系统需要 Skills？

A：

因为很多稳定有效的经验不应长期埋在 prompt 和代码里。Skill 能把这些经验资产化、版本化、评测化，实现跨项目复用和平台沉淀。

Q27：Skill 和 MCP、A2A 是什么关系？

A：

MCP 更像能力接入协议，A2A 更像远端 Agent 协作协议，Skill 更像能力使用方法包。Skill 可以建立在 MCP 或 A2A 提供的能力之上，但它本身不是协议层。

追问1: 为什么 progressive disclosure / 动态加载很重要?

因为 skill 一多, 如果每次都把全部正文塞进上下文, 会浪费 token、污染注意力并影响选择质量。更好的方式是先看 skill 元数据, 再按需加载正文。

追问2: Skill 为什么也要版本化和 eval?

因为 skill 不是长 prompt, 而是可复用能力资产。它是否值得保留, 要看是否稳定提升任务效果、降低成本、提高一致性, 这些都需要版本和评测来证明。

追问3: Skill 会带来什么安全风险?

典型风险包括隐式权限放大、过时知识、注入与供应链风险、误调用。所以 skill 也需要最小化依赖、版本管理和回滚能力。

01 Function Calling 与 Tool Schema（重要）

这篇主要考察什么

Function Calling 是 Agent 系统里最容易被说得很浅的主题。很多候选人会说：模型识别到需要查天气，就调用 `get_weather` 函数。这句话当然没错，但只够及格。面试官真正想知道的是：

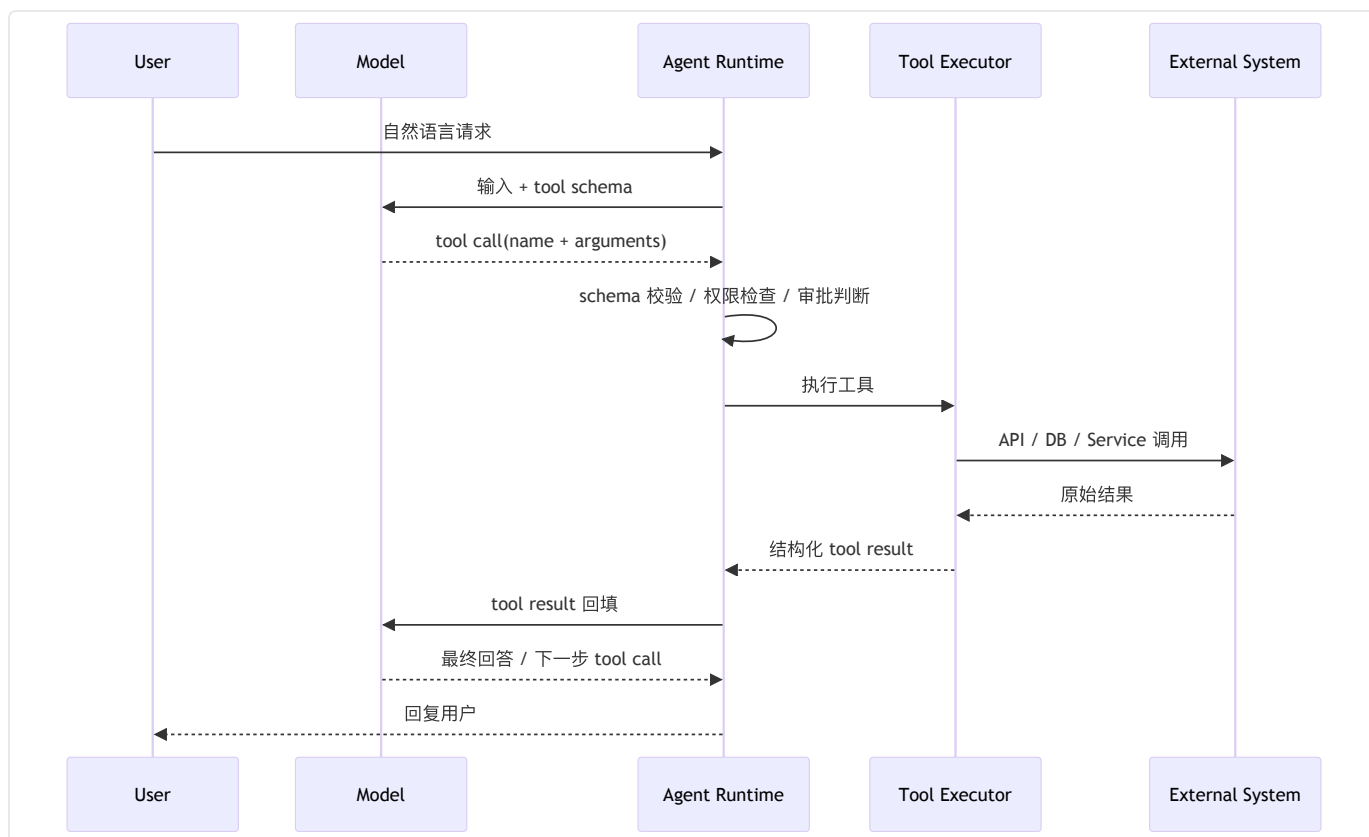
1. 你是否理解 模型输出的是动作意图，而不是函数本身在执行；
2. 你是否知道 Tool Schema 是模型与系统之间的契约，而不是随便写几个字段；
3. 你是否能解释 Function Calling、Structured Outputs、JSON mode、Custom Tools 的边界；
4. 你是否考虑过 参数校验、错误恢复、权限控制、幂等保护、并行调用、副作用管理；
5. 你是否知道在工具数量大、上下文紧张时，如何通过 namespace、tool search、deferred loading、缓存友好设计 做工程优化。

所以这道题的本质，是你会不会把模型的自由输出，变成受约束、可观测、可治理的系统动作。

面试里可以这么说

Function Calling 本质上是让模型在自然语言推理之后，输出一个符合预定义 schema 的动作请求；真正的执行由应用侧完成。Schema 设计得越清楚，系统越稳；运行时控制得越严，副作用越安全。生产里要把它看成协议契约 + 执行循环 + 安全治理三件事，而不是一个方便的 API 小功能。

一图看懂 Function Calling 的生产执行链



这张图里最容易被忽略的一点，是中间那一步schema 校验 / 权限检查 / 审批判断。很多 Demo 直接在收到 tool call 后就执行，但真正可上线的系统一定要在模型和外部副作用之间插一个受控 runtime。

Function Calling 到底解决什么问题

从系统设计角度看，Function Calling 解决的是让语言模型用一种机器可解析、可校验、可执行的方式表达动作意图。

如果没有 Function Calling，你通常会遇到下面几种糟糕情况：

- 模型输出我建议调用 `search_orders(order_id=123)`这类伪代码，你还得靠字符串解析；
- 模型用自然语言描述参数，结果日期、枚举、单位、时间区间都解析错；
- 工具越来越多以后，模型容易乱猜工具名、漏字段、字段拼错；
- 每个项目都写一套把模型输出转成函数参数的胶水代码，系统可维护性很差。

Function Calling 的价值，就在于把这些不确定性收敛成一个结构化接口：

模型只负责选哪个工具、填什么参数，应用负责能不能执行、什么时候执行、执行后怎么处理。

你要特别强调：模型不会真正调用函数。函数是你的系统、容器、服务、数据库、第三方 API 在执行。模型只是在建议一个结构化动作。这句话在面试里非常重要，因为它直接体现你是否理解LLM 是控制面，不是数据面。

Tool Schema 是什么，为什么它比想象中更重要

Tool Schema 不是给机器看的注释，而是一个既给模型看、也给运行时看、还给开发者看的**动作契约**。它至少承担三层作用：

第一层：给模型看，帮助模型正确选择工具

模型需要理解：

- 工具叫什么；
- 这个工具能做什么，不能做什么；
- 参数有哪些；
- 参数含义、单位、枚举值是什么；
- 哪些参数是必须的；
- 什么情况下应该调用，什么情况下不该调用。

这就是为什么 tool name、description、parameter description 都不能乱写。

一个差的描述会直接导致模型选错工具；一个模糊的字段名会直接导致参数歧义。

例如，`query` 这个字段在一个搜索工具里可能表示用户原始查询，在另一个报表工具里可能表示SQL过滤条件，如果不写清楚，模型经常会混。

第二层：给运行时看，作为严格校验依据

在生产系统里，tool schema 不是提示词的一部分而已，它还是运行时做参数校验、权限治理和审计记录的基础。

你至少要做到：

- 不允许未知字段悄悄混进来；
- 不允许必填字段缺失；
- 不允许字符串本应是枚举却随便写；
- 不允许日期、金额、ID、分页参数超出约束；
- 不允许模型绕过审批直接传危险参数。

如果你的运行时只把 schema 当作给模型参考，不拿它做真正校验，那么系统的安全性和可预测性会很差。

第三层：给平台看，形成可复用的能力注册

当系统只有 3 个工具时，你可以在代码里手写；当系统有 30 个、300 个、3000 个工具时，你必须把 schema 当成资产。

它会影响：

- 工具注册中心如何组织；
- namespace 如何拆分；
- 是否支持动态发现和延迟加载；
- 是否支持跨团队共享；
- 是否支持版本化、灰度、回滚；
- 是否支持自动生成 SDK、文档和测试用例。

所以，Schema 不是细节，而是整个能力平台的元数据入口。

一个好 Tool Schema 应该长什么样

你可以从下面七条回答：

1. 名字要像动作，不要像实现细节

好名字示例：

- `search_customer_orders`
- `get_invoice_status`
- `create_meeting_invite`

差名字示例：

- `tool1`
- `query`
- `run_api`
- `execute_order_service_v2`

原则是：模型理解的是语义，不是你的内部模块名。工具名要体现业务动作，而不是内部实现路径。

2. description 要写边界，不要只写功能

差的 description：

搜索订单信息。

更好的 description:

按订单号、用户邮箱或时间区间检索订单。适合查询订单状态、发货时间和支付结果；不用于修改订单，也不用于模糊生成报表。

这种写法的好处在于，它告诉模型这个工具**适合做什么，不适合做什么**。很多误调用不是因为功能描述太少，而是因为边界描述太少。

3. 参数名要业务语义清晰

`start_date` 比 `from` 清楚；

`user_email` 比 `user` 清楚；

`currency_code` 比 `currency` 清楚；

`page_size` 比 `limit` 在很多业务里更容易懂。

校招生常见错误是把参数名写成后端实现风格，模型会更偏好业务语义明确的字段。

4. 枚举和约束能写就写

如果一个字段只能是 `paid / pending / failed`，就不要让模型自由填字符串。

如果分页上限是 100，就写在 schema 里；

如果日期必须是 ISO 格式，也要约束。

这里的核心思想是：**能前移到契约层约束的问题，不要留到执行层兜底。**

5. 严格模式比尽量解析更适合生产

现在主流平台已经支持更严格的 schema 约束。你在回答时要强调：

生产里更推荐让模型输出严格匹配 schema，而不是让服务端写一堆容错解析 JSON 的补丁代码。后者初期看起来灵活，长期却会把脏数据和隐性故障引进来。

一个很重要的工程点是：如果启用严格 schema，schema 本身也必须满足更严格要求，比如对象通常要显式禁止额外字段，并把字段必填关系描述完整。面试里说到这里，说明你不是停留在知道功能，而是关心它在生产里的可验证性。

6. 把可选性、默认值、空值语义提前讲清楚

很多工具调用出错，不是字段缺了，而是有值 / 没值 / 用户没说 / 用户明确说不限四种语义混在一起。例如查询报销单：

- 用户没说时间范围；

- 用户说最近一个月；
- 用户说全部时间；
- 用户只说开始时间没说结束时间。

这些语义如果不在 schema 和调用约定中设计清楚，模型填参会反复出错。

7. 工具返回值也应该结构化

很多人只关注入参，不关注出参。实际上，tool result 的结构化同样重要，因为它会影响模型下一步推理。

一个好的 tool result 应该区分：

- 业务数据；
- 错误码 / 错误类型；
- 是否可重试；
- 是否需要用户补充信息；
- 是否需要审批；
- 是否包含敏感字段被脱敏。

如果工具返回一大段自由文本，模型下一步经常会误判状态、重复调用或忽略错误。

Function Calling、Structured Outputs、JSON mode、Custom Tools 的边界

这部分是高频考点，必须说清。

1. Function Calling

适合场景：模型需要触发外部动作，或者访问应用侧工具。

核心是：模型选工具并给参数，应用执行后再把结果喂回模型。

典型例子：

- 查数据库；
- 调外部 API；
- 执行搜索；
- 发起工单；
- 创建日程；

- 调 MCP server；
- 触发代码执行或 shell。

2. Structured Outputs

适合场景：你想约束的是模型最终给你的输出结构，而不是要它调外部工具。

比如：

- 把简历解析为结构化 JSON；
- 把用户请求抽成 DSL；
- 把客服意图输出为分类 + 槽位；
- 把审阅结果输出为固定 schema。

一句话记忆：

Function Calling 是输出动作；Structured Outputs 是输出结构。

3. JSON mode

JSON mode 只能提高输出像 JSON 的概率，但不等于严格遵守你的业务 schema。

所以在生产场景里，如果你真的要依赖结构稳定性，应该优先用更严格的 schema 机制，而不是只求它长得像 JSON。

4. Custom Tools / Free-form Tools

有些场景下，工具的输入不适合被强行拆成很多字段，例如：

- 代码补丁；
- shell 命令；
- 正则表达式；
- SQL 草案；
- 长文本搜索表达式；
- 复杂脚本片段。

这时可以让模型输出一段自由格式文本交给工具。但你必须知道：自由度越高，校验难度越大，安全风险也越大。因此它更适合受控场景，如代码 agent、受限 shell、专门的 DSL 解析器，而不适合随意暴露给有副作用的业务系统。

运行时如何执行一次安全的 Tool Call

面试时，建议你用下面这个执行循环来回答：

第一步：把可用工具集合发给模型

这里不是把全世界工具都给模型。而是给它当前回合可用、当前租户可见、当前权限允许、当前业务阶段需要的工具集合。

你可以顺带讲两个工程点：

- 对大工具集，要做 namespace 或按场景裁剪；
- 如果平台支持动态工具发现或 tool search，可以避免一开始把所有 schema 塞进上下文，减少 token 和缓存失效。

第二步：模型输出 tool call

模型会输出：

- 工具名；
- 参数对象；
- 有时会有多个调用；
- 有时会选择不调工具直接回答。

此时还不能直接执行，因为模型只是提出意图。

第三步：运行时做校验与治理

这是最关键的一步。至少包括：

- **Schema 校验**：字段名、类型、枚举、必填、范围；
- **权限校验**：这个用户 / 租户 / 角色是否能执行；
- **业务校验**：比如会议时长上限、金额上限、目标环境限制；
- **审批校验**：删除、扣费、发通知、写生产库等危险动作是否需要人审；
- **幂等校验**：是否会重复执行；
- **上下文校验**：是否需要额外确认，比如是要给全部用户群发吗？；
- **安全校验**：是否存在 prompt injection 诱导危险调用。

第四步：调用工具执行器

执行器通常不直接等于业务函数本体。更稳妥的设计是：

- 模型层只看到 tool schema；
- runtime 调用一个 tool executor；
- executor 再去访问内部 service / API / queue / database；
- 所有副作用、签名、超时、重试、日志都放在 executor 侧。

这样模型和内部系统之间有一个明确的隔离层。

第五步：把工具结果作为结构化信息回填

不要只返回执行成功四个字。要让模型理解：

- 成功还是失败；
- 失败是什么类型；
- 是否可重试；
- 是否需要用户补充信息；
- 产物 ID 是什么；
- 是否已经产生副作用。

第六步：让模型基于结果继续推理

模型可能：

- 直接给最终答案；
- 继续调用下一个工具；
- 请求用户补充信息；
- 发起审批；
- 结束当前任务但产出 artifact。

这就是典型的 tool loop。

优秀的候选人会补一句：tool loop 需要终止条件和最大步数上限，否则会进入错误的循环调用。

Parallel Tool Calls 什么时候能开，什么时候该关

这也是高频追问。标准回答是：

可以开并行的场景

- 多个只读查询，彼此独立；
- 多个数据源的聚合查询；
- 批量召回再 rerank 前的独立检索；
- 多个无副作用、无顺序依赖的工具。

例如：

- 同时查天气、交通、会议室空闲情况；
- 同时调用多个搜索源；
- 同时取用户画像、订单历史、会员等级。

不该开并行的场景

- 有副作用的写操作；
- 后一步依赖前一步结果；
- 有共享资源竞争；
- 容易产生重复动作；
- 审批链条尚未完成。

例如：

- 同时创建工单和发送通知，但通知内容依赖工单 ID；
- 同时对同一订单做取消和退款；
- 同时更新配额和发放优惠券。

一句话记忆：

只读、独立、可交换的动作，适合并行；有副作用、有关联、有顺序的动作，默认串行。

工具很多时，为什么不能全塞给模型

这是现在越来越新的考法。因为工具规模一大，就会带来三类问题：

1. 上下文成本变高

每个 tool schema 都占 token。工具一多，系统提示和工具定义本身就会吃掉大量上下文，影响真正业务内容。

2. 选择精度下降

当大量工具描述彼此接近时，模型会更容易选错。例如：

- `search_customer_orders`
- `search_orders_by_status`
- `query_order_summary`
- `lookup_order_details`

这些工具如果都长得差不多，模型的选择负担会明显上升。

3. Prompt caching / KV cache 更容易失效

如果每一轮都把一大坨工具定义重新发给模型，缓存命中率会受影响。现在很多平台开始提供 tool search、延迟加载和 namespace 机制，本质上就是在解决大工具集的成本和稳定性问题。

所以你可以给一个很工程化的回答：

小工具集适合静态注册；大工具集应该分 namespace、按场景裁剪，必要时做动态发现或延迟加载。

安全：为什么 Function Calling 是 Agent 风险的第一入口

Function Calling 能让模型连接世界，也就意味着它会成为风险入口。面试官如果问安全，不要只说做一下参数校验，要从五层讲：

1. 能力最小化

不要把不需要的工具暴露给当前任务。

用户只是问订单状态，不该把批量退款导出全部客户数据也给模型。

2. 参数最小化

Schema 不要暴露危险自由度。

例如删除文件工具不要让模型任意传绝对路径；SQL 工具不要直接让模型写裸 SQL 到生产库。

3. 执行隔离

工具最好经过 sandbox、proxy 或 service layer，而不是让模型直接连内网高权限系统。

4. 人审与审批

有副作用、高风险、跨租户、越权级别高的动作，必须可中断并要求 human-in-the-loop。

5. 审计与回放

每次工具调用都要能追溯：

- 谁触发的；
- 模型当时看到了什么；
- 选了哪个工具；
- 传了什么参数；
- 运行时做了哪些校验；
- 最终是否执行成功；
- 是否产生了副作用。

如果你能答到Function Calling 的最大风险不是参数格式错，而是模型把不该执行的动作合法化，通常会给面试官留下很好的印象。

常见失败模式

失败模式 1：工具描述太短，模型老是选错

症状：模型总是调用相似工具中的错误一个。

根因：description 只写了查询订单，没有写适用边界。

修复：把能做什么 / 不能做什么 / 什么时候用写清楚。

失败模式 2：Schema 太松，解析端补丁越来越多

症状：后端写一堆 if-else 修字段名、补默认值、猜枚举。

根因：把宽松误当成鲁棒。

修复：严格 schema + 明确错误返回 + 让模型重新填参，而不是后端偷偷猜。

失败模式 3：工具返回自由文本，模型误判下一步

症状：工具失败了，但模型以为成功继续往下走。

根因：tool result 不结构化。

修复：显式返回 status / error_type / retryable / artifact_id 等字段。

失败模式 4：模型可以直接执行高风险动作

症状：用户一句 prompt injection，模型开始调用危险工具。

根因：运行时没有审批和权限边界。

修复：把敏感工具放进 approval gate，动作前强校验。

失败模式 5：重复调用导致重复副作用

症状：同一邮件发了两次、同一订单扣了两次。

根因：没有幂等键，或者失败重试与成功返回混淆。

修复：副作用工具必须有幂等策略和去重记录。

高频面试题与标准回答

1. Function Calling 和普通 prompt 输出 JSON 有什么区别？

标准回答：普通 prompt 输出 JSON 依然是自由生成文本，系统通常还要自己解析和容错；Function Calling 则把模型输出约束到工具名和参数 schema 上，更适合承载外部动作。生产里二者最大的区别是：前者更像格式约束，后者更像动作契约。

2. Structured Outputs 和 Function Calling 怎么选？

标准回答：如果我要的是最终答案的结构，用 Structured Outputs；如果我要的是让模型触发外部能力，用 Function Calling。前者约束输出，后者组织执行。

3. 什么时候不能相信模型生成的参数？

标准回答：任何涉及副作用、权限边界、金额、资源 ID、系统命令、生产环境写入的参数都不能只因为看起来合法就直接执行，必须经过 schema 校验、业务校验和权限校验。

4. 为什么副作用型工具要更严格？

标准回答：查询类工具最多影响结果正确性，副作用型工具会改变外部世界，一旦重复执行、越权执行或顺序错误，代价更大。因此副作用工具要默认串行、默认幂等、默认可审计、必要时需要审批。

5. 如果模型连续两次都选错工具怎么办？

标准回答：不要盲目继续试。应该从三层排查：第一看 schema / description 是否有歧义；第二看 runtime 是否给了过多相似工具；第三看是否要增加一个上游 routing / tool selection 阶段。必要时加 tool choice 限制、namespace 或更强的场景裁剪。

6. Tool Schema 设计里最重要的一条经验是什么？

标准回答：把业务边界写进 schema，而不是只写字段。模型最怕的不是字段多，而是边界模糊。

7. 为什么工具结果最好也做 schema？

标准回答：因为工具结果不是给人看的说明文，而是给模型作为下一步决策依据。如果结果结构化，模型更容易判断是否继续、是否重试、是否向用户追问。

8. 当工具很多时，你怎么优化？

标准回答：按业务域拆 namespace，按场景裁剪工具集合；对超大工具集使用延迟加载 / tool search；尽量稳定高频工具定义，提升缓存命中；减少近义工具，防止模型混淆。

9. Function Calling 最大的工程价值是什么？

标准回答：它把模型与系统之间的接口标准化了。这样我们可以把校验、权限、日志、审批、幂等和回放都统一放到 runtime 层，而不是散落在每个 prompt 和每个业务函数里。

10. Function Calling 最大的风险是什么？

标准回答：模型通过看起来结构化的方式触发高风险动作。如果系统把 schema 合法误当成业务合法，就会把安全问题从自然语言层搬到结构化调用层，风险并没有消失。

项目回答模板

你可以这样讲一个自己的项目：

在我们的 Agent 项目里，模型不会直接访问后端服务，而是通过 Function Calling 输出结构化动作。我们先按业务域注册工具，比如检索、工单、日程和通知，每个工具都定义清晰的 name、description 和参数 schema。运行时收到 tool call 后，不会立刻执行，而是先做 schema 校验、权限判断和审批判断。对查询类工具我们允许并行，对副作用类工具默认串行并带幂等键。工具执行结果也不是一段自由文本，而是返回结构化状态和业务字段，方便模型决定下一步是继续调用、向用户追问还是直接结束。工具规模大了之后，我们又引入 namespace / 动态裁剪，避免把全部工具都塞进上下文，降低 token 成本并提高选工具准确率。整个过程都打 trace 和审计日志，这样遇到误调用时可以回放到具体的 schema、参数和执行链路。

既有概念，也有工程细节，还带上了 trade-off 和治理视角。

你在面试里最值得主动补充的观点

第一，Function Calling 不是让模型会编程，而是让模型会表达受控动作。

第二，Tool Schema 的设计质量，直接决定模型是否懂你的系统。

第三，生产系统的关键不在于模型会不会调工具，而在于 runtime 会不会拒绝不该执行的工具。

第四，工具调用是 Agent 可靠性的第一层地基。上层再复杂的 Planning、Memory、Multi-Agent，如果底层 tool contract 混乱，系统就不可能稳定。

小结

把这篇内容浓缩成三句话，就是：

1. Function Calling 让模型输出结构化动作意图，真正执行在应用侧；
2. Tool Schema 是动作契约，既服务模型理解，也服务运行时校验和平台治理；
3. 生产可用的关键不在调起来，而在调得准、调得稳、调得可控、调得可审计。

如果你能把这三句话展开成完整的工程回答，这一题在校招面试里通常已经足够强。

延伸阅读（OAI官方为主）

- [OpenAI Function Calling](#)
- [OpenAI Structured Outputs](#)
- [OpenAI Tool Search](#)
- [OpenAI Skills](#)

- [OpenAI Agents SDK](#)

02 MCP 原理与实践（重要）

这篇主要考察什么

MCP 是这两年 Agent 面试里非常容易被问到、也非常容易被答虚的一道题。很多同学会说：MCP 就是统一工具接入标准。这句话方向没错，但太薄。面试官真正想考的是：

1. 你是否知道 MCP 想统一的不只是调工具，而是 AI 应用如何连接外部系统、资源、提示和工作流能力；
2. 你是否理解 MCP 的核心角色不是一个 server，而是 host / client / server 三者协作；
3. 你是否知道 MCP 为什么要有 生命周期、能力协商、传输层、认证框架、资源与工具区分；
4. 你是否明白 MCP 和传统 HTTP API、自定义 function calling、A2A 的边界；
5. 你是否能讲清楚本地 stdio、远程 HTTP、网关代理、多租户隔离、审计和安全落地模式。

所以这道题不是背诵名词，而是考察你是否具有协议意识。协议意识强的人，面对生态演进时会思考可移植性、互操作性和治理成本；协议意识弱的人，往往只会在单项目里写私有 glue code。

面试一句话版本：

MCP (Model Context Protocol) 本质上是一个连接 AI 应用与外部系统的开放协议。它把能力如何被发现、描述、协商、调用、授权和返回标准化了，使工具、资源和提示不必为每个模型平台各写一套接入方式。小项目可以不用 MCP，但一旦你想做跨应用复用、跨团队共享、生态互通和企业级治理，MCP 的价值会迅速放大。

为什么 MCP 会出现

如果你只在一个项目里、只接一个模型、只写几个工具，那么 Function Calling 已经足够好用。问题出在规模和生态一增长，私有方案会出现四个明显痛点：

1. 每个平台都要接一遍

假设你写了一个企业知识库检索能力。

如果没有统一协议，你可能要分别为：

- OpenAI 的 function calling；
- Anthropic 的 tool use；

- 某个 IDE 插件系统；
- 某个内部 agent runtime；
- 某个桌面客户端；

都写一套适配层。能力本身没变，胶水代码却复制了很多份。

2. 能力描述不可迁移

不同系统对工具描述、资源发现、认证、版本、流式返回、订阅更新的要求不一致，导致你的能力难以在别的宿主里复用。

也就是说，能力资产无法沉淀成可插拔模块，只能沉淀成和某平台强绑定的一份实现。

3. 上下文和资源接入缺少统一抽象

很多真实能力并不是一个函数，而是：

- 一组可以浏览的资源；
- 一类参数化模板；
- 一个需要订阅变化的数据源；
- 一个需要代理认证的远程服务；
- 一个由宿主提供根目录 / 工作区上下文的能力。

如果只用函数 + 参数抽象，会显得很勉强。

4. 企业治理成本越来越高

一旦涉及：

- 多租户；
- 多模型平台；
- 外部 SaaS；
- 内部敏感系统；
- 审计和审批；
- 认证与授权；
- 代理和网关；
- 版本兼容；

私有协议很快就会变成平台债务。
MCP 的价值，正是在于把这些每家公司都要重新发明一次的问题标准化。

一图看懂 MCP



一句话解释这张图：
Host 是宿主应用，MCP Client 是宿主里的协议适配层，MCP Server 是能力提供方。协议本身定义的不是某个具体工具，而是三者之间如何发现能力、协商能力、调用能力和处理结果。

MCP 的核心角色：Host、Client、Server

这是高频基础题，必须清楚。

1. Host

Host 指最终承载模型交互的应用环境，例如：

- Chat 产品；
- IDE / AI 编程工具；
- 企业内部 Agent 平台；
- 桌面应用；
- 浏览器插件；
- 任何会把模型和外部能力连接起来的宿主。

Host 的职责不是直接理解 MCP 协议细节，而是把我需要连接哪些能力、给模型暴露哪些上下文、当前用户有何权限这些事情管理起来。

2. MCP Client

MCP Client 是 Host 里负责说 MCP 协议的那一层。

它的职责包括：

- 连接 server；
- 发起初始化和能力协商；
- 列出 tools / resources / prompts；
- 处理调用请求和返回结果；
- 管理认证；
- 处理日志、错误、事件、通知。

面试中一个很容易加分的点是：**Client 并不等于模型**。

模型只是通过 Host / Runtime 间接利用这些能力；真正跟 server 说话的是 client 这一层。

3. MCP Server

MCP Server 是能力提供方。它可以暴露：

- **Tools**：让模型可以发起动作；
- **Resources**：让应用 / 模型可以读取外部上下文；
- **Prompts**：让能力提供方把一段可参数化的提示逻辑标准化输出。

你可以把它理解成能力适配器，但又比传统 adapter 更强，因为它不是某个平台私有插件，而是一个面向开放生态的标准服务端接口。

MCP 不只是 Tools：Tools、Resources、Prompts 分别是什么

这部分面试特别喜欢问，因为很多人只知道 tools。

1. Tools：模型驱动的动作能力

Tools 是 MCP 里最接近 Function Calling 的部分。

特点是：模型可以选择并调用它，通常用于触发动作，例如：

- 搜索；
- 查询数据库；
- 调 API；
- 执行脚本；

- 创建工单；
- 修改任务状态。

一句话记忆：

Tool 更偏做事。

2. Resources：标准化上下文入口

Resources 提供的是可读取的上下文资源，通常由应用侧主动获取或挂载，例如：

- 文件内容；
- 数据库表快照；
- 文档页面；
- 项目元数据；
- 仓库状态；
- 可订阅更新的资源流。

MCP 明确区分 resource 和 tool，很重要，因为不是所有外部能力都适合建模成动作调用。很多时候你只是要把某些上下文安全、标准地提供给模型或宿主，而不是触发执行。

一句话记忆：

Resource 更偏给上下文。

3. Prompts：可参数化的提示模板

Prompts 允许 server 暴露一些可参数化的 prompt 模板。

听起来像小事，但它对平台化很重要，因为很多团队不是只输出一个函数，而是希望把：

- 分析模板；
- 审核模板；
- 代码修复模板；
- 特定领域任务模板；

作为可复用资产暴露出去。这样，能力的边界不再只是能做什么动作，还包括能提供什么结构化 prompt 入口。

一句话记忆：

Prompt 更偏给方法。

MCP 和 Function Calling 的关系

面试里千万别把它们讲成对立关系。更准确的说法是：

- Function Calling 解决的是 **模型如何表达动作意图**；
- MCP 解决的是 **外部能力如何被标准化暴露给 AI 应用**。

所以它们经常是配合关系，而不是替代关系。

你可以把 MCP server 暴露的 tools，最终映射成模型可调用的工具；也可以把远程 MCP server 当成一种能力来源，让 runtime 以统一方式接入。

如果面试官追问那为什么不只用 Function Calling 就好，你可以这样答：

如果只有单应用、少量工具、没有生态互通诉求，只用 function calling 完全可以；但一旦你要跨宿主、跨平台、跨团队复用能力，并且希望统一认证、资源发现和上下文供给，MCP 会更有长期价值。

MCP 协议里为什么有生命周期和能力协商

这部分很容易拉开差距。

很多同学会把工具协议想成拿到 schema 就调用，但 MCP 更像一个真正的会话协议，因此需要 lifecycle。

初始化 (Initialization)

初始化阶段至少要解决三件事：

1. **版本协商**：双方支持哪个协议版本；
2. **能力协商**：client 和 server 各自支持哪些特性；
3. **实现信息交换**：让对方知道自己是谁、能做什么。

为什么这很重要？因为 MCP 不是一个固定死的、永远只有一种能力的接口。不同 server 可能支持：

- tools；
- resources；
- prompts；
- sampling；
- roots；
- logging；
- completion；
- tasks；
- 订阅变化通知。

如果没有初始化和能力协商，client 只能靠猜，生态就不稳。

运行阶段（Operation）

运行阶段包括：

- 列出能力；
- 读取资源；
- 调用工具；
- 获取提示；
- 处理日志与事件；
- 接收通知；
- 执行扩展能力。

这里真正体现了 MCP 作为协议的价值：你不是在对接一个孤立 API，而是在进行一轮标准化会话。

关闭阶段（Shutdown）

很多 Demo 完全不考虑关闭，但生产里连接生命周期管理很重要，尤其是：

- 本地 stdio 进程；
- 长连接 HTTP / SSE；
- 代理层；
- 资源订阅和清理。

面试里如果你能主动提到连接建立、能力协商、运行、关闭这四段生命周期，说明你理解的是协议，不只是概念。

传输：stdio 与 Streamable HTTP 怎么选

这是 MCP 的高频工程题。

1. stdio

适合：

- 本地工具；
- IDE 插件；

- 桌面应用；
- 单机开发环境；
- 本地进程内隔离相对容易的场景。

优点：

- 实现简单；
- 本地性能好；
- 不一定要暴露网络服务；
- 适合开发者工具和本地 agent。

缺点：

- 进程管理复杂；
- 分布式部署不方便；
- 企业认证、统一代理、网关和可观测通常不如 HTTP 友好；
- 对容器编排和远程服务治理不够自然。

一个很经典的落地点是：**stdio server 不要往 stdout 打普通日志**，否则会污染协议流；日志应该走 stderr 或单独通道。这个点很细，但很能体现你看过实践文档。

2. Streamable HTTP

适合：

- 远程 MCP server；
- 多租户平台；
- 企业内网服务；
- SaaS 能力提供；
- 需要统一认证、代理、网关和安全治理的场景。

优点：

- 更适合生产网络架构；
- 更容易接入 OAuth、反向代理、审计与 WAF；
- 更容易跨语言和跨环境部署；
- 更符合服务化治理方式。

缺点：

- 网络链路更复杂；

- 远程调用延迟更高；
- 需要额外处理认证、连接恢复和多租户隔离。

一句话选择建议：

本地开发者工具偏 `stdio`，企业级远程能力偏 `HTTP`。

认证与授权：为什么 MCP 不只是连上就行

面试里答 MCP，如果完全不提 `auth`，基本不够完整。

因为 MCP 很多时候连的是：

- 企业知识库；
- 内部业务系统；
- 第三方 SaaS；
- 带用户身份的资源；
- 高敏感数据与动作。

`HTTP` 传输下，MCP 已经把授权问题纳入标准考虑。

你需要知道三点：

1. 认证不是工具参数的一部分

不要把 `access token` 混在业务参数里传来传去。

认证应该在连接层、传输层或 `server` 元数据协商层处理，而不是让模型直接接触认证细节。

2. 用户身份和模型能力要分离

很多场景下，MCP `server` 代表的是当前用户在这个外部系统上的权限，而不是 Agent 拥有一个超级账号。

这意味着：

- 认证上下文要和用户会话绑定；
- 不同租户和不同用户的能力可见性应不同；
- 模型不能跨会话复用不该复用的 `token`。

3. 企业落地要考虑代理与网关

真实生产里很少是客户端直连每一个 MCP server。

更常见的是：

- 统一 gateway；
- 企业 proxy；
- 审计与访问策略中间层；
- 密钥托管；
- 统一注册和健康检查。

所以，一个成熟回答应该说到：

MCP 的问题不只是怎么接上，而是怎么在企业环境里安全、可审计、可治理地接上。

Sampling、Roots、Completion、Tasks：为什么这些细节值得知道

校招生不一定要背所有扩展，但知道这些关键词，会让你显得更懂协议。

1. Sampling

Sampling 允许 server 请求 client 侧再发起一次模型交互。

这说明 MCP 不是单向的 client 调 server，而可能出现 server 请求宿主帮它完成某种模型推理的模式。

但这里一定要强调：涉及额外模型调用和潜在敏感上下文时，**必须有人类可控边界**，不能让 server 无限向 client 索取更多推理权限。

2. Roots

Roots 可以理解为宿主向 server 暴露的一组根上下文，例如允许访问的根目录、工作区范围等。

这对 IDE、代码 agent、本地文件操作场景很关键，因为它决定了 server 能在什么边界内工作。

3. Completion

Completion 允许对 prompt 参数或 resource 模板参数做自动补全。

它的意义不在炫技，而在于工具和资源模板体验可以做得更友好、可引导、可发现。

4. Tasks

随着协议演进，MCP 不再只适合短平快函数调用，也开始吸收更长任务、更异步化的一些能力表达。你不一定要把 Tasks 细节背完，但知道 MCP 正在往更完整的能力交互协议演进，会让你的回答更贴近 2026 的生态现实。

MCP 和 A2A 的边界怎么讲

这是当前最容易问混的一道题。推荐你直接用一句话先定锚：

MCP 更偏 agent-to-tool / app-to-context；A2A 更偏 agent-to-agent。

展开讲：

- MCP 里的能力提供者通常是工具、资源、提示模板、上下文接口；
- A2A 里的对端更像一个完整 Agent，它有自己的状态、任务、artifact、生命周期，甚至有自己的内部工具和推理逻辑；
- MCP 更强调我怎么把外部能力接进来，A2A 更强调多个 agent 怎么协同分工。

你也可以举例子：

- 企业报表系统提供查报表、读资源、取模板，很适合 MCP；
- 法务 Agent 接收合同审查任务，产出 artifact，等待补充资料，再继续任务，比较适合 A2A。

什么时候应该上 MCP，什么时候没必要

适合上 MCP 的场景

1. 你要做平台化能力复用，不想每个模型平台各接一遍；
2. 你要接的不只是函数，还有资源、提示模板、文件工作区等上下文；
3. 你有多个宿主：IDE、Chat、后台 Agent、桌面端等；
4. 你要做企业级认证、审计和网关治理；
5. 你希望能力提供方和能力消费方之间通过开放协议解耦。

暂时没必要的场景

1. 单应用、单模型、工具很少；
2. 纯内部项目，没有生态互通诉求；
3. 只是验证业务可行性，协议成本还不值得承担；
4. 团队对协议治理、版本兼容、认证管理还没有准备。

一个成熟的回答不是所有项目都应该用 MCP，而是能说出：
MCP 有长期复用价值，但也有引入协议复杂度的成本。

2025—2026 的 MCP 新变化，面试里可以怎么用

如果你想体现不是只看过早期博客，可以提到下面几件事：

1. 当前规范版本已经推进到 2025-11-25

这意味着你不能再把 MCP 只理解成 2024 年初那个本地工具适配的早期形态。
它已经逐渐成熟为一个更完整的开放协议，有版本化、授权、扩展、治理和生态路线。

2. 2026 路线图更强调治理和企业准备度

MCP 现在讨论的不只是能不能接上，还包括：

- 正式治理机制；
- 工作组与提案流程；
- 企业级审计与可观测；
- 企业管理认证；
- gateway / proxy 模式；
- 配置可移植性。

这说明 MCP 已经从开发者方便使用走向企业愿不愿意用。

3. SDK 开始有 tiering 思路

这反映出生态成熟度的另一个信号：并不是所有 SDK 都同样完整和同样稳定，社区开始明确不同层级的支持与兼容要求。

对于面试来说，这意味着你不应把支持 MCP 简单理解成能列个工具就算支持。

4. MCP Apps 与富 UI 能力

MCP 甚至开始探索工具返回富交互 UI 的能力。这很值得你在面试中顺带一提，因为它说明 MCP 不只是文本工具协议，而在往更丰富的人机交互形态扩展。

如果你的项目涉及审批卡片、交互表单、嵌入式操作面板，这个点会特别加分。

企业落地模式：我建议你背三种

模式一：本地开发者工具模式

- Host: IDE / 本地 agent
- Client: 内嵌在应用
- Server: 本地 stdio 进程
- 能力: 文件、代码库、shell、git、issue tracker
- 优点: 快、简单、适合开发体验
- 风险: 本地权限过大时，安全边界薄弱

模式二：远程服务模式

- Host: 企业 Chat / Agent 平台
- Client: 平台统一 MCP 连接层
- Server: HTTP 远程服务
- 能力: 知识库、CRM、报表、工单、审批系统
- 优点: 统一认证、统一治理、易于运维
- 风险: 网络链路复杂、延迟和多租户隔离要求高

模式三：网关代理模式

- Host: 多个上层应用
- Client: 统一 gateway / proxy
- Server: 后面挂多个 MCP server
- 能力: 集中注册、统一审计、策略编排、密钥管理
- 优点: 平台化最强，治理能力最好
- 风险: 架构复杂度和平台投入更高

面试里如果你能把这三种模式讲出来，已经明显不是只停留在知道 MCP 这个词。

常见误区

误区 1: MCP 就是统一函数调用

不完整。

MCP 的范围比函数调用大，因为它还处理 resources、prompts、生命周期、认证与扩展能力。

误区 2：上了 MCP 就不需要业务治理

错误。

协议统一的是接口，不是替你做权限、审计、审批和幂等。真正的治理还得由 runtime、gateway 和业务系统承担。

误区 3：MCP 一定比 HTTP API 更好

不一定。

如果你只是两个服务之间的固定集成，普通 HTTP API 可能已经足够。MCP 的优势在于开放生态和标准化能力接入，而不是替代所有内部服务调用。

误区 4：只要工具能列出来，就叫 MCP 化完成

远远不够。

真正成熟的 MCP 化还要看：

- schema 质量；
- 认证方式；
- 资源表达；
- 可观测性；
- 错误处理；
- 版本兼容；
- 多租户隔离；
- SDK 支持度。

误区 5：MCP 和 A2A 二选一

错误。

二者解决的层次不同，很多系统会同时用：

Agent 内部通过 MCP 接工具，Agent 之间通过 A2A 协作。

高频面试题与回答模板

1. MCP 和普通 HTTP API 的核心差异是什么？

标准回答：HTTP API 是通用服务接口，MCP 是面向 AI 应用能力接入的标准协议。MCP 不只关心请求和响应，还关心能力发现、上下文资源、提示模板、生命周期协商、认证和生态互操作。

2. MCP 和 Function Calling 是什么关系？

标准回答：Function Calling 主要是模型输出动作意图的方式，MCP 主要是外部能力暴露给 AI 应用的方式。前者更偏模型接口，后者更偏能力接入协议。它们常常协同使用。

3. MCP 为什么要区分 tools 和 resources？

标准回答：因为做动作和给上下文是两类不同能力。把资源都包装成工具，会让只读上下文获取变得别扭，也不利于订阅、浏览和标准化上下文管理。

4. 什么时候用 stdio，什么时候用 HTTP？

标准回答：本地开发者工具、IDE 场景适合 stdio；远程服务、多租户、企业治理和统一认证更适合 HTTP。

5. 企业里直接让客户端连每个 MCP server 可行吗？

标准回答：通常不优雅。更常见是通过网关或代理统一接入，集中做认证、审计、策略、配额和注册发现。

6. MCP 最大的工程价值是什么？

标准回答：把能力接入从项目私有 glue code 提升为生态可互通、平台可治理的标准接口，让能力真正能跨宿主复用。

7. MCP 最大的挑战是什么？

标准回答：引入协议本身就会增加复杂度。你要面对版本兼容、认证、SDK 成熟度、多租户和网关治理，因此它更适合有复用诉求和平台诉求的系统。

8. 如果面试官问你你们项目为什么没用 MCP，该怎么答？

标准回答：可以直接说当前项目规模和边界下，function calling 加统一 tool registry 已经够用，MCP 的复用收益还没覆盖其协议复杂度。但我们在设计上保留了能力注册和 schema 标准化，为后续迁移或对外开放留接口。这样的回答比我们不会要成熟很多。

项目表达模板

我们评估过 MCP，最后把系统分成两层：项目内部的轻量工具先用统一 schema 和 tool executor 管理，跨团队共享、需要资源发现和统一认证的能力则逐步 MCP 化。这样做的原因是，小范围功能用 MCP 会增加不必要复杂度，但平台级能力如果不标准化，后续每接一个宿主都要重写适配层。我们的落地方式是平台内有一个统一 MCP client / gateway，负责租户隔离、权限代理、审计和 trace，把知识库检索、报表查询、工单系统等能力以 MCP server 形式暴露出来。模型本身不直接碰认证，所有调用都经过 runtime 和策略层。这样后续无论接聊天产品、办公助手还是代码 agent，都可以共享同一套能力资产。

这个模板会给人一种你理解渐进式引入协议的感觉，而不是为了追热点而追热点。

小结

把 MCP 这道题压缩成四句话：

1. MCP 是 AI 应用连接外部系统的开放协议，不只是统一函数调用；
2. 它通过 host / client / server、tools / resources / prompts、lifecycle / auth / transport 建立标准化能力接入层；
3. 它和 Function Calling、A2A 不是互斥关系，而是位于不同层次；
4. 它最适合能力复用、生态互通和企业治理诉求强的场景。

如果你能把这四句话展开成概念—边界—工程—落地模式—trade-off 的完整回答，这一题已经足够在校招中脱颖而出。

延伸阅读与更新依据（官方为主）

- [MCP Introduction](#)
- [MCP Specification 2025-11-25](#)
- [MCP Lifecycle](#)
- [MCP Transports](#)
- [MCP Authorization](#)
- [MCP Server Tools](#)

- [MCP Roadmap](#)
- [MCP Apps](#)

03 A2A 与 Agent 互操作（前沿了解）

这篇主要考察什么

A2A (Agent2Agent Protocol) 是 2025—2026 年多 Agent 方向最值得关注的开放标准之一。面试里一旦问到 A2A，通常不是为了让你背术语，而是要看你有没有真正思考过下面几个问题：

1. 当一个 Agent 不是直接调用工具，而是把任务委托给另一个 Agent 时，交互对象为什么已经不是函数了？
2. 跨 Agent 协作时，为什么不能只靠一个普通 RPC 接口？
3. 任务状态、上下文、Artifact、异步执行、用户补充输入这些东西，应该如何在协议层表达？
4. A2A 和 MCP 为什么是互补关系，而不是替代关系？
5. 一个多 Agent 系统如何在开放生态下做发现、协作、治理和审计？

所以，A2A 这道题考察的不是你懂不懂多 Agent 这个热点，而是你是否理解 **Agent 作为一等公民** 时，协议设计会发生什么变化。（可以去看下linear之类的产品）

面试一句话版本：

A2A 本质上是一个面向 Agent 协作的开放协议。它假设交互对端不是一个简单工具，而是一个有身份、有能力说明、有任务状态、有异步生命周期、能产出 artifact、也可能要求补充输入的完整 Agent。因此 A2A 关心的不只是调用一个函数，而是把一个任务安全、可追踪地委托给另一个 Agent 并持续跟踪其进度。

为什么 另一个 Agent 不能简单当成一个 Tool

这是理解 A2A 的起点。

如果你把另一个 Agent 当成一个普通工具，会立刻遇到几个问题：

1. 它不一定同步返回结果

一个工具调用往往是：

- 给参数；
- 执行；
- 回结果。

但一个 Agent 可能：

- 先接收任务；
- 进入 working 状态；
- 中途产出部分 artifact；
- 要求用户补充资料；
- 等待授权；
- 在很久之后才完成。

这明显比函数调用复杂得多。

2. 它可能有自己的内部推理和工具系统

远端 Agent 不是一个黑盒函数，而是一个自治实体。

它内部可能：

- 自己做 planning；
- 自己调工具；
- 自己维护短期状态；
- 自己产出多版本 artifact；
- 自己决定何时请求补充信息。

这意味着调用方不应该强行要求它暴露所有内部细节，而应该通过协议只约定**协作边界**。

3. 它需要被发现和识别

如果是一个工具，你通常只要知道函数名和参数。

但如果是一个 Agent，你还需要知道：

- 它是谁；
- 它擅长什么；
- 接收什么类型的任务；
- 用什么输入输出格式；
- 是否支持 streaming；
- 是否支持 push notification；
- 认证要求是什么；
- 能否处理多轮继续任务。

这就是为什么 A2A 需要 Agent Card 这类能力说明。

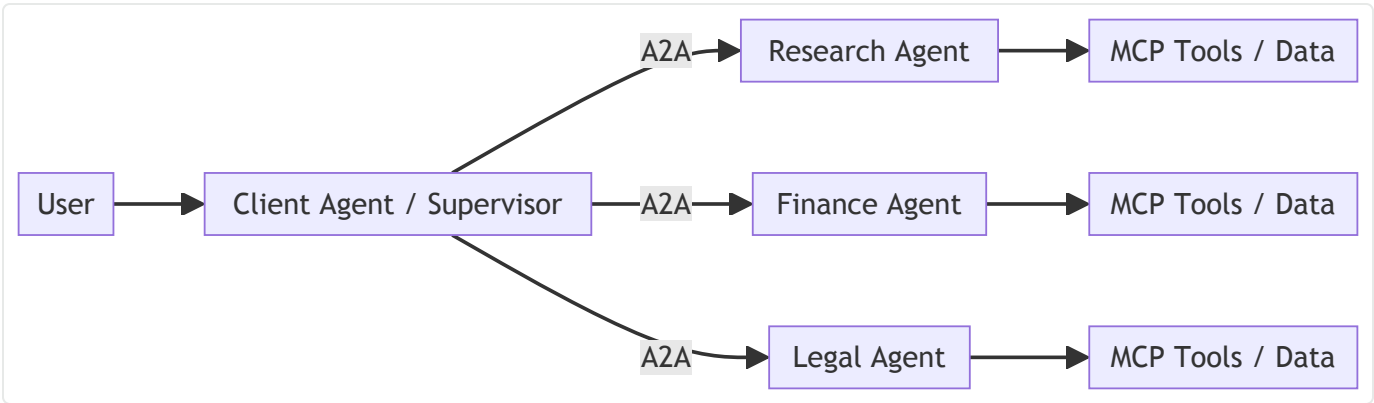
4. 它会产出 Artifact 而不是只返回一个值

一个 Agent 的结果可能不是简单 JSON，而可能是：

- 报告草稿；
- 代码补丁；
- 审核意见；
- 合同批注；
- 多版本文档；
- 图表文件；
- 需要后续继续修改的工作产物。

这类产物通常需要版本和持续更新语义，不适合被压扁成一个函数返回值。

一图看懂 A2A 的定位



这张图表达的核心是：

- MCP 更适合 Agent 连接工具和资源；
- A2A 更适合 Agent 之间分工协作；
- 一个完整系统里，这两者经常同时存在：Agent 内部通过 MCP 获得外部能力，Agent 之间通过 A2A 协调任务。

A2A 的核心对象有哪些

这部分是面试里的基本盘，建议用发现—发消息—跟踪任务—处理产物的顺序讲。

1. Agent Card: Agent 的能力名片

Agent Card 可以理解成远端 Agent 的公开说明书，它告诉调用方：

- 这个 Agent 的名称、描述、版本；
- 它擅长什么任务；
- 支持什么输入输出；
- 是否支持 streaming；
- 是否支持 push notifications；
- 认证和连接信息；
- 可能还有更详细的扩展元数据。

面试里你可以直接说：

Agent Card 的作用相当于让 Agent 先完成自我介绍，调用方再决定要不要把任务委托给它。

这个设计很重要，因为多 Agent 生态里，调用方不能靠硬编码记住所有远端能力，它需要一个标准化发现和描述机制。

2. Message: 一次交互输入

Message 表示一次发送给远端 Agent 的消息。

但要注意，A2A 里的 message 不只是聊文本，它可能包含不同类型的 Part，例如：

- 文本；
- 结构化数据；
- 文件引用；
- 表单数据；
- 其他协议定义的部件。

这说明 A2A 的交互模型比简单的文本 RPC 更丰富。

3. Task: 被委托出去的工作单元

Task 是 A2A 最重要的概念之一。

当 client agent 把一件事情交给 remote agent 时，本质上是在创建或推进一个 task。

Task 之所以重要，是因为它把下面这些东西统一挂在一起：

- 当前状态；
- 输入上下文；

- 相关 artifact;
- 错误信息;
- 后续继续交互所需的 taskId / contextId;
- 可能的进度与历史。

一句话记忆:

Message 更像一次通信, Task 更像一件持续存在的工作。

4. Artifact: 工作产物

Artifact 是远端 Agent 产出的结果对象。

它和 tool result 最大的不同在于:

- 可能有多个版本;
- 可能是中间产物, 不是最终结果;
- 可能在任务过程中持续被更新;
- 可能需要人类或另一个 Agent 再次编辑。

例如:

- 一个调研报告草稿;
- 一份合同修订版;
- 一段生成的代码;
- 一个分析表格;
- 一个带批注的文档。

在 A2A 里, Artifact 把 Agent 不只是回答一句话, 而是在协作中生产工作成果这件事表达出来了。

5. Context: 任务继续所依赖的上下文

多轮任务推进时, 只靠上一条消息文本是不够的。

你还需要知道:

- 这是哪个任务上下文;
- 当前轮是在继续哪个 task;
- 之前已经产出了什么 artifact;
- 是不是等待用户补充;
- 是不是某个审批还没完成。

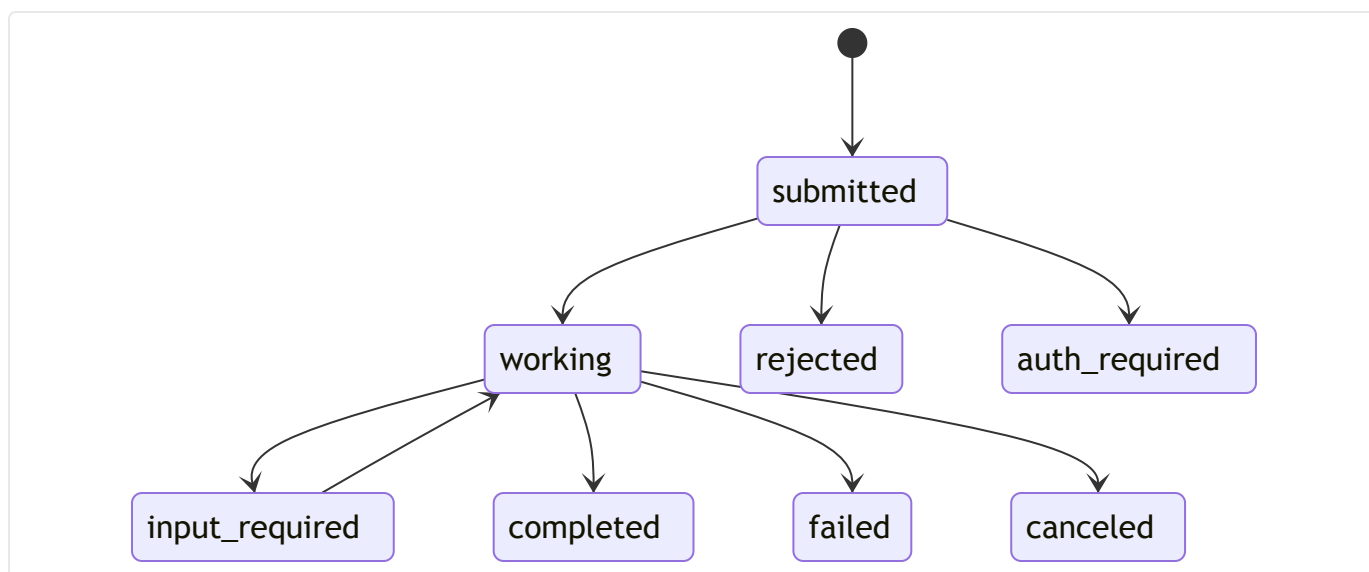
因此，A2A 里的 context 不是上下文窗口意义上的 prompt，而更接近协议层任务上下文标识。

A2A 的任务状态为什么是关键设计

这是面试里最能区分理解深度的地方。

A2A 明确把 task state 提上来了，因为 Agent 任务天然长生命周期的。

一个典型状态流可以这样理解：



各状态怎么理解

- **submitted**: 任务已提交，远端 Agent 已经接收；
- **working**: 正在处理；
- **input-required**: 远端 Agent 不能继续，需要补充输入；
- **completed**: 任务完成，通常伴随最终 artifact；
- **failed**: 任务失败；
- **canceled**: 任务被取消；
- **rejected**: 任务被拒绝，通常表示能力不适配或策略不允许；
- **auth-required**: 需要认证或额外授权；
- **unknown**: 调用方无法准确判断的保底状态。

为什么这很重要？

因为一旦你把远端 Agent 看成完整工作单元，系统就必须显式区分：

- 它是还在跑；
- 还是缺信息；

- 还是需要认证；
- 还是彻底失败；
- 还是可以继续推进。

如果没有这些状态，你就没法做真正的多 Agent 协作，只能做发一条消息碰碰运气。

A2A 的标准协作流程

可以在面试里按下面这条链讲

第一步：发现远端 Agent

通过 Agent Card 或目录服务，知道有哪些 Agent 可用、各自擅长什么。

第二步：发送任务消息

Client agent 把用户需求或子任务封装成 message / task 请求发给远端 agent。

第三步：远端进入任务状态机

远端可能立刻完成，也可能进入 working 状态持续执行。

如果是长任务，它可能通过 polling、streaming 或 push notification 持续更新状态。

第四步：产出 artifact 或请求补充输入

如果任务执行中遇到信息不足、权限不足、认证不足、边界不明确，远端 agent 可以进入 input-required 或 auth-required，而不是瞎猜着继续干。

第五步：继续任务或结束任务

Client agent 可以基于 taskId / contextId 继续推进同一任务，而不是从头新开一轮。

这条流程的本质，是把跨 Agent 协作从一次性请求，升级为有状态任务协作。

A2A 为什么需要 Streaming、Polling、Webhook / Push Notification

真实系统里，跨 Agent 任务经常很长。
如果只支持同步请求-响应，会有三个问题：

1. 用户体验差

你不可能让用户一直盯着一个同步请求等 2 分钟、5 分钟甚至更久。

2. 中间结果无法暴露

很多复杂任务中间就已经有价值，例如：

- 已完成第一轮信息收集；
- 已生成报告初稿；
- 已发现风险点；
- 已完成 70% 子任务。

如果没有 streaming 或异步状态更新，调用方无法利用这些中间进展。

3. 网络与前端连接不可靠

长任务执行期间，前端页面关闭、网络波动、连接断开都很常见。

协议必须支持：

- 重新订阅；
- 查询 task 状态；
- 接收推送；
- 从中断处恢复。

所以 A2A 对 streaming 和 async 的关注，不是高级特性，而是它把 Agent 当作长期协作对象的自然结果。

A2A v1.0 值得知道的更新点

如果想体现跟得上最新生态，可以在面试里提到一些 v1.0 之后的重要点。这里不要求你背协议字段，但应该知道方向：

1. v1.0 走向稳定与生产可用

这意味着 A2A 不再只是概念验证，而是开始强调：

- 稳定语义；
- 版本迁移；
- 多语言 SDK；
- 企业部署需求。

2. 交互协议有过迁移与升级

例如 Part、流式事件解析、Agent Card 解析、版本头、分页等都做了更明确的规范化。
你不一定需要背每个字段，但要知道：
A2A 正在从能跑通走向行为更可预测、更适合生产。

3. SDK 生态更完整

Python、Go、Java、JavaScript、.NET 等多语言 SDK 的稳定度提升，说明协议正在从少数示范项目进入更广泛采用阶段。
这个点在面试中很有用，因为它说明你知道一个协议是否能真正落地，不只看概念，还看 SDK 和生态。

A2A 和 MCP 怎么讲最清楚

推荐用下面这个对照说法：

维度	MCP	A2A
主要对象	工具、资源、提示	完整 Agent
关注点	接入外部能力	委托和协作任务
状态模型	相对轻	明确 task / artifact / async state
典型场景	检索、文件、数据库、业务系统连接	专家 Agent 协作、跨团队智能体协作
关系	Agent-to-tool / app-to-context	Agent-to-agent

你甚至可以直接说：
MCP 像 USB-C，A2A 像互联网协议。
前者让 Agent 接能力，后者让 Agent 彼此通信。

A2A 和普通微服务调用有什么差别

这是非常好的深入题。

普通微服务调用通常假设：

- 服务边界明确；
- 入参出参相对固定；
- 调用方知道对方 API；
- 返回值是业务响应；
- 不太强调继续任务的多轮语义。

而 A2A 假设：

- 对端是自治 Agent，不一定暴露内部细节；
- 能力往往以自然语言任务和 artifact 协作为主；
- 需要发现机制和能力描述；
- 需要状态化任务推进；
- 需要 input-required、auth-required 这类协作状态；
- 需要更强的异步与 streaming 语义。

所以 A2A 不是替代微服务，而是给智能代理之间的协作定义一个更贴合的语义层。

企业级 A2A 落地要注意什么

1. 身份与信任

跨组织或跨系统协作时，远端 Agent 不能只是一个 URL。

你需要知道：

- 它是谁；
- 其 Agent Card 是否可信；
- 是否经过签名验证；
- 是否在可信目录中；
- 该不该被授权访问某类任务。

2. 数据最小化

不要把本地全部上下文都丢给远端 Agent。

应该只发送完成当前子任务所需的最小数据集，并明确敏感字段脱敏或省略策略。

3. 全链路追踪

A2A 场景里，一个用户请求可能穿过多个 Agent。

因此必须有：

- correlation ID；
- taskId / contextId；
- trace / span；
- 请求者与被请求者身份信息；
- 明确的日志和审计记录。

4. 多租户与策略

远端 Agent 可能是多租户服务。

你必须确认：

- 是否按租户隔离任务；
- artifact 是否可能串租户；
- 调用额度和频率如何控制；
- 是否支持策略拒绝和审批。

5. 失败与补偿

如果远端 Agent 已经做了部分动作，又在后续失败，你要能知道：

- 失败发生在哪一阶段；
- 是否已产出部分 artifact；
- 是否需要补偿；
- 是否需要重新委托或人工接管。

常见使用模式

模式一：Supervisor + Expert Agents

主 Agent 负责理解用户问题和任务分解，把子任务交给不同专家 Agent，例如：

- 调研 Agent；
- 法务 Agent；
- 财务 Agent；
- 数据分析 Agent。

这是最典型的 A2A 使用方式。

模式二：跨组织能力协作

本公司 Agent 不直接调用外部企业内部系统，而是把任务交给合作方的 Agent，由对方在自己的安全边界内完成工作并返回 artifact。

这个模式很符合 A2A 的保留内部逻辑不暴露的理念。

模式三：人机混合协作

某个远端 Agent 在执行过程中进入 input-required，请求用户补充信息；用户补充后，再由本地 client agent 继续推进任务。

这类多轮、多主体协作，正是 A2A 比普通 API 更自然的地方。

常见误区

误区 1：A2A 就是把 HTTP API 换个名字

错误。

A2A 强调的是 Agent 的身份、发现、任务状态、artifact 和异步协作语义，而不是简单的接口封装。

误区 2：A2A 可以替代 MCP

错误。

MCP 更适合连接工具与上下文，A2A 更适合连接 Agent 与 Agent。它们经常同时使用。

误区 3：多 Agent 一定要用 A2A

不一定。

如果只是单项目内、单团队、少数子代理，而且完全受同一 runtime 控制，用框架内 handoff 或内部消

息协议也可以。A2A 的价值在于开放互操作，不是所有多 Agent 场景都必须上。

误区 4：远端 Agent 一定要暴露内部推理过程

不需要。

A2A 的一个重要价值就是允许 Agent 以黑盒但可协作的方式存在，只暴露协作边界和任务结果，而不是内部工具链。

误区 5：Artifact 只是附件

太浅。

Artifact 是多 Agent 协作里的工作产物抽象，它承载的是持续演化的结果，而不是简单文件上传。

高频面试题与回答模板

1. 为什么 A2A 不能简单用 Tool Calling 代替？

标准回答：Tool Calling 更适合短、同步、边界清晰的动作。A2A 面向的是完整 Agent 的协作，它需要任务状态、artifact、多轮继续、异步更新和能力发现，这些都不是简单函数调用能自然表达的。

2. A2A 和 MCP 的区别是什么？

标准回答：MCP 解决 agent-to-tool / app-to-context 的标准接入问题，A2A 解决 agent-to-agent 的标准协作问题。前者接能力，后者协同工作。

3. Agent Card 的作用是什么？

标准回答：它是 Agent 的能力名片，帮助调用方在调用前知道对方是谁、会什么、支持什么交互能力，从而进行发现、选择和策略判断。

4. 为什么 Task 比 Message 更重要？

标准回答：Message 是一次通信，Task 是一件持续存在的工作。跨 Agent 协作的关键不是发出一句话，而是把任务在多个状态之间推进并跟踪产物和进度。

5. 为什么 A2A 强调 Artifact？

标准回答：因为 Agent 的协作结果经常不是一句文本，而是报告、文档、代码、表格等工作产物。只有把 Artifact 抽象成一等公民，才能支持版本、持续更新和后续协作。

6. A2A 最大的工程挑战是什么？

标准回答：信任和治理。因为远端是自治 Agent，你需要处理身份、授权、数据最小化、全链路追踪、多租户和失败恢复，而不是只关注一个请求有没有返回 200。

7. 什么场景下没必要上 A2A？

标准回答：单团队、单 runtime、内部子代理 handoff 就能解决的问题，未必需要上 A2A。A2A 更适合开放生态、多组织协作和跨框架互操作。

项目表达模板

在我们的系统里，主 Agent 负责理解用户需求和任务编排，但有些子任务是由独立专家 Agent 完成的，比如法务审查和财务核验。我们没有把这些专家能力简单包装成普通工具，因为它们本身有较长生命周期，会产出报告、需要补充资料、可能等待认证或审批，因此更适合按 Agent 任务协作建模。我们用标准化的 Agent Card 做能力发现，用 taskId 跟踪任务状态，用 artifact 管理中间和最终产物，用 streaming / polling 反馈进度；而专家 Agent 自己内部再通过 MCP 去连接知识库和业务系统。这样做的好处是，每个 Agent 可以保持自治和安全边界，同时整个系统仍然可追踪、可恢复、可治理。

小结

把 A2A 这篇压缩成四句话：

1. A2A 解决的是 Agent 与 Agent 的协作，而不是简单函数调用；
2. 它把 Agent 看成有身份、能力说明、任务状态、artifact 和异步生命周期的完整实体；
3. 它与 MCP 互补：MCP 接能力，A2A 协作 Agent；
4. 它最适合多 Agent、跨团队、跨组织和开放生态的协作场景。

如果你能围绕这四句话展开到任务状态、artifact、发现机制、异步交互和企业治理层面，A2A 这一题就已经答得很有层次。

延伸阅读与更新依据（官方为主）

- [A2A Protocol Home](#)

- [A2A v1.0 Announcement](#)
- [A2A SDK](#)
- [A2A and MCP](#)
- [A2A Enterprise Features](#)
- [Google Blog: Interactions API and A2A](#)

04 Session-State 与 Context 生命周期

这篇主要考察什么

如果说 Function Calling 考的是模型如何表达动作，那么 Session-State 与 Context 生命周期考的就是系统如何记住过去，并决定下一步带什么进去。

这道题之所以高频，是因为几乎所有 Agent 产品一旦进入真实使用，都会遇到下面的问题：

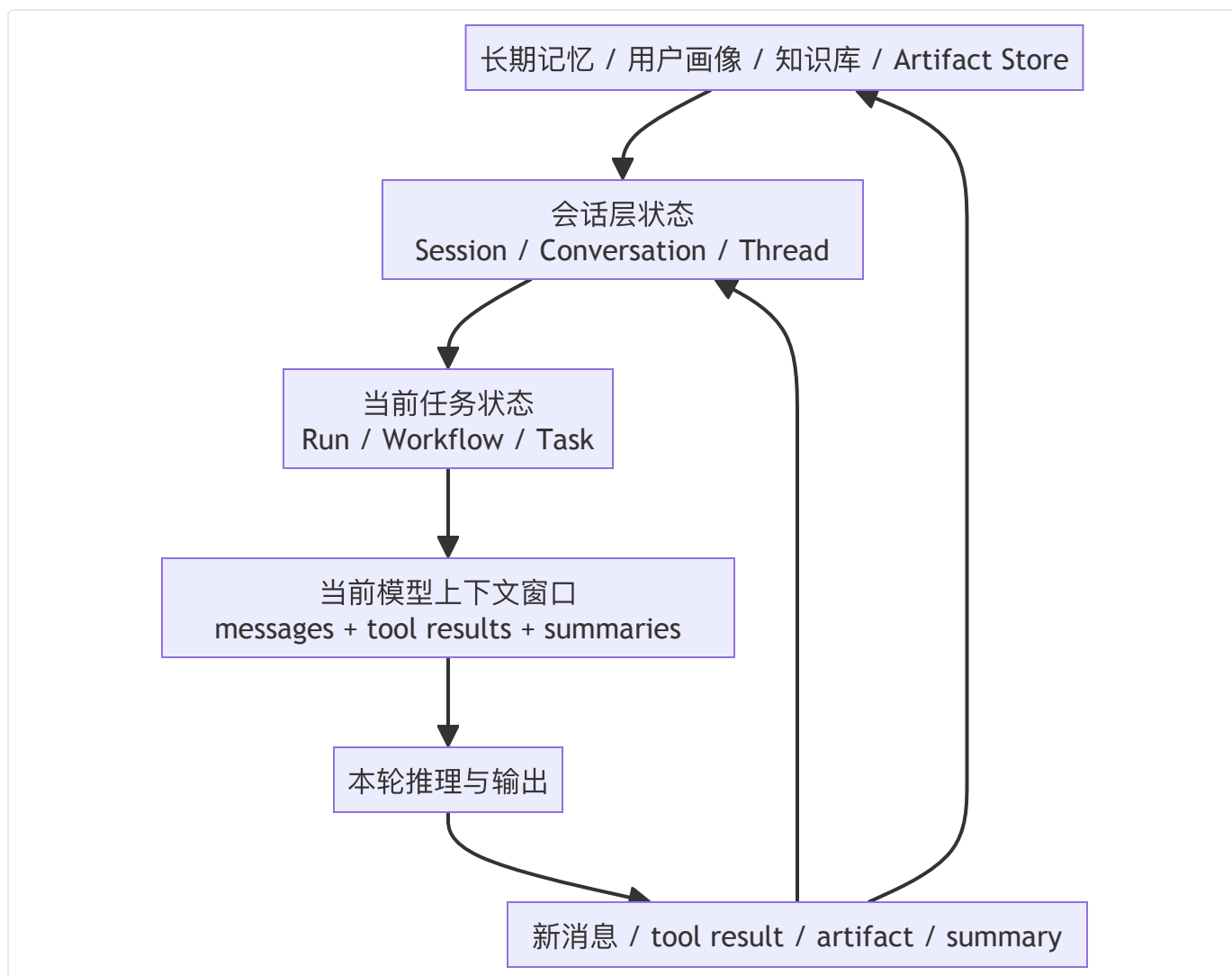
1. 用户说继续刚才那个任务，系统怎么知道刚才是什么？
2. 一个任务跑了 5 分钟，页面关掉之后，状态是丢了，还是还能继续？
3. 工具调用结果、用户消息、系统摘要、记忆、artifact、审批记录，哪些应该进上下文，哪些应该只放状态库？
4. 如果对话很长，怎么压缩？是自己做摘要，还是依赖平台的 compaction？
5. Session、Conversation、Thread、Memory、Context Window 到底分别指什么？
6. 当前轮该带哪些历史，带多少，谁来决定？

所以这篇文章的本质，是帮你把状态和上下文从一个模糊概念，拆成可设计、可治理的几个层次。

面试一句话版本：

Session-State 解决的是系统在多轮、多任务、多步骤里要记住什么；Context 生命周期解决的是这些信息何时进入模型上下文、何时被裁剪、何时被摘要、何时只留在外部状态存储中。一个成熟的 Agent 系统一定会把持久状态和当前窗口里的上下文分开设计，因为它们不是一回事。

一图看懂状态与上下文的层次



这张图非常关键。它说明：

- 不是所有被记住的信息都会进入模型上下文窗口；
- 不是所有进入窗口的信息都应该永久持久化；
- 你必须同时设计外部状态存储和当前轮上下文组装。

先把最容易混的几个词拆开

1. Context Window

Context Window 是模型当前一次调用能看到的输入窗口。

它只和本次模型调用有关，不等于你的全部会话历史，更不等于长期记忆。

一句话记忆：

Context Window 是当前看见什么。

2. Session

Session 通常表示某个连续交互会话的状态容器。

它强调的是：多次 run / turn 之间如何自动保留历史，常见于 SDK 层或应用层会话记忆。

一句话记忆：

Session 是这一串交互怎么串起来。

3. Conversation

Conversation 更偏平台或服务端管理的一段对话对象。

它通常是一个有 durable identifier 的会话实体，能把消息、工具调用和其他上下文项都组织起来。

一句话记忆：

Conversation 是平台侧保存的一段对话实体。

4. Thread

Thread 常见于工作流或图执行框架中，表示某条执行脉络的持久状态线。

例如在图工作流里，同一个 thread 会不断积累状态 checkpoint。

一句话记忆：

Thread 是某条执行脉络的持久轨迹。

5. Memory

Memory 是比 session 更泛的概念。

它可以包括：

- 短期记忆：本会话里刚发生的事；
- 长期记忆：跨会话的用户偏好、画像、稳定事实；
- 工作记忆：当前任务的中间结论；
- 总结记忆：把长历史压缩后的摘要。

一句话记忆：

Memory 是系统决定留下来的可复用信息。

6. State

State 是更工程化的词。

它不只包括聊天历史，还可能包括：

- 当前任务阶段；
- 审批状态；
- tool call 历史；
- artifact 指针；
- retry 计数；
- 幂等键；
- 上次中断位置；
- 权限上下文。

一句话记忆：

State 是让系统能继续往下执行所需的一切结构化事实。

为什么把聊天记录存数据库远远不够

这是很多校招生会掉进去的坑。

你说我们把消息存 MySQL 就行，这当然是状态管理的一部分，但远远不够，因为真正的问题不是有没有存，而是：

1. 谁来组织它？是按用户、按会话、按任务还是按 thread？
2. 哪些要进模型？历史消息全部塞进去肯定不行。
3. 哪些要摘要？太长会爆窗口。
4. 哪些不能进模型？比如部分日志、密钥、内部错误堆栈。
5. 哪些要跨会话记住？比如用户偏好。
6. 系统重启后如何恢复？只存 message 不一定足够。
7. 工具结果怎么处理？全量原文往往又长又脏。
8. 多租户怎么隔离？不同用户和不同团队的状态不能混。

所以，状态管理一定要回答两个问题：

- 持久层存什么；
- 模型调用时取什么。

当前主流的几种状态管理思路

思路一：客户端 / 应用侧自己维护历史

最传统的做法是：应用自己存消息历史，每一轮调用时把需要的历史重新拼进去。

优点：

- 可控；
- 容易和业务库整合；
- 平台无绑定。

缺点：

- 自己要处理裁剪、摘要、去重、重放、恢复；
- 很容易把历史组装逻辑写乱；
- 长对话和长任务管理成本高。

这种方式在小系统里很常见，但随着功能变复杂，会越来越难维护。

思路二：服务端 / 平台管理 conversation state

一些平台支持通过 conversation 对象或 `previous_response_id` 之类的 continuation 机制，让服务端负责把前序上下文串起来。

优点：

- 调用简单；
- 平台能帮你保留上下文链；
- 和平台的 compaction、background、tool execution 等能力结合更自然。

缺点：

- 平台绑定更强；
- 如果你还自己重复维护一套历史，容易造成双份状态；
- 迁移到其他平台时要重新设计。

这类方案的优势很明显：**开发体验好**。但面试里要体现成熟度，最好补一句：

便利性与可迁移性要权衡，不能一边让平台管 conversation，一边又在应用层盲目重复拼接完整历史。

思路三：SDK Session 抽象

还有一些 SDK 提供 session 抽象，自动帮你在多轮 run 之间维护历史。

这一层通常更偏**客户端记忆容器**，可以接 SQLite、Redis、SQLAlchemy、Dapr 等后端。

它的好处是：

- 比自己手工 `.to_input_list()` 更稳；
- 可替换不同存储；
- 对应用开发者更友好。

但也要注意：如果 SDK 的 session 和记在服务端的 conversation continuation 机制同时启用，往往会冲突或语义重复。

所以一个很重要的面试点是：

不要把 client-side session 和记在 server-side conversation 的 continuation 机制叠在一起，除非你非常清楚各自责任边界。

OpenAI 这一侧，最新值得知道的状态管理能力

面试里不需要背 API 参数，但最好知道几个关键词和边界。

1. `previous_response_id`：以响应链串联上下文

这种方式本质上是告诉平台：

当前这次调用接在上一条响应之后。

好处是：

- 简化多轮状态续接；
- 不用每轮都手动发完整历史；
- 和平台的上下文管理机制耦合更紧。

你可以这样解释给面试官：

这更像告诉平台从哪条链继续，而不是我自己手工回放全部历史。

2. Conversation 对象：把会话变成可持久化的一等对象

更进一步的平台做法，是直接把 conversation 作为 durable entity 管起来。

这样 conversation 不只是上一条响应的指针，而是一个完整对话容器，里面可保存：

- 消息；
- 工具调用；
- 工具结果；
- 其他上下文项。

这对于长对话、异步任务和多次恢复非常重要。

3. Sessions: SDK 侧自动维护历史

在 SDK 层, session 的价值是减少手工拼接历史。

它通常会做两件事:

- 在本轮执行前取回该 session 的历史;
- 在本轮执行后把新产生的 items 存进去。

但这里要记住一个高频点:

SDK session 和 server-managed continuation 通常不是叠加使用的。

因为两者都在试图回答历史由谁来带, 如果同时开, 容易语义重叠甚至重复注入历史。

4. Compaction: 不是简单摘要, 而是对长上下文的结构化压缩

Compaction 很值得讲, 因为它体现你对长对话管理的理解不是停留在做个总结。

真正成熟的 compaction 不是随便生成一段摘要, 而是把:

- 已经不必逐 token 保留的历史;
- 仍需对后续推理保持影响的上下文;
- 某些隐含推理或中间信息;

压缩成更小但仍可续接的表示。

如果平台已经支持 server-side compaction, 就不要自己同时盲目手工删历史, 否则可能破坏 continuation 语义。

5. Prompt Caching: 状态管理之外的另一条降本路径

Prompt caching 不是会话状态, 但和上下文生命周期紧密相关。

原因是: 如果你每轮都改动前缀、改动工具定义、改动大段系统提示, 缓存命中率会下降。

因此一个成熟系统会同时考虑:

- 状态怎么存;
- 上下文怎么裁;
- 前缀怎么稳定, 以便缓存复用。

这也是为什么大量工具 schema 不要每轮乱变会影响成本和时延。

Gemini / Interactions 这一侧可以怎么理解

不同平台叫法不同，但核心问题相似。

例如一些平台会提供：

- `previous_interaction_id` 之类的方式表示接着上一轮；
- 原生状态管理；
- 后台执行；
- 将思考过程和最终响应分层管理。

你在面试里不需要背所有字段，但可以体现一个成熟认知：

不同平台的 API 形式不一样，但状态管理本质都在回答同一件事：系统是否把多轮交互当成一等对象，还是要求调用方每次自己重放全部历史。

有这个认知就够了。

LangGraph / 工作流框架里的 Thread、Checkpoint 和 Memory

当系统不只是聊天，而是要执行一个有节点、有状态转移的流程时，状态语义会更像图执行引擎。

1. Thread：一次执行脉络

在图工作流里，thread 类似某个会话 / 某个任务的状态线。

同一个 thread 上，会不断追加 checkpoint。

2. Checkpoint：每一步执行后的持久快照

Checkpoint 的意义在于：

- 失败后能恢复；
- 支持 human-in-the-loop 中断再继续；
- 支持 time travel / replay 调试；
- 支持多次 run 在同一 thread 上推进。

3. Short-term Memory 与 Long-term Memory

图框架里常常会把短期记忆放在线程状态里，而把长期记忆放到外部 store。

这和我们前面画的那张层次图是完全一致的：

- 线程状态里放当前执行必须知道的东西；
- 外部 store 放跨会话或跨线程可复用的东西。

这个区分很重要，否则你会把 thread 搞得越来越重，恢复和迁移都会变麻烦。

上下文生命周期：一条信息会经历什么阶段

这是整篇最核心的设计题。

你可以把任何一条信息生命过程概括成下面几个阶段：

阶段 1：产生（Generate）

信息可能来自：

- 用户输入；
- 模型输出；
- 工具结果；
- 审批动作；
- 子任务产物；
- 记忆召回；
- 系统生成摘要。

阶段 2：分类（Classify）

系统要判断它属于哪一类：

- 需要进当前上下文；
- 只需持久化；
- 需要脱敏后持久化；
- 只用于审计，不给模型看；
- 需要总结后再进入上下文；
- 需要进入长期记忆。

阶段 3：注入（Inject）

真正进入模型上下文的内容，应该按角色和优先级组织，例如：

- system / developer instructions;
- 当前用户消息;
- 必要历史摘要;
- 必要 tool result;
- 必要 artifact 摘要;
- 检索结果;
- 安全 / 权限提示。

阶段 4：压缩 (Compact / Summarize)

随着会话变长，不可能保留所有原文。

这时系统要判断：

- 哪些历史可只保留摘要;
- 哪些必须保留原文;
- 哪些只需保存在外部状态库，不再带入窗口。

阶段 5：归档 (Archive)

一些信息不再需要参与推理，但需要留作：

- 审计;
- 回放;
- 用户下载;
- 结果复盘;
- 训练样本;
- 故障分析。

这一步常被忽视，但在企业系统里非常重要。

一个成熟的上下文组装策略应该包含什么

1. 固定前缀层

包括：

- 系统指令;

- 产品策略；
- 安全规则；
- 稳定工具 / namespace 信息。

这一层应尽量稳定，有利于缓存。

2. 当前任务层

包括：

- 当前用户请求；
- 当前 workflow 状态；
- 当前待解决目标；
- 当前审批结论；
- 当前 task 的关键变量。

这一层是本轮最重要的。

3. 历史摘要层

不是完整历史，而是能够帮助模型继续任务的摘要，例如：

- 已经确认过哪些事实；
- 已完成哪些步骤；
- 用户偏好；
- 当前未决问题。

4. 外部召回层

包括：

- RAG 检索结果；
- 长期记忆召回；
- Artifact 摘要；
- 外部上下文资源。

5. 工具反馈层

最近一次或最近几次对本轮决策有价值的 tool result。
注意不是所有工具结果都要塞回去，尤其是长日志和大文本。

常见错误：把工具结果当聊天记录原封不动塞回去

这真的是工程上很常见的问题。

例如一次搜索工具返回 20 页长文本；一次 SQL 查询返回 500 行表格；一次网页抓取返回整页 HTML。

如果你把这些原样放进会话历史，会带来：

- 上下文爆炸；
- 模型注意力被噪声占据；
- 后续每轮成本上升；
- 隐私与安全风险扩大；
- 真正关键事实反而被淹没。

更好的做法是：

- 持久化完整原始结果到 artifact / object store；
- 为模型生成结构化摘要或关键字段；
- 在需要时再按需加载原文。

一句话记忆：

模型上下文里应放决策所需信息，不是所有你曾经拿到的信息。

人机中断、恢复与长任务状态

一旦涉及审批或长任务，状态设计会更复杂。

例如：

- 模型请求用户补充日期范围；
- 系统要求用户确认是否真的发给全部成员；
- 长任务跑到一半，用户离开页面；
- 任务在后台继续，完成后再通知用户。

这时你至少要保存：

- task / run ID；
- 当前阶段；

- 等待的输入类型；
- 已生成 artifact；
- 已执行过的副作用步骤；
- 可恢复位置；
- 相关审计信息。

如果只保存聊天文本，你根本无法可靠恢复执行。

多租户与隔离

面试里只要提到状态，就值得顺带提一句多租户。

因为状态系统一旦设计差，很容易出现最危险的问题：串上下文、串记忆、串工具权限。

必须隔离的对象至少包括：

- session / conversation ID 命名空间；
- state store；
- memory store；
- artifact store；
- tool visibility；
- cache key；
- trace 与日志检索范围。

一个很成熟的回答是：

状态隔离不只是数据库表里加 tenant_id，而是整个上下文装配链都要 tenant-aware。

常见误区

误区 1: Session 就等于 Memory

不对。

Session 更偏当前会话级历史管理；Memory 还可能包含跨会话长期记忆。

误区 2: Conversation 越长越好，说明记住更多

错误。

长历史不等于高质量上下文。真正好的系统会做裁剪、摘要和分层状态管理。

误区 3：摘要就是 Compaction

不完全对。

摘要只是压缩的一种方式；平台级 compaction 还可能保留更多结构化续接语义，不是随手总结几句就能替代。

误区 4：工具结果都应该回填进上下文

错误。

应该只回填对下一步决策有价值、且结构化过的结果。

误区 5：状态管理只是后端数据库问题

错误。

它同时是：

- 模型输入问题；
- 成本与时延问题；
- 安全与隐私问题；
- 可靠性与恢复问题；
- 平台耦合问题。

高频面试题与回答模板

1. Session、Conversation、Thread 有什么区别？

标准回答：Session 更偏应用或 SDK 层的会话记忆容器；Conversation 更偏平台侧持久化的对话实体；Thread 更偏工作流或图执行中的一条持久执行脉络。三者都和连续性有关，但语义层次不同。

2. 为什么要把持久状态和模型上下文分开？

标准回答：因为很多信息需要被系统记住，却不需要每轮都给模型看。把两者混在一起会导致上下文膨胀、成本上升、噪声变多和安全风险变大。

3. 什么情况下适合用平台的 server-managed conversation state？

标准回答：当你希望快速获得多轮续接、长任务和上下文管理能力，且能接受一定平台绑定时，这种方式很高效。但要避免同时再手工重复组装完整历史，防止双重状态。

4. 为什么 Compaction 很重要？

标准回答：因为真实对话和任务会变得很长，不能无限制保留原始历史。Compaction 的价值是保留后续推理需要的关键信息，同时减少 token 成本和噪声。

5. 为什么不能把所有 tool result 都放进下一轮？

标准回答：因为原始 tool result 常常冗长且噪声大，会拖垮上下文质量。更好的方式是把完整结果外置存储，只把结构化摘要或关键字段注入模型。

6. 长任务中断后恢复，最关键要保存什么？

标准回答：不仅是消息历史，还包括 task / run 标识、当前阶段、已完成步骤、已产生副作用、待补充输入、artifact 指针和可恢复位点。

7. 状态管理最大的工程风险是什么？

标准回答：串状态和错状态。包括多租户串上下文、重复注入历史、过度压缩导致遗忘关键事实、平台和应用层同时维护状态导致语义冲突。

项目表达模板

我们把状态管理拆成三层：第一层是外部持久状态，存 session、task、artifact、审批记录和完整工具结果；第二层是会话摘要和当前任务关键变量，用于跨轮续接；第三层才是本轮真正进入模型上下文的内容。这样做的原因是，不是所有被系统记住的信息都应该进上下文。对长会话我们会做摘要或 compaction，对大体积工具结果只存外部对象并给模型一份结构化摘要。对于 SDK session 和平台 conversation continuation，我们只选一种主导机制，避免双重管理历史。整个上下文装配过程都带 tenant 和 permission 约束，防止串租户和越权召回。

小结

把这篇压缩成四句话：

1. Session-State 解决的是系统要记住什么，Context 生命周期解决的是这些信息何时进窗口、何时

被压缩、何时只存外部；

2. Session、Conversation、Thread、Memory、State、Context Window 不是一个概念，必须分层；
3. 长会话的关键不是全保留，而是分层保存、按需注入、持续压缩；
4. 真正可靠的 Agent 系统，恢复执行依赖的从来不只是聊天记录，而是结构化状态。

如果你把这四句话讲顺，再辅以一个你自己的上下文装配策略，这一题已经会很有说服力。

延伸阅读与更新依据（官方为主）

- [OpenAI Conversation State](#)
- [OpenAI Compaction](#)
- [OpenAI Background Mode](#)
- [OpenAI Agents SDK](#)
- [OpenAI Sessions](#)
- [LangGraph Persistence](#)
- [Google Blog: Interactions API](#)
- [Claude Prompt Caching](#)

05 Workflow-Orchestration 与 Durable Execution

这篇主要考察什么

当面试官开始问 Workflow、Orchestration、Durable Execution，说明他已经不满足于你只会做一个会调工具的聊天机器人了。他想看的是：你是否能把一个 Agent 任务做成一个能跨步骤、跨时间、跨中断、可恢复、可审批、可观测的生产流程。

这道题通常会围绕下面几个问题展开：

1. Workflow 和 Agent 是什么关系？是替代还是组合？
2. 为什么复杂 Agent 任务不能只靠一个 while-loop + function call？
3. 什么叫 durable execution？它和失败后重试一下有什么本质区别？
4. LangGraph 这种图式 workflow 和 Temporal 这种 durable workflow 系统各自解决什么问题？
5. 什么场景用图编排就够，什么场景必须上真正的 durable workflow 引擎？
6. 如果系统崩了、worker 挂了、用户半路审批、任务跑了三天，你的流程还能不能接着走？

所以，这一题本质上考察的是 Agent 从交互程序走向业务流程系统时，你有没有系统工程视角。

在面试时你可以这么一句话表达：

Workflow Orchestration 解决的是多步骤任务如何按状态、依赖和策略被有序推进；Durable Execution 解决的是这些推进过程如何在失败、重启、中断和长时间等待后仍然可以可靠继续，而不是从头再来。前者强调流程控制，后者强调执行可靠性。真正生产可用的 Agent 往往两者都需要。

为什么 Agent 系统会自然走向工作流编排

一个单次问答 Agent 可以靠大提示词 + 一轮模型调用 + 几个工具完成，但只要任务复杂度上来，系统就会出现以下特征：

- 需要多步推理与多步执行；
- 步骤之间有显式依赖；
- 某些步骤要等外部结果；
- 某些步骤要等待人工审批；

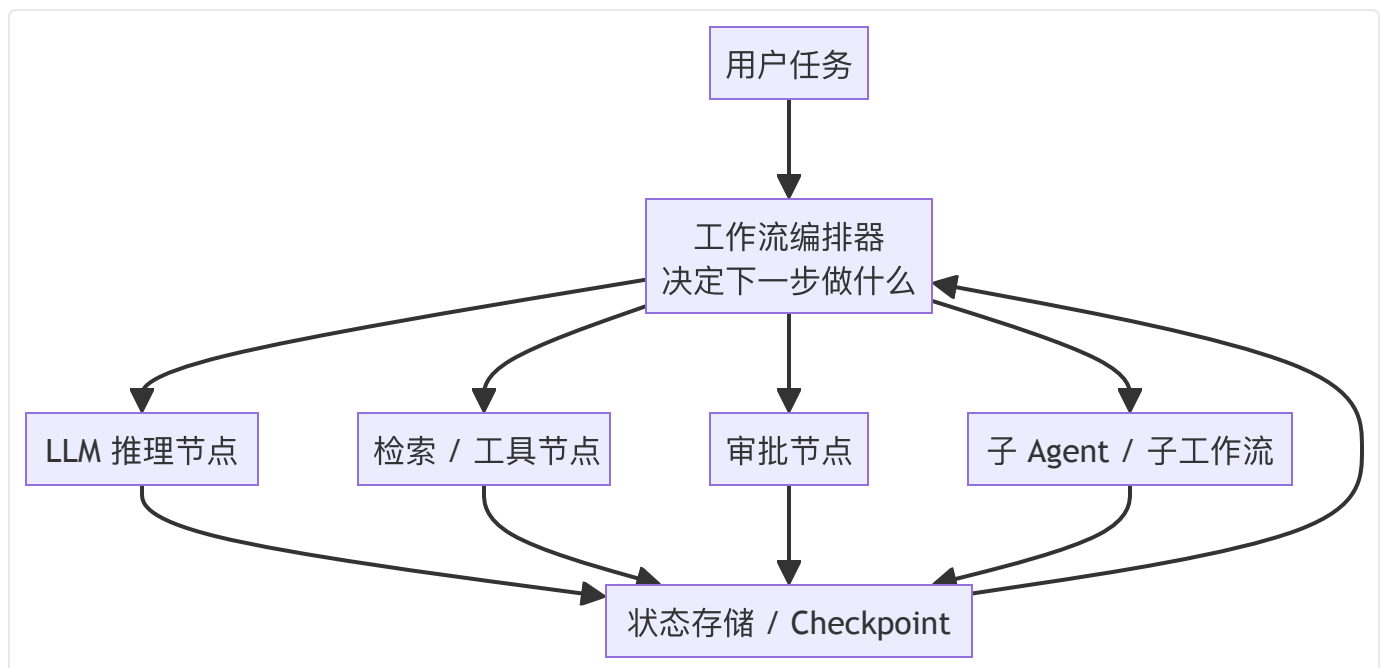
- 某些步骤失败可重试，某些失败必须停止；
- 某些任务可能持续几分钟、几小时、几天；
- 某些步骤有副作用，不能重复执行。

这时如果你还用一个纯粹循环调用模型直到结束的框架，就会出现：

- 状态难追踪；
- 中断难恢复；
- 审批难插入；
- 错误处理散落在各处；
- 日志难回放；
- 执行边界不清晰。

这就是 Workflow Orchestration 需要登场的原因。

一图看懂：为什么 Workflow 和 Durable Execution 是两层问题



这张图里的关键是：

编排器决定下一步是什么，状态存储保证下一步还能继续。

很多人只讲前者，不讲后者，就停留在流程图层面，而不是运行时层面。

Workflow、Agent、Orchestrator 三者关系怎么讲

1. Agent 是能根据上下文决定行动的执行体

Agent 具有一定自治性，可能会：

- 选择工具；
- 决定下一步；
- 进行推理；
- 在模糊问题上做探索。

2. Workflow 是步骤关系和状态转移的框架

Workflow 更强调：

- 步骤有哪些；
- 依赖关系是什么；
- 什么条件下从 A 到 B；
- 什么条件下进入审批或结束；
- 哪些步骤能并行；
- 哪些步骤失败后怎么处理。

3. Orchestrator 是把流程推进下去的控制层

Orchestrator 的职责包括：

- 读取当前状态；
- 决定执行哪个节点；
- 调用节点；
- 持久化结果；
- 处理重试与超时；
- 接入人工中断；
- 记录 trace。

所以，Agent 和 Workflow 不是非此即彼的关系。

一个非常成熟的回答是：

Agent 适合解决步骤内部的不确定性，Workflow 适合解决步骤之间的确定性。Orchestrator 把这两者组织起来。

这句话在面试里非常好用。

Workflow vs Agent：什么时候该让模型自己决定，什么时候该由流程提前写死

这是最经典的追问之一。

更适合 Workflow 的场景

- 流程有明确业务规则；
- 步骤顺序相对固定；
- 合规要求强；
- 需要可解释、可审计；
- 需要审批和回滚；
- 失败处理要非常稳定；
- 任务结果比探索空间更重要。

例如：

- 报销审批；
- 工单分派；
- 数据同步；
- 测试发布流程；
- 采购申请流程。

更适合 Agent 的场景

- 问题空间开放；
- 需要探索和试错；
- 工具选择不固定；
- 用户需求模糊；
- 更强调推理与规划能力。

例如：

- 开放式调研；
- 代码修复探索；
- 复杂问答；
- 任务分解与方案生成。

最常见的生产组合方式

最实际的答案不是二选一，而是：

- 用 Workflow 决定阶段；
- 在某些节点里调用 Agent 做局部自治推理；
- Agent 产出的结论再回到 Workflow 中继续执行。

例如：

合同审查流程里，整体阶段是固定的：上传 → 解析 → 初审 → 风险项汇总 → 人审 → 出具意见；但风险项提取这一步可以交给 Agent。

什么是 Durable Execution

这道题必须答得明确。

Durable Execution 不是失败了自动重试那么简单，它的核心在于：

系统能够在执行过程中把状态和进度以可恢复的方式保存下来，因此即使进程崩溃、机器重启、任务等待很久、人工中断后再继续，也不需要从头重新执行已经完成的步骤。

这里有两个关键词：

1. 已完成工作不会白做

假设一个 20 步任务跑到第 17 步，worker 挂了。

如果没有 durable execution，你可能只能：

- 从头跑；
- 或者靠人工判断从哪接。

如果有 durable execution，系统应该知道：

- 前 16 步已经成功；
- 当前卡在第 17 步；
- 第 17 步是否产生副作用；
- 是否需要重放还是继续；
- 下次恢复时从哪里接。

2. 状态恢复是系统级能力，不是业务代码临时补丁

很多同学会说：我们把中间结果存数据库，出错再读回来。

这当然是思路的一部分，但 durable execution 更强调运行时级别的恢复语义，而不是每个业务节点各自随手存点东西。

换句话说，Durable Execution 是把恢复能力提升为平台能力。

LangGraph：图式编排与持久执行

LangGraph 很适合用来解释 Workflow 和 Agent 的结合。

它的核心抽象是什么

可以浓缩成三个词：

- **State**：共享状态；
- **Node**：执行节点；
- **Edge**：状态转移与流程连接。

这种模型很适合 Agent 场景，因为：

- 可以把规划、检索、工具调用、审批、结束等都做成节点；
- 可以在节点之间显式表达条件和分支；
- 可以让部分节点由 LLM 自治决定；
- 可以持久化每一步的 checkpoint。

Persistence / Checkpoint 的价值

图工作流如果没有 checkpoint，很快就会出现两个问题：

- 失败后只能重跑；
- 人工中断后无法继续。

有了 checkpoint，每一步执行完都能把状态快照存下来。这样可以支持：

- fault tolerance；
- human-in-the-loop；
- time travel / replay 调试；
- 多轮任务延续；
- 会话线程级状态管理。

这是 LangGraph 与只有一个 agent loop 的根本区别之一。

LangGraph 更适合什么

- 需要把 Agent 拆成显式节点；
- 希望保留一定自治能力；
- 想快速搭建可中断、可恢复、可调试的图式流程；
- 任务主要围绕 LLM 推理、工具调用和状态转移；
- 对 workflow 历史和调试可视化有需求。

一句话记忆：

LangGraph 更像为 Agent 设计的状态图运行时。

Temporal：Durable Workflow 的经典代表

Temporal 很适合解释 durable execution 的工程本质。

Workflow Execution 是什么

Temporal 把 workflow execution 当成主执行单元。

它可以持续很久，可以有本地状态，可以接收 signals，可以调用 activities，可以在 worker 崩溃后通过事件历史恢复。

这里最重要的认知是：

- Workflow 不是普通进程；
- 它的进度由事件历史和运行时语义保护；
- Worker 掉了可以换另一个继续，不需要人工找回状态。

Activities 是什么

Activities 通常表示真正去做外部副作用或 IO 的工作，例如：

- 调支付接口；
- 查数据库；
- 发消息；
- 调第三方服务；
- 写文件；

- 调 shell 或远端执行器。

为什么 Temporal 强调 Activity 幂等？

因为 Activity 可能被重试。

只要有重试，就有重复执行风险；只要有重复执行风险，副作用就必须设计幂等。

Signals、Queries、Updates 为什么重要

复杂流程里，Workflow 往往不是启动后静静跑到结束，而是会在中途被外部交互影响，例如：

- 用户补充资料；
- 管理员审批通过；
- 运营强制取消；
- 前端查询当前状态；
- 外部事件到达后继续。

这些能力使得 Workflow 更像一个长期存活、可交互的业务流程实体。

LangGraph 和 Temporal 怎么选

这是非常高频的系统设计题。推荐你按下面思路回答：

更偏 LangGraph 的情况

- 你关心的是 Agent 节点编排；
- 核心步骤大多是 LLM 推理、工具调用和对话状态推进；
- 你想快速构建图式 Agent；
- 需要 checkpoint、HITL、状态图调试；
- 工作流生命周期相对中等，不一定跨很多天。

更偏 Temporal 的情况

- 你有很强的业务流程可靠性诉求；
- 任务可能持续很久；
- 需要强恢复语义；
- 需要严肃处理重试、超时、定时器、补偿、事件驱动；
- 需要把 workflow 当成生产级业务实体管理。

实际工程里非常常见的组合方式

外层用 Temporal 这种 durable workflow 管理业务流程阶段、重试、等待和补偿；
内层某些智能步骤用 LangGraph 或自定义 agent runtime 跑推理图。

例如：

- 外层工作流：接单 → 数据准备 → 调用代码修复 Agent → 等待人工 review → 发布；
- 内层 Agent：分析报错 → 检索仓库 → 生成补丁 → 运行测试 → 形成建议。

这样的分层往往比所有东西都交给一个 agent loop 稳定得多。

Durable Execution 和普通重试有什么区别

很多候选人会把两者混。

你可以这样区分：

普通重试

重点是：

某个动作失败了，再试一遍。

它通常关注：

- 失败类型；
- 重试次数；
- 退避时间；
- 幂等保护。

Durable Execution

重点是：

整个流程即使被打断，也能从正确位置继续。

它关注：

- 已完成步骤的持久记录；
- 当前状态与转移点；
- 外部事件恢复；
- 多天任务等待；

- worker 崩溃后的恢复；
- 人工中断后的继续执行。

一句话记忆：

重试解决一个动作失败怎么办，Durable Execution 解决整个过程断了怎么办。

Agent 场景里最需要 Durable Execution 的地方

1. 长时间工具或后台任务

例如：

- 大规模代码库扫描；
- 多网页长链调研；
- 大文件解析；
- 报表生成；
- 批量自动化操作。

2. 需要人工审批的步骤

例如：

- 删除数据；
- 发外部通知；
- 合同提交；
- 批量执行脚本；
- 进入生产环境。

这类流程会被人为暂停，没有 durable execution 就很容易丢状态。

3. 多 Agent 协作链

一个 Agent 把子任务委托给另一个 Agent，再等待返回。

如果调用关系拉长，没有可靠状态记录和恢复机制，系统很容易变成只在 demo 里能跑通。

4. 带副作用的多步流程

例如：

- 创建工单；
- 更新数据库；
- 发邮件；
- 再同步第三方系统。

中间任一步失败，都需要知道前面做到了哪里，以及后面是重试、补偿还是人工接管。

workflow 状态设计：校招生最容易忽略什么

1. 终止条件

流程什么时候结束？

是模型说我觉得结束了就结束，还是必须满足：

- 某个 artifact 已生成；
- 某个审批已通过；
- 某个状态已写回；
- 某个验收条件已满足？

没有显式终止条件，Agent 很容易在工具循环里打转。

2. 中断点

流程在哪些位置允许暂停？

- 等用户；
- 等审批；
- 等外部事件；
- 等定时器；
- 等远端 Agent 返回。

这些位置最好在设计时就是一等状态，而不是临时 if-else。

3. 幂等边界

哪些节点可以重复执行？

哪些节点一旦重试就要特殊保护？

比如生成摘要可以重复，发送通知就必须小心。

4. 错误分级

并不是所有错误都应导致整个 workflow fail。

你应该区分：

- 可重试错误；
- 不可重试错误；
- 需要人工介入的错误；
- 可降级继续的错误。

5. 观测与回放

没有 trace、checkpoint 和事件历史，出问题时你根本不知道：

- 是模型推理错；
- 工具挂了；
- 状态没存；
- 分支条件写错；
- 还是审批没回来。

常见误区

误区 1: Workflow 就是写一串 if-else

太浅。

真正的编排系统要考虑状态转移、并行、超时、中断、恢复和可观测。

误区 2: Durable Execution 就是把结果存数据库

不够。

存数据库是手段之一，Durable Execution 更强调运行时语义：怎么 checkpoint、怎么 replay、怎么恢复、怎么等待事件、怎么保证步骤边界清晰。

误区 3: Agent 越自主越好，不需要 Workflow

错误。

很多业务场景恰恰需要 Workflow 来限制 Agent 的自由度，使过程可审计、可解释、可治理。

误区 4: 上了 Workflow 就不需要 Agent

也不对。

Workflow 适合管理确定性流程，Agent 适合处理不确定性探索。很多系统最优解是二者组合。

误区 5: 只要能恢复，就不需要幂等

错误。

恢复不等于重复执行安全。只要某步可能被重放或重试，副作用节点仍然必须设计幂等。

高频面试题与回答模板

1. Workflow 和 Agent 是什么关系？

标准回答：Workflow 管步骤之间的确定性与状态转移，Agent 管步骤内部的不确定性与推理。生产系统里通常是 Workflow 包裹 Agent，而不是二选一。

2. 什么是 Durable Execution？

标准回答：它让流程在崩溃、重启、长时间等待或人工中断后，仍能从已知正确状态继续，而不是从头执行。重点是持久化进度和恢复语义，不只是失败重试。

3. LangGraph 和 Temporal 的区别是什么？

标准回答：LangGraph 更偏 Agent 状态图和节点编排，适合 LLM / tool / HITL 场景；Temporal 更偏生产级 durable workflow，适合长生命周期、强恢复语义、复杂重试和补偿控制。很多系统会组合使用。

4. 为什么复杂 Agent 任务不能只靠 agent loop？

标准回答：因为一旦任务跨多步、有审批、有长等待、有副作用和失败恢复需求，单纯 loop 很难显式管理状态、中断、重试和回放，系统会非常脆弱。

5. Durable Execution 和 Retry 的区别是什么？

标准回答：Retry 关注某个动作失败后再试一次，Durable Execution 关注整个过程断了以后还能否从正确位置继续。前者是局部容错，后者是流程级可靠性。

6. 什么时候一定要上真正的 workflow 引擎？

标准回答：当任务持续时间长、业务影响大、步骤多且带副作用、需要审批和恢复、需要严肃观测和补偿时，最好上专门的 workflow / durable execution 引擎，而不是靠临时脚本。

项目表达模板

我们的 Agent 不是一轮模型调用就结束，而是一个多阶段流程：任务解析、资料准备、模型推理、工具执行、人工确认和结果交付。早期我们用简单 agent loop，很快遇到状态难恢复、审批难插入、失败后容易重跑的问题。后来我们把系统改成工作流编排：每一步都是显式节点，节点之间通过共享状态流转；关键节点后做 checkpoint，长等待和审批用持久状态保存。对偏智能的步骤，我们在节点内部调用 Agent；对业务可靠性强的外层流程，则交给 durable workflow 运行时管理。这样即使 worker 挂掉，系统也能从已知步骤继续，而且全链路有 trace 和回放。

小结

把这篇压缩成四句话：

1. Workflow Orchestration 管流程推进，Durable Execution 管流程能否可靠继续；
2. Agent 适合步骤内部的不确定性，Workflow 适合步骤之间的确定性；
3. LangGraph 更像 Agent 状态图运行时，Temporal 更像生产级 durable workflow 系统；
4. 真正上线的 Agent 任务，一旦跨时间、跨步骤、跨审批、跨副作用，就离不开编排与持久恢复。

如果你能围绕这四句话讲清楚状态、checkpoint、replay、activity、审批和恢复，这一题在系统设计面里就会很扎实。

延伸阅读与更新依据（官方为主）

- [LangGraph Workflows and Agents](#)
- [LangGraph Persistence](#)
- [Temporal Workflow Execution](#)
- [Temporal Activity Definition](#)
- [Temporal Error Handling](#)

06 任务调度、重试、幂等、回滚（重要）

这篇主要考察什么

如果说前几篇更多在讲协议、状态和执行框架，那么这一篇就是面试里最能体现“工程味”的部分。因为一个 Agent 只要真的去碰外部世界，就一定会遇到：

- 工具调用超时；
- 网络抖动；
- 第三方接口偶发失败；
- 任务重复投递；
- 用户重复点击；
- Worker 崩溃重启；
- 模型重试导致重复副作用；
- 某一步已经成功，后一步失败；
- 外部系统已修改，内部状态还没写回。

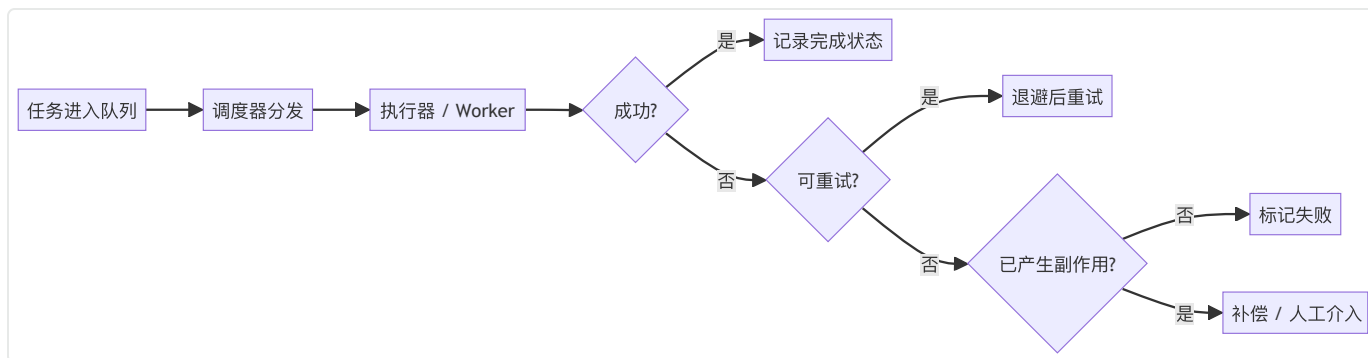
很多同学能把 Agent demo 做得很炫，但一问“重复发邮件怎么办”“已经扣款了后来失败怎么办”“消息重放怎么办”，立刻就露底了。

所以这篇的本质，是让你具备一套可靠性思维：**调度怎么做、失败怎么分级、什么时候重试、怎么防重复、出了事故怎么补偿。**

可以在面试的时候这样说：

任务调度解决的是“什么任务在什么时候、以什么优先级、由谁执行”；重试解决的是“临时失败时怎么再试”；幂等解决的是“再试了也不能产生重复副作用”；回滚或补偿解决的是“前面已经产生副作用、后面失败时如何把系统恢复到可接受状态”。这四件事必须配合设计，缺一不可。

一图看懂可靠性链路



这张图说明一个非常关键的点：

重试之前先问“可不可以重试”，不可重试之前先问“有没有副作用”。

任务调度到底在解决什么问题

很多人把调度理解成“丢进队列就行”，这太窄了。

调度至少回答下面这些问题：

1. 任务什么时候执行：立刻、延迟、定时、满足条件后；
2. 任务由谁执行：哪个 worker、哪个资源池、哪个地域、哪个租户隔离单元；
3. 任务优先级如何排序；
4. 同一类任务是否有限流、并发上限或配额；
5. 任务是否允许重复排队；
6. 一个任务失败后是立即重试、延迟重试还是进入死信；
7. 同一个业务实体上能否并发跑多个任务；
8. 长任务是否支持取消、恢复和续跑。

在 Agent 系统里，调度对象不只是“业务 job”，还可能包括：

- 模型调用任务；
- 工具调用任务；
- A2A 子任务；
- 后台研究任务；
- 审批等待后的恢复任务；
- 定时回访任务。

因此调度层实际上是 Runtime 的心脏之一。

先分清三类失败：不是所有失败都能重试

优秀回答一定先分失败类型，而不是上来就“指数退避”。

第一类：瞬时失败（Transient Failure）

例如：

- 网络抖动；
- 短暂超时；
- 下游服务 5xx；
- 限流窗口暂时关闭；
- 容器冷启动过慢；
- 远程连接暂时中断。

这类失败通常适合重试。

第二类：确定性失败（Deterministic Failure）

例如：

- 参数非法；
- 权限不足；
- 资源不存在；
- schema 校验失败；
- 业务规则不满足；
- 输入缺失。

这类失败重试通常没有意义，因为不改变输入，结果不会变。

第三类：危险失败（Side-effectful Failure）

例如：

- 邮件可能已经发出，但客户端没收到成功响应；
- 第三方支付可能已扣款，但本地事务失败；
- 工单可能已创建，但回执丢失；
- 脚本可能已执行一半，连接却断了。

这类失败最危险，因为你既不能简单当成功，也不能简单重试。它们需要幂等标识、状态核对、补偿策略或人工介入。

一句话记忆：

先问失败是不是暂时的，再问有没有副作用。

重试设计：不是“失败就多试几次”这么简单

1. 重试单位要明确

你在重试什么？

- 整个 workflow？
- 某个 activity？
- 某个 tool call？
- 某个模型调用？
- 某个数据库写入？

重试单位越大，误伤范围越大；

重试单位越小，控制越精细，但实现更复杂。

一般来说：

- 外层 workflow 少做整流程重试；
- 内层副作用步骤单独做受控重试；
- 模型调用和纯读操作更适合局部重试。

2. 退避策略要合理

最常见是指数退避加抖动（jitter），原因很简单：

- 避免立即重试把下游压垮；
- 避免大量任务同一时刻齐刷刷重试形成雪崩；
- 给限流和临时故障留恢复时间。

你在面试里不需要背公式，但要能说出：

- 不能无间隔重试；
- 不能所有任务固定一秒后一起重试；
- 不能无限重试不设上限。

3. 重试上限和截止时间

一个成熟系统会同时定义：

- 最大重试次数；
- 最大总耗时 / deadline；
- 单次执行超时；
- 任务整体 TTL。

因为“试到成功为止”在很多系统里是灾难：

- 消耗资源；
- 持续打爆下游；
- 让用户一直不知道结果；
- 把僵尸任务堆满队列。

4. 非重试错误要快速失败

例如 schema 不合法、权限不足、租户配置错误，这类错误应该尽快暴露出来，而不是进入漫长重试。

5. 重试必须和幂等绑定

如果一个动作可能被重试，那么它就必须被当成“可能执行多次”来设计。

这就是为什么“重试”和“幂等”在工程里必须一起讨论。

幂等：为什么它是生产系统的生命线

幂等的核心定义

同一个请求执行一次和执行多次，系统最终外部效果应保持一致，或者至少在可接受的受控范围内一致。

举例：

- 创建工单请求重复提交两次，最终只能有一个工单；
- 发优惠券请求重复执行两次，用户不能收到两张；
- 同一封通知消息重放，最终也只能发送一次；
- 删除一个已删除资源，再删一次不应造成额外伤害。

为什么 Agent 系统特别需要幂等

因为 Agent 天然更容易触发重复执行：

- 模型自己可能重复发同样的 tool call；
- 人工审批后可能再次恢复执行；
- 后台任务重试；
- 网络问题导致调用方不确定是否成功；
- A2A / 工作流引擎重放某个步骤；
- 前端用户重复点击。

如果副作用工具没有幂等保护，系统一定会出事，只是早晚问题。

幂等通常怎么做

1. 幂等键 (Idempotency Key)

最常见的方法。

为某个“应视为同一业务动作”的请求生成唯一键，例如：

- user_id + business_object_id + action_type
- workflow_run_id + step_id
- external_request_id
- client_generated_uuid

执行前先查是否已经成功处理过该幂等键：

- 如果处理过，直接返回之前结果；
- 如果没处理过，进入执行并记录处理状态。

2. 去重表 / 幂等表

对于有副作用的操作，常见做法是维护一张去重表，记录：

- 幂等键；
- 当前状态 (processing / success / failed) ；
- 结果摘要；
- 外部系统回执；
- 重试次数；
- 更新时间。

这张表非常有用，因为它可以帮助你处理“请求发出去了，但结果不确定”的尴尬场景。

3. 资源唯一约束

有些场景可以直接借助数据库唯一约束或业务唯一键，例如：

- 一个订单只能创建一个退款单；
- 一个任务只能生成一份最终报告；
- 一个用户某天只能领取一次福利。

这种方式简单有效，但要注意唯一约束只是底线，通常还需要配合业务状态表和错误处理。

4. 状态机防重入

某些流程可以通过显式状态机来防止重复进入，例如：

- 只有 `PENDING` 才能进入 `APPROVED` ；
- 已经 `COMPLETED` 的任务不允许再次执行完成逻辑；
- 已 `CANCELED` 的任务不允许再调起副作用步骤。

5. 外部系统对账 / 查询确认

对于“可能已经成功，但本地不知道”的场景，最稳妥的方式常常不是立刻重试，而是先去外部系统查询：

- 邮件是否已发送；
- 工单是否已存在；
- 交易是否已落单；
- 文件是否已上传。

很多事故都是因为把“不确定”误当成“失败”，然后盲目重试。

“精确一次”为什么往往是幻觉

这是一个很好的加分点。

在分布式系统里，真正的 `exactly-once` 常常代价极高，很多时候我们实现的是：

- 传输层可能 `at-least-once`；
- 处理层通过幂等和去重实现“外部效果看起来接近 `exactly-once`”。

也就是说，工程上更现实的目标不是“绝对不重复”，而是：

- 承认可能重复投递；
- 承认可能重试和重放；
- 用幂等和状态机保证最终副作用不重复或可接受。

你如果能在面试里主动说出这一点，会显得非常有工程意识。

回滚与补偿：为什么很多时候回不去，只能补

回滚（Rollback）

回滚更接近把系统恢复到之前状态。

它适合：

- 同一个事务边界内；
- 数据库可回滚；
- 副作用还没真正对外可见；
- 本地一致性较强的场景。

补偿（Compensation）

补偿是承认前面的副作用已经发生，然后执行一个反向或平衡动作。

例如：

- 已发通知后再发撤销通知；
- 已创建工单后再关闭工单；
- 已发券后再作废券；
- 已预留库存后再释放库存；
- 已创建外部资源后再发起删除。

在多步 Agent 流程里，真正“回滚到什么都没发生”往往做不到，所以更常见的是补偿。

一句话记忆：

数据库里回滚，外部世界里常常只能补偿。

Saga 思想为什么适合多步 Agent 流程

当一个流程由多个带副作用的步骤组成时，可以把每步拆成：

- 正向动作；
- 对应补偿动作。

例如一个“自动化会议安排”流程：

1. 创建候选日程 → 补偿：删除日程
2. 预定会议室 → 补偿：释放会议室
3. 发送邀请邮件 → 补偿：发送取消通知
4. 同步 CRM → 补偿：写入撤销记录

Agent 系统尤其适合用 Saga 思维，因为：

- 步骤经常跨系统；
- 没有一个全局数据库事务；
- 中间还会掺杂模型推理和人工审批；
- 失败处理不能只靠 try-catch。

Temporal / Durable Workflow 视角下的重试与幂等

这部分如果讲出来，说明你不是只停留在概念层。

1. Activity 应设计为幂等

因为 Activity 可能被重试。

如果 Activity 有副作用但不幂等，那么在网络抖动、worker 迁移或超时场景下，很容易重复执行。

2. Workflow Retry 和 Activity Retry 要区分

很多时候更推荐把重试放在具体 activity 或工具步骤上，而不是整个 workflow 级别无脑重试。

原因是整个 workflow 重试可能导致：

- 前面已经成功的副作用又被执行一次；
- 排查复杂度更高；
- 代价更大。

3. Signals / Updates 与幂等边界

当外部信号或人工操作可以改变流程时，也要注意：

- 重复信号是否会重复触发状态变化；
- 一个审批通过是否会被处理两次；
- 取消操作是否可重入。

也就是说，幂等不仅是“外部 API 调用”的问题，也是“流程状态转移”的问题。

Agent 场景下的典型可靠性案例

案例一：自动发邮件 Agent

问题：模型决定给用户发送一封总结邮件。

风险：

- 模型重复触发同一发送动作；
- 网络超时，客户端不知道邮件是否已发；
- 恢复执行时再次发送。

设计：

- 以 `task_id + recipient + template_version` 作为幂等键；
- 发送前检查幂等表；
- 发送成功后记录外部 `message_id`；
- 结果不确定时先向邮件服务查询状态；
- 如后续流程失败，发送一封补偿通知或撤回说明。

案例二：自动创建工单 Agent

问题：Agent 检测到故障并自动创建工单。

风险：

- 重试导致多个重复工单；
- 工单创建成功但本地状态未更新；
- 下游系统慢导致超时。

设计：

- 用故障事件唯一 ID 做幂等键；
- 创建前先根据唯一标识查现有工单；
- 本地记录 `CREATING / CREATED / UNKNOWN` 状态；

- UNKNOWN 状态优先对账查询，而非直接重试创建。

案例三：多步销售线索处理

流程：抽取线索 → 写 CRM → 通知销售 → 更新报表。

风险：

- CRM 写成功后通知失败；
- 报表更新失败但线索已落库；
- 用户重复导入同一批数据。

设计：

- 每一步都有明确状态；
- 对写 CRM 和通知分别设置幂等键；
- 通知失败只重试通知，不重试整条线索创建；
- 报表失败可补偿或异步修复；
- 批量导入使用批次号和明细去重。

常见误区

误区 1：只要有 retry 就可靠

错误。

如果没有幂等，retry 只会把错误放大。

误区 2：幂等就是“数据库里查一下”

太浅。

真实系统里还要处理处理中状态、未知结果状态、外部系统对账和状态机约束。

误区 3：所有失败都应该自动重试

错误。

参数错误、权限错误、配置错误通常越重试越浪费。

误区 4：回滚就是删掉数据

不对。

外部世界里很多动作是不可逆的，往往只能补偿。

误区 5：消息队列天然防重

错误。

很多队列系统只保证 at-least-once 或至少不保证绝对无重复，因此消费端仍然要做幂等。

高频面试题与回答模板

1. 为什么重试一定要和幂等一起设计？

标准回答：因为只要允许重试，就意味着同一动作可能执行多次。没有幂等保护，重复执行就会造成重复副作用，比如重复发邮件、重复扣费、重复创建工单。

2. 什么错误适合重试，什么错误不适合？

标准回答：网络抖动、临时超时、下游 5xx、短期限流更适合重试；参数非法、权限不足、资源不存在、业务规则不满足通常不适合自动重试。

3. 怎么处理“可能成功但我不确定”的请求？

标准回答：不要直接当失败重试。应该先用幂等键和外部系统查询做状态确认，必要时进入 UNKNOWN 状态，再决定是返回已有结果、继续等待还是补偿。

4. 什么是幂等键？怎么设计？

标准回答：幂等键是用来标识“同一个业务动作”的唯一标识，通常由业务实体 ID、动作类型、步骤 ID、调用方请求 ID 等组成。设计关键是让真正重复的动作共享同一个键，而不同动作不会撞键。

5. 为什么很多时候要补偿而不是回滚？

标准回答：因为外部副作用一旦发生，通常无法像数据库事务一样直接回退。更现实的做法是执行一个语义上的反向动作，把系统恢复到可接受状态。

6. Agent 系统里最容易忽略的可靠性问题是什么？

标准回答：模型重复触发副作用。很多人只盯着模型答得对不对，却忽略它可能在异常恢复、工具循环或审批恢复时重复执行同一动作。

项目表达模板

“我们的 Agent 会调用外部系统做写操作，所以我们在运行时里专门设计了可靠性层。所有副作用型工具都带幂等键，执行前先查去重表；失败时先分级，只有瞬时错误才进入指数退避重试。对结果不确定的情况，不直接重试，而是先对账查询外部系统状态。多步流程采用状态机和补偿动作设计，确保前面某一步已经成功时，后续失败不会把系统留在不一致状态。这样我们既避免了重复副作用，也让任务失败后可恢复、可审计、可人工接管。”

小结

把这篇压缩成四句话：

1. 调度决定任务什么时候、由谁执行；重试决定临时失败时怎么再试；
2. 幂等保证再试不会产生重复副作用，是一切自动重试的前提；
3. 多步外部副作用流程里，很多时候做不到真正回滚，只能做补偿；
4. 可靠性不是“写几个 retry”，而是调度、状态、幂等、补偿的一整套系统设计。

如果你能把这四句话展开成失败分级、幂等键、UNKNOWN 状态、Saga 补偿和 workflow 结合的完整答案，这一题会非常有工程说服力。

延伸阅读与更新依据（官方为主）

- [Temporal Activity Definition](#)
- [Temporal Error Handling](#)
- [Temporal Workflow Execution](#)
- [OpenAI Background Mode](#)

07 异步任务、长任务、流式执行（重要）

这篇主要考察什么

大模型应用的一个现实问题是：很多任务根本不适合用用户发请求，服务同步返回结果这一种模式来做。

例如：

- 深度调研任务可能要跑几分钟；
- 多网页抓取和整理可能涉及几十次工具调用；
- 代码 Agent 可能要不断 build、test、patch；
- 多 Agent 协作可能存在多个远端等待点；
- 文件解析、批处理、报表生成、批量操作更不可能在一个同步请求里优雅完成。

所以，当面试官问长任务怎么做、异步怎么设计、流式输出怎么配合后台执行，本质上是在看你是否理解：**Agent 不只是一个 HTTP handler，而是一个可能跨时间运行的交互系统。**

可以在面试里这么表述：

同步适合短、确定、结果马上可用的任务；异步适合长时间、可能中断、需要后台继续的任务；流式执行适合把中间进展持续反馈给用户。三者不是互斥关系：一个任务可以既是异步后台执行的，又对前端提供流式进度，还支持取消和恢复。

为什么 Agent 特别需要异步与流式

1. 模型和工具链本身耗时不可忽略

即使单次模型推理不算太慢，一旦叠加：

- 检索；
- 网页抓取；
- 文件读取；
- 多次 tool loop；
- rerank；
- 多 Agent 协作；

- 审批等待；

整体时长很容易超出用户可接受的同步等待时间。

2. 用户对正在做什么很敏感

如果前端只是转圈，用户会非常焦虑：

- 它是不是卡死了？
- 到底在查什么？
- 有没有拿到部分结果？
- 需要我补充信息吗？

流式状态和阶段性反馈可以显著改善体验。

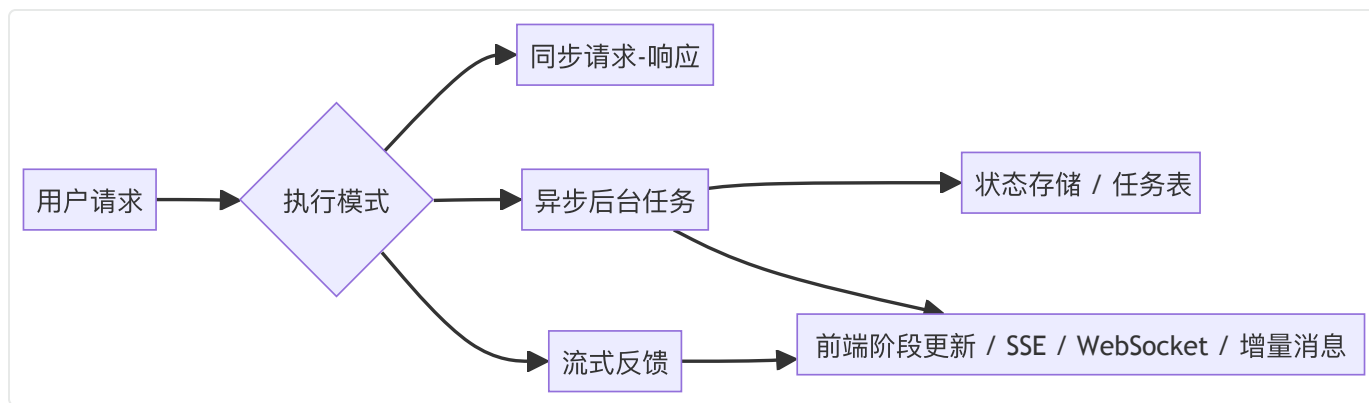
3. 长任务会遇到连接不可靠

网页关闭、网络抖动、移动端切后台都很常见。

如果任务只能绑在一次前端连接上，可靠性会很差。

所以必须把任务执行与前端连接分离开。

一图看懂三种执行形态



一句话解释：

同步是结果交付方式，异步是执行方式，流式是反馈方式。

三者可以组合，而不是只能三选一。

同步、异步、流式分别适合什么场景

1. 同步

适合：

- 简单问答；
- 少量工具查询；
- 结果可在几秒内稳定返回；
- 不涉及审批和长等待；
- 用户必须立即得到最终答案。

优点：

- 实现简单；
- 产品心智直观；
- 状态管理较轻。

缺点：

- 一旦任务变长，体验和可靠性都迅速变差；
- 不适合断点恢复；
- 难插入复杂等待。

2. 异步后台

适合：

- 长时间任务；
- 需要在用户离开页面后继续；
- 需要和队列、调度器、工作流结合；
- 可能依赖外部慢系统；
- 需要取消、恢复、回调通知。

优点：

- 执行与前端连接解耦；
- 更适合资源调度、超时、重试和恢复；
- 能承载真正的后台 Agent。

缺点：

- 需要任务状态存储；
- 用户如何得知进展要额外设计；

- 一致性和取消语义更复杂。

3. 流式反馈

适合：

- 任务过程本身对用户有价值；
- 需要逐步展示 reasoning phase、阶段进展或部分结果；
- 希望提升体感速度；
- 需要在任务很长时持续建立用户信任。

优点：

- 反馈快；
- 适合展示中间结果；
- 便于提示用户何时需要补充输入。

缺点：

- 前端和协议更复杂；
- 需要定义进度事件与断流恢复；
- 不是所有内容都适合实时暴露。

OpenAI 背景任务（Background Mode）可以怎么讲

这是当前很新的高频点。

你不需要背全部 API 细节，但建议知道以下几个要点。

1. Background Mode 解决什么问题

它把需要较长时间完成的模型任务从同步请求中拆出来，让任务在后台继续运行。

这非常适合：

- 深度调研；
- 长 reasoning；
- 多步工具调用；
- 需要几分钟的复杂任务。

2. 基本交互模型

典型模式是：

1. 发起一个后台响应任务；
2. 立即拿到任务标识或响应对象；
3. 后续通过轮询状态、流式恢复或 webhook 获取完成结果；
4. 必要时发起取消。

这个设计的关键价值是：

任务继续跑，但前端连接不需要一直悬挂。

3. Background 和 Streaming 可以组合

一个很值得你在面试里主动提的点是：后台执行不代表完全没有实时反馈。

它可以和 streaming 结合：

- 一边后台跑；
- 一边输出阶段性事件；
- 连接断了后还能从 cursor 继续取增量。

这体现了执行方式和反馈方式是两件事。

4. Cancel 要做成幂等操作

长任务一旦支持取消，就要考虑：

- 用户可能点两次取消；
- 前端和后端可能重复发取消；
- 任务已经完成后又收到取消。

因此取消动作本身也要设计成幂等且可安全落空。

5. Webhook 可以让任务完成通知更自然

对真正的后台任务来说，轮询虽然简单，但并不总是最优。

如果平台支持 webhook，任务完成、失败或需要人工介入时，可以主动通知应用侧。

这对于集成工作流和企业通知系统很有价值。

流式执行到底流什么

很多人一提 streaming，只想到模型 token 一点点吐字。
但在 Agent 系统里，更有价值的流式内容往往包括：

1. 文本增量

最直观的一种。

适合：

- 逐字输出答案；
- 展示草稿形成过程；
- 给用户体感上的即时反馈。

2. 阶段事件

例如：

- 已完成需求理解；
- 正在搜索网页；
- 已找到 12 个候选结果；
- 正在调用财务 Agent；
- 等待用户确认；
- 已进入最终生成阶段。

这种事件比纯文本 token 更有产品价值，因为它让用户知道系统当前在哪个阶段。

3. 工具事件

例如：

- 某个 tool call 发起；
- tool result 返回；
- 某个并行子任务完成；
- 某个远端 Agent 返回 artifact。

这类事件对调试和高级用户很有用，也适合作为 trace 的用户可见子集。

4. 部分结果 / Artifact 增量

例如：

- 已生成报告大纲；
- 已完成代码补丁初稿；
- 已汇总前 20 条调研结果；
- 已生成表格第一页。

这在长任务中很重要，因为用户经常不需要全做完才开始利用结果。

流式输出不等于暴露全部 Chain-of-Thought

这是一个需要小心说的点。

工程上，流式阶段更新和进度可见性非常重要，但不代表你要把模型的全部内部思维原样暴露给用户。更合理的做法是输出：

- 阶段描述；
- 已执行动作；
- 可理解的中间状态；
- 用户需要知道的下一步。

也就是说，流式是产品反馈机制，不等于把内部推理全部公开。

长任务的核心设计：执行与呈现分离

这是非常重要的系统设计原则。

执行层负责

- 真正跑模型与工具；
- 维护任务状态；
- 写 checkpoint / trace；
- 处理重试、取消、超时和恢复；
- 记录 artifact。

呈现层负责

- 把当前状态映射成用户可见进度；
- 展示阶段消息、部分结果、最终结果；

- 处理断线重连；
- 订阅 webhook / polling / stream；
- 给用户提供取消、继续、补充输入入口。

如果你把这两层混在一起，例如执行线程直接维护前端长连接并把所有状态都绑死在连接上，系统会很脆弱。

Polling、Streaming、Webhook 怎么选

1. Polling

适合：

- 实现简单优先；
- 前端或调用方暂时不方便接 webhook；
- 任务频率不高；
- 能接受秒级状态刷新。

优点：

- 简单稳定；
- 容易调试；
- 很适合作为基础兜底机制。

缺点：

- 有额外无效请求；
- 状态更新不够实时；
- 轮询间隔要权衡时延与成本。

2. Streaming

适合：

- 用户在线等待时；
- 中间进度很重要；
- 希望体感更丝滑；
- 前端能稳定处理增量事件。

优点：

- 延迟低；
- 用户体验好；
- 适合多阶段 Agent 可视化。

缺点：

- 连接管理复杂；
- 需要处理断流和恢复；
- 前后端事件协议要设计好。

3. Webhook / Push Notification

适合：

- 任务可能很久；
- 应用侧是服务对服务集成；
- 需要被动接收完成通知；
- 不希望靠频繁轮询。

优点：

- 更节省查询流量；
- 适合后台集成；
- 与工作流和事件总线结合自然。

缺点：

- 需要安全验证；
- 回调失败要重试；
- 复杂度高于简单 polling。

成熟系统通常是：

- 在线场景用 streaming；
- 离线场景用 webhook；
- 始终保留 polling 作为保底查询。

取消（Cancel）为什么比想象中更复杂

很多人以为 cancel 就是停掉任务。

真实系统里，取消至少要回答：

1. 当前任务是还没开始，还是正在执行，还是已经完成？
2. 如果正在执行一个长外部调用，能否真正中断？
3. 如果某一步已经产生副作用，取消后是否需要补偿？
4. 如果任务是多子任务并行，取消是全取消还是局部取消？
5. 取消请求重复到达时怎么处理？

所以一个成熟回答会说：

取消本质上是状态转移，不一定保证物理中止所有动作，但必须保证系统进入一致的 canceled / canceling 语义。

恢复 (Resume) 和断点续跑

异步长任务一旦存在，就必须考虑恢复。

恢复通常分三种：

1. 前端恢复观察

用户刷新页面后，继续查看任务当前状态和历史进度。

2. 执行恢复

Worker 崩溃或连接断开后，系统从 checkpoint 或持久状态继续执行任务。

3. 人工继续

任务进入 input-required，用户补充资料后，同一 task 再次继续。

你在面试里可以直接说：

查看状态和继续执行是两种不同恢复。前者恢复观察通道，后者恢复执行语义。

长任务里最容易被忽略的五个状态字段

这部分很适合拿来做工程化加分：

1. **phase**：当前阶段，如 planning / searching / waiting_input / finalizing；
2. **progress**：阶段进度，可选百分比或可读进度；
3. **last_event_id / cursor**：用于流式恢复；

4. `cancel_requested`: 取消意图标记, 避免硬杀一切;

5. `resume_token / task_id`: 恢复执行所需标识。

这些字段让长任务从只有一个 `pending / done` 变成真正可管理的任务实体。

A2A、MCP 与异步流式的关系

A2A

A2A 天然适合异步和流式, 因为远端 Agent 任务本来就可能持续很久, 并且有 `task state`、`artifact`、`push notification`、`resubscribe` 等语义。

所以如果面试问多 Agent 长任务怎么做, A2A 是很好的切入点。

MCP

MCP 更常和能力接入相关, 但在远程 HTTP 与事件流场景下, 也会涉及长连接、流式事件、工具返回 UI 或任务状态等问题。

你不必把它讲得和 A2A 一样复杂, 但要知道 MCP 也不是只能短平快同步调用。

常见误区

误区 1: 异步就是把请求丢进线程池

太浅。

真正的异步任务需要状态存储、取消、恢复、通知和失败处理, 而不是只把请求搬到后台线程。

误区 2: 流式只是逐字输出

不完整。

在 Agent 里, 流式更有价值的是阶段事件、工具事件和部分结果。

误区 3: 长任务可以一直靠前端连接等着

不现实。

前端连接会断, 用户会离开页面, 所以必须把执行和连接解耦。

误区 4：支持取消就等于能立即停机

不一定。

很多外部调用无法瞬时中断，取消更常见的是让系统进入一致的取消状态，并停止后续步骤。

误区 5：有 streaming 就不需要 webhook / polling

不对。

流式适合在线观察，webhook 适合后台通知，polling 适合保底查询。三者往往是互补的。

高频面试题与回答模板

1. 为什么 Agent 特别适合异步执行？

标准回答：因为 Agent 任务经常是多步、多工具、可能很长、可能等待外部系统或人工输入。同步请求-响应模式不适合承载这类长生命周期任务。

2. 流式执行和异步执行是什么关系？

标准回答：异步是执行方式，流式是反馈方式。一个任务可以在后台异步跑，同时把阶段性进度流式推给前端。

3. 为什么长任务不能只靠轮询？

标准回答：可以用轮询兜底，但仅靠轮询会带来时延和无效请求。在线场景通常配合 streaming，后台集成通常配合 webhook。

4. Cancel 最重要的设计点是什么？

标准回答：把取消看成状态转移而不是简单杀进程，并处理重复取消、部分已执行副作用和后续补偿逻辑。

5. Resume 需要保存哪些信息？

标准回答：至少要有 task_id / run_id、当前阶段、最近事件 cursor、已完成步骤、已生成 artifact、等待输入状态和取消标记。

6. Background Mode 的价值是什么？

标准回答：它把长时间模型任务从同步请求中解耦出来，让任务能后台继续执行，并通过 polling、streaming、webhook 等方式获取进度和结果，更适合长 reasoning 和复杂工具链任务。

项目表达模板

我们的 Agent 有一些任务会跑很久，比如深度调研和批量自动化处理，所以我们没有把所有执行都绑在一次前端请求里。系统会把长任务提交到后台运行时，生成 task 状态并持续写入 phase、progress、artifact 和 trace。用户在线时，我们通过流式事件返回阶段进展和部分结果；用户离线时，可以靠 webhook 通知或后续轮询恢复查看。取消操作只修改任务状态并停止后续步骤，已执行副作用则通过补偿逻辑处理。这样执行层和呈现层被彻底分开，系统既能长时间运行，也能保持良好交互体验。

小结

把这篇压缩成四句话：

1. 同步适合短任务，异步适合长任务，流式适合持续反馈；
2. Agent 的长任务本质上要求执行与连接分离，不能绑在一个前端请求上；
3. Polling、Streaming、Webhook 不是替代关系，而是不同场景下的组合；
4. 取消、恢复、进度、部分结果和任务状态，是长任务设计的核心。

如果你能围绕这四句话讲清后台执行、流式事件、取消恢复和任务状态字段，这一题就已经非常像真正做过系统的人在回答。

延伸阅读与更新依据（官方为主）

- [OpenAI Background Mode](#)
- [OpenAI Webhooks](#)
- [OpenAI Conversation State](#)
- [A2A Protocol](#)
- [A2A v1.0](#)
- [MCP Transports](#)

08 Agent Runtime 设计（重中之重）

这篇主要考察什么

很多面试官喜欢问一句非常开放的问题：“如果让你设计一个 Agent Runtime，你会怎么做？”
这道题之所以难，不在于概念陌生，而在于范围很大。候选人很容易答成两种极端：

- 一种是答得太浅，只说“一个模型 + 一些工具 + 一个数据库”；
- 另一种是堆一堆 buzzword，但没有主干，听不出系统是怎么跑起来的。

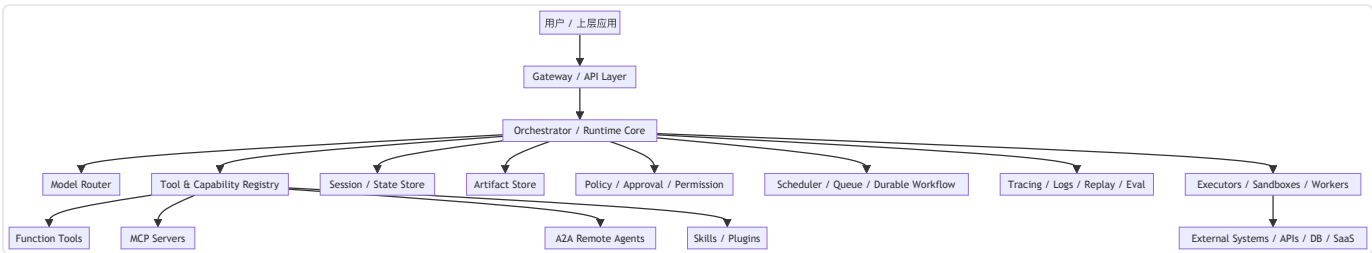
真正好的回答，应该能把 Runtime 讲成一套分层清晰的执行底座，它至少要回答：

1. 模型意图如何被解释和执行；
2. 工具、MCP、A2A、Skills 等外部能力如何注册和调用；
3. Session、状态、artifact、memory 如何管理；
4. 多步任务如何调度、重试、取消、恢复；
5. 权限、审批、审计和安全怎么插入；
6. Trace、日志、回放、评测怎么串起来；
7. 多租户和资源隔离怎么做（实习可以不用讲）。

所以，这道题的本质是：你能不能把 Agent 想成一个真正的“运行时系统”，而不是一个 prompt 脚本。
怎么回答：

Agent Runtime 是介于模型与外部世界之间的执行系统。它负责把模型的输出解释成受控动作，把工具、协议、状态和策略组织起来，并提供调度、恢复、安全、观测和治理能力。没有 Runtime，Agent 只是一个会说话的模式；有了 Runtime，Agent 才能成为一个可上线、可运营、可审计的软件系统。

一图看懂 Agent Runtime 的核心组成



这张图建议你在脑子里背住。

因为面试里一旦问 Runtime，你完全可以按图来讲：入口层、编排核心、能力注册层、状态层、执行层、策略层、观测层。

Agent Runtime 到底负责什么

一句最简单的定义是：

Runtime 把“模型能想”变成“系统能做”。

展开之后，至少有下面八类职责：

1. 输入规范化

把来自用户、API、上游 workflow、Webhook、外部事件的输入统一成 Runtime 可处理的任务对象。这一步通常会做：

- 身份识别；
- 会话绑定；
- 租户绑定；
- 参数校验；
- 速率限制；
- 路由准备。

2. 上下文装配

Runtime 负责决定：

- 当前模型调用要看到哪些历史；
- 哪些 memory / RAG 结果要注入；
- 哪些工具 / namespace 可见；
- 是否带上审批结果、系统策略、artifact 摘要。

这一步本质上是“将状态转成模型上下文”的过程。

3. 模型调用与路由

不是所有请求都发给同一个模型。

Runtime 可能根据任务类型选择：

- 推理更强的模型；
- 成本更低的模型；
- 支持 tool search / MCP 的模型；
- 支持长上下文的模型；
- 支持后台长任务的模型。

这就是 Model Router 的意义。

4. 动作解释与执行

模型可能输出：

- 最终答案；
- function call；
- structured output；
- 需要继续提问；
- handoff 到另一个 agent；
- 调 MCP server；
- 调 A2A remote agent；
- 触发 skill 或脚本。

Runtime 要把这些不同形式的输出解释成可执行动作，并在执行前做治理校验。

5. 状态与 Artifact 管理

Runtime 不只是“对话引擎”，还是状态机执行器。

它要管理：

- session / conversation；
- 当前任务状态；
- workflow 节点状态；
- tool call 历史；
- artifact 引用与版本；
- 审批状态；
- 取消 / 恢复状态；
- trace 关联 ID。

6. 调度与可靠性

Runtime 需要决定：

- 哪些任务同步执行；
- 哪些任务异步后台执行；
- 失败是否重试；
- 是否允许并行；
- 是否进入 durable workflow；
- 是否需要等待外部事件或人审。

7. 安全与策略控制

Runtime 是所有外部动作的闸门。

它必须在模型与副作用之间插入：

- 权限校验；
- 审批门；
- 敏感操作拦截；
- 数据脱敏；
- 多租户隔离；
- 速率限制与配额；
- 允许 / 禁止工具集合。

8. 可观测与回放

一旦系统出问题，Runtime 必须能回答：

- 当时模型看到了什么；
- 选了什么工具；
- 传了什么参数；
- 哪一步失败；
- 哪个状态转移出了问题；
- 是否产生副作用；
- 是否有人审批；
- 如何重放这条执行链路。

没有可观测，Runtime 就不可运营。

用“控制面 / 数据面”理解 Runtime，很容易说清楚

这是一个非常适合系统设计面试的表达方式。

控制面（Control Plane）

控制面更关注“决定怎么跑”，包括：

- 工具注册；
- Skill 管理；
- 模型路由策略；
- 权限和审批策略；
- 租户配置；
- 流量开关；
- 版本管理；
- 灰度发布；
- 评测与回滚。

数据面（Data Plane）

数据面更关注“实际在跑什么”，包括：

- 当前请求执行；
- 模型调用；
- 工具执行；
- 队列消费；
- 状态写入；
- artifact 生成；
- trace 上报；
- 事件流输出。

这样讲的好处是：

你可以把 Runtime 解释成一套长期演进的平台，而不仅是某个请求处理函数。

Runtime Core：编排器是中枢

如果必须只挑一个 Runtime 核心组件，那就是 Orchestrator / Runtime Core。

它通常承担下面这些职责：

1. 读取当前任务状态；
2. 组装输入上下文；
3. 调模型；
4. 解释模型输出；
5. 发起工具 / MCP / A2A / Skill 调用；
6. 写回结果与新状态；
7. 判断终止、继续、等待还是转人工；
8. 触发流式事件和 trace。

也就是说，它是一个“状态驱动的执行循环”。

但和最初级的 `while not done` 不同，它需要：

- 显式状态；
- 错误分级；
- 中断点；
- 最大步数；
- 幂等边界；
- 副作用控制；
- 多任务调度能力。

Capability Registry：工具、协议和能力资产如何统一

Runtime 里一个经常被忽略但很重要的模块，是能力注册中心。

它不只是存函数列表，而是统一管理多种能力来源：

1. Function Tools

最基础的一层。

包括名称、schema、权限要求、执行器绑定、幂等类型、风险等级等。

2. MCP Servers

对于远程或标准化能力接入，注册中心要知道：

- server 地址；
- 支持哪些 tools / resources / prompts；
- 认证方式；
- 可见租户和用户；
- 健康状态；
- 命名空间与发现方式。

3. A2A Remote Agents

如果 Runtime 允许委托给远端 Agent，还需要注册：

- Agent Card；
- 可接收任务类型；
- streaming / push notification 能力；
- 信任与认证要求；
- 调用策略与配额。

4. Skills / Plugins

Skill 更像高层能力封装，不一定直接对应一次外部调用。

Registry 需要知道：

- skill 元数据；
- 适用场景；
- 调用方式；
- 版本；
- 是否可在容器或 shell 中挂载；
- 评测结果。

统一能力注册的价值在于：

Runtime 才能按统一策略做发现、授权、路由、观测和版本控制。

Model Router：为什么它不只是“选一个模型”

一个成熟 Runtime 的 Model Router 通常会考虑：

1. 任务类型

- 普通问答；
- 复杂推理；
- 代码任务；
- 长上下文总结；
- 工具密集型任务；
- 多语言任务。

2. 预算与 SLA

- 这类请求预算多少；
- 延迟要求多高；
- 是否允许后台执行；
- 是否走高质量模型或低成本模型。

3. 工具能力兼容性

不是所有模型都支持相同的：

- tool calling；
- strict schema；
- tool search；
- MCP；
- 流式；
- background；
- 长上下文；
- 多模态输入。

4. 风险等级

高风险操作可能要求：

- 更稳定的模型；
- 更严格的 schema 模式；
- 更低的 hallucination 容忍度；

- 更明确的审批前置。

所以 Router 本质上是策略引擎的一部分，而不是简单 switch-case。

Session / State Store: Runtime 里最容易被低估的模块

前一篇已经讲了状态分层，这里从 Runtime 角度再看一遍。

State Store 至少应该支持：

- session / conversation 级历史；
- workflow / task 级状态；
- checkpoint；
- tool call 与结果索引；
- artifact 引用；
- 审批和中断状态；
- 恢复标记；
- 租户与用户绑定；
- TTL / 归档 / 删除策略。

一个成熟 Runtime 不会把所有状态都塞进一张 messages 表，而会根据语义分层存储。

Artifact Store: 为什么它要单独拿出来

很多 Agent 系统的结果不是一句文本，而是：

- 报告；
- 表格；
- 代码补丁；
- 网页抓取结果；
- 截图；
- 结构化中间数据；
- 大体积工具原始输出。

这些东西如果都塞进 session 历史，会拖垮上下文和数据库。

所以 Runtime 通常需要独立的 Artifact Store 来保存：

- 大体积对象；
- 版本化结果；

- 部分结果与最终结果；
- 可供用户下载或继续加工的产物。

然后在上下文里只放它们的摘要、引用或关键字段。

Policy / Approval / Permission: Runtime 的安全边界

很多同学把安全理解成“在接口层做鉴权”。

但 Agent Runtime 的安全边界更深，因为真正的风险出现在“模型选了一个结构化动作之后”。

Policy 层至少要管四类东西：

1. 工具可见性

当前用户 / 租户能看到哪些工具、MCP server、A2A agent、skills。

2. 执行权限

看得见不等于能执行。

某些工具只允许查询，不允许修改；

某些动作只有审批通过后才能执行。

3. 数据访问范围

即使是查询，也要限制：

- 哪些资源 URI 可读；
- 哪些文件路径可访问；
- 哪些数据库 schema 可查；
- 哪些租户数据可见。

4. 风险动作审批

例如：

- 发消息；
- 删数据；
- 批量操作；

- 写生产环境；
- 调高权限 API；
- 对外发布内容。

这些都应该由 Runtime 在执行前插入 human-in-the-loop，而不是让模型自由直达。

Executors / Sandboxes: 为什么 Runtime 里要有执行隔离层

模型输出 tool call 后，不应该直接触达核心系统。

更稳妥的架构是：

- Runtime Core 解释动作；
- Executor 负责真正执行；
- 高风险能力在 sandbox / container / isolated worker 中运行；
- 外部系统访问走代理和凭据管理层。

这样设计的原因有三个：

1. **安全**：限制模型影响面；
2. **可靠性**：把执行重试、超时和幂等逻辑集中；
3. **可观测**：所有副作用动作都有统一日志与 trace。

对代码 Agent、Shell、浏览器自动化这类高风险能力，这层尤其重要。

Scheduler / Queue / Durable Workflow: Runtime 不能只做同步调用

Agent Runtime 一旦支持长任务，就必须纳入调度层。

否则：

- 所有执行都绑死在请求线程；
- cancel / resume 很难做；
- 多任务并发没有资源隔离；
- worker 崩溃后无法恢复。

因此成熟 Runtime 往往会和：

- 队列系统；
- 背景任务系统；

- durable workflow 引擎；
- cron / event scheduler；

结合在一起。

也就是说，Runtime Core 负责编排逻辑，Scheduler / Workflow 负责长期执行语义。

Tracing、Logs、Replay、Eval：为什么观测是 Runtime 的内建能力

如果一个 Agent Runtime 没有内建观测能力，出了问题几乎无从排查。

至少要有：

1. Trace

串起一次请求从入口到模型、到工具、到子任务、到远端 Agent 的完整链路。

2. Structured Logs

不是简单打印文本日志，而是结构化记录：

- tenant_id；
- session_id；
- task_id；
- tool_name；
- tool_args 摘要；
- 状态转移；
- error_type；
- cost / latency；
- approval actor。

3. Replay

能在开发或调试环境中复现一次执行，帮助判断问题出在：

- 模型输出；
- 上下文装配；
- 工具异常；

- 状态机逻辑；
- 人工审批环节；
- 远端协议调用。

4. Eval Hooks

Runtime 最好天然支持把某些执行链导入：

- benchmark harness；
- 回归测试；
- 线上 sample review；
- trace grading。

这样你才能持续优化而不是靠拍脑袋。

多租户与资源隔离：平台化 Runtime 的必答题

只要是平台化 Runtime，就一定要回答多租户。

至少包括：

1. 命名空间隔离

- 工具；
- session；
- artifact；
- memory；
- traces；
- webhook；
- 配置。

2. 配额和限流

不同租户、不同用户、不同 agent、不同工具应该有独立额度和并发限制。

3. 凭据隔离

一个租户的 API key、OAuth token、MCP 连接信息不能泄漏到另一个租户。

4. 缓存隔离

如果有 prompt cache、tool metadata cache、retrieval cache，也要 tenant-aware。否则会出现最隐蔽也最危险的串数据问题。

5. 版本与灰度

Runtime 平台通常要允许：

- 某租户先用新版本模型；
- 某租户切换某组工具；
- 某租户试运行新的 skill；
- 出问题时快速回滚。

Runtime 状态机：什么时候继续、什么时候停

一个成熟 Runtime 必须有清晰终止条件。

常见终止原因包括：

- 模型给出最终答案；
- 达到目标状态；
- 完成最终 artifact；
- 进入等待用户输入；
- 进入等待审批；
- 任务取消；
- 达到最大步数；
- 遇到不可恢复错误；
- 达到时间 / 成本预算上限。

这一点特别重要，因为很多初级 Agent 框架的失败根因就是：
没有明确终止条件，导致无限 tool loop 或错误自我修复循环。

一个适合校招生面试的 Runtime 分层回答模板

你可以用下面这套五层法来答，很稳：

第一层：入口与身份层

负责 API、用户身份、租户、限流和请求规范化。

第二层：编排与状态层

负责 session、workflow、task、checkpoint、终止条件、取消恢复。

第三层：能力层

负责 tool registry、MCP、A2A、skills、模型路由和 capability discovery。

第四层：执行与隔离层

负责 executor、sandbox、queue、durable workflow、外部系统调用。

第五层：治理与观测层

负责 permission、approval、audit、trace、logs、replay、eval、灰度与回滚。

这五层讲下来，Runtime 的主干就很完整了。

常见误区

误区 1：Runtime 就是一个 Agent 类封装

太浅。

真正 Runtime 要解决的是执行系统问题，而不是面向对象封装问题。

误区 2：把工具执行直接写在模型回调里

风险很大。

这样会丢失权限边界、审计能力和执行隔离。

误区 3：把状态、日志、artifact 全塞一个库

短期能跑，长期会很乱。

这些对象生命周期和访问模式不同，应该分层存储。

误区 4：有 Workflow 就等于有 Runtime

不完全对。

Workflow 是 Runtime 的一部分，但 Runtime 还包括能力注册、模型路由、安全治理和观测系统。

误区 5：Runtime 只关心后端，不关心前端

错误。

长任务的 phase、流式事件、取消恢复都要求 Runtime 和前端交互契约设计一致。

高频面试题与回答模板

1. Agent Runtime 至少包含哪些核心组件？

标准回答：至少包括编排核心、状态存储、工具 / 能力注册、模型路由、执行器 / 隔离层、调度与恢复、权限审批、观测与回放。

2. 为什么 Runtime 需要单独的 Capability Registry？

标准回答：因为能力来源不再只是本地函数，还可能是 MCP server、A2A remote agent、skill 和插件。统一注册后才能做发现、授权、路由、版本控制和观测。

3. 为什么 Runtime 要把执行与模型解耦？

标准回答：模型只负责表达意图，不应该直接触达外部副作用。Runtime 通过执行层和策略层把模型的自由输出变成受控动作，才能保证安全、可靠和可审计。

4. Runtime 和 Workflow 引擎是什么关系？

标准回答：Workflow 引擎常常是 Runtime 的执行底座之一，尤其在长任务、审批和 durable execution 场景下。但 Runtime 的范围更大，还包括能力管理、模型路由、状态装配和安全治理。

5. 多租户 Runtime 最危险的问题是什么？

标准回答：串租户状态和串租户权限。因为一旦 session、memory、artifact 或凭据没有 tenant-aware，后果比普通系统更严重。

6. Runtime 如何支持线上优化?

标准回答：通过内建 trace、结构化日志、回放、评测 hook、灰度发布和版本管理，让每一次执行都可以被观测、复盘和对比优化。

项目表达模板

“如果让我设计一个 Agent Runtime，我会把它拆成五层。入口层负责身份、租户、限流和请求规范化；编排层负责 session、任务状态、终止条件和中断恢复；能力层统一管理 function tools、MCP、A2A 和 skills，并做模型路由；执行层通过 sandbox、worker 和 durable workflow 跑真正的工具和长任务；治理层负责 permission、approval、audit、trace、replay 和 eval。这样模型不会直接碰外部系统，所有副作用都会经过受控执行器和策略层。系统既能支持单轮问答，也能支持多步、多协议、可恢复、可观测的生产任务。”

小结

把这篇压缩成四句话：

1. Agent Runtime 是模型与外部世界之间的执行系统，不是一个简单 SDK 封装；
2. 它至少要负责编排、状态、能力接入、执行隔离、安全治理和可观测；
3. 它的关键价值在于把模型的自由输出转成受控、可恢复、可审计的系统动作；
4. 一旦系统走向平台化，多租户、版本化、灰度和能力资产化就会成为 Runtime 的一部分。

如果你能围绕这四句话，再辅以一张架构图和五层法回答，这一题在系统设计面里会非常稳。

延伸阅读与更新依据（官方为主）

- [OpenAI Agents SDK](#)
- [OpenAI Sessions](#)
- [OpenAI Conversation State](#)
- [OpenAI Background Mode](#)
- [OpenAI Tool Search](#)
- [LangGraph Workflows and Agents](#)
- [LangGraph Persistence](#)
- [Temporal Workflow Execution](#)

推荐阅读

此处为语雀内容卡片，点击链接查看：

https://www.yuque.com/u63312805/df29bk/yx5erdi9q4ghzg3c?view=doc_embed

09 Skills、插件化与能力封装（重要）

这篇主要考察什么

很多校招生做 Agent 项目时，会把所有逻辑都写在 prompt、tool 函数和几段 glue code 里。短期看当然能跑，但只要项目一多、团队一大，就会出现典型问题：

- 相似场景每个项目都重写一次 prompt；
- 领域知识散落在代码和文档里，无法复用；
- 一些复杂能力要复制脚本、模板和示例；
- 不同 Agent 对同类任务的做法不一致；
- 上线后很难做版本管理、评测和回滚。

这时就会自然走向一个问题：能不能把“零散的能力”沉淀成标准化、可复用、可版本化的资产？这就是 Skills、插件化和能力封装要解决的核心问题。

面试官问这一题，真正想看的是：

1. 你是否理解“能力资产化”的价值；
2. 你能否区分 Tool、Plugin、Skill、MCP Server、A2A Agent 的层次；
3. 你是否知道 Skill 不只是 prompt snippet，而是可能包含指令、脚本、资源、模板和执行环境要求；
4. 你是否考虑过版本、评测、安全、动态加载和平台复用。

可以这样一句话描述：

Skill 是把一类可重复的做事方法、领域经验和配套资源，封装成一个可发现、可调用、可版本化的能力包。它通常高于单个 Tool，低于完整 Agent，目标不是替代工具，而是把“如何解决某类问题”的经验资产化、模块化和平台化。

先把几个常混概念拆开

这是这篇最关键的基础题。

1. Tool

Tool 是最细粒度的动作接口。通常代表一次能力调用，例如：

- 查订单；
- 搜网页；
- 读文件；
- 发邮件；
- 执行 shell；
- 调数据库。

它更像“一个动作”。

2. Plugin

Plugin 更像扩展点的通称。它可能包含：

- 一组工具；
- 一套 UI；
- 一段集成逻辑；
- 某个平台的扩展能力。

Plugin 是一个偏产品 / 平台视角的词，粒度不固定。

3. Skill

Skill 更强调“可复用的方法包”或“经验包”。它往往不仅有工具调用说明，还会包含：

- 任务指令；
- 示例；
- 最佳实践；
- 脚本；
- 参考资料；
- 执行环境依赖；
- 适用和不适用场景。

一句话记忆：Tool 是一次动作，Skill 是一套做法。

4. MCP Server

MCP Server 是一种标准化能力提供方式。它可以暴露 tools、resources、prompts，供宿主或 Agent 统一接入。它关注的是“能力如何被开放协议接入”，而不一定直接表达“解决某类任务的方法论”。

5. A2A Agent

A2A Agent 是更高层的能力体。它是一个完整自治 Agent，有身份、任务状态、artifact 和协作语义。Skill 通常不会有这么重的自治与长期任务语义。

所以这几个概念的关系大致可以这样理解：

- Tool：最细粒度动作
- Skill：一组动作 + 指令 + 资源 + 经验封装
- MCP Server：标准化接入这些能力的协议载体之一
- A2A Agent：更高层次、可协作的自治能力体

为什么 Skills 会成为 Agent 时代的重要能力抽象

1. Prompt 迟早会变成知识垃圾场

很多项目早期的习惯是：有一个新的业务场景，就在系统提示词里再加一段说明。久而久之 prompt 里会堆满：

- 业务规则；
- 例子；
- 操作步骤；
- 异常处理建议；
- 语气约束；
- 小技巧。

这会带来两个后果：

- prompt 越来越长，越来越难维护；
- 同类经验难以在不同 Agent 和项目间共享。

Skill 的价值就在于把这类“经验性内容”从超级 prompt 中拆出来。

2. 复杂能力通常不只是一个函数

例如“修复 Python 测试失败”这个能力，往往不只是一个 `run_tests()` 工具，而是一整套方法：

- 先收集错误日志；
- 再定位失败测试；
- 再检查最近改动；
- 再生成最小 patch；
- 再重新运行测试；
- 再整理修复说明。

这种能力显然更适合封装成 Skill，而不是散落在多个工具定义和 prompt 里。

3. 经验需要版本化和评测

一旦某类能力被多人反复使用，你就会关心：

- 哪个版本效果更好；
- 哪个版本更省 token；
- 哪个版本在某些 benchmark 上通过率更高；
- 出问题如何回滚；
- 新版本能否灰度给部分团队。

这些都要求 Skill 成为一等资产，而不是“谁在 prompt 里改了几句”。

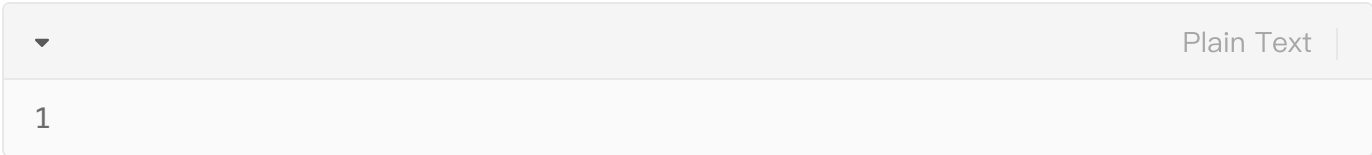
4. 技能复用是平台竞争力

真正的平台化价值，不是“接了很多模型”，而是“沉淀了很多好用的能力资产”。一个成熟平台会把：

- 代码审查；
- 竞品调研；
- 会议纪要整理；
- 报销合规审查；
- 法务条款风险检查；
- 客诉归因分析；

都沉淀为可以反复复用的 Skill，而不是每个项目重写一次。

一图看懂 Skill 的位置



这张图表达的是：Skill 往往位于 Tool 和完整 Agent 之间。它比 Tool 更高层，强调“怎么做”；又比完整 Agent 更轻，不一定具备完整自治和长期协作能力。

Agent Skills 标准：为什么它值得关注

现在 Skill 已经不只是某家平台的私有概念，而正在形成更开放的格式生态。这一点对面试很重要，因为它意味着 Skill 不再只是“某个平台里的小功能”，而是在往跨 Agent 复用标准演进。

一个比较清晰的抽象是：一个 Skill 目录通常至少包含一个 `SKILL.md` 作为描述入口，此外还可以带：

- 脚本；
- 参考资料；
- 资源文件；
- 示例；
- 辅助资产。

也就是说，Skill 天生就比“几段 instruction”更丰富。

一个 Skill 应该封装什么内容

这是很好的设计题。你可以从六个方面回答：

1. 适用任务

明确这个 Skill 是干什么的，例如：

- “总结 PR 变更风险”
- “定位 Python 测试失败”
- “撰写周报草稿”
- “分析客服工单根因”

2. 调用条件

什么时候应该用它，什么时候不该用。这和 Tool schema 的边界描述类似，但更偏任务层。

3. 操作步骤或策略

Skill 的核心不只是告诉模型“去做”，而是提供一套可复用方法，例如：

- 优先查哪些信息；
- 如何分步骤；
- 如何输出结果；
- 常见失败如何处理；
- 遇到歧义如何追问。

4. 依赖资源与工具

Skill 可能需要：

- 某几个工具；
- 某个 MCP server；
- 某个 shell 环境；
- 一些示例文件或参考模板。

5. 输出标准

一个好的 Skill 会规定：

- 结果结构；
- 质量要求；
- 风险提示；
- 验收清单。

6. 示例与反例

示例非常重要，因为它能把隐性的经验显化出来。尤其对校招生项目来说，加入几组“输入—输出示例”和“常见错误案例”，会极大提升 Skill 的稳定性。

Skills 与 Prompt Engineering 的关系

一个非常值得主动补充的观点是：

Skill 不是 Prompt Engineering 的替代品，而是 Prompt Engineering 的资产化形式。

也就是说，你在某个场景里做出的好 prompt、好流程、好工具组合，并不应该永远停留在一段字符串里，而应该沉淀为一个可维护、可复用的 Skill。

这个观点很适合面试，因为它把“会调 prompt”提升成了“会沉淀能力资产”。

Skills 与 Runtime 的关系

Skill 很少独立存在，它通常由 Runtime 发现、挂载、调度和观测。Runtime 可能会做：

- 根据任务类型选择是否启用某个 Skill；
- 把 Skill 需要的文件和脚本挂到执行环境里；
- 根据 namespace / tool search 动态加载；
- 把 Skill 的元数据暴露给模型；
- 记录某次 Skill 调用的 trace 和评测结果。

所以 Skill 的平台价值，很大程度取决于 Runtime 是否把它当作一等资产来管理。

OpenAI 这一侧的 Skills，可以怎么理解

当前一个明显趋势是：平台不仅提供单个 tool，还开始支持 Skill 这种更高层的封装方式。在 OpenAI 这一侧，你至少可以理解到下面这些点：

1. Skill 是版本化能力包

它不只是几句描述，而是可以作为一个有版本的对象管理。这意味着：

- 可以创建 / 更新 / 获取 Skill；
- 不同版本可以比较；
- 更适合平台化演进。

2. Skill 能和执行环境结合

在容器或 shell 场景中，Skill 可以作为一组文件、说明和资源被挂载到环境里。这对代码 Agent、自动化执行类任务非常重要，因为很多知识和脚本不是纯文本提示能完整表达的。

3. Skill 与 Tool Search / Deferred Loading 形成互补

如果工具和能力很多，不应该全部一股脑塞给模型。Skill 可以承担“更高层任务包”的角色，而 tool search / deferred loading 解决“运行时按需发现和注入具体工具”的问题。两者结合，能让系统既保持丰富能力，又避免上下文膨胀。

4. Skill 不只是给模型看，也给执行环境看

一个成熟回答可以提到：Skill 既是模型侧的指令资产，也是运行时侧的执行资产，因为它可能包含脚本、参考文件、示例和环境要求。

Claude / Anthropic 这一侧的 Skills 可以怎么理解

Anthropic 在 Claude Code / Agent SDK 方向上也在持续强化 Skill 这一层抽象。一个值得你知道的点是：Skill 同样通常以 `SKILL.md` 为入口，并可结合更丰富的目录、脚本、上下文注入和子代理调用等能力。这说明“Skill 作为跨平台能力格式”的趋势不是单家产品自说自话，而是生态正在汇聚。

面试里可以用一句话表达这个观察：

Skill 正在从“某个产品里的技巧包”演化成“跨 Agent 生态可复用的能力封装格式”。

Skills、MCP、A2A 怎么搭配

这是很有深度的系统问题。

Skill + Tool

最常见。Skill 给出方法，Tool 负责做动作。例如“代码修复 Skill”内部会指导如何使用 `read_file`、`run_tests`、`apply_patch` 等工具。

Skill + MCP

很适合做平台能力复用。例如“企业知识检索 Skill”可能说明：

- 先读某类资源；
- 再调某个 MCP search tool；
- 再根据模板输出总结。

MCP 提供能力接口，Skill 提供使用方法。

Skill + A2A

当某种方法不是简单调用本地工具，而是需要委托给某类专家 Agent 时，Skill 也可以把“什么时候交给哪个远端 Agent”这种经验封装进去。例如“法务审查 Skill”可能规定：

- 金额大于某阈值时转交法务 Agent；
- 无风险模板合同时本地直接处理。

所以 Skill 往往是上层策略封装，可以把 Tool、MCP、A2A 都串起来。

一个值得背的 Skill 目录示例

▼ Plain Text |

```
1  skills/
2    code-debug-python/
3      SKILL.md
4    scripts/
5      run_tests.sh
6      collect_failure_logs.py
7    references/
8      pytest-triage-checklist.md
9      common-failure-patterns.md
10   examples/
11     input-example.md
12     output-example.md
```

这个目录结构非常有启发性。它说明一个 Skill 不是只有说明文，而是可以携带真正帮助执行的资源。

Skill 设计时最重要的七条原则

1. 任务边界明确

不要做“万能 Skill”。边界越清晰，调用效果越稳，评测也越容易。

2. 方法可复述

Skill 的价值在于可复制的做法，而不是“某个高手当时灵光一现写出来的 prompt”。

3. 尽量和具体模型解耦

好的 Skill 应尽量表达任务方法，而不是过度依赖某个模型的偶然行为。这样迁移和复用才更容易。

4. 工具依赖显式化

如果 Skill 依赖特定工具、特定 MCP server、特定执行环境，要写清楚。不要把这些依赖隐含在某句模糊提示里。

5. 输出标准明确

如果没有输出标准，Skill 很容易“看起来会做，实际每次做法都不一样”。

6. 可评测

Skill 应该有办法被 benchmark、回归测试或人工评审。否则就无法知道版本升级到底有没有变好。

7. 可观测和可回滚

每次 Skill 调用最好都能追踪到：

- 用了哪个版本；
- 耗时多少；
- 调了哪些工具；
- 是否成功；
- 产出质量如何。

这样出问题时才能快速回滚。

Skill 的动态加载与发现

当 Skill 越来越多时，你不能让模型每轮都看见所有 Skill。常见做法包括：

- 按任务域分组；
- 按租户启用；
- 根据入口意图召回候选 Skill；
- 用工具搜索或命名空间缩小范围；
- 在运行时按需挂载 Skill 到容器或环境。

这背后的核心逻辑和“大工具集不能全塞进上下文”完全一致。所以 Skill 平台一旦规模化，**发现与动态加载** 就会变得很关键。

Skill 的评测：怎么知道它真的有价值

面试里如果能把“怎么评估 Skill”讲出来，会很强。可以从三个维度回答：

1. 效果提升

例如：

- 任务完成率提升；
- benchmark 通过率提升；
- 错误率下降；
- 人工返工率下降。

2. 成本与效率

例如：

- 平均 token 降低；
- 平均耗时下降；
- 减少不必要工具调用；
- 减少人工干预。

3. 一致性与可维护性

例如：

- 同类任务输出格式更稳定；
- 新人上手更快；
- 改一次 Skill，多处受益；
- 回归测试更容易做。

所以 Skill 的价值不只是“好像更聪明了”，而是要能被量化和对比。

安全：为什么 Skill 也会带来风险

Skill 不是天然安全的。它可能带来：

1. 隐式权限放大

一个 Skill 可能通过组合多个工具，实现比单个工具更强的能力。因此 Skill 也需要权限审查，而不是只审单工具。

2. 过时知识

如果 Skill 内置的规则、链接、脚本、模板过时，可能持续输出错误行为。所以 Skill 需要版本管理和维护者。

3. 注入与供应链风险

如果 Skill 包含脚本、外部资源引用或第三方模板，就可能出现供应链风险。平台需要：

- 审核 Skill 来源；
- 控制执行环境；
- 限制高风险脚本权限；
- 保留审计记录。

4. 误调用

如果 Skill 的适用边界写得不清楚，模型可能在不合适场景下调用它。所以 Skill 元数据设计和触发条件同样重要。

常见误区

误区 1: Skill 就是 prompt 模板

太窄。Skill 更像“任务方法包”，可能包含脚本、资源、示例和执行要求。

误区 2: 有了 Skill 就不需要工具

错误。Skill 负责“怎么做”，工具负责“真正去做”。

误区 3: Skill 越大越好

不对。过大的 Skill 会边界模糊、难以评测、难以动态加载。通常更适合按明确任务边界拆分。

误区 4: Skill 不需要版本管理

错误。一旦 Skill 被多人使用，它就是平台资产，必须版本化、可灰度、可回滚。

误区 5: Skill 是纯模型侧概念

不完整。Skill 也涉及运行时、容器环境、工具依赖和平台治理。

高频面试题与回答模板

1. Tool 和 Skill 的区别是什么？

标准回答：Tool 是单次动作接口，Skill 是围绕某类任务的一整套方法封装，通常包含指令、示例、资源和脚本，目标是复用做事方法，而不是只暴露动作。

2. 为什么 Agent 系统需要 Skills？

标准回答：因为很多稳定有效的经验不应长期埋在 prompt 和代码里。Skill 能把这些经验资产化、版本化、评测化，从而实现跨项目复用和平台沉淀。

3. Skill 和 MCP 是什么关系？

标准回答：MCP 更像能力接入协议，Skill 更像能力使用方法包。Skill 可以建立在 MCP 提供的 tools / resources 之上。

4. Skill 和 A2A Agent 的区别是什么？

标准回答：Skill 通常更轻，偏方法与资源封装；A2A Agent 是完整自治体，具有任务状态、artifact 和协作语义。Skill 可以决定何时调用 A2A Agent，但它本身不一定是一个 Agent。

5. Skill 最重要的设计原则是什么？

标准回答：边界清晰、方法可复用、依赖显式、输出标准明确、可评测、可版本化。

6. 怎么评估一个 Skill 是否值得保留？

标准回答：看它是否稳定提升任务效果、降低成本、提高一致性，并且这种收益是否能在 benchmark、回归测试或人工评审中被量化出来。

项目表达模板

“我们发现很多 Agent 项目虽然工具集相似，但真正拉开效果差距的是一些可重复的方法论，比如如何分析日志、如何整理 PR 风险、如何生成业务周报。最初这些经验都散在 prompt 里，后来我们把它们沉淀成 Skills：每个 Skill 都有清晰的任务边界、步骤说明、示例、参考资料和依赖工具列表，必要时还会附带脚本与模板。Runtime 会根据任务类型动态暴露候选 Skills，并记录使用的是哪个版本、调了哪些工具、效果如何。这样同类能力可以跨项目复用，出问题也能快速回滚到旧版本，而不是每个团队各自维护一份 prompt。”

小结

把这篇压缩成四句话：

1. Tool 是动作接口，Skill 是方法与经验的能力包；
2. Skill 的价值在于把 Prompt Engineering 和领域经验沉淀成可复用、可版本化、可评测的资产；
3. Skill 与 MCP、A2A、Tool、Runtime 是互补关系，而不是替代关系；
4. 真正的平台竞争力，往往来自高质量能力资产的持续积累，而不是只接了多少模型。

如果你能围绕这四句话，再补充目录结构、版本管理、动态加载和评测方式，这一题会非常完整。

延伸阅读与更新依据（官方为主）

- [OpenAI Skills Guide](#)
- [OpenAI Tool Search](#)
- [Agent Skills Standard](#)
- [Claude Code Skills](#)
- [OpenAI Agents SDK](#)